DX Framework полная книга по движку и построчному разбору runtime

Полное справочное издание 12.0

C++ математика ECS DirectX 11 HLSL Bullet assets и анимация

Построчный разбор собственного runtime кода и порядка кадра

Оружие физика геометрия комнаты энергия ремонт и эффекты

NPC одиночная тактика cover navigation crew и безопасная архитектура AI

Editor и устаревшая система затопления осознанно исключены

Собрано по текущему снимку репозитория

1 ноября 2026 года

Как пользоваться изданием 12.0

Книга остаётся единым учебником, а новое издание добавляет проверяемый справочник исходников. Для изучения системы сначала прочитай теорию, затем открой профильную runtime главу и пройди метод сверху вниз вместе с кодом.

Маршрут	Раздел	Что станет понятно
Основа	Математика и C++	память типы матрицы лучи время и ошибки
Каркас	Core Application ECS	инициализация кадр ownership input scene jobs
Gameplay	Игрок оружие мир	movement damage inventory rooms power repair effects
Графика	DirectX и HLSL	renderer passes resources shaders PBR skinning и тени
Assets	Импорт и анимация	Assimp GLB bones clips IK profiles и Studio
Физика	Bullet и geometry	bodies rays controller mesh cutting и repair
AI	NPC runtime	perception tactics cover navigation crew и squad layer
Проверка	Полный индекс	каждый собственный runtime модуль и его глава

Как выглядит построчное чтение

Для каждого метода отмечай пять вещей: что пришло на вход, где ранний выход, какие данные прочитаны, кто изменил состояние и какой следующий модуль увидит результат. Скобки и одинаковые Release объединены, значимые проверки и записи объясняются в порядке выполнения.

Граница этого издания

Editor не разбирается. FloodingSystem не разбирается как рудимент ранней концепции подлодки. Сохранившиеся полезные room, pressure, power и structural модули объясняются с пометкой исторических имён.

Полный справочник runtime систем

Издание 12 продолжает единую книгу и добавляет подробный разбор собственного runtime кода DX Framework. Формат изменён по просьбе автора проекта: главы объясняют не только взаимодействие систем, но и то, что делает каждая значимая строка внутри методов.

Границы разбора

В справочник входят собственные системы движка, игровые системы, физические адаптеры, загрузчики, renderer, shader classes, AI и инструменты runtime диагностики.

Осознанно исключены:

- каталог `Editor` и большой `EditorUI`, кроме уже имеющихся объяснений границы с runtime;
- `FloodingSystem`, оставшийся от ранней концепции игры о подлодках;
- исходники сторонних библиотек ImGui, Assimp, stb, Bullet и tinygltf;
- механические include, одиночные скобки и повторяющиеся `Release`, если они уже объяснены как одна операция.

Собственные обёртки над сторонними библиотеками разобраны. Например, книга не объясняет внутренний код Assimp, но подробно объясняет `AssimpModelClass` и `AssetLoadSystem`.

Что означает построчный разбор

Одна запись справочника соответствует одной значимой строке или маленькой неразделимой группе строк. В ней указано:

- какое значение читается;
- что вычисляется;
- какое состояние изменяется;
- почему существует проверка;
- кто владеет вызванным внешним объектом;
- какой результат увидит следующая стадия.

Строки `if`, вычисление условия и тело ветви объясняются отдельно. Последовательность однотипных освобождений COM ресурсов объясняется одним блоком, потому что повторение одной фразы для каждой строки не добавляет понимания.

Исторические имена

В коде всё ещё встречаются `Submarine`, `SUBMARINE_EXT`, `waterLevel` и связанные названия. Если такой код продолжает участвовать в текущей физике, комнатах или соединениях, он разобран как существующий runtime контракт и помечен историческим именем. Модель затопления не развивается и не предлагается как направление игры.

Как читать справочник

Сначала открой файл рядом с книгой. Прочитай один метод целиком. Затем пройди пояснения сверху вниз и после каждой ветви предскажи состояние полей. Не нужно читать весь том подряд. Это справочник для возвращения к системе перед её изменением.

Главный вопрос каждой функции: какое состояние она имеет право изменить и какое обещание оставляет следующему коду.

Математика движка с нуля

Эта глава не является списком формул. Каждая тема проходит один и тот же путь: смысл, ручной расчёт, запись на C++, применение в DX Framework и проверка результата. Если ручной расчёт не совпал с кодом, не переходи дальше.

1 Числа единицы измерения и погрешность

`float` хранит приближённое число. Поэтому координаты, длины и таймеры почти никогда не сравнивают с нулём через `==`.

```
constexpr float epsilon = 0.0001f;
const bool almostZero = std::fabs(value) <= epsilon;
```

Всегда подписывай единицы мысленно или в имени:

Величина	Единица	Пример
позиция	метры	`position = { 4, 1, -2 }`
скорость	метры в секунду	`runSpeed = 5.5f`
время кадра	секунды	`deltaTime = 0.016f`
угол DirectXMath	радианы	`XMConvertToRadians(45.0f)`
здоровье	условные единицы	`health = 100.0f`

Если скорость равна 5 м/с, а кадр длится 0.02 с, перемещение за кадр равно:

$$\text{deltaPosition} = \text{speed} \cdot \text{deltaTime} = 5 \cdot 0.02 = 0.1$$

Ошибка новичка: прибавлять `speed` каждый кадр. Тогда результат зависит от FPS.

2 Точка и вектор

Точка отвечает на вопрос «где». Вектор отвечает на вопрос «куда и насколько».

Пусть NPC находится в точке A = (2, 1, 3), а цель в B = (8, 1, 7). Направление от NPC к цели получается вычитанием начала из конца:

$$d = B - A = (8-2, 1-1, 7-3) = (6, 0, 4)$$

```
XMFLOAT3 direction{
    target.x - npc.x,
    target.y - npc.y,
    target.z - npc.z
};
```

Перепутанный порядок даст вектор от цели к NPC. Это не «почти тот же результат»: знак определяет направление.

Сложение векторов

Складываем соответствующие координаты:

$$(a_x, a_y, a_z) + (b_x, b_y, b_z) = (a_x + b_x, a_y + b_y, a_z + b_z)$$

Пример: движение вперёд (0, 0, 3) и вправо (2, 0, 0) даёт (2, 0, 3).

Умножение на число

Каждая координата умножается на одно число:

$$2 \cdot (3,0,-1) = (6,0,-2)$$

Так из единичного направления получают скорость, а из скорости — смещение за кадр.

3 Длина нормализация и расстояние

Длина вектора вычисляется по теореме Пифагора:

$$|v| = \sqrt{(x^2 + y^2 + z^2)}$$

Для d = (6, 0, 4):

$$|d| = \sqrt{(6^2 + 0^2 + 4^2)} = \sqrt{(52)} \approx 7.211$$

Нормализация оставляет направление, но делает длину равной единице:

$$\text{unit d} = (d)/(|d|) \approx (0.832,0,0.555)$$

```
XMVECTOR raw = XMLoadFloat3(&direction);
const float length = XMVectorGetX(XMVector3Length(raw));

if (length > 0.0001f)
{
    XMVECTOR unit = XMVector3Normalize(raw);
}
```

Нулевой вектор нормализовать нельзя: деление на ноль породит `NaN`. Поэтому сначала проверяется длина.

Расстояние без квадратного корня

Если нужно только сравнить расстояние с радиусом, корень не нужен:

$$\text{distanceSquared} = dx^2 + dy^2 + dz^2$$

```
const float distanceSquared = dx * dx + dy * dy + dz * dz;
if (distanceSquared < attackRange * attackRange)
{
    // Цель внутри дальности.
}
```

Это дешевле и хорошо подходит для массового поиска кандидатов. Настоящую длину вычисляй только для финального кандидата или интерфейса.

4 Скалярное произведение и поле зрения

Скалярное произведение двух нормализованных векторов показывает, насколько одинаково они направлены:

$$a \cdot b = a_xb_x + a_yb_y + a_zb_z$$

Значение	Геометрический смысл
1	направления совпадают
0	угол 90 градусов
-1	направления противоположны

Связь с углом:

$$a \cdot b = |a||b|\cos(\theta)$$

Если оба вектора единичные, остаётся `cos(theta)`. Для поля зрения шириной 90 градусов половинный угол равен 45 градусов:

$$\text{threshold} = \cos(45°) \approx 0.707$$

```
const XMVECTOR forward = XMVector3Normalize(forwardVector);
const XMVECTOR toTarget = XMVector3Normalize(targetVector);
const float dot = XMVectorGetX(XMVector3Dot(forward, toTarget));
const float threshold = std::cos(XMConvertToRadians(45.0f));
const bool insideFov = dot >= threshold;
```

Важно: поле зрения ещё не доказывает видимость. После него нужен raycast, иначе NPC увидит цель сквозь стену.

5 Векторное произведение

Векторное произведение создаёт вектор, перпендикулярный двум исходным:

$$a \times b = (a_yb_z - a_zb_y, a_zb_x - a_xb_z, a_xb_y - a_yb_x)$$

В движке оно полезно, чтобы получить правое направление относительно `forward` и мирового `up`:

```
XMVECTOR right = XMVector3Normalize(XMVector3Cross(up, forward));
```

Порядок важен: `Cross(a, b)` и `Cross(b, a)` смотрят в противоположные стороны. Если правая точка выглядывания оказалась слева, первым делом проверь порядок аргументов и систему координат.

6 Что такое матрица

Матрица — прямоугольная таблица чисел. В 3D часто используется матрица 4 на 4: она помещает поворот, масштаб и перенос в одну операцию.

$$M = [\ m_(11), m_(12), m_(13), m_(14);\ m_(21), m_(22), m_(23), m_(24);\ m_(31), m_(32), m_(33), m_(34);\ m_(41), m_(42), m_(43), m_(44)\ ]$$

Индекс `m23` означает строку 2, столбец 3. Прежде чем программировать матрицы, научись уверенно находить ячейку по двум индексам.

Единичная матрица

Единичная матрица ничего не меняет:

$$I = [\ 1, 0, 0, 0;\ 0, 1, 0, 0;\ 0, 0, 1, 0;\ 0, 0, 0, 1\ ]$$

Она играет ту же роль, что число 1 при обычном умножении.

7 Как складывать матрицы

Матрицы можно складывать, только если у них одинаковое число строк и столбцов. Складывается каждая пара соответствующих ячеек.

$$A = [\ 1, 2;\ 3, 4\ ], B = [\ 5, 6;\ 7, 8\ ]$$

$$A + B = [1 + 5, 2 + 6;\ 3 + 7, 4 + 8] = [6, 8;\ 10, 12]$$

Псевдокод показывает главный принцип:

```
for (int row = 0; row < rowCount; ++row)
    for (int column = 0; column < columnCount; ++column)
        result[row][column] = a[row][column] + b[row][column];
```

Сложение матриц редко используется для построения `world`-матрицы. Трансформации обычно соединяются умножением. Но сложение важно знать для накопления поправок, смешивания данных и понимания линейной алгебры.

Умножение матрицы на число

Число умножается на каждую ячейку:

$$2A = [2, 4;\ 6, 8]$$

8 Как умножать матрицы

Чтобы вычислить одну ячейку результата, берём строку левой матрицы и столбец правой, перемножаем пары и складываем.

$$C_(row, column) = \Sigma_k\ A_(row, k) B_(k, column)$$

Рассмотрим полный расчёт:

$$A = [1, 2;\ 3, 4], B = [5, 6;\ 7, 8]$$

$$C_(11) = 1 \cdot 5 + 2 \cdot 7 = 19$$

$$C_(12) = 1 \cdot 6 + 2 \cdot 8 = 22$$

$$C_(21) = 3 \cdot 5 + 4 \cdot 7 = 43$$

$$C_(22) = 3 \cdot 6 + 4 \cdot 8 = 50$$

$$AB = [19, 22;\ 43, 50]$$

Матрицы не перестановочны: обычно `A * B != B * A`. Поэтому порядок `scale * rotation * translation` является частью поведения, а не косметикой.

```
for (int row = 0; row < 2; ++row)
{
    for (int column = 0; column < 2; ++column)
    {
        result[row][column] = 0.0f;
        for (int k = 0; k < 2; ++k)
            result[row][column] += a[row][k] * b[k][column];
    }
}
```

9 Масштаб поворот и перенос

В `TransformComponent::GetWorldMatrix` текущего проекта операции создаются отдельно:

```
XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
XMMATRIX rotX = XMMatrixRotationX(rotation.x);
XMMATRIX rotY = XMMatrixRotationY(rotation.y);
XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
XMMATRIX rotationMatrix = rotX * rotY * rotZ;
XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
return scaleMatrix * rotationMatrix * translationMatrix;
```

Читаем не как «набор непонятных матриц», а как конвейер:

1. локальная вершина получает масштаб;
2. результат поворачивается по X, затем Y, затем Z в принятой проектом конвенции;
3. результат переносится в позицию сущности;
4. получается мировая координата.

Если сначала перенести объект, а затем повернуть, перенос тоже может повернуться вокруг начала координат. Поэтому объект начнёт двигаться по дуге вместо поворота на месте.

Координата и направление преобразуются по разному

`XMVector3TransformCoord` применяет перенос. Для точки дула это правильно. Направление не должно получать перенос; для него используют `XMVector3TransformNormal` или только поворотную часть.

```
XMVECTOR worldPoint = XMVector3TransformCoord(localPoint, worldMatrix);
XMVECTOR worldDirection = XMVector3TransformNormal(localDirection, worldMatrix);
```

10 Обратная матрица

Обратная матрица отменяет преобразование:

$$M^{(-1)}M = I$$

Если `world` переводит локальную точку в мир, `inverseWorld` возвращает мировую точку в локальное пространство.

```
XMMATRIX world = transform.GetWorldMatrix();
XMMATRIX inverseWorld = XMMatrixInverse(nullptr, world);
XMVECTOR localPoint = XMVector3TransformCoord(worldPoint, inverseWorld);
```

Практическое применение: определить, где попадание находится относительно отсека или корпуса, а не относительно всей сцены.

Не всякая матрица обратима. Нулевой масштаб по оси делает преобразование вырожденным. Поэтому масштаб `(0, 1, 1)` опасен для кода, которому нужна обратная матрица.

11 Пространства координат

Одна и та же точка проходит несколько пространств:

Пространство	Вопрос
локальное	где точка относительно модели
мировое	где точка на уровне
вид камеры	где точка относительно камеры
clip	как точка прошла проекцию
NDC	где точка в нормализованном экране
экранное	какой это пиксель

Типовой конвейер:

$$p_{(clip)} = p_{(local)} \cdot World \cdot View \cdot Projection$$

Если модель видна, но raycast идёт не туда, часто смешаны локальная и мировая координаты. Всегда спрашивай: «в каком пространстве находится эта переменная?»

12 Что такое луч

Луч задаётся началом O и нормализованным направлением D:

$$P(t) = O + tD, t \geq 0$$

`t` — расстояние вдоль луча, если D единичный. Для конечного отрезка вводится `0 <= t <= maxDistance`.

Пример: O = (1, 2, 3), D = (0, 0, 1). При t = 5 получаем P = (1, 2, 8).

```
XMFLOAT3 origin = muzzleWorldPosition;
XMFLOAT3 rawDirection{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};

XMVECTOR direction = XMVector3Normalize(XMLoadFloat3(&rawDirection));
```

Создание луча всегда состоит минимум из трёх вопросов:

1. Откуда начинается луч?
2. Куда он направлен?
3. Какова максимальная длина и кого он должен игнорировать?

Для винтовки начало должно быть у дула, а не у центра NPC или камеры. Иначе визуальный ствол и физический выстрел разойдутся.

13 Луч из курсора камеры

Для выбора объекта мышью нужно построить две мировые точки: на ближней и дальней плоскости камеры.

1. Перевести пиксель мыши в NDC.
2. Создать точки near и far в clip-пространстве.
3. Умножить их на обратную `ViewProjection`.
4. Разделить на `w`.
5. Началом сделать near, направлением — `normalize(far - near)`.

$$x_(ndc) = 2(x_(screen)/width) - 1$$

$$y_(ndc) = 1 - 2(y_(screen)/height)$$

Y переворачивается, потому что экранные координаты обычно растут вниз, а NDC — вверх.

14 Пересечение луча с плоскостью

Плоскость задаётся точкой Q и нормалью N. Подставляем луч в уравнение плоскости:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Алгоритм:

1. вычислить denominator = dot(D, N);
2. если модуль denominator почти нулевой, луч параллелен плоскости;
3. вычислить t;
4. если t < 0, пересечение позади начала луча;
5. вернуть O + tD.

```
const float denominator = Dot(rayDirection, planeNormal);
if (std::fabs(denominator) <= 0.0001f)
    return false;

const float t = Dot(planePoint - rayOrigin, planeNormal) / denominator;
if (t < 0.0f)
    return false;

hitPoint = rayOrigin + rayDirection * t;
```

Применение: поставить маркер на полу, найти точку приказа, пересечь прицел с условной тактической плоскостью.

15 Пересечение луча со сферой

Сфера задаётся центром C и радиусом r. Ищем точку луча, расстояние которой до C равно r:

$$|O + tD - C|^2 = r^2$$

После раскрытия получается квадратное уравнение. Удобная реализация для нормализованного D:

```
const Vec3 oc = origin - center;
const float b = Dot(oc, direction);
const float c = Dot(oc, oc) - radius * radius;
const float discriminant = b * b - c;

if (discriminant < 0.0f)
    return false;

const float root = std::sqrt(discriminant);
float t = -b - root;
if (t < 0.0f)
    t = -b + root;
return t >= 0.0f;
```

Отрицательный дискриминант означает промах. Два корня — вход и выход из сферы. Обычно нужен ближайший неотрицательный.

16 Пересечение луча с AABB

AABB — коробка, выровненная по мировым осям. Slab-алгоритм рассматривает диапазон допустимых `t` для каждой оси и пересекает диапазоны.

```
float tMin = 0.0f;
float tMax = maxDistance;

for (int axis = 0; axis < 3; ++axis)
{
    if (std::fabs(direction[axis]) < epsilon)
    {
        if (origin[axis] < boxMin[axis] || origin[axis] > boxMax[axis])
            return false;
        continue;
    }

    float t1 = (boxMin[axis] - origin[axis]) / direction[axis];
    float t2 = (boxMax[axis] - origin[axis]) / direction[axis];
    if (t1 > t2)
        std::swap(t1, t2);

    tMin = (std::max)(tMin, t1);
    tMax = (std::min)(tMax, t2);
    if (tMin > tMax)
        return false;
}
```

Ключевая мысль: луч должен одновременно находиться внутри допустимого интервала по X, Y и Z.

17 Raycast в физическом мире

Математический луч только описывает линию. Raycast задаёт миру вопрос: «какой физический объект первым встретится на этом отрезке?»

В проекте при выстреле передаются:

Аргумент	Назначение
`origin`	мировая позиция дула
`direction`	направление выстрела
`world`	Bullet physics world
`range`	конечная дальность
`shooterEntity`	сущность, которую нельзя принять за цель
`shooterObject`	физическое тело стрелка для исключения
`rayHit`	структура результата

Не используй заранее выбранного `targetEntity` как доказательство попадания. Урон должен получить первый объект луча: стена, союзник или цель.

18 Угол поворота без скачка через 360 градусов

Обычное вычитание углов ломается рядом с границей `-pi` и `pi`. Кратчайшая разница вычисляется так:

$$\text{error} = \operatorname{atan2}(\sin(\text{target} - \text{current}), \cos(\text{target} - \text{current}))$$

```
const float yawError = std::atan2(
    std::sin(targetYaw - currentYaw),
    std::cos(targetYaw - currentYaw));
```

Результат всегда лежит в диапазоне от `-pi` до `pi`. Знак говорит, в какую сторону вращаться.

19 Время таймеры и плавное изменение

Таймер обратного отсчёта:

```
timer = (std::max)(0.0f, timer - deltaTime);
if (timer <= 0.0f)
{
    // Событие разрешено.
}
```

Линейная интерполяция:

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

```
const float value = a + (b - a) * t;
```

Ограничивай `t` диапазоном 0..1, если не нужна экстраполяция. Для плавности, одинаковой при разном FPS, полезно экспоненциальное приближение:

$$\mathrm{alpha} = 1 - e\text{^}(-\mathrm{speed} \cdot \ \mathrm{deltaTime})$$

```
const float alpha = 1.0f - std::exp(-responseSpeed * deltaTime);
current += (target - current) * alpha;
```

20 Как математика связана с текущим NPC

Кодовая задача	Математика	Проверка в отладчике
найти ближайшего врага	квадрат расстояния	линия до выбранной цели
проверить поле зрения	нормализация и dot	конус или граничные лучи
подтвердить видимость	raycast	зелёный или красный луч
направить ствол	матрицы кости и мира	точка muzzle
запретить выстрел из стены	короткий ray chest to muzzle	луч упирается в укрытие
проверить линию огня	конечный raycast	первый пересечённый entity
оценить укрытие	несколько лучей на высотах тела	трассеры к кандидату
повернуться к угрозе	atan2 и кратчайшая разница	yaw error

21 Практика по математике

Задание 1 направление

NPC находится в `(3, 0, 2)`, цель в `(11, 0, -4)`. Вручную вычисли направление, длину и единичный вектор. Затем сравни с DirectXMath.

Задание 2 поле зрения

Направление NPC `(0, 0, 1)`. Проверь цели в направлениях `(0, 0, 1)`, `(1, 0, 1)` и `(1, 0, 0)` для FOV 90 градусов. Сначала предскажи результат.

Задание 3 матрицы

Вручную сложи и умножь матрицы:

$$A = [2, 1;\ 0, 3], B = [4, -1;\ 2, 5]$$

Для умножения выпиши все четыре ячейки отдельно.

Задание 4 порядок трансформаций

Возьми точку `(1, 0, 0)`. Сравни два случая: сначала перенос `(5, 0, 0)`, потом поворот на 90 градусов; и сначала поворот, потом перенос. Нарисуй результат сверху.

Задание 5 собственный луч

Сделай маленькую функцию, которая принимает начало и конечную точку, возвращает нормализованное направление и длину. Она обязана отклонять почти нулевой отрезок.

Задание 6 визуальная проверка

Нарисуй в отладке три разных цвета:

- жёлтый от глаз NPC к кандидату;
- красный от груди к дулу;
- зелёный от дула до первого попадания.

По одному кадру должно быть понятно, какой именно тест запретил или разрешил выстрел.

22 Ответы для самопроверки

Для задания 1 направление равно `(8, 0, -6)`, длина равна `10`, нормализованный вектор равен `(0.8, 0, -0.6)`.

Для задания 2 первая цель точно внутри. Вторая лежит на границе 45 градусов и должна считаться внутри с небольшим допуском. Третья находится под 90 градусов к forward и снаружи.

Для задания 3:

$$A + B = [6, 0;\ 2, 8]$$

$$AB = [10, 3;\ 6, 15]$$

23 Контрольный список

Перед следующей частью ты должен уметь без подсказки:

- объяснить разницу между точкой и вектором;
- получить направление как `end - start`;
- безопасно нормализовать вектор;
- сравнить расстояния без квадратного корня;
- объяснить dot через угол;
- вручную сложить две матрицы;
- вручную вычислить каждую ячейку произведения матриц;
- объяснить, почему порядок матриц меняет результат;
- построить конечный луч от дула к точке;
- объяснить разницу между математическим лучом и физическим raycast;
- найти кратчайшую ошибку угла;
- обновить таймер независимо от FPS.

C++ для разработчика игрового движка

Эта глава не пытается пересказать весь язык. Она объясняет ту часть C++, от которой напрямую зависит надёжность DX Framework: где живут данные, кто их уничтожает, почему указатель может стать недействительным и каким образом шаблонный ECS превращает тип компонента в набор сущностей.

1 От исходного файла до работающей программы

Файл `.cpp` сам по себе не является программой. Компилятор обрабатывает каждую единицу трансляции отдельно. Упрощённо единица трансляции — это `.cpp` после текстовой подстановки всех `#include`.

Процесс состоит из четырёх стадий.

1. Препроцессор раскрывает include, условную компиляцию и макросы.
2. Компилятор проверяет типы и превращает одну единицу трансляции в объектный файл.
3. Линкер соединяет объектные файлы и библиотеки, находя определения функций и глобальных данных.
4. Загрузчик операционной системы размещает готовый модуль и его зависимости в памяти.

Отсюда происходят типичные ошибки.

- `identifier not found` обычно означает, что объявление не видно компилятору.
- `unresolved external symbol` означает, что объявление было видно, но линкер не нашёл определение.
- `already defined` означает, что одно внешнее определение попало в несколько единиц трансляции.
- Неполный тип допустим для указателя или ссылки, но недостаточен, когда компилятору нужен размер объекта или его поля.

1.1 Объявление и определение

Объявление сообщает форму сущности:

```
class NpcPlanner;
void UpdateNpc(float deltaTime);
```

Определение создаёт функцию или полностью описывает класс:

```
void UpdateNpc(float deltaTime)
{
    // Здесь существует тело функции.
}
```

Header-only системы проекта, например `InventorySystem`, помещают определения методов в `.h`. Это удобно, когда не хочется добавлять новую единицу трансляции в проект Visual Studio. Цена — увеличение времени компиляции и более сильная связанность include. Большие устойчивые реализации лучше переносить в `.cpp`, оставляя в заголовке публичный контракт.

1.2 `#pragma once` и include

`#pragma once` не даёт одному заголовку раскрыться дважды внутри одной единицы трансляции. Он не лечит циклический дизайн. Если `A.h` включает `B.h`, а `B.h` включает `A.h`, стоит спросить, нужны ли им полные определения или достаточно предварительного объявления.

Практическое правило движка: заголовок включает то, что необходимо его публичным полям и сигнатурам; `.cpp` включает детали реализации. Чем меньше случайных транзитивных include, тем легче понять зависимости.

2 Память и время жизни

У каждого объекта есть время жизни. Ошибка чаще возникает не потому, что адрес неправильный, а потому, что объект по этому адресу уже перестал существовать.

Основные области хранения:

- автоматическая память локальных переменных до выхода из блока;
- динамическая память объектов, которыми владеет контейнер или умный указатель;
- статическая память глобальных и `inline static` данных до завершения программы;
- память внешней системы, например COM-ресурса Direct3D или объекта Bullet.

2.1 Значение ссылка и указатель

```
NpcTacticalComponent value;
NpcTacticalComponent& reference = value;
NpcTacticalComponent* pointer = &value;
```

`value` является самим объектом. `reference` — обязательное второе имя существующего объекта, которое нельзя переназначить на другой объект. `pointer` — число-адрес, которое может быть `nullptr` и может начать указывать на другой объект.

В ECS проекта `GetComponent<T>(entity)` возвращает указатель, потому что компонента у сущности может не быть:

```
auto* tactical = registry.GetComponent<NpcTacticalComponent>(entity);
if (!tactical)
    return;
```

Проверка не является лишним шумом. Она формализует вариант «данных нет». После проверки `tactical->status` обращается к реальному компоненту.

2.2 Когда указатель из ECS опасен

Даже ненулевой указатель не вечен. Контейнер компонента может перераспределить память после добавления новых элементов. Сущность может быть уничтожена. Поэтому не стоит хранить сырой `T*` между кадрами. Надёжнее хранить `Entity`, а в момент использования снова запрашивать компонент.

Опасный шаблон:

```
class SomeSystem
{
    NpcBrainComponent* cachedBrain = nullptr;
};
```

Более устойчивый шаблон:

```
class SomeSystem
{
    Entity npc = INVALID_ENTITY;

    void Update(ECSRegistry& registry)
    {
        auto* brain = registry.GetComponent<NpcBrainComponent>(npc);
        if (!brain)
            return;
    }
};
```

2.3 `const` как контракт

`const` сообщает не только компилятору, но и читателю: функция не должна менять объект через этот доступ.

```
const auto* weapon = registry.GetComponent<WeaponComponent>(weaponEntity);
```

Такой указатель позволяет читать боезапас, но не менять его. Если система только оценивает решение, это полезная граница. Решающий модуль может получить `const WorldState&`, вернуть намерение и не иметь физической возможности телепортировать NPC.

Сравни:

```
NpcCoverRequestDecision EvaluateCoverRequest(
    const NpcTacticalComponent& tactical,
    const NPCComponent& npc);
```

Функция текущего `NpcTacticalDecisionSystem` читает тактическую память и здоровье, а наружу возвращает маленький результат. По сигнатуре видно, что она не меняет состояние мира.

3 Владение и RAII

Владение отвечает на вопрос: кто обязан освободить ресурс. RAII связывает освобождение со временем жизни C++ объекта.

`std::vector`, `std::string` и `std::unique_ptr` освобождают свои ресурсы автоматически. Поэтому контейнеры компонентов и `ShaderManager` с `std::unique_ptr<ShaderBaseClass>` безопаснее ручного `new/delete`.

3.1 `unique_ptr` и единственный владелец

```
std::unordered_map<std::string,
    std::unique_ptr<ShaderBaseClass>> shaders;
```

У каждого шейдера один владелец — карта менеджера. При удалении элемента вызывается деструктор. Копирование запрещено, зато владение можно переместить.

```
auto shader = std::make_unique<PBRShaderClass>();
shaders.emplace("PBR", std::move(shader));
```

После `std::move` переменная `shader` больше не владеет объектом. Сам объект не копируется: передаётся право его уничтожить.

3.2 COM ресурсы Direct3D

Текущий `ShaderBaseClass` хранит сырые COM-указатели и освобождает их в `Shutdown`. Это рабочий старый стиль DirectX 11. Он требует дисциплины: каждый успешно созданный ресурс должен быть освобождён один раз, а указатель после `Release()` надо обнулить.

Современный вариант — `Microsoft::WRL::ComPtr<T>`:

```
Microsoft::WRL::ComPtr<ID3D11Buffer> matrixBuffer;
```

`ComPtr` автоматически вызывает `Release`, корректно переживает ранний `return` и упрощает обработку частично успешной инициализации. Миграцию лучше выполнять по одному классу, сохраняя поведение.

3.3 Внешнее владение Assimp

`AssimpModelClass` хранит `Assimp::Importer`. Возвращённый `aiScene*` принадлежит importer и действителен, пока жив importer или пока не загружена новая сцена. Нельзя освободить узлы Assimp вручную или сохранить указатель на них после `Shutdown`.

Это общий принцип адаптеров: C++ класс движка владеет внешним контекстом, а наружу копирует только стабильные данные — вершины, индексы, имена костей, ключи и текстуры.

4 Классы структуры и инварианты

`struct` и `class` отличаются только доступом по умолчанию. В проекте компоненты естественно оформлены как `struct`: они являются наборами данных. Системы оформлены как `class` со статическими методами: они являются поведением над данными.

Инвариант — условие, которое должно оставаться истинным после любой публичной операции. У `InventoryComponent` это, например, ограниченный индекс активного слота и отсутствие одной сущности оружия сразу в двух слотах. `InventorySystem::SanitizeSlots` восстанавливает эти условия после удаления сущностей или загрузки старых сцен.

Хороший публичный метод либо сохраняет инвариант, либо явно сообщает ошибку:

```
InventoryOperation result = InventorySystem::PickupWeapon(
    registry, npc, weapon);

if (!result)
{
    // status объясняет причину отказа.
}
```

Возвращаемая структура лучше одного `bool`, потому что для AI важна причина: инвентарь полон, предмет занят другим владельцем или сущность уже уничтожена.

5 Шаблоны и ECS

Шаблон создаёт семейство функций или классов. Вызов `GetComponent<NpcBrainComponent>` сообщает тип во время компиляции. Registry может использовать `std::type_index` как ключ к правильному пулу компонентов.

Упрощённая идея:

```
template<class T>
T* GetComponent(Entity entity)
{
    auto* pool = FindPool<T>();
    return pool ? pool->Get(entity) : nullptr;
}
```

При первом использовании `T = NpcBrainComponent` компилятор создаёт соответствующую версию функции. Ошибка типа обнаруживается до запуска.

5.1 `ForEach` и лямбда

```
registry.ForEach<NPCCombatComponent>(
    [&](Entity npc, NPCCombatComponent& combat)
    {
        // Один вызов для каждой подходящей сущности.
    });
```

`[&]` захватывает используемые внешние переменные по ссылке. Это удобно, но широко. Для учебного и критичного кода полезно иногда перечислить захват явно:

```
[&registry, deltaTime](Entity npc, NPCCombatComponent& combat)
```

Так читатель видит входы лямбды. Нельзя сохранять такую лямбду дольше объектов, захваченных по ссылке.

5.2 Вложенный `ForEach`

Во внешнем цикле `npcEntity` — наблюдатель. Во внутреннем цикле `candidateEntity` — проверяемая цель. Ошибка в именах быстро превращает условие самопроверки в бессмыслицу.

```
registry.ForEach<NPCCombatComponent>(
    [&](Entity observerEntity, NPCCombatComponent& combat)
    {
        registry.ForEach<FactionComponent, TransformComponent>(
            [&](Entity candidateEntity,
                FactionComponent& candidateFaction,
                TransformComponent& candidateTransform)
            {
                if (candidateEntity == observerEntity)
                    return;
            });
    });
```

Здесь `return` выходит только из текущего вызова внутренней лямбды, то есть пропускает кандидата. Он не завершает всю систему.

6 Контейнеры и сложность

`std::vector` хранит элементы подряд. Это хорошо для обхода каждого кадра: процессор заранее подгружает соседние данные. Добавление в конец иногда вызывает перераспределение и делает старые указатели недействительными.

`std::unordered_map` даёт ожидаемый поиск за постоянное время, но элементы разбросаны по памяти и обход дороже. Она удобна для поиска по имени шейдера, кости или снимка NPC.

`std::deque` подходит журналу `NpcSimulationStudio`: добавление события в конец и удаление старого спереди не требует сдвигать весь массив.

6.1 Big O без мистики

- `O(1)` — число операций примерно не зависит от размера данных.
- `O(n)` — просмотр всех элементов.
- `O(n log n)` — типичная сортировка.
- `O(n²)` — каждый объект сравнивается с каждым.

Поиск целей «каждый NPC против каждого кандидата» имеет квадратичную природу. Для десятка NPC это нормально. Для сотен понадобится пространственное разбиение: uniform grid, quadtree, BVH или запрос широкофазной структуры Bullet.

Оптимизировать стоит после измерения. Сначала правильная модель знания цели, затем профилирование.

7 Значения перемещение и копирование

Копирование создаёт независимый объект. Перемещение передаёт внутренние ресурсы, оставляя источник в корректном, но неопределённом содержательном состоянии.

```
std::string reason = BuildReason();
event.details = std::move(reason);
```

После перемещения `reason` можно уничтожить или присвоить новое значение, но нельзя рассчитывать на прежний текст.

Для маленьких `Entity`, `float` и `XMFLOAT3` копирование дёшево. Для больших `std::vector` и строк перемещение полезно. Не следует механически ставить `std::move` везде: он может запретить оптимизацию возврата значения и ухудшить читаемость.

8 Ошибки и способы сообщить о них

В игровом кадре исключение редко является лучшим способом обычного отказа. «У NPC нет оружия» — не авария, а состояние мира. Его лучше выразить enum или `std::optional`.

Исключение уместно на границе загрузки инструмента, если верхний слой гарантированно ловит его и сообщает путь файла. Текущий `SaveAnimationStudio` использует `try`, потому что JSON и файловая система могут выбросить исключение.

Три уровня реакции:

1. Ожидаемый отказ возвращается как статус.
2. Нарушение внутреннего предположения проверяется `assert` в debug-сборке.
3. Ошибка внешних данных записывается в лог и показывает понятный fallback.

Не стоит молча продолжать с частично корректными данными. Например, constant buffer обязан иметь размер, кратный 16 байтам. Создающая его функция должна округлить размер и проверить HRESULT.

9 Числа время и устойчивость

`float` не представляет большинство дробей точно. Сравнение таймера с точным нулём после серии вычитаний ненадёжно. В проекте используется насыщение:

```
timer = (std::max)(0.0f, timer - (std::max)(0.0f, deltaTime));
```

После этого `timer <= 0.0f` имеет понятный смысл.

Не все величины должны использовать игровой `deltaTime`. UI может продолжать работать на паузе, а физика — выполнять фиксированный шаг. NPC Logic Studio отдельно управляет паузой, одиночным тиком и масштабом симуляционного времени. Это пример правильного разделения часов.

9.1 `std::isfinite`

Перед использованием внешних или накопленных данных полезно проверять `std::isfinite`. `NaN` заражает дальнейшие вычисления: любое сравнение с ним ложно, а матрица может исчезнуть с экрана без очевидной ошибки.

10 Матрицы и типы DirectXMath в C++

`XMFLOAT3` — удобное хранилище трёх чисел. `XMVECTOR` и `XMMATRIX` предназначены для SIMD-вычислений. Обычный путь:

```
using namespace DirectX;

XMVECTOR position = XMLoadFloat3(&transform.position);
XMMATRIX world = XMMatrixRotationRollPitchYaw(
    transform.rotation.x,
    transform.rotation.y,
    transform.rotation.z) *
    XMMatrixTranslationFromVector(position);

XMFLOAT4X4 stored{};
XMStoreFloat4x4(&stored, world);
```

Важно определить соглашение умножения. Текущие HLSL-шейдеры используют строковый вектор через `mul(position, matrix)`, поэтому порядок преобразований читается слева направо: local → world → view → projection. Смешивание этого соглашения с примерами, где пишут `matrix * columnVector`, даёт визуально правдоподобные, но неправильные результаты.

11 Многопоточность и границы данных

Нельзя просто отправить любую систему в `JobSystem`. Параллельный код безопасен, если известны наборы чтения и записи.

Пример независимой задачи: декодировать файл текстуры во временный CPU-буфер. Пример опасной задачи: одновременно добавлять компоненты в Registry, пока основной поток его обходит.

Удобная модель:

- фоновые потоки читают неизменяемый снимок и создают результат, которым владеют сами;
- очередь главного потока принимает результат и создаёт GPU-ресурсы или компоненты;
- runtime-системы одного кадра исполняются в известном порядке, пока нет явного графа зависимостей.

Direct3D 11 immediate context и ImGui обычно принадлежат главному потоку. Физический мир Bullet тоже не следует менять параллельно без отдельного контракта синхронизации.

12 Производительность через данные

Главная цена кадра часто не в арифметике, а в промахах кэша, выделениях памяти и повторной работе.

Практические правила:

- не строить диагностическую строку каждый кадр, если статус не менялся;
- не искать компонент несколько раз в одной ветке, если указатель уже получен и контейнер не менялся;
- резервировать `vector`, когда примерное число элементов известно;
- сравнивать квадрат расстояния, если само расстояние не нужно;
- отделять частое обновление от редкого решения таймером;
- не выполнять полный поиск пути, если цель и навигационная ревизия не изменились.

Текущий AI уже использует часть этих идей: `decisionTimer`, `repositionCooldown`, `tacticCommitTimer`, `rejectedCoverTimer`. Таймер не просто оптимизация. Он также создаёт устойчивое поведение вместо дрожания между почти равными вариантами.

13 Как читать сложную функцию построчно

Не начинай с деталей. Разметь функцию пятью вопросами.

1. Какие guard clauses прекращают работу?
2. Какие данные только читаются?
3. Какие поля изменяются?
4. Где вызывается внешний мир: физика, файл, GPU, Registry?
5. Что функция обещает вызывающему коду?

Например, `InventorySystem::PickupWeapon` читается фазами:

- проверяет владельца и сущность предмета;
- убеждается, что предмет является оружием и не занят другим;
- создаёт или санирует инвентарь;
- ищет свободный слот;
- записывает отношение владения;
- синхронизирует `EquippedWeaponComponent`;
- возвращает подробный статус.

После такого скелета отдельные `if` перестают быть стеной текста.

14 Практика без изменения проекта

Задание 1 Владение

Открой `ShaderManager.h`. Для каждого поля ответь, кто создаёт объект, кто им пользуется и кто уничтожает. Отдельно объясни, почему `Get` возвращает невладеющий указатель.

Задание 2 Время жизни

Открой `NpcSimulationStudio.h`. Найди данные с длительностью один кадр, одну сессию Studio и всю программу. Объясни, почему история ограничена `MaxEvents`.

Задание 3 Чистая функция

Возьми одно правило из `NpcCombatDecisionRules.h`. Выпиши входы, выход и пограничные значения. Придумай пять тестов, не запуская игру.

Задание 4 Контейнер

Сравни `vector`, `unordered_map` и `deque` на примере костей модели, снимков NPC и журнала. Объясни, почему один контейнер не является лучшим для всех трёх задач.

Задание 5 Дизайн API

Спроектируй на бумаге результат поиска оружия. Одного `Entity` недостаточно: добавь причину отказа, стоимость пути, тип оружия и необходимость освобождения резервации. Реализацию пока не пиши.

15 Критерий понимания C++ в этом проекте

Знание языка проявляется не в количестве запомненных ключевых слов. Ты понимаешь код, если можешь до запуска предсказать:

- сколько живёт объект;
- кто имеет право его менять;
- что станет недействительным после добавления или удаления компонента;
- где произойдёт копирование или передача владения;
- какую ошибку увидит вызывающая сторона;
- в какой стадии кадра изменение станет видимым.

С этими вопросами даже большой класс раскладывается на проверяемые части.

Как читать и менять DX Framework с нуля

Этот раздел рассчитан на человека, который умеет открыть проект, но хочет заново понять собственный C++ код. Здесь не требуется предварительно помнить названия паттернов. Мы будем идти от одного значения в памяти к компоненту, от компонента к системе, затем к целому кадру. Если встретилось незнакомое слово, сначала выясни, какие данные оно обозначает и кто ими владеет. Только после этого переходи к следующему методу. Файлы и имена ниже описывают состояние проекта на 29 октября 2026 года; строки могут сдвинуться после изменения исходников, поэтому ищи указанное имя метода.

1. Как разобрать строку C++ и не потерять смысл

1.1. Тип, имя, значение и время жизни

Возьми простую строку `float deltaTime = 0.016f;`. `float` задаёт тип числа с плавающей точкой, `deltaTime` — имя, `0.016f` — начальное значение. Суффикс `f` означает литерал типа float, иначе дробный литерал в C++ обычно имеет тип double. Но тип не сообщает единицы измерения. В игровом кадре `deltaTime` — секунды, а `position` обычно измеряется в метрах. Если прибавить `deltaTime` к позиции без скорости, объект будет двигаться на численно похожее расстояние, но игровое правило окажется неявным. При чтении каждой арифметической строки подпиши единицы аргументов и результата.

Локальная переменная создаётся при входе в область видимости и уничтожается при выходе из неё. Поле объекта живёт столько же, сколько объект. Динамически созданный ресурс живёт до освобождения владельцем. Это различие важно в `ApplicationClass::Frame`: локальный `registry` как ссылка на `m_sceneManager.GetECSRegistry()` не создаёт второй мир. Когда функция заканчивается, сама ссылка исчезает, но Registry остаётся полем SceneManager. Нельзя вернуть указатель на локальную временную переменную из функции и продолжать пользоваться им в следующем кадре.

1.2. Значение, ссылка и указатель

`Entity e` передаёт числовой идентификатор по значению: изменение локальной копии не меняет ID у вызывающего кода. `ECSRegistry& registry` означает ссылку на существующий Registry: система работает с тем же объектом, а не копирует все компоненты. `const RepairableComponent* repairable` — указатель, который может быть `nullptr`, и через который нельзя менять компонент, пока действует `const`. Звёздочка в объявлении указывает тип указателя; `*repairable` в выражении разыменовывает его. `repairable->progress` — доступ к полю через указатель. Разыменовать `nullptr` нельзя.

Если `GetComponent<T>(entity)` возвращает `T*`, следующий безопасный шаг — проверить результат. Сам `Entity` может быть живым, но компонента T у него нет. Для примера, у декоративного ящика есть Transform, но нет Repairable. Правильное условие `if (auto* repairable = registry.GetComponent<RepairableComponent>(e))` связывает проверку и использование. Однако даже ненулевой указатель не обещает вечной жизни: после `DestroyEntity`, `RemoveComponent` или структурной операции с pool проверь его заново. Особо опасно хранить `T&` между вызовами систем, которые могут удалить цель.

1.3. `const`, `auto` и `enum`

`const` не означает, что весь игровой мир заморожен; оно ограничивает изменение через конкретный доступ. `const auto* brain = registry.GetComponent<NpcBrainComponent>(e)` позволяет читать brain, но не записывать state через этот указатель. `auto` выводит тип выражения, а не превращает язык в

динамический: если справа `T*`, тип остаётся указателем. При чтении сложного кода при необходимости выпиши настоящий тип `auto`, иначе легко перепутать копию с ссылкой. `auto&` создаёт ссылку, `auto` в range loop может копировать элемент.

`enum class` задаёт именованные варианты состояния: `NpcState::Idle`, `NpcState::Attacking` и так далее. Он помогает компилятору отличать состояние NPC от произвольного числа. Но enum сам не определяет переходы: их задают условия систем. Если персонаж одновременно хранит `state=Attacking` и `target=INVALID_ENTITY`, это логически противоречивое состояние, даже если оба поля по отдельности имеют допустимые типы. Составь инварианты: при Attacking цель жива и враждебна; при Repairing есть действительная цель и достижимая рабочая точка. Claim нужен для автоматического назначения; для оборудования может использоваться WorkZone, а пробоина в текущем коде обрабатывается без него.

1.4. `std::vector`, `unordered_map` и итерация

`std::vector<T>` хранит последовательность. Добавление элемента иногда перераспределяет память, поэтому старые указатели и ссылки на элементы могут стать недействительными. `std::unordered_map<Key,Value>` хранит значения по ключу; изменение структуры контейнера влияет на итераторы по правилам конкретной операции. В ECS Registry каждый тип компонента имеет pool, а `ForEach` итерирует его. Поэтому не удаляй текущий компонент из того же pool прямо во время `ForEach`, если контракт `ComponentManager` этого явно не гарантирует. Безопасный подход: во время обхода складывай Entity ID в `pendingDestroy`, после обхода применяй удаления. Это не просто стиль — так сохраняется корректность итератора и ссылок.

1.5. Лямбда и захват

`[&](Entity e, auto& component) { ... }` — функция без имени, которую можно передать `ForEach`. `[&]` разрешает использовать внешние локальные переменные по ссылке. Это удобно для накопления результатов, но опасно для фоновой задачи: после выхода из вызывающей функции захваченная ссылка может стать висячей. В синхронном `ForEach`, который вызывает лямбду до возврата, захват по ссылке обычно соответствует времени жизни. При `JobSystem::SubmitJob` код может выполниться позже; копируй нужные входные данные и применяй итог в главном потоке после проверки живости Entity и версии сцены.

1.6. RAII и владение ресурсом

RAII означает, что освобождение ресурса связано с уничтожением объекта-владельца. `std::unique_ptr<IComponentPool>` внутри `ECSRegistry` ясно выражает единоличное владение pool. Если Registry уничтожается, unique_ptr освободит pool. Bullet body, GPU buffer и job queue живут по другим правилам: удалить C++ объект недостаточно, если он всё ещё зарегистрирован в physics world или привязан к GPU command. Для каждого такого ресурса составь пару «создание — удаление» и момент отписки от внешнего мира. Если scene reload оставил collider без Entity, это нарушение жизненного цикла, а не визуальный баг.

Проверочное упражнение. Прочитай сигнатуру `void StateSystem::Update(ECSRegistry& registry, float deltaTime, JobSystem& jobs)`. Назови, какие аргументы копируются, какие ссылаются на существующий объект, где ожидаются единицы, какие изменения возможны. Затем найди один вызов функции из `ApplicationClass::Frame`. Проверь, передан ли `simulationDeltaTime`, когда Play поставлен на паузу. Не переходи к телу метода, пока сигнатура не понятна.

2. Разбираем `ECSRegistry.h` буквально по шагам

2.1. `CreateEntity`: от ID до живой сущности

Файл `src/ECS/ECSRegistry.h`, метод `CreateEntity` (в текущем файле около строк 15–22). Строка с `if (nextEntity == INVALID_ENTITY)` защищает от выдачи зарезервированного значения. `throw std::overflow_error` останавливает операцию явной ошибкой; без этой проверки переполнение ID могло бы дать новое значение, совпадающее с «нет сущности». `const Entity entity = nextEntity++;` сначала берёт прежний номер, затем увеличивает счётчик. `m_liveEntities.insert(entity)` фиксирует, что ID существует. Возврат отдаёт номер вызывающему коду. В этот момент у сущности ещё нет Transform, RenderComponent или NPCBrain: CreateEntity создаёт идентичность, а не готовый игровой объект.

Проверь на бумаге: если `nextEntity=10`, первый вызов возвращает 10 и оставляет 11; второй возвращает 11. `IsAlive(10)` после первого вызова истинно, но `HasComponent<TransformComponent>(10)` может быть ложно. Это объясняет, почему одно лишь создание Entity не рисует объект. Если игра удалит 10, текущая реализация не переиспользует этот номер немедленно; generation handle здесь не реализован. При добавлении повторного использования в будущем понадобятся поколения, чтобы старый сохранённый ID не указывал на новую сущность.

2.2. `DestroyEntity`: удаление из набора и pools

Около строк 24–28 `DestroyEntity` сначала делает `m_liveEntities.erase(e)`, затем проходит `for (auto& [ti, pool] : m_pools)` и вызывает `pool->Remove(e)` у каждого типа компонента. Это универсальная очистка ECS данных: Registry не обязан знать заранее, какие компоненты были у Entity. Но у внешних систем есть дополнительные ресурсы. Если компонент содержал `btRigidBody*`, перед уничтожением нужно убедиться, что тело исключено из Bullet world и освобождено по договору владельца. Если async job хранит ссылку на компонент, удаление до завершения job опасно. Поэтому в gameplay коде зачастую нужен более высокий `DespawnEntity` путь, который согласует ECS, physics, renderer, jobs и временные reservations.

На проверочной сцене создай Entity с двумя компонентами, вызови Destroy, затем проверь `IsAlive=false`, `GetComponent<T>=nullptr` для обоих. Потом проверь физический мир: число тел тоже должно уменьшиться, если Entity владело телом. Если не уменьшилось, Registry выполнил свой контракт, а вызывающий lifecycle путь — нет. Важно различать эти уровни ответственности.

2.3. `AddComponent` и две перегрузки

Около строк 33–43 есть две формы. `AddComponent(Entity e, const T& comp)` копирует объект и разрешена только для copy-constructible T. `AddComponent(Entity e, T&& comp)` пересылает временное значение или объект, который можно переместить; это нужно для некопируемых компонентов, например тех, что владеют физическим ресурсом. `GetOrCreatePool<T>()` находит pool этого типа либо создаёт его. Обрати внимание: в этих строках не видно проверки `IsAlive(e)`. Следовательно, вызывающий код должен соблюдать контракт живой Entity или нужно усилить Registry; не предполагай невидимую защиту. Если компонент добавлен на несуществующий ID, pool может хранить мусорную запись, которая не соответствует `m_liveEntities`.

Для новичка полезна аналогия: Entity ID — номер папки, component pool — отдельная таблица одного вида данных. `AddComponent` добавляет строку в таблицу по номеру папки. Если номер папки неверен, таблица не обязательно сама поймёт это. Поэтому фабрика объекта должна создавать Entity один раз, затем последовательно добавлять обязательные компоненты и валидировать готовую конфигурацию.

2.4. `GetComponent`, `HasComponent`, `RemoveComponent`

`GetComponent<T>` находит pool типа T. Если pool отсутствует, возвращает `nullptr`; иначе вызывает `pool->Get(e)`. `HasComponent<T>` проверяет наличие pool и Entity в нём. `RemoveComponent<T>` удаляет только этот тип у выбранной Entity. Ни один из этих методов не меняет `m_liveEntities`. Следовательно, NPC может оставаться живым, но временно потерять оружие, и системы должны обрабатывать это как допустимое или ошибочное состояние по своему правилу. Нельзя писать `*registry.GetComponent<T>(e)` без проверки, если отсутствие T возможно.

`GetComponents<T>()` в отличие от `GetComponent` создаёт pool, если его ещё не было. Это полезно для системного обхода, но имеет побочный эффект создания пустого pool. При диагностике «почему появился pool» учитывай это. Для read-only запроса к одному объекту предпочитай `GetComponent`/`HasComponent`, чтобы чтение не изменяло структуру Registry без необходимости.

2.5. `ForEach<T1,T2>` построчно

Около строк 45–59 берутся два указателя на pools: `FindPool<T1>()` и `FindPool<T2>()`. Если один отсутствует, `return` заканчивает обход без вызова callback. Затем цикл проходит каждый элемент первого pool. Для текущего Entity проверяется `pool2->Has(entity)`. Если второй компонент найден, берётся ссылка `comp2` и вызывается `fn(entity, comp1, comp2)`. Это значит, что порядок типов влияет на количество итераций: `ForEach<Редкий,Частый>` обычно делает меньше проверок, чем `ForEach<Частый,Редкий>`. Но менять порядок нужно только если callback аргументы и логика соответственно переставлены.

Ключевое ограничение: callback получает ссылки на элементы pools. Если внутри callback удалить текущий компонент или Entity, цикл может потерять валидность итератора. Даже если конкретный контейнер иногда переживает удаление другого элемента, не строй на этом неявную гарантию. Собирай команды на структурные изменения отдельно. Вопрос для самопроверки: `ForEach<NpcBrainComponent,TransformComponent>` найдёт ли Entity только с Transform? Нет: она отсутствует в первом pool и не участвует в цикле. Найдёт ли Entity только с Brain? Цикл её увидит, но проверка второго pool не пропустит callback.

2.6. `type_index` и закрытая часть класса

В `private` находятся `nextEntity`, набор живых ID и `unordered_map<std::type_index,std::unique_ptr<IComponentPool>>`. `typeid(T)` превращается в ключ вида компонента. `FindPool<T>` ищет ключ и приводит базовый `IComponentPool*` к конкретному `ComponentManager<T>*`; корректность зависит от того, что под данным ключом всегда лежит pool именно T. `GetOrCreatePool<T>` создаёт его при первом обращении. Это простая реализация ECS, которую можно понять без магии: каждый тип имеет собственную таблицу, а Registry связывает таблицы общим Entity ID.

Упражнение: вручную нарисуй три pools — Transform у Entity 1 и 2, Repairable у Entity 2 и 3, NPCBrain у Entity 1. Что вызовет `ForEach<TransformComponent,RepairableComponent>`? Только Entity 2. Что произойдёт после `DestroyEntity(2)`? ID 2 удалён из набора живых и обоих pools. После этого попытка `GetComponent<RepairableComponent>(2)` должна вернуть null. Проверь тем же набором в маленьком тестовом запуске.

3. Главный кадр: читаем порядок, а не названия систем

3.1. Вход `Frame(float dt)` и режимы времени

`ApplicationClass::Frame` начинается около строки 1023 `src/Core/ApplicationClass.cpp`. Параметр `dt` — длительность кадра. В отдельном режиме проверки оружия код принудительно ставит `dt=1/60`, поэтому измерение FPS в этой ветке не отражает реальное время. Затем обновляется DebugHud, input manager и очищаются события текущего кадра. Уже здесь читатель должен спросить: `m_events.Clear()` удаляет события прошлого кадра до или после того, как их потребители обработали? Ответ ищи в местах публикации и чтения событий, а не предполагай. Для нового события выбери конкретную фазу кадра, чтобы оно не исчезало сразу после создания.

Переключение F5 проверяет не только `IsKeyDown`, но и `!m_wasF5Down`: так одно долгое удержание клавиши не переключает режим каждый кадр. При входе в Play `EnterPlayMode` проверяет сцену, затем сбрасываются timer, event log, trigger state, открытый UI и режим камеры. В AI-only сцене без игрока код отдельно не создаёт фиктивный инвентарь для `INVALID_ENTITY`. Это пример защитной проверки на границе режимов. При выходе в Editor нужно очистить накопленное намерение движения character controller; иначе физика продолжает шагать, используя последнюю команду Play. Этот комментарий в коде показывает, что режим Editor не является полной остановкой всех подсистем.

3.2. От ввода игрока до намерения движения

В Play при работающей симуляции `m_gameplayController.Update` получает Registry, `simulationDeltaTime`, input, режим захвата UI и yaw камеры. Он не обязан непосредственно перемещать mesh. Его задача — превратить действие игрока и направление камеры в намерение. Далее системы движения и Bullet определят фактическое положение. Если игрок нажал W, а персонаж стоит, проверь по цепи: получен ли GameAction; заблокировал ли его UI; записана ли желаемая скорость; вызван ли character controller; есть ли контакт с collider; изменился ли Transform после physics. Если первый разрыв на входе, физику менять рано.

3.3. Почему NPC pipeline стоит именно так

В текущем кадре перед выбором и выстрелами уменьшаются таймеры оружия (`WeaponSystem::UpdateTimers`). Затем `NpcUtilitySystem` рассчитывает автоматическое действие; `CrewTaskCoordinator` закрепляет работу за конкретным членом экипажа; `NpcCombatSystem` обновляет боевую видимость и память; `NpcCoverSystem` выбирает индивидуальное укрытие; `StateSystem` исполняет итоговое состояние, строит путь и заполняет движение; `CharacterFacingSystem` поворачивает персонажа. Групповой `NpcSquadCombatSystem::Update` в этой ревизии закомментирован. Позднее, после обновления позы и оружейных данных, `NpcRangedCombatSystem` может фактически стрелять. Не говори «NpcCombatSystem стреляет» только потому, что он выбирает Attacking; выстрел имеет отдельные условия и место кадра.

На бумаге проследи одного инженера. До кадра у него Idle и нет цели. Utility видит пробоину и рассчитывает полезность; Coordinator ставит claim; StateSystem в этом же кадре может увидеть назначение и запросить путь; Facing поворачивает модель. Фактическая капсула может сдвинуться позже на physics step. Если поменять местами Coordinator и StateSystem, первое движение задержится до следующего кадра. Это не обязательно катастрофа, но меняет наблюдаемое поведение. Любая перестановка вызовов должна сопровождаться таблицей «кто пишет поле → кто читает».

3.4. Кто владеет состоянием и что значит «на один кадр старое»

В одном кадре разные поля могут отражать разные моменты. У NPC `brain.state` уже новый, `desiredVelocity` вычислена по новому маршруту, а `Transform` ещё хранит позицию до physics step. Renderer после physics может увидеть обновлённое положение, а HUD, если был заполнен раньше, покажет старую дистанцию. Это обычный вопрос фаз, пока задержка определена и мала. Для диагностики показывай timestamp/fixed tick рядом с данными. Если поле используется через несколько кадров без инвалидации, это уже ошибка контракта, особенно для raycast muzzle и lastKnownPosition.

4. Как добавлять систему так, чтобы она заработала во всей игре

4.1. Сперва сформулируй правило без C++

Выберем пожар. Правило для игрока: повреждённый прибор может загореться; огонь расходует ресурс и нагревает комнату; инженер с огнетушителем может потушить его; закрытая дверь влияет на доступность; HUD сообщает причину, если работа невозможна. Запиши, как игрок видит начало, развитие и конец события. Затем выпиши состояния `Igniting`, `Burning`, `Dying`, `Extinguished`; для каждого — кто меняет его, что видно, можно ли сохранять. Если эту схему нельзя изложить словами, реализация тоже будет противоречивой.

Данные огня принадлежат `FireComponent` на Entity очага: интенсивность, топливо, room ID, рабочая зона и временной параметр. Температура комнаты принадлежит комнате, заряд огнетушителя — инвентарю NPC. Не дублируй одно и то же число в нескольких местах без явного правила синхронизации. Например, если и FireComponent, и RoomComponent хранит «текущую температуру пожара», один из них неизбежно станет устаревшим. Выбери источник правды и производные показатели.

4.2. Реализация минимального тика

Создай систему, которая работает с одним очагом без NPC и рендера. Пример ниже — учебный псевдокод, потому что конкретных `FireComponent` и `RoomTemperatureComponent` в текущем проекте ещё может не быть. Он показывает контракт обновления, а не готовую вставку:

```
void FireSystem::Update(ECSRegistry& ecs, float dt) {
    ecs.ForEach<FireComponent>([&](Entity fireEntity, FireComponent& fire) {
        if (fire.state != FireState::Burning) return;
        const float burn = std::min(fire.fuel, fire.burnRate * dt);
        fire.fuel -= burn;
        fire.intensity = std::clamp(fire.intensity + fire.growthRate * dt,
                                    0.0f, fire.maxIntensity);
        if (fire.fuel <= 0.0f) fire.state = FireState::Dying;
    });
}
```

Разбор. Проверка state предотвращает рост потушенного очага. `burnRate*dt` переводит расход «в секунду» в расход за шаг. `std::min` не позволяет забрать больше топлива, чем есть. Clamp удерживает интенсивность в допустимых границах. Переход в Dying выполняется при исчерпании топлива. Но этот код ещё не нагревает комнату, не наносит урон и не создаёт задачу NPC. Добавляй по одному звену и каждый раз проверяй инвариант, чтобы при сбое знать, на каком этапе он появился.

4.3. Место в кадре и зависимости

Если Coordinator должен выбирать пожар по текущей интенсивности, `FireSystem::Update` должен выполняться до него либо решение сознательно использует предыдущий tick. Если тушение снижает интенсивность после работы NPC, порядок должен не удваивать изменение за один кадр. Отдельно реши, когда атмосфера реагирует на кислородный расход: до или после FireSystem, и нужна ли

задержка. Напиши таблицу входов и выходов каждой функции. Не вставляй новую систему «куда удобно» между случайными вызовами `ApplicationClass::Frame`.

4.4. Сцена, Inspector, сохранение, UI

После работающего тика нужно дать автору игры создать очаг в Editor. Добавь сериализацию только authoring полей, например начальная интенсивность, запас топлива и room ID. Runtime указатель на комнату, временные claim, path cache, Bullet pointer и GPU handle не записывай как числа в JSON. Для продолжаемой игры отдельно реши, какие runtime поля нужно сохранять: оставшееся топливо и текущий state обычно важны. После LoadScene восстанови Entity references и runtime ресурсы. В Inspector показывай и редактируемые authoring поля, и read-only live данные, чтобы пользователь не путал сохранённый параметр со значением текущего кадра.

4.5. Связь с NPC и контроль отказов

Новая работа требует обнаружения очага, оценки полезности, claim, доступной WorkZone, маршрута, расхода огнетушителя, прогресса тушения и события завершения. Каждое из этих звеньев имеет отдельную причину отказа: нет очага, он уже занят, путь закрыт, нет ресурса, комната опасна, WorkZone внутри collider. Покажи причину в debug UI. Начни с одного NPC и одного очага, затем два NPC и две цели. Смерть исполнителя должна освободить claim. Ручная команда должна вытеснить automatic работу по ясному правилу. После тушения удалённый Entity не должен оставаться в чужом `repairTarget`.

4.6. Проверка результата в игре

Построй маленькую сцену с двумя комнатами, дверью и одним очагом. Запиши прогноз на 10 секунд: сколько топлива сгорит, до какой интенсивности вырастет пожар, когда инженер войдёт в WorkZone, сколько заряда потратит и когда огонь перейдёт в Extinguished. Выполни прогон при 30, 60 и 120 FPS на одинаковое simulation time. Если числа кратно различаются, проверь `dt`. Сохрани игру посередине, загрузи и убедись, что огонь не создался второй раз, ресурс не вернулся, claim не дублировался. Это минимальный стандарт новой системы до включения её в большую миссию.

Построчный разбор ядра времени задач файлов и ресурсов

Эта глава проходит путь от `WinMain` до готового GPU mesh. Она дополняет прежний разбор `ApplicationClass`: здесь подробно разобраны окружающие его службы, которые раньше были только пунктами архитектурной карты.

1 main cpp

`WinMain` является точкой входа Windows приложения.

1. Параметры `HINSTANCE` и командной строки передаёт Windows. Текущий код не использует старый instance и строку запуска, но сохраняет стандартную сигнатуру.
2. Создаётся локальный `SystemClass system`. Его время жизни охватывает весь запуск программы.
3. `system.Initialize()` последовательно создаёт окно, ввод и `ApplicationClass`.
4. Если инициализация вернула `false`, выполнение завершается без входа в цикл.
5. `system.Run()` держит message loop и кадры до `WM_QUIT` или отказа `Frame`.
6. `system.Shutdown()` безопасно повторяет очистку. Деструктор также вызывает `Shutdown`, поэтому операция обязана быть идемпотентной.
7. Возвращаемое значение сообщает Windows код завершения процесса.

Главное: `main.cpp` не должен знать о сцене, NPC или renderer. Он владеет только корневым объектом приложения.

2 SystemClass Initialize построчно

Файл `src/Core/SystemClass.cpp` является мостом WinAPI и объектной части движка.

2.1 Глобальный мост

`SystemClass* g_applicationHandle = nullptr` хранится в безымянном namespace. Внешние translation units не видят символ.

Глобальная `WndProc` нужна потому, что Windows вызывает C-style callback. Она не может напрямую вызвать нестатический метод без адреса объекта. `g_applicationHandle` предоставляет этот адрес.

2.2 Конструктор и деструктор

Default constructor полагается на инициализаторы полей заголовка. Деструктор вызывает `Shutdown`. Поэтому любой ресурс обязан либо быть пустым, либо корректно переживать повторную очистку.

2.3 Начало Initialize

`Shutdown()` в первой строке делает повторную попытку детерминированной. Если предыдущая инициализация завершилась на середине, старые окно и подсистемы не смешаются с новыми.

`screenWidth` и `screenHeight` создаются нулевыми. `InitializeWindows` заполняет их фактическим размером.

Проверка `!m_hwnd` отделяет ошибку окна от ошибки Direct3D. Повторный `Shutdown` очищает зарегистрированный класс окна и глобальный bridge.

2.4 Ввод

`std::make_unique<InputClass>()` создаёт единственного владельца ввода. `Initialize` приводит массивы клавиш и mouse state в начальное состояние.

`unique_ptr` важен при раннем выходе: если последующая графическая инициализация не удастся, `reset` уничтожит input без ручного `delete`.

2.5 Application

Следующий `make_unique` создаёт `ApplicationClass`. Его `Initialize` получает размеры и `HWND`, потому что swap chain привязывается к окну.

Проверка результата не пытается продолжать с частично готовым renderer. Она вызывает полную обратную очистку.

`SetInput(m_Input.get())` передаёт невладеющий указатель. `ApplicationClass` пользуется вводом, но не удаляет его. Владелец остаётся `SystemClass`.

3 SystemClass Shutdown

`m_Application.reset()` выполняется раньше input и окна. Это соответствует зависимостям: Application освобождает swap chain, ImGui и ресурсы, которым ещё нужны `HWND` и контекст.

`m_Input.reset()` уничтожает input после прекращения кадров.

`ShutdownWindows()` восстанавливает display mode, уничтожает `HWND`, снимает регистрацию window class и обнуляет bridge.

Повторный вызов безопасен, потому что каждая операция проверяет наличие ресурса и обнуляет поле после освобождения.

4 SystemClass Run

`MSG msg = {}` обнуляет WinAPI структуру. `done` является единственным условием внешнего цикла.

`TimeManager::UpdateFrame()` вызывается в начале каждой итерации, даже если в очереди есть сообщение. Поэтому unscaled time продолжает измеряться во время UI и pause.

`PeekMessage(... PM_REMOVE)` неблокирующе забирает одно сообщение. В отличие от `GetMessage`, игра не ждёт пользовательского события перед следующим кадром.

В editor mode Space переключает `Running` и `Paused`. Эта ветвь намеренно недоступна в Play, где Space принадлежит Jump.

Клавиша K в неработающем времени ставит `StepMode` и вызывает `StepFrame`. На текущей итерации появляется фиксированный игровой delta.

`WM_QUIT` ставит `done`. `TranslateMessage` создаёт текстовые сообщения из клавиш, а `DispatchMessage` вызывает глобальную `WndProc`.

Даже при обработанном сообщении вызывается `Frame`. Это поддерживает непрерывный render. Если `Frame` возвращает `false`, цикл заканчивается.

`ResetStep` выполняется после кадра, чтобы фиксированный delta не протёк в следующую итерацию.

5 SystemClass Frame и сообщения

`TimeManager::DeltaTime()` возвращает масштабированное игровое время. Это значение передаётся в `ApplicationClass::Frame`.

Escape закрывает приложение только в editor mode. В Play та же клавиша сначала принадлежит action frame и может закрыть interaction panel.

`MessageHandler` переводит WinAPI события в состояние `InputClass`.

- `WM_KEYDOWN` и `WM_KEYUP` меняют массив клавиш.
- `WM_MOUSEMOVE` извлекает signed координаты из `LPARAM`.
- При captured mouse позиция не обновляется из сообщений: движение сэмплируется один раз за кадр. Это предотвращает очередь искусственных сообщений от recenter.
- В свободном режиме координаты сохраняются для UI и picking.
- Шесть сообщений кнопок мыши меняют соответствующие флаги.
- Неизвестные сообщения передаются `DefWindowProc`.

Глобальная `WndProc` сначала отдаёт сообщение ImGui. Если UI обработал его, gameplay bridge не получает повторное событие.

`WM_DESTROY` и `WM_CLOSE` публикуют `WM_QUIT`. Остальные сообщения идут в `SystemClass`, если bridge существует.

6 Создание окна

`InitializeWindows` сохраняет `this` в bridge и получает module handle текущего exe.

`WNDCLASSEX` заполняется явно:

- `CS_HREDRAW | CS_VREDRAW` просит перерисовку при изменении размеров;
- `CS_OWNDC` закрепляет device context за окном;
- callback указывает на глобальную `WndProc`;
- иконка, cursor и чёрный background получают стандартные WinAPI ресурсы;
- menu отсутствует;
- имя класса и размер структуры обязательны для регистрации.

`RegisterClassEx` регистрирует описание до создания окна.

Размер desktop читается через `GetSystemMetrics`. В fullscreen код меняет display settings и ставит позицию `(0,0)`. В оконном режиме желаемые 1920×1280 ограничиваются desktop и центрируются.

`CreateWindowEx` использует popup style без рамки. `WS_CLIPSIBLINGS` и `WS_CLIPCHILDREN` уменьшают лишнюю перерисовку пересекающихся окон.

После создания окно показывается, становится foreground и получает focus. Cursor остаётся видимым до явного captured режима.

7 TimeManager

7.1 Singleton

`Instance` содержит function local static object. Начиная с C++11 его первое создание thread-safe. Копирование и присваивание удалены, поэтому второго таймера создать нельзя.

7.2 Constructor

Поля получают нулевые delta, scale 1, состояние Running и сброшенный step flag.

`QueryPerformanceCounter` запоминает стартовую отметку. Без этого первый delta измерялся бы от нуля системы.

`QueryPerformanceFrequency` сохраняет число тиков high-resolution counter в секунду.

7.3 UpdateFrame

Новая отметка считывается в `currentTime`.

`CalculateDelta` вычитает прошлый counter и делит на frequency. Результат — реальные секунды.

`m_lastTime = currentTime` передвигает начало следующего интервала.

В Running `m_gameDt = realDt * m_timeScale`. В Paused и StepMode обычный update даёт ноль. StepMode получает ненулевое время только через явный `StepFrame`.

`m_unscaledDt = realDt` сохраняется независимо от pause. UI, cursor animation и диагностика могут продолжать обновляться.

7.4 StepFrame и ResetStep

`StepFrame` работает только в StepMode. Он записывает фиксированное значение, по умолчанию 1/60 секунды, и ставит `m_wasStepped`.

`ResetStep` после кадра возвращает delta к нулю и очищает флаг. Без него один запрос выполнял бы много simulation frames.

7.5 Риск SetTimeScale

Текущий setter принимает любое число. Безопасное развитие — отвергать `NaN`, отрицательный scale и ограничивать максимум. Иначе системы с обратным временем или бесконечностью нарушат таймеры.

8 GameTimer

`GameTimer` является независимыми часами конкретной игровой симуляции или Studio.

`Update` сначала заменяет нечисловой или отрицательный delta нулём. Если paused, итоговый delta равен нулю; иначе умножается на неотрицательный scale.

Результат накапливается в `m_totalSeconds`, а также возвращается вызывающему коду.

`SetTimeScale` проверяет конечность и насыщает отрицательное значение до нуля.

`SetPaused` меняет только флаг. Он не стирает total time.

`Reset` задаёт начальное total time и обнуляет delta. Это создаёт новую логическую сессию без пересоздания объекта.

Различие: `TimeManager` обслуживает корневую итерацию приложения, `GameTimer` — управляемую simulation clock внутри `ApplicationClass` и NPC Studio.

9 PhysicsWorld

9.1 Initialize

Проверка существующего `m_dynamicsWorld` не создаёт второй мир.

Локальные `unique_ptr` последовательно создают collision configuration, dispatcher, broadphase, solver, ghost callback и world. Пока инициализация не завершена, поля singleton не изменяются.

Gravity устанавливается в Bullet world. Solver parameters задают число итераций, split impulse, penetration threshold и ERP. Эти числа управляют устойчивостью контактов и constraints, а не игровым ускорением персонажа.

Ghost pair callback регистрируется в broadphase, чтобы character ghost objects получали overlapping pairs.

Только после полного успеха локальные владельцы перемещаются в поля. Внутренние невладеющие указатели Bullet остаются корректны, потому что перемещение `unique_ptr` не перемещает сами объекты.

9.2 Update

Отсутствующий world завершает метод.

Фиксированный timestep равен 1/60. Положительный `deltaTime` добавляется в accumulator.

Accumulator ограничивается 0.25 секунды. Это защищает от spiral of death после долгого зависания: движок не пытается отыграть бесконечное число старых шагов.

Цикл выполняет Bullet step, пока накоплено не меньше одного timestep. Каждый шаг уменьшает accumulator и увеличивает монотонный `stepIndex`.

После цикла `interpolationAlpha = accumulator / timestep`. Renderer может интерполировать между прошлым и текущим fixed state на дисплее быстрее 60 Гц.

9.3 Shutdown

Сначала из мира удаляются constraints в обратном порядке. Затем удаляются collision objects. Мир не владеет ECS objects, поэтому он только снимает регистрации.

После этого уничтожаются world, callback, solver, broadphase, dispatcher и configuration в порядке обратных зависимостей.

Accumulator, alpha и step index обнуляются для повторной инициализации.

10 JobSystem

10.1 Initialize

Worker count ограничивается хотя бы одним потоком. Если система уже работала, повторное создание не допускается.

`stopping` сбрасывается под mutex. Затем в vector создаются threads, каждый выполняет `WorkerLoop`.

Если создание потока выбрасывает исключение, система должна остановить уже созданных workers. Этот путь важно сохранять при рефакторинге.

10.2 SubmitJob

Под queue mutex проверяется `stopping`. Новая `std::function<void()>` кладётся в FIFO очередь.

После освобождения mutex `condition_variable.notify_one` будит одного worker. Уведомление вне lock уменьшает немедленную конкуренцию за mutex.

Возвращаемый `bool` сообщает, принята ли задача. `AssetLoadSystem` превращает отказ в `AssetState::Error`.

10.3 Main thread queue

`SubmitMainThread` кладёт callback в отдельную очередь под отдельным mutex.

`ExecuteMainThreadJobs` меняет общую очередь с локальной. Lock удерживается только на swap, а сами callbacks выполняются без mutex. Поэтому callback может поставить новую main-thread задачу без deadlock; она останется до следующего вызова.

10.4 WorkerLoop

Worker ждёт condition variable, пока не появится job или `stopping`.

Если stopping и очередь пуста, поток заканчивается. Если jobs ещё есть, он завершает накопленную работу.

Job извлекается под lock, затем исполняется без lock. После выполнения уменьшается active count и condition сообщает `Wait`.

10.5 Wait и Shutdown

`Wait` блокируется, пока очередь не пуста или есть active workers. Он не обязан выполнять main-thread callbacks: их должен качать главный поток.

`Shutdown` ставит stopping, уведомляет всех, join каждого thread, очищает containers и main-thread queue.

Опасность: job захватывает `this`. Владелец `AssetLoadSystem` обязан дождаться jobs до собственного уничтожения.

11 FileSystem

11.1 Constructor и root

Root path нормализуется через `weakly_canonical`. Ошибка хранится и выводится, а объект не должен выполнять операции вне корректного root.

11.2 ScanFileSystem

Старый cache очищается. Recursive directory iterator проходит root с `error_code`, чтобы ошибка разрешений не выбросила приложение.

Для каждой записи формируется `FileEntry`: full path, relative path, name, extension, directory flag и размер.

Extension переводится в lower case. Это делает поиск `.PNG` и `.png` одинаковым.

После прохода entries сортируются для стабильного UI. Стабильность важна: один scan не должен случайно менять selection.

11.3 Поиск

`FindByPath` нормализует разделители и сравнивает cached paths.

`FindByName` вызывает case-insensitive `MatchesName` и возвращает невладеющие указатели. Они действуют до следующего `ScanFileSystem`, который перераспределит vector.

`GetFilesByExtension` нормализует точку и регистр расширения.

`GetEntriesInDirectory` сначала безопасно разрешает relative dir внутри root, затем сравнивает parent path, а не простой prefix. Иначе `assets/a` ошибочно включал бы `assets/ab`.

11.4 ResolvePath

Абсолютный input отвергается. `root / relative` проходит `lexically_normal`, затем `weakly_canonical`.

`IsWithinRoot` сравнивает path components case-insensitive на Windows. Простая строковая проверка prefix небезопасна: `C:\game\assets2` начинается похоже на `C:\game\assets`.

Флаг `allowRoot` запрещает операции удаления над самим root.

11.5 DeleteFile и DeleteDirectory

Обе функции сначала получают безопасный resolved path. Ошибка resolution завершает операцию до filesystem mutation.

`DeleteFile` вызывает `fs::remove`, проверяет `error_code` и результат, затем удаляет entry из cache.

`DeleteDirectory` использует `remove_all`, после чего удаляет все cached entries внутри подтверждённого пути.

Эти операции принадлежат editor/file tooling, но сам boundary безопасности является runtime core и разобран полностью.

11.6 CreateDirectory

Resolved path проверяется внутри root. `create_directories` создаёт промежуточные каталоги. Если каталог уже существовал, функция всё равно может считать результат успешным через `exists`.

Cache получает directory entry только после подтверждённого filesystem результата.

12 AssetLoadSystem структуры

`AssetState` задаёт конечный автомат: `Unloaded → Loading → CpuLoaded → Ready` или `Error`.

`MeshCpuVertex` повторяет GPU input: position, UV, normal, tangent, binormal и четыре bone channels.

`MeshLoadTracker` использует atomics, потому что worker пишет progress, а main/UI читает. `memory_order_relaxed` достаточен для независимых диагностических чисел; готовность данных передаётся через main-thread queue.

`MeshGpuData` владеет сырыми COM buffers. `MeshAsset` объединяет path, state, CPU/GPU данные, error и shared tracker.

Поле `gpuModel` сейчас не участвует в показанном пути и остаётся legacy заготовкой. Это стоит либо удалить, либо оформить владельцем.

13 RequestMeshLoad построчно

Карта сначала ищет path.

Если asset уже существует и не Error, повторный запрос игнорируется. Если Error, GPU ресурсы освобождаются, запись удаляется и разрешается retry того же пути.

Новый `MeshAsset` получает path, Loading, null model и новый shared tracker. Перемещение в map устанавливает runtime запись до запуска worker.

`tracker` копируется отдельно, чтобы worker не хранил ссылку на map element, который может перераспределиться.

`SubmitJob` захватывает `this`, path и tracker. Worker вызывает `LoadMeshFromFile`, создавая только CPU данные.

После завершения worker ставит main-thread callback. Result перемещается в lambda capture, чтобы не копировать большие vectors.

Main-thread callback снова получает map asset. При failure записывает Error и строку. При успехе перемещает mesh, ставит CpuLoaded и progress 0.90.

Если job queue отвергла задачу, asset немедленно становится Error. Запись не зависает в Loading.

14 AssetLoadSystem Update и snapshots

`Update(device)` перебирает assets только на главном потоке. Для CpuLoaded вызывается `CreateGpuMesh`, потому что Direct3D device integration не оставляется worker без контракта.

Успех переводит state в Ready и progress 1. Ошибка переводит в Error и создаёт fallback message.

`GetLoadSnapshots` копирует только диагностические поля. Atomics читаются relaxed. Строка stage выводится из state, а не хранится как вторая потенциально противоречивая истина.

15 CreateGpuMesh построчно

Null device, пустые vertices или indices немедленно возвращают false.

Counts преобразуются в `uint32_t`. До production масштаба полезно проверить переполнение `size_t`.

Vertex descriptor задаёт `D3D11_USAGE_DEFAULT`, byte width и `D3D11_BIND_VERTEX_BUFFER`. Initial data указывает на непрерывный vector.

После успешного vertex buffer аналогично создаётся index buffer с 32-bit indices.

Если второй CreateBuffer падает, `ReleaseGpuMesh` освобождает первый. Asset не остаётся частично готовым.

После успеха CPU vectors очищаются и `shrink_to_fit` освобождает память. Это экономит RAM, но запрещает последующее CPU mesh editing для этого loader.

16 LoadMeshFromFile построчно

Progress 0.03 сообщает начало.

Локальный `Assimp::Importer` владеет scene до конца функции.

Flags триангулируют, генерируют normals и tangents, объединяют одинаковые vertices и применяют node transforms к геометрии. `PreTransformVertices` делает этот loader статическим: hierarchy и полноценный skinning здесь не сохраняются. Для анимированных моделей используется `AssimpModelClass`.

Scene, root и incomplete flag проверяются вместе. Error string копируется до уничтожения importer.

Total mesh count и progress 0.12 публикуются после validation.

Для каждого mesh запоминается `baseVertex`, потому что все meshes складываются в один общий vertex vector.

Каждая вершина value-initialized. Position копируется обязательно. UV, normal, tangent и bitangent копируются только при наличии channel; иначе остаются нули.

Четыре bone slots обнуляются. Из-за `PreTransformVertices` этот путь не загружает weights и предназначен для статической геометрии.

Face с числом индексов не равным трём пропускается, хотя triangulate должен был устранить такой случай. Три индекса получают `baseVertex`.

После каждого mesh atomics обновляют count и progress от 0.12 до 0.85.

В конце `success = true`. Пустая геометрия будет отклонена позднее `CreateGpuMesh`; улучшение — сообщить более точную ошибку здесь.

17 Bind и RenderMeshAsset

`BindMeshAsset` требует Ready и два buffers. Затем IA получает vertex buffer, stride, index buffer R32 и triangle list. Out count заполняется только при успехе.

`RenderMeshAsset` требует material и shader. Texture выбирается из slot 0 либо main texture.

Switch по `ShaderType` выполняет типизированный вызов Light или Texture shader. Для Light дополнительно передаётся selected factor и arrays lights.

Остальные shader types возвращают false. Это осознанное ограничение helper, а не универсальный renderer.

18 Shutdown AssetLoadSystem

Каждый asset освобождает GPU buffers. Затем map очищается.

Перед этим владелец обязан дождаться worker jobs: lambda захватила `this`. Сам `Shutdown` не вызывает `JobSystem::Wait`, потому что порядок владения определяется `ApplicationClass`.

19 WorldEventLog

`Push` добавляет `WorldEvent` в deque. Если превышен предел, старое событие удаляется спереди.

`GetEvents` возвращает const reference: UI читает, но не меняет историю.

`Clear` начинает новую сессию.

World event отличается от NPC debug event: это общий gameplay факт. Для масштабирования им нужны sequence, simulation time и typed payload вместо одной строки.

20 DebugHud

Singleton `DebugHud` хранит один краткий message, kind, duration и enabled flag.

`Show` заменяет текущий message и timer. Это deliberately last-message HUD, не журнал.

`Update` уменьшает timer только положительным delta. По нулю очищает текст.

`Render` ограничивает panel width и позицию экраном, выбирает цвет по kind, преобразует UTF-8 в UTF-16 и рисует через `DWriteTextSystem`.

`Utf8ToWide` сначала спрашивает нужную длину `MultiByteToWideChar`, затем выделяет string и выполняет conversion. Ошибка возвращает пустую строку.

21 Что исправлять первым в core

1. Привести повреждённые комментарии `TimeManager` и части runtime к UTF-8.
2. Добавить validation time scale.
3. Сделать lifetime barrier между AssetLoadSystem и JobSystem явным.
4. Заменить raw COM buffers в assets на `ComPtr`.
5. Разделить статический async loader и animated Assimp loader понятными именами.
6. Записывать точную stage/error при пустой модели.
7. Добавить unit tests path containment для FileSystem.

Системы DX Framework: чтение реального кода от кадра до пикселя

Эта часть учебника учит читать существующий движок как цепь причин и следствий. Если объект переместился, надо выяснить, кто записал `TransformComponent`; если погас свет, пройти от генератора через сеть питания к `LightComponent`; если пропал предмет после загрузки, проверить перечисление сущностей в `SaveScene`, поле JSON, обратное чтение и remap ссылок. Здесь описано **состояние кода**, проверенное по файлам проекта, а изменения, которых в нём пока нет, помечены **«Проектное предложение»**. Пути вроде `src/Core/ApplicationClass.cpp` отсчитываются от `DX_FrameWork/` внутри репозитория. Номера строк помогают найти блок в текущей версии, но после правок будут сдвигаться.

Ключевой принцип для переучивания: у каждого значения есть владелец, единицы измерения, время жизни и момент записи. `Entity` — числовой ключ, `TransformComponent` — авторское или итоговое положение, Bullet хранит физическое тело, `PowerConsumerComponent::hasPower` — вычисленное за кадр состояние, JSON — снимок части компонентов, а видеопамять и потоковые задания — временные ресурсы. Когда эти категории смешиваются, появляются ошибки, которые выглядят как «физика дрожит», «шейдер не работает» или «сохранение иногда ломается».

1. Один кадр целиком: как читать `ApplicationClass::Frame`

1.1. Входные данные, режим и два времени

Точка входа кадра — `ApplicationClass::Frame(float dt)` (`src/Core/ApplicationClass.cpp:1023`). `dt` выражает прошедшее время кадра в секундах. В начале есть специальная ветка проверки оружия: при `m_weaponQA` значение принудительно становится `1/60`. Затем `DebugHud` обновляется даже при паузе игрового времени. Вызовы `m_inputManager.Update(m_input)` и `m_events.Clear()` (`:1033–1034`) означают соответственно новый снимок ввода и пустой список событий на новый кадр. Событие, оставшееся только в прошлой очереди, здесь исчезнет; производитель и потребитель события должны работать в ожидаемом порядке в одном кадре либо хранить долгоживущее состояние отдельно.

F5 обрабатывается по фронту нажатия (`f5Down && !m_wasF5Down`, `:1078`). При входе в Play вызывается `m_gameplayController.EnterPlayMode`, затем сбрасываются таймер, журнал событий мира, trigger-состояние, взаимодействие и камера. Для игрока создаётся инвентарь и тестовая винтовка; сцена для наблюдения за AI без игрока отдельно защищена проверкой `player != INVALID_ENTITY`. При выходе из Play очищаются команды движения у `CharacterMovement` и контроллеров. Важная привычка: читать **условие вызова** вместе с вызываемым методом. Комментарий возле выхода из Play о том, что физика продолжает шагать в Editor, устарел относительно фактического вызова `UpdatePhysics(isPlayMode ? simulationDeltaTime : 0.0f)` и раннего возврата при `dt <= 0` (`:1570`, `:2276`). По текущему вызову Editor не шагает Bullet и не выполняет затопление/давление.

На строках `1185–1186` вводятся `simulationDeltaTime` и `simulationRunning`. В обычном Play это время симуляции, в NPC Studio его может изменить пауза или одиночный шаг (`:1279–1283`); `simulationRunning = simulationDeltaTime > 0` (`:1352`). Поэтому `dt` не всегда равно времени игрового мира. UI, камера и диагностические сообщения могут обновляться по кадру, а оружие, AI и физика — по симуляционному времени. Когда добавляешь таймер перезарядки, выбери время осознанно: при паузе Studio перезарядка должна остановиться, значит ей нужен `simulationDeltaTime`. Когда обновляешь анимацию воды в окне редактора, используется `dt`, чтобы визуал оставался живым (`:1571`). Если перепутать их, ошибка обнаружится лишь в паузе или при нестандартной частоте кадров.

1.2. Порядок стадий и зависимость от предыдущей записи

После ввода главный поток выполняет `m_jobSystem.ExecuteMainThreadJobs()`, `m_assetLoadSystem.Update(device)` и `UpdateAsyncShipImport(...)` (`:1228–1234`). Первый вызов переносит готовые фоновые результаты на главный поток; второй может создать буферы Direct3D из подготовленных CPU-данных. Затем идут редакторский ввод, взаимодействие, подбор оружия, двери и ремонт. Камера до движения считывает новый yaw мыши (`:1405–1442`), чтобы направление намерения игрока и взгляда было согласовано. Блок `isPlayMode && simulationRunning` (`:1448`) формирует намерения управления и последовательность NPC. Таймеры оружия обновляются **до** попыток стрелять (`:1467`); AI принимает решение, движение и контроллеры получают команду, анимация выбирает клипы; `DamageSystem` потребляет события урона (`:1549–1551`). Отдельная глава книги разбирает NPC, здесь важно лишь место их записи в общей цепи.

Далее идут `SyncCompartmentPositions`, `UpdatePhysics`, `UpdateWaterAnimation` (`:1566–1571`). В `UpdatePhysics` строго задана последовательность: затопление, создание соединений, обновление корня подлодки, физика Bullet, проверка сломанных соединений, давление (`:2274–2302`). Это не косметический порядок. Затопление изменяет объём воды до расчёта выталкивания; соединение создаётся до физического шага; разрушение соединения проверяется после нагрузки; давление читает уже новую позицию тела. Значит новый ущерб от давления станет виден следующей фазе, которая его читает, а не «одновременно» со старым состоянием.

После физики камера следует итоговому `TransformComponent` игрока (`:1583–1613`). Далее `SceneManager::UpdateAll` получает ноль игрового времени в Editor (`:1628–1631`), обновляет анимационные и корабельные системы, а `UpdateLights` формирует положение светильников. Лишь затем камера строит view-матрицу (`:1634–1635`). Отдельный блок позднее вычисляет луч оружия: прицел берётся из камеры, но фактический выстрел начинается от дула. `WeaponSystem::Update` и `NpcRangedCombatSystem::Update` идут после позы и физики (`:1757–1769`), поэтому положение дула соответствует уже актуальной руке. Наконец `m_renderer->RenderFrame` (`:2010`) читает подготовленную сцену. Если новую систему помещают до той, чьи данные она читает, она увидит вчерашнее состояние; если после рендера, эффект станет заметен лишь через кадр.

Практика чтения кадра: выпиши для новой системы четыре пункта — вход, источник времени, изменяемые компоненты, первый читатель результата. Например «электроснабжение»: вход — мощности генераторов и потребностей, время — `deltaTime` из `SceneManager::UpdateAll`, запись — `hasPower`, читатели — свет, кислородный генератор, насос. Затем найди все вызовы этих читателей, а не только название класса. Так выявляется реальный pipeline.

1.3. Почему один кадр не равен одному шагу физики

`PhysicsWorld::Update` (`src/Core/PhysicsWorld.h:86–113`) складывает `deltaTime` в аккумулятор, ограничивает его `0.25` секунды и, пока хватает времени, шагает Bullet фиксированным `1/60` секунды. В кадре длительностью `1/120` секунды физический шаг произойдёт через два визуальных кадра; при резком лаге может произойти несколько шагов подряд, но не бесконечная «догонялка». `m_interpolationAlpha = accumulator / timestep` предназначен для сглаживания визуала между фиксированными состояниями. Это причина, почему нельзя рассчитывать игровую скорость как «два метра за вызов Update»: нужно умножать скорость в м/с на секунды. И наоборот, нельзя в коде силы умножить на `dt`, если затем эту величину как силу передать в `applyCentralForce` и Bullet уже интегрирует её по шагу: тогда время фактически учитывается дважды. Для конкретной силы проверяй единицы и API тела.

2. ECS: сущность, компонент, система и владение состоянием

2.1. `Entity` — ключ, а не объект

В `src/ECS/ECSRegistry.h:15-21` метод `CreateEntity` проверяет исчерпание пространства ID, выдаёт `nextEntity++` и помещает ID в `m_liveEntities`. У сущности нет встроенной позиции, здоровья или меша. Это числовая связь между отдельными хранилищами компонентов. `DestroyEntity` (`:24-28`) убирает ключ из множества живых и вызывает `Remove` во всех пулах. `IsAlive` проверяет именно множество живых (`:30`). В этой реализации не видно generation counter: после полного `Clear` реестр создаётся заново и ID опять начинаются с нуля. Поэтому долгоживущую ссылку на `Entity` нельзя переносить через `LoadScene` или `Clear`, даже если после загрузки цифра случайно совпала. До использования сохраняемой ссылки обычно нужны `IsAlive`, наличие нужного компонента и, если это сериализованная ссылка, корректный remap.

`AddComponent<T>` делегирует вставку пулу (`:33-43`). У метода нет проверки `IsAlive(e)`: вызывающая сторона обязана создать сущность и не добавлять компонент к уничтоженному ID. Более того, `ComponentManager<T>::Add` сначала удаляет прежнюю запись, затем делает `emplace` (`src/ECS/Components/ComponentManager.h:29-42`). Это **замена всего компонента**, а не изменение одного поля. Если код сохранил `T*` и затем повторно вызвал `AddComponent<T>` для того же Entity, старый указатель уже недействителен. Для обычного изменения состояния бери существующий компонент через `GetComponent` и меняй конкретное поле; `AddComponent` используй при создании или осознанной полной замене.

`GetComponent<T>` возвращает указатель или `nullptr` (`ECSRegistry.h:61-65`), `HasComponent<T>` даёт булево (`:67-71`), `RemoveComponent<T>` удаляет запись при наличии пула (`:73-77`). `GetComponents<T>` вызывает `GetOrCreatePool<T>` даже при намерении только прочитать пул (`:79-82`). `ForEach<T>` не создаёт пустое хранилище, если тип ещё нигде не использовался (`:83-92`). Двухкомпонентный `ForEach<T1,T2>` (`:45-59`) перебирает **первый** пул и ищет такой же Entity во втором. Поэтому выбери более редкий тип первым: если `PressureHullComponent` имеют 20 сущностей, а `TransformComponent` — 3000, `ForEach<PressureHullComponent,TransformComponent>` делает существенно меньше итераций. Это вывод из структуры обхода, а не гарантия порядка: внутри `unordered_map` порядок сущностей не задан.

`ActiveComponent` хранится особым образом как множество ID (`ComponentManager.h:69-96`). У него нет индивидуальных полей на каждую сущность; `Get` возвращает указатель на один `m_dummy`. Для проверки активности используй `HasComponent<ActiveComponent>`. Нельзя записывать в найденный через `Get` экземпляр и ожидать уникального состояния объекта. И важное отличие: живость и активность независимы. Неактивный объект остаётся сущностью, может сохраняться в сцену и иметь все компоненты; системы сами решают, пропустить его или нет. Например физика отключает контактную реакцию и движение неактивного RigidBody (`PhysicsSystem.cpp:1007-1027`), а `PressureSystem::Update` явной проверки `ActiveComponent` не делает. Если правило нового эффекта требует паузы для неактивных объектов, добавь проверку в этом эффекте.

2.2. Где живёт положение объекта

`TransformComponent` содержит `position`, `rotation`, `scale` и вычисляет мировую матрицу как `scaleMatrix * rotationMatrix * translationMatrix` (`src/ECS/Components/Components.h:27-40`). Углы вращения применяются в порядке X, затем Y, затем Z. Для соглашения `mul(position, matrix)` в шейдере позиция вершины сначала масштабируется, потом вращается, потом переносится. `SceneManager::UpdateAll` прямо не поддерживает отдельную копию трансформации в старом `GameObject`; `RenderAllECS` читает `transform->GetWorldMatrix()` (`SceneManager.cpp:2457`). Значит обычный путь изменения позиции для отображения — компонент, а не

ручной сдвиг матрицы только перед одним draw call. Исключение — объекты с Bullet: там надо понимать, кто хозяин на данном этапе.

Кинематическое тело получает положение из ECS каждый кадр; динамическое обычно получает положение из Bullet, а внешнее редактирование `TransformComponent` распознаётся и отправляется обратно в Bullet. Статическое тело не интегрируется динамикой, но может быть явно пересинхронизировано при изменении трансформации. В `PhysicsSystem.cpp:965–982` сравниваются `lastSyncedPosition`, `lastSyncedRotation`, `lastSyncedScale` с текущим `Transform`. Это не «почти равно», а прямое сравнение записанных значений. Для внешней телепортации динамического тела (`:1080–1113`) обновляются world transform, motion state, interpolation transform и AABB, затем сбрасываются скорости и силы. Поэтому нельзя использовать прямую запись `Transform.position` как способ плавного толчка: это будет воспринято как телепорт. Для толчка применяй силу/импульс к Bullet-телу в подходящей фазе. После шага динамическое тело пишет позицию и вращение обратно в ECS (`:1353–1394`). Именно это означает «владелец Transform» в этой системе: постоянное хранилище отображения — ECS, но авторитет движения динамического объекта во время симуляции — Bullet.

2.3. Как безопасно итерировать и удалять

Обычные пулы — `std::unordered_map<Entity,T>` (`ComponentManager.h:15–56`). `ForEach` держит итератор пула, пока выполняет callback. Удаление текущей записи внутри callback инвалидирует текущий итератор; добавление того же типа может вызвать rehash и сломать обход. Поэтому для массового удаления сначала собери ID в отдельный `std::vector<Entity>`, закончи обход, затем удали. Если создаёшь компоненты другого типа, проверь, не держит ли параллельный обход указатели в этом пуле. Здесь не требуется универсальный запрет на любую запись, но структурную модификацию **перебираемого** пула нужно считать потенциально опасной.

Удаление сущности сцены — больше чем `registry.DestroyEntity`. `SceneManager::RemoveObject` (`src/Scene/SceneManager.cpp:2092–2122`) сначала убирает соединения Bullet для обоих видов joint, удаляет `ActiveComponent`, освобождает уникальный материал, уничтожает runtime character controller, затем удаляет компоненты из реестра. Если обойти `RemoveObject` и удалить только ECS ID, constraints могут сохранить указатель на уже уничтоженное тело. Для новой системы заранее реши, есть ли у компонента внешний ресурс: Bullet body, GPU buffer, открытый дескриптор, очередь job или кэш NavMesh. Если есть, у удаления нужен явный этап освобождения. `Clear` (`:2128–2161`) сбрасывает асинхронный импорт, инвалидирует NavMesh, вызывает `ReleasePhysicsRuntime`, удаляет материалы и кэш моделей, после чего целиком заменяет `m_registry = ECSRegistry{}`. `ReleasePhysicsRuntime` (`:2163–2178`) сначала отсоединяет constraints, затем контроллеры персонажей — порядок обусловлен зависимостями Bullet. Это полезный образец для любого ресурса, который ссылается на другой.

3. Сцена и сохранение: путь от Entity к JSON и обратно

3.1. Что именно сохраняет `SaveScene`

`SceneManager::SaveScene` (`src/Scene/SceneManager.cpp:3270`) создаёт JSON с `scene_name`, `version = 2` и массивом `objects`. Лямбда `saveEntity` проверяет `IsAlive`, пропускает `NpcWeaponVisualComponent` (визуал экипировки восстанавливается позже) и `ShipPartComponent` (импортированные части восстанавливаются из корня GLB), а множество `savedEntities` защищает от дублирования одной сущности, найденной в нескольких пулах (`:3277–3289`). Затем метод перечисляет корни кораблей, объекты с мешем, отсеки, комнаты, поверхности NavMesh, физические тела, пробоины, NPC без меша, оружие, области мира, взаимодействия, триггеры, инвентарь, предметы и именованные логические сущности (`:3292–3415`). Это список **владельцев включения** в файл. Если ты создашь новый «чисто логический» компонент без

меша, имени или любого уже перечисляемого признака, он может не попасть в JSON, даже если `SerializeObject` умеет записать его поля. Нужно проверить и перечисление, и сериализацию.

На `:3417–3420` файл открывается и записывается `scene.dump(4)`. Возврат `true` означает, что `ofstream` удалось открыть и запись дошла до конца функции; явной проверки ошибки после `file <<` здесь нет. `SerializeObject(entity,id)` (`:3428–3433`) пишет два разных идентификатора: `id` — последовательный индекс объекта в документе, `entity_id` — реальный runtime ID на момент сохранения. Ссылки других компонентов ремапятся по второму полю. Не путай индекс в массиве с ключом ECS. Хранить в ручном JSON ссылку на `id` вместо `entity_id` — значит потерять цель при загрузке.

В коде чтения есть блок `scene["world"]` для fog и skybox (`LoadScene:4314–4326`), но в текущем `SaveScene` нет присваивания `scene["world"]`. Следствие: новые файлы, записанные этим методом, не несут мировые настройки этим путём, хотя старые или внешне подготовленные файлы могут их восстановить. Это факт конкретной реализации, а не обещание, что какой-то другой редакторский механизм обязательно исправляет пробел. **Проектное предложение:** включить `WorldSettings` в `SaveScene` симметрично `LoadScene`, затем проверить round-trip тумана и пути skybox на новой сцене.

3.2. Почему загрузка не может просто восстановить старые числа ID

`LoadScene` (`:4301–4455`) сначала открывает файл и парсит JSON, после этого очищает старую сцену (`Clear`). Затем создаёт таблицу `old entity_id → new Entity` и набор `restoredEntities`. Для корня корабля с `has_ship_root` вызывается `ImportShipFromGlb`, после чего возвращённому корню применяются позиция, вращение, масштаб, состояние аккумулятора и сохранённые данные генераторов; части корабля создаются импортом заново. Обычный объект идёт через `DeserializeObject`. На `:4476–4490` десериализатор вызывает `CreateEntity`, создаёт `TransformComponent` и по полям JSON добавляет `ActiveComponent`, `InvulnerableComponent`, `NameComponent`. Далее большой метод восстанавливает остальные поддержанные типы. Важно: даже если прежний ID был `41`, новый может оказаться `76`: импорт корня GLB и его частей расходует ID в другом порядке. Поэтому ссылка «дверь ведёт в комнату 41» не готова к употреблению до remap.

Когда все объекты созданы, `RemapEntityReferences(remap,restoredEntities)` (`:4433–4437`, определение `:5890`) проходит по поддержанным ссылочным полям. Обрабатываются только объекты из сериализованного набора, чтобы случайно не переназначить новые ID импортированных деталей корабля, которые уже указывают друг на друга правильно. Затем физические тела, сохранённые лишь как данные компонента, получают новые Bullet runtime bodies (`:4439–4448`), разрешаются зоны ремонта и соединения комнат. Это двухфазный протокол: сначала создание, потом связи и ресурсы. Если добавить компонент, хранящий `Entity target`, но забыть `RemapEntityReferences`, после загрузки цель может стать другой сущностью или не существовать. Для новых ссылок удобно требовать `INVALID_ENTITY` при отсутствующей цели.

В `RemapEntityReferences` есть два варианта: `remapEntity` при отсутствии записи печатает диагностику и возвращает старый ID (`:5894–5905`), `remapEntityOrInvalid` возвращает `INVALID_ENTITY` (`:5906–5912`). Первый вариант допускает, что старое число случайно совпадёт с новым живым Entity; для связи, которая должна быть гарантированно сериализована, это риск. **Проектное предложение:** для каждой ссылки явно определить политику «цель отсутствует» и использовать её, а не оставлять старый ID по умолчанию; проверить на сцене, где целевой объект специально удалён из JSON. Парсер `file >> scene` не обёрнут локальной обработкой исключений и `LoadScene` не проверяет схему до `Clear`. Битый JSON бросит исключение; неверное поле может привести к частично восстановленной сцене. Это не повод дописывать «магический» catch вслепую: сначала нужны тестовые файлы для отсутствующего `objects`,

неверного типа координат и отсутствующего asset, затем явная политика ошибки и безопасная точка замены сцены.

3.3. Упражнение: добавляем сохраняемый «маяк тревоги»

Проектное предложение, не существующая функция. Допустим, `AlarmBeaconComponent` содержит `float radiusMeters` и `Entity owner`. Путь реализации: создать компонент; решить, кто создаёт его Entity (редактор, импорт корабля или gameplay); обеспечить перечисление сущности в `SaveScene`, если она может быть без меша и имени; записать `has_alarm_beacon`, радиус и старый `owner` в `SerializeObject`; прочитать поля в `DeserializeObject`; добавить `owner` в remap; после загрузки пересоздать любые runtime-ресурсы (например, свет) и проверить `IsAlive(owner)`. На тестовой сцене разместить два маяка, один выключенный, сохранить, очистить и загрузить. Сравни не только количество компонентов, но и радиус, активность, связь с правильным кораблём и видимость в рендере. Это цепочка проверки, а не один тест функции сериализации.

4. Bullet и ECS: два представления одного физического объекта

4.1. Создание мира и тела

`PhysicsWorld::Initialize` (`src/Core/PhysicsWorld.h:20–84`) создаёт collision configuration, dispatcher, broadphase, constraint solver и `btDiscreteDynamicsWorld`; включает гравитацию `(0,-9.81,0)`, ghost callback для персонажей и GImpact для динамических вогнутых мешей. Настроены 50 итераций солвера, split impulse и ERP `0.2`. Эти значения — параметры текущей реализации, а не универсальные константы физики. Они влияют на устойчивость соединений тяжёлых отсеков и стоимость шага. Метод сначала создаёт зависимости во временных `unique_ptr` и переносит их в поля лишь после успешной настройки. Это уменьшает риск мира, который наполовину инициализирован. Повторная инициализация при уже существующем мире возвращает успех без пересоздания: нельзя ожидать, что вызов `Initialize` «обнулит» тела.

`PhysicsSystem::CreateRigidBody` (`src/ECS/System/PhysicsSystem.cpp:554–960`) требует `RigidBodyComponent` и `TransformComponent`; дальше выбирает форму по `shapeType`, строит Bullet shape, local inertia для массы, motion state, `btRigidBody`, collision group/mask и добавляет тело в мир. Внутри есть ветки Sphere, Box, Cylinder, Capsule, Plane, ConvexHull, TriangleMesh, GImpactMesh, составные элементы и особые внутренние коллизии корабля. При чтении этого длинного метода не пытайся запомнить каждую строку. Сначала выдели инвариант: на входе у сущности есть авторские параметры формы/массы и world transform; на выходе `rb.body` существует в Bullet и `lastSynced...` соответствует `Transform`. Если редактируется форма, старое тело нужно снять из мира и создать заново; в `RefreshRigidBody` (`:491–554`) сохранён старый transform, удаляются body и motion state, затем вызывается создание. Поэтому изменение масштаба нельзя считать обычным сдвигом матрицы: collision shape часто зависит от масштаба и требует перестройки.

В `PhysicsWorld` fixed step `1/60` секунд, но `PhysicsSystem::Update` получает кадровый `deltaTime` и до физического шага применяет силы плавучести. Если рендер идёт быстрее или медленнее 60 FPS, здесь стоит отдельно проверять, сколько раз силы ставятся относительно реальных Bullet шагов. Код уже ограничивает накопление времени, но сила задаётся на вызов `PhysicsSystem::Update`, а шаги выполняются внутри `PhysicsWorld::Update`. **Проектное предложение:** для воспроизводимой симуляции перенести расчёт сил, зависящих от физического шага, в цикл fixed step или явно проверить текущую схему на 30/60/144 FPS. На тесте одинаковый корабль должен иметь близкую траекторию через десять секунд игрового времени при разных частотах отображения.

4.2. Разбор `PhysicsSystem::Update` по фазам

На `PhysicsSystem.cpp:984–988` берутся пулы плавучести, тел и трансформаций. Первая фаза (`:1003–1145`) идёт по `RigidBodyComponent`. Если `rb.body == nullptr`, объект не участвует в Bullet и пропускается. Если нет `ActiveComponent`, тело получает флаг `CF_NO_CONTACT_RESPONSE`, обнуляет скорости и силы, переводится в `DISABLE_SIMULATION`; прежние collision flags и activation state сохранены для восстановления при включении (`:1007–1040`). Это runtime-остановка, не удаление авторского компонента. При повторной активации флаги и состояние возвращаются, тело будится.

Далее требуется `TransformComponent`. Для кинематического тела (`:1045–1076`) ECS всегда авторитетен: позиция и Euler-вращение преобразуются в `btTransform`, записываются в body и motion state, обновляются interpolation transform и AABB. Если изменился scale, вызывается `RefreshRigidBody` и текущая итерация заканчивается, поскольку форма зависит от размера. Для динамического тела (`mass > 0`, `:1077–1116`) обычное движение живёт в Bullet. Только если `TransformChangedExternally` обнаружил отличия от последней обратной синхронизации, ECS принудительно отправляется в Bullet: это телепорт/редакторское изменение, со сбросом скоростей и сил. Для тела с нулевой массой есть отдельная ветка внешнего изменения (`:1117–1144`) без динамической интеграции. Следовательно, при добавлении новой системы движения сначала реши: объект физически движется из-за сил либо следует программному пути? Для первого варианта не перезаписывай `Transform` каждый кадр; для второго явно выбирай кинематику.

Вторая фаза (`:1149–1344`) рассчитывает плавучесть по `BuoyancyComponent`. Третья синхронизирует ghost-персонажей из ECS до шага, вызывает `PhysicsWorld::Get().Update(deltaTime)` и читает ghost-позиции после шага (`:1345–1348`). `SyncCharacter` (`:32–217`) учитывает смещение центра капсулы относительно визуальных ног, отслеживает внешнюю правку позиции, защищает от NaN и сглаживает визуальную позицию при простое. Комментарий `:87–93` объясняет, почему визуальный поворот персонажа не следует каждый кадр вталкивать в симметричную Y-up капсулу: лишний warp может потянуть контроллер назад из-за интерполяции. Это пример разделения «где смотрит меш» и «как столкновения видят капсулу».

Четвёртая фаза (`:1353–1395`) берёт динамические активные тела, пропускает кинематические и статические, получает Bullet world transform и записывает позицию/вращение в ECS. Перед записью есть защита от NaN координат и от `abs(y)>10000`, в проблемных случаях тело останавливается. После успешной записи обновляются `lastSynced...`: на следующем кадре `TransformChangedExternally` не примет собственный результат Bullet за внешнюю телепортацию. Проверка NaN здесь касается только позиции, а не всех полей quaternion или масштабов; это граница защиты, о которой важно помнить при поиске неисправной физики.

4.3. Пример: импульс, телепорт и статическая преграда

Представь коробку массой 10 кг с `RigidBodyComponent::Box` на координатах `(0,2,0)`. Если нужно «толкнуть» её взрывом, игровая система получает `rb.body` и применяет Bullet импульс в выбранном направлении. Затем шаг Bullet меняет скорость и координаты, а обратная синхронизация обновляет ECS. Если вместо этого каждый кадр писать `transform.position.x += speed*dt`, `TransformChangedExternally` будет запускать ветку телепорта и обнулять скорости; объект может проходить сквозь динамические ожидания и разрушать естественную реакцию столкновений. Телепорт же при респавне как раз должен обновить world transform, motion state, AABB и сбросить старую скорость: в таком сценарии ветка текущего кода соответствует смыслу действия. Статическая преграда с массой ноль не падает под гравитацией; если дизайнер подвинул её в редакторе, Bullet должен получить новый transform, иначе луч попадания и визуальная стена разойдутся.

Для отладки ставь рядом три наблюдения: `TransformComponent::position`, `rb.body->getWorldTransform().getOrigin()` и `rb.lastSyncedPosition`. Если первое меняется до шага, а второе нет, ищи проверку активности, отсутствие body или неверную ветку sync. Если второе меняется, а первое нет, ищи пропуск обратной фазы, статическое/кинематическое тело или защиту NaN. Если визуал меняется, а луч Bullet попадает в старое место, проверь AABB и перерождение формы после scale. Такое сравнение локализует проблему лучше, чем перестановка `Update` наугад.

5. Вода, глубина, пробоины и атмосферное давление: не смешивать среды

5.1. Текущая физическая среда проекта

Хотя проект развивается в сторону космического симулятора, конкретные `PressureSystem`, `FloodingSystem`, `BuoyancyComponent`, `SubmarineComponent` и часть renderer описывают **подводную** среду. У них есть горизонтальный `waterLevel`, глубина `waterLevel - worldY`, плотность воды и выталкивающая сила. В вакууме формула давления от глубины неприменима: наружное давление близко к нулю, для повреждения от разгерметизации важна разность между внутренним давлением отсека и внешним. Эти модели можно использовать как отдельный «водный режим» корабля, но учебник не должен выдавать их за уже реализованную космическую разгерметизацию. Внутренние комнаты `RoomEnvironmentSystem` и кислород — другая подсистема (`SceneManager.cpp:2287–2291`), её состояние нужно соединять с пробоинами отдельным контрактом.

`PressureSystem::Update` (`src/ECS/System/PressureSystem.cpp:5–79`) объявляет `P0=101325 Pa`, `ρ=1000 kg/m³`, `g=9.81 m/s²`. Перебирает `PressureHullComponent`. Неуязвимость убирает флаги течи и пропускает урон, но наличие/отсутствие `ActiveComponent` в этом методе не проверяется. Объект без `RigidBodyComponent` или `rb.body` пропускается; глубина берётся из **Bullet**, а не из ECS. `depth=max(0,waterLevel-bodyY)`. Давление записывается в `hull.lastPressure` и считается как `P=P0+ρgh`. При превышении `hull.maxPressure` отношение `pressure/maxPressure` формирует `damagePerSecond=(ratio-1)*20`, здоровье убывает на `dps*deltaTime`, ниже нуля не идёт, утечка включается при здоровье меньше 100. Иначе здоровье медленно растёт на `repairRate*deltaTime*0.1` до 100. Это игровая кривая повреждения, а не формула прочности реального корпуса. `maxPressure` по умолчанию `202000 Pa` (`Components.h:778–786`); при 10 м глубины давление `101325+1000·9.81·10=199425 Pa`, ещё ниже порога, а около 10.3 м порог будет превышен. Если `maxPressure<=0`, код делит на него без валидации — проектная задача для конфигурационного контроля.

5.2. Почти построчный путь воды через отсек

`FloodingSystem::Update` (`src/ECS/System/FloodingSystem.cpp:5–143`) берёт отсеки, transform и связи с подлодкой. Отсек без `SubmarineComponent`, без валидного корня, сам корень или без `TransformComponent` пропускается. Это важно при создании нового корабля: одной пробоины в списке `CompartmentComponent::breaches` недостаточно; нужна правильная привязка к корню. Для каждой незапечатанной `BreachData` позиция хранится локально в отсеке. `XMVector3TransformCoord(localPos, compartmentWorldMatrix)` превращает её в world position (`:29–38`). Затем глубина пробоины `globalWaterLevel - breachWorldPos.y`. Над поверхностью расход ноль, под поверхностью вычисляется `pressure=breachDepth*9.81f`, затем `CalculateFlowRate`. В `CalculateFlowRate` (`:227–237`) площадь считается как `π·size²`, скорость как `sqrt(2·waterPressure/1000)`, расход как `area·velocity`. Именно здесь видно несогласование единиц: если `waterPressure` должен быть в паскалях, для глубины `h` нужно `ρgh`, а сейчас передаётся лишь `gh`. Получаемый расход примерно в `sqrt(1000)` раз меньше физически согласованного с такой интерпретацией. Возможно, это намеренно ослабленная игровая скорость, но тогда переменная `pressure` и комментарий о Па вводят в заблуждение; коэффициент баланса надо выделить явно. Не меняй это число автоматически в готовой сцене: течение может резко ускориться.

Например, пробоина радиуса `0.1 м` на глубине `10 м`: площадь `π·0.1²≈0.0314 м²`. Текущий код передаст `98.1`, даст `v≈0.443 м/с` и `Q≈0.0139 м³/с`. При гидростатическом перепаде `98100 Па` и той же формуле `v≈14.0 м/с`, `Q≈0.439 м³/с` до коэффициента расхода и иных ограничений. Для игры может быть разумным расход между ними, но его надо выбирать как параметр дизайна в единицах, а не получать из пропущенной плотности. **Проектное предложение:** ввести `pressureDifferencePa`, `dischargeCoefficient`, ограничение по доступному объёму/размеру шага и тесты на глубинах 0, 1, 10, 100 м.

Насос на `:73–77` откачивает `comp.pumpRate`, только если `comp.hasPower` и воды больше нуля. Чистый объём за кадр равен `(totalInflow-totalOutflow)*deltaTime`; `waterVolume` ограничивается `[0,comp.volume]` (`:84–86`). Высота воды выводится как доля заполнения, умноженная на геометрическую высоту отсека (`:91–99`). Для этого нужны `comp.volume>0` и положительная высота; в коде проверена высота, но в ветке `waterVolume/comp.volume` нет отдельной проверки ненулевого объёма. Потом создаётся `DamageDecalComponent`, если его не было, и синхронизируется с пробоинами (`:101–106`). Это перевод логического повреждения в визуальные данные для шейдера. `CreateBreachAtWorldPosition` (`:175–193`) использует обратную мировую матрицу: координата попадания мира превращается в координату отсека и сохраняется локально, поэтому маркер движется вместе с корпусом. `SealBreach` (`:198–204`) только выставляет `isSealed`; накопленная вода не исчезает и должна быть откачана отдельно.

Текущая связь насоса с сетью питания неполная. `CompartmentComponent::hasPower` по умолчанию `true` (`Components.h:692`) и сохраняется/читается как `compartment_has_power` (`SceneManager.cpp:4221`, `:4963`). По поиску в текущем `src/` нет записи из `PowerConsumerComponent::hasPower` обратно в `CompartmentComponent::hasPower`; `ShipPowerGridSystem` меняет свои consumer-поля, а не это поле. Поэтому нельзя утверждать, что отключение реактора автоматически остановит откачку каждого отсека. **Проектное предложение:** назначить насос отдельным consumer, определить поведение при частичном `powerRatio`, а вычисленное питание использовать до `FloodingSystem`. Сейчас `SceneManager::UpdateAll` с расчётом сети вызывается **после** физической стадии `FloodingSystem`; при исправлении зависимости надо осознанно перестроить порядок, иначе насос увидит состояние прошлого кадра.

5.3. Плавучесть и два противоречащих действия кода

`PhysicsSystem::Update` (`:1149–1344`) вычисляет погружение по вертикальной области тела `[objectY-height/2, objectY+height/2]` относительно поверхности. Если объект внутри отсека, вместо внешней поверхности берётся уровень воды отсека (`:1175–1202`). Для корня подлодки суммируется вода во всех отсеках. Сглаженный коэффициент погружения — смесь старого значения и нового с коэффициентом `0.3`; это уменьшает скачок силы, но вводит временную задержку и зависит от количества кадров. Выталкивающая сила считается как `fluidDensity * g * buoyancy.volume * submersionRatio * buoyancyMultiplier` (`:1271–1272`), добавочный вес воды — `totalWaterInCompartments * 1000 * g`, итоговая вертикальная сила — разность (`:1274–1286`). Затем есть сопротивление по скорости, ослабление вертикального drag при затоплении, пружинная стабилизация при малом затоплении и угловое демпфирование. Всё это смешивает физическое приближение с игровыми стабилизаторами: формулу можно объяснять, но коэффициенты `50`, `5`, `0.3` нельзя называть законом природы.

Важная несогласованность: комментарий и формула в `PhysicsSystem.cpp:1253–1279` говорят, что внешний вытесняемый объём корпуса при заливании воды **не уменьшается**, а вода добавляет вес. Но `FloodingSystem.cpp:116–140` при `lastCalculatedVolume>0` прямо записывает `buoy->volume = max(0,lastCalculatedVolume-totalWaterVolume)`. Следовательно, в таком состоянии вода одновременно уменьшает выталкивающую силу и добавляет вес — двойной штраф. Пока `lastCalculatedVolume<=0`, уменьшения объёма не будет, поэтому проявление зависит от того, кто и когда вычислил это поле. **Проектное предложение:** выбрать единую модель: внешний объём остаётся постоянным, а масса

внутренней воды добавляется один раз; либо явно описать и проверить альтернативу для открытого корпуса. Создать диагностическую сцену с известной массой и объёмом, по шагам прибавлять 0, 1, 2 м^3 воды, записывать `buoy.volume`, `waterWeight`, `netUpForce` и высоту через 10 секунд при разных FPS. Только после этого править формулы.

Ещё одна деталь: `FloodingSystem::CreateBreach` (`:148–169`) проверяет, выходит ли локальная точка за границы `localMin/localMax`, но тело ветки пустое — фактического запрета или clamp нет. Если учебный пример «стреляем в стену» создаёт дырку вне отсека, функция всё равно добавит её в `breaches`; мировой уровень воды будет вычисляться из этой точки. Следует либо явно разрешать внешнюю точку с объяснением, либо возвращать ошибку/подрезать координату. Такое исправление относится к проектному предложению, а не к текущему поведению.

5.4. Космический режим как отдельный контракт

Проектное предложение. Для космического симулятора добавь тип среды `EnvironmentMode` или отдельную конфигурацию мира. В режиме Space наружное давление задаётся в Па, гравитация берётся из локального источника, нет глобальной `waterLevel`, `FloodingSystem` и плавучесть не запускаются. Комната хранит объём в м^3, количество газа/давление, температуру, соединения дверей и пробоины в вакуум. Поток газа определяется разностью давлений и геометрией отверстия, а здоровье экипажа — парциальным давлением кислорода и временем воздействия. `PressureHullComponent` в таком режиме сравнивает **разность** внутреннего и наружного давления с прочностью, а не глубину. Один и тот же `HullBreachComponent` может быть входом двум моделям, но алгоритмы и единицы должны быть разделены. Начни с простого теста: отсек 100 м^3 при 101325 Па, закрытые двери, пробоина в вакуум — давление должно монотонно падать, масса газа не возрастать, а без пробоины оставаться постоянной. Реалистичные скорость звука и истечение можно добавить после проверки сохранения массы; пока это проект, не заявляй, что он уже реализован.

6. Электросеть, генератор и зависимые системы корабля

6.1. Генератор производит мощность, сеть её распределяет

В `SceneManager::UpdateAll` (`src/Scene/SceneManager.cpp:2237–2250`) сначала выключаются генераторы у уничтоженных объектов, затем вызывается `GeneratorSystem::Update` и только после него `ShipPowerGridSystem::Update`. `GeneratorSystem` (`src/Gameplay/Power/GeneratorSystem.h:13–113`) пропускает неактивную сущность, обнулив текущую выходную мощность и расход топлива, но сохранив настройки. При `deltaTime<=0` не расходует топливо и не меняет тепловое состояние. Для активного генератора он ограничивает `maxOutputMw`, `targetOutputMw` и остатки дейтерия/трития допустимыми диапазонами; проверяет наличие обоих видов топлива; при превышении `scramTemperature` ставит `isScrammed=true`, `isOnline=false`, а повторный запуск разрешается только после охлаждения до `restartTemperature`. `currentOutputMw` приближается к эффективной цели максимум на `outputRampMwPerSecond*deltaTime`, то есть не прыгает мгновенно. Затем из отношения текущей мощности к максимуму выводятся расход топлива в кг/с и изменение температуры. Формула температурного сглаживания `1-exp(-response*dt)` даёт более устойчивый результат при разном `dt`, чем постоянное «добавь 10% каждый кадр».

`ShipPowerGridSystem::Update` (`src/Gameplay/Power/ShipPowerGridSystem.h:18–62`) собирает временные `WorkingGrid` отдельно по корню корабля. `FindGridRoot` (`:74–84`) возвращает собственный Entity, если на нём `PowerGridComponent`, иначе `ShipPartComponent::root`, иначе `INVALID_ENTITY` для локальной сети вне корабля. Это защищает от питания одного корабля генератором другого, но все потребители без корня попадают в один условный `INVALID_ENTITY` grid: эту семантику стоит учесть при добавлении независимых

станций. Активный генератор добавляет `max(0,currentOutputMw)` только если `isOnline && !isScrammed`. Каждый consumer перед сбором сбрасывает `suppliedMw`, `powerRatio`, `hasPower`; неактивный остаётся обесточенным. Его `requiredMw` добавляется к demand без отрицательных значений. Если ошибочно считать `hasPower` сохранённым авторским переключателем, сброс каждый кадр будет выглядеть багом. Это вычисленная телеметрия, которую нужно заново получить от сети.

`UpdateGrid` (`:86–154`) начинает с мощности генераторов. Если `demandMw` больше `availableMw` и есть `PowerGridComponent`, вызывается разряд аварийной батареи на дефицит. Затем приоритеты `0..3` обходятся по порядку; в каждом собираются потребители с положительной потребностью и вызывается `SupplyEvenly`. Значит низшее число получит ресурс раньше. `SupplyEvenly` (`:156–189`) делит имеющееся поровну между потребителями данного приоритета; потребитель с малой нуждой насыщается, остаток перераспределяется на следующем круге. После распределения `powerRatio=clamp(supplied/required,0,1)`, `hasPower` становится true только от `0.999`. У потребителя с `requiredMw<=0` считается полное питание (`:119–123`). Телеметрия grid записывает generation, demand, supplied, output батареи, резерв, load ratio и brownout. Система электричества не знает о насосе или лампе по имени — она выставляет общий контракт consumer.

Пример. Генератор дал 8 MW, батарея недоступна. Приоритет 0: устройства требуют 3 и 2 MW — оба получают полностью, остаётся 3 MW. Приоритет 1: устройства требуют 4 и 1 MW. Первая равная доля 1.5 MW; маленькое устройство берёт только 1 MW, большое 1.5 MW, остаётся 0.5 MW. Во втором круге оно идёт большому: итог 2 MW из 4, `powerRatio=0.5`, `hasPower=false`. `suppliedMw` сети равна 8 MW, `demandMw=10 MW`, `brownout=true`. Если добавить аккумулятор, который выдаёт 2 MW, дефицит может исчезнуть на этот шаг — пока хватает энергии и не превышен предел мощности. Именно поэтому для части устройств полезно использовать `powerRatio`, а для бинарных устройств — `hasPower`. Текущий код лампы пользуется бинарным признаком (`SceneManager.cpp:2253–2267`), так что лампа погаснет и при 99% номинала.

6.2. MW, MWh и правильное время батареи

`EmergencyBatterySystem::Discharge` (`src/Gameplay/Power/EmergencyBatterySystem.h:9–23`) возвращает минимум из требуемой мощности, аппаратного лимита разряда и ограничения по запасённой энергии `batteryChargeMwh*3600/deltaTime`. Затем вычитает `suppliedMw*deltaTime/3600` MWh. Это размерно корректно: MW — скорость передачи энергии, MWh — запас; час содержит 3600 секунд. Например, батарея 1 MWh при отдаче 2 MW теоретически выдержит 0.5 часа, то есть 1800 секунд, если нет других ограничений. При `dt=1 c` расход `2/3600≈0.000556 MWh`. `Charge` симметрично ограничивает мощность surplus, лимитом зарядки и свободной ёмкостью (`:25–46`). Если в новом коде вычесть `MW*dt` прямо из MWh, ресурс сгорит в 3600 раз быстрее. Если применить этот же расход при `deltaTime=0`, батарея не должна измениться — методы уже имеют ранние проверки.

После сети `SceneManager::UpdateAll` связывает питание с отдельными приборами: свет с `PowerConsumerComponent` включается лишь при живом объекте и полном питании (`:2253–2267`); кислородному генератору выставляются `hasPower` и `broken`, затем обновляются `RoomEnvironmentSystem`, `OxygenGeneratorSystem`, `PlayerVitalsSystem`, `ShipAlertSystem` (`:2269–2292`). Обрати внимание на временную причинность: физическое затопление сейчас уже прошло до `UpdateAll`, а питание сети рассчитывается здесь. Если насос станет настоящим consumer, последовательность надо перестроить. Также `UpdateAll(0)` в Editor вызывает `ShipPowerGridSystem::Update`: её телеметрия может измениться без расхода топлива, поскольку сеть пересчитывает текущие мощности, а генератор при нулевом времени просто возвращается. Это допустимо как preview, но при расширении системы следи, чтобы новый consumer не расходовал ресурсы в Editor.

6.3. Добавление новой системы жизнеобеспечения

Проектное предложение. Пусть нужен фильтр CO_2. Компонент содержит заданный расход воздуха и энергоёмкость; `PowerConsumerComponent.requiredMw` показывает запрос. `FilterSystem::Update` читает `powerRatio`, состояние повреждения, текущую концентрацию CO_2 и `deltaTime`, рассчитывает удалённую массу и записывает её в комнату. Помести фильтр после `ShipPowerGridSystem`, но до расчёта витальных эффектов игрока; иначе здоровье получит концентрацию до очистки. Питание фильтра сделай либо непрерывным (`effectiveFlow = nominalFlow * powerRatio`), либо пороговым с документированным порогом; для двоичного `hasPower` при небольшом brownout фильтр полностью выключится. При нулевом времени функция не должна менять массу газа. На интеграционном тесте сравни комнату с фильтром при 100%, 50% и 0% питания, затем сохранение/загрузку значения авторского расхода и отдельно вычисляемого runtime-состояния. Всё это решение дизайна; текущего `FilterSystem` в коде нет.

7. Renderer и шейдеры: от Transform до цвета пикселя

7.1. Что делает CPU перед GPU

`Renderer::RenderFrame` (`src/Render/Renderer.cpp:203–514`) получает Direct3D context, view и projection, выбранный Entity и параметры отладки. В начале обновляется контекст вида от первого лица, создаётся snapshot кандидатов видимости для фоновой frustum-проверки (`:213–222`), собирается статистика активных мешей и обнаруживаются материалы, которым нужны reflection/refraction. Метод `InspectSceneRenderFeatures` (`:34–59`) считает reflection нужным для Water/Reflection, refraction — для Glass/Water. Это означает, что дорогие дополнительные проходы запускаются по **реально активным материалам**, а не всегда. Однако сама проверка проходит по пулу мешей, значит при большом числе объектов это тоже измеряемая работа на CPU.

Далее `RenderFrame` обновляет направленный свет и environment map, рисует shadow map, при нужде reflection и refraction в отдельные render textures, при наличии editor callback — preview, потом основную сцену в backbuffer (`:263–448`). Shadow pass использует выбранные видимые комнаты. Reflection pass строит отражённую view-матрицу для плоскости `y=0`; это конкретное соглашение текущей водной сцены, поэтому для произвольной наклонной отражающей поверхности его не хватает. Refraction pass пропускает Glass/Water, чтобы текстура преломления содержала фон за ними (`:316–382`). После каждого render texture вызываются восстановление backbuffer/viewport и `ClearSRVSlots`: в D3D11 один ресурс нельзя бездумно держать одновременно как вход SRV и выход RTV. Main pass очищает кадр цветом fog или чёрным, рисует skybox с view-матрицей без переноса камеры (`:415–445`), обычные 3D объекты, sprites, частицы, отладочные объёмы и в конце first-person view model (`:447–510`). Для оружия от первого лица очищается depth buffer и задаётся отдельная near/far projection (`:516–550`), чтобы руки не исчезали за мировой стеной из-за глубины. Это визуальный приём, а не изменение попаданий: луч выстрела отдельно вычисляется из камеры/дула в мире.

`Render3DObjects` (`:556–606`) включает Z-buffer, выключает alpha blending, передаёт fog в шейдеры, привязывает shadow map, затем просит `SceneManager::RenderAllECS`. Тот (`src/Scene/SceneManager.cpp:2393–2550`) собирает свет, строит frustum, список видимых комнат и проходит по `MeshRendererComponent`. Для непрозрачного прохода он пропускает неактивные объекты, исключённые из мирового вида, неподходящие кандидаты, editor gizmo, отсутствие Transform/model/material, невидимые комнаты и объекты вне frustum (`:2434–2462`). Лишь после этих фильтров `GetWorldMatrix()` становится матрицей конкретного draw. Далее `PreparePbrDrawState` привязывает decal-данные и вызывается renderer меша. Если объект не виден, искать причину нужно последовательно: `ActiveComponent`, видовой слой,

готовность модели, комната, frustum, затем шейдер. Замена текстуры не поможет, если draw вообще не дошёл до GPU.

7.2. Три пространства координат и вершинный этап

Для одного треугольника CPU берёт локальные вершины меша, `TransformComponent` даёт world matrix, камера даёт view и projection. В `PBRShaderClass::Render` (`src/Render/Shader/PBRShaderClass.cpp:143–176`) C++ транспонирует `world`, `view`, `projection` перед отправкой в VS constant buffer `b0`; `SetMatrixBuffers` делает то же (`:127–140`). Это контракт C++/HLSL о памяти матриц, а не лишнее вращение объекта. Если убрать transpose только в одном месте, модель может исчезнуть или вести себя зависимо от камеры. В `pbr.vs` (`src/Render/Shader/Vs/pbr.vs:7–12`) тот же `b0` объявляет три `matrix` в том же порядке. Затем код использует row-vector умножение:

```
float4 worldPos = mul(finalPos, worldMatrix);
float4 viewPos  = mul(worldPos, viewMatrix);
output.position = mul(viewPos, projectionMatrix);
```

Здесь `finalPos` — координата в пространстве модели, `worldPos` — в мире, `viewPos` — относительно камеры, `SV_POSITION` — координата для растеризации. Позиция `(1,0,0)` в меше сначала масштабируется и поворачивается вокруг локального начала, затем переносится в мир. Если свет имеет позицию в мире, расстояние до него надо вычитать из `worldPos`, а не из `SV_POSITION`: последняя уже зависит от камеры и перспективы. Pixel shader получает `worldPos` отдельно через `TEXCOORD5` (`pbr.vs:50–64`, `pbr.ps:80–94`). Между VS и PS rasterizer интерполирует атрибуты по поверхности треугольника; это не новый вызов C++ для каждого пикселя.

У анимированной модели до world/view/projection выполняется skinning (`pbr.vs:71–116`). Каждый входной vertex хранит до четырёх индексов костей и весов. Шейдер пропускает индекс вне `[0,128)`, неположительный вес, суммирует преобразованные позиции и направления, затем делит на общую сумму весов. При отсутствии валидных весов он использует исходную вершину, а не случайную `boneTransforms[0]`, которая могла остаться после предыдущего skinned draw (`:98–114`). Если модель «разъехалась», проверяй `boneIDs`, веса, bind pose и тот факт, что статический меш действительно пришёл без валидных весов. Нельзя тестировать skinning только на одной позе: ошибки иногда скрываются в bind pose и проявляются при повороте руки.

Нормаль — направление поверхности, не точка. При неравномерном scale простое `mul(normal,world)` даёт неверную ориентацию. `normal_transform.hlsli` строит normal matrix через кофакторы, соответствующую обратной транспонированной линейной части, нормализует нормаль и переортогонализует tangent/binormal. Вершинный шейдер вызывает `BuildWorldTangentBasis` (`pbr.vs:127–136`). На CPU тот же принцип используется в `PBRShaderClass::SetDamageDecals` для нормали трещины (`:327–335`): позиция `XMVector3TransformCoord` учитывает перенос, нормаль `XMVector3TransformNormal` — нет. Тест: поверхность с scale `(2,1,0.5)` и нормалью не по оси. Если освещение сохраняется при вращении и scale, преобразование согласовано; если блик внезапно перескакивает, сначала проверь normal/TBN, а не roughness.

7.3. Пиксельный этап: текстуры, материал и свет

`pbr.ps` (`src/Render/Shader/Ps/pbr.ps`) объявляет albedo `t0`, normal `t1`, metalness `t2`, roughness `t3`, crack `t6`, shadow `t7`, environment `t8`. C++ в `PBRShaderClass::Render` выставляет `t0..t3`, crack и environment, вызывает `DrawIndexed`, затем отвязывает `t0..t8` (`PBRShaderClass.cpp:210–245`). Это точное соответствие слотов: если HLSL читает `t8`, а C++ привязал новый ресурс к `t5`, пиксель не «догадается» о другом месте. `m_defaultNormalSRV`, `m_defaultMetalnessSRV`, `m_defaultRoughnessSRV`, `m_defaultCrackSRV` создаются как 1×1 fallback (`:91–95`) для отсутствующих карт. Albedo при `nullptr` получает предупреждение после draw (`:247–`

249); это не полноценная гарантия корректного цвета, поэтому путь создания материала надо проверять отдельно.

PS берёт albedo, metalness из синего канала, roughness из зелёного (pbr.ps:220–255); это соответствует metallic-roughness упаковке glTF и отдельным grayscale картам, если все каналы одинаковы. Нормаль из карты переводится из [0,1] в [-1,1], затем через TBN — в мир (:257–268). Вектор к наблюдателю V=normalize(cameraPos-worldPos) и нормаль N должны быть в одном пространстве. F0 смешивает 0.04 для диэлектрика с albedo для металла (:270–274). Направленный свет и до четырёх локальных источников используют GGX distribution, Smith geometry и Schlick Fresnel (:188–218, :282–342). Физический смысл: NDF отвечает за распределение микрофасеток, geometry term — за взаимное затенение, Fresnel — за долю отражения при данном угле. Диффузная часть уменьшается для металла через (1-metalness), чтобы поверхность не получала одновременно весь металлический блик и полный диффузный цвет. Точечный свет затухает с нормализованным расстоянием до радиуса, spot дополнительно проверяет угол конуса (:304–340). Это художественная модель затухания проекта; её нельзя автоматически считать точной обратноквадратичной физикой.

Ambient и отражение окружения добавляются на :344–381. Sky visibility приближённо выводится из shadowFactor, так что освещённость закрытых комнат уменьшается без полноценной системы light probes. Затем идут tone mapping и gamma (:383–386), alpha стекла (:391–408), трещины (:416–473), подсветка выделенного объекта и fog (:476–489). Последовательность имеет последствия: трещина в текущем коде затемняет **уже посчитанный color** после освещения. Комментарий о «повышении roughness» есть, но вычисленная ранее переменная roughness после трещины не меняется и BRDF повторно не считается (:466–472 прямо это признают). **Проектное предложение:** если нужен физически изменённый блик на повреждении, перенеси смешивание карты трещин до расчёта света, отдельно проверь влияние на metalness/roughness и сохранение визуального совпадения с пробоиной.

7.4. Как менять контракт CPU/GPU без случайной поломки

Для нового параметра сначала выбери, где он хранится: авторская настройка материала, вычисленное состояние компонента или глобальное состояние кадра. Затем выбери HLSL register(bN/tN/sN) и зеркальную привязку в C++. В PBRShaderClass.h:60–93 структуры C++ соответствуют MatrixBuffer, PBRParamsBuffer, DamageBuffer HLSL; порядок float3 + float или массивы float4 нужен для 16-байтной упаковки constant buffer. Проверяй sizeof и offsets, особенно при добавлении bool или структуры с другим выравниванием. Если параметр per-object, обновляй буфер перед **каждым** draw; иначе второй объект унаследует состояние первого. PreparePbrDrawState (SceneManager.cpp:2295–2391) именно перед draw отправляет трещины выбранного объекта, в том числе переводит локальные decal координаты панелей в координаты batch-меша. Тестируй сперва постоянным ярким цветом без освещения, потом одной текстурой, затем комбинацией света, тени и анимации. Наконец проверь выключенный объект и следующий draw: старый ресурс в слоте может создать «призрачный» эффект даже при правильном шейдере.

8. Выстрел, Bullet raycast и `DamageEvent`: что происходит с «пулей»

8.1. Текущая модель — мгновенный луч

Слово «Bullet» в проекте имеет два возможных значения: библиотека физики Bullet и пуля оружия. Текущая ветка WeaponSystem::TryFire (src/Gameplay/Combat/WeaponSystem.h:75–155) реализует **hitscan**: в момент выстрела бросает луч в btDynamicsWorld, находит первое столкновение и создаёт событие урона. Отдельной ECS-сущности летящего снаряда, его позиции/скорости и физического времени полёта в этой ветке нет. Видимый muzzle flash — частица, а отладочная линия — диагностическое изображение

луча; ни одно из них не определяет попадание. Это критично для будущего космического оружия: лазер или сверхскоростной выстрел может оставаться hitscan, ракета и медленный плазменный шар потребуют отдельной реализации.

Игрок и NPC используют общий механический метод `TryFire`. Владелец и оружие — разные сущности: `GetEquippedWeaponEntity` читает `EquippedWeaponComponent`, с совместимостью для старой сцены, где `WeaponComponent` лежит на самом персонаже (`WeaponSystem.h:20–29`). `UpdateTimers` один раз за симуляционный кадр сбрасывает `triggerHeld` и продвигает cooldown/reload у всех оружий (`:63–71`), поэтому нельзя дополнительно вызывать `AdvanceTimers` в каждом NPC: ускорится стрельба и перезарядка. `WeaponFiringRules::AdvanceTimers` (`src/Gameplay/Combat/WeaponFiringRules.h:9–33`) уменьшает cooldown, recoil и reload timer на `dt`; по окончании перезарядки переносит из резерва только недостающие патроны. `CanFire` (`:59–66`) проверяет конечные положительные fireRate/range, готовый cooldown, наличие патрона и отсутствие перезарядки. `CommitShot` (`:80–90`) выставляет `1/fireRate` секунд до следующего выстрела, вычитает патрон, увеличивает счётчик, flash timer и recoil. При `fireRate=4` это `0.25 с` между разрешёнными выстрелами, не четыре вызова кадра.

8.2. Проверка стрелка, разброс и первый контакт

Вход `TryFire` — registry, стрелок, world-координата начала, направление, указатель на Bullet world, DamageSystem, debug, device и jobs. Ранние проверки `WeaponSystem.h:82–93` отклоняют отсутствующий мир, мёртвого/неактивного стрелка, некорректный луч, отсутствующее оружие, пустой магазин и неготовое оружие. Пустой магазин пытается начать механическую перезарядку. Затем `direction` нормализуется, читается `CharacterMovement::isMoving`, вычисляется множитель разброса. Две случайные выборки дают направление внутри конуса через `WeaponFiringRules::ConeDirection` (`:93–110`): радиальная выборка задаёт cos угла между центральным лучом и выстрелом, азимут — поворот вокруг оси. Код строит ортонормированный базис `right/up/forward` в мировых координатах и возвращает нормализованный результат. Подвижность и накопленная отдача увеличивают полуугол конуса. Случайность берётся **один раз на состоявшийся выстрел**, и этот же луч используется для debug, урона и разрезания корпуса (`WeaponSystem.h:102–110`); иначе линия трассера могла бы показывать одно, а пробоина появлялась бы в другом месте.

`CommitShot` вызывается **до** raycast (`:110`), поэтому и промах тратит патрон и запускает cooldown; это ожидаемая игровая семантика. После необязательной вспышки `PhysicsRaycast::CastRay` проверяет мир до `weapon->range` (`:119–131`). Сам `CastRay` (`src/Physics/PhysicsRaycast.h:72–123`) обнуляет `outHit`, требует положительную дистанцию, нормализует направление, проверяет конечность его компонент, строит точки начала и конца `origin + normalized*distance`, вызывает `world->rayTest`. Специальный callback (`:20–53`) игнорирует собственный ghost/body стрелка, объекты с тем же `userIndex` Entity и тела в `DISABLE_SIMULATION`. Результат содержит первый collision object, Entity из Bullet `userIndex`, мировую точку и нормаль попадания. Возврат false при объекте с `INVALID_ENTITY` означает: геометрический контакт мог быть, но игровой Entity не разрешён; debug при этом может показывать контакт, а урона не будет. При диагностике такого случая проверяй установку `setUserIndex` при создании RigidBody, а не сразу множитель damage.

Стена перед NPC остановит луч: `ClosestRayResultCallback` отдаёт ближайший допустимый контакт, `TryFire` не переносит урон на выбранную AI цель за стеной. Если `CastRay` вернул false, `TryFire` всё равно возвращает true — выстрел состоялся, только не было игровой цели. Если есть hit, создаётся `DamageEvent`: source = Entity оружия, target = hit Entity, damageAmount, physical type, world-space hit position/normal и breachRadius (`WeaponSystem.h:133–154`). Для панели корпуса `CanDamageHull` true только у `WeaponType::ShipCannon` (`src/Gameplay/Combat/WeaponComponent.h:79–82`). `CalculateHullDamage` оценивает, дойдёт ли здоровье панели до порога; при разрешённом D3D device и возможности пробоины

вызывается `MeshCuttingTest::CutByRayAsync`. Его возвращаемое bool означает принятую/запущенную операцию разрезания, не то, что новый меш уже готов. `meshCutOperationId` связывает runtime-операцию с будущей ремонтной логикой. Обычный ручной выстрел по броне может показать искру, но не снимать structural HP.

8.3. Когда урон реально применяется

`DamageSystem::AddDamageEvent` (`src/Gameplay/Combat/DamageSystem.h:15–18`) лишь добавляет копию в `std::deque`. `DamageSystem::Update` (`src/Gameplay/Combat/DamageSystem.cpp:104–371`) проходит очередь, проверяет валидную цель, положительный урон, активность. Тактическая реакция NPC на попадание может зарегистрироваться до проверки `InvulnerableComponent`; затем неуязвимость отменяет уменьшение здоровья и пробоины (`:123–194`). Для игрока меняется `PlayerComponent::health`, для NPC без общего `DamageableComponent` — `NPCComponent::health`, для панели корпуса — её собственное здоровье/пробоины, для остальных — `DamageableComponent`. По окончании очередь очищается (`:371`). Если событие поставлено **до** вызова `m_damageSystem.Update` в `ApplicationClass::Frame:1549`, оно будет применено в этом кадре. Выстрел игрока и NPC через `WeaponSystem::Update` вызывается позже, на `:1757–1769`, значит его событие обычно дождётся обработки следующего кадра. Это небольшой, но реальный сдвиг: UI, который читает здоровье сразу после `TryFire`, увидит прежнее число. Если нужна обработка в тот же кадр, меняй порядок или добавляй вторую строго определённую фазу обработки, а не вызывай `DamageSystem::Update` случайно из оружия.

Для `HullPanelComponent` (`DamageSystem.cpp:215–328`) capability «может повредить броню» и факт созданной пробоины различны. Если capability false, рождается только metal impact. Иначе здоровье уменьшается на эффективный урон. Если `canBreachHull` true, создаётся новый `HullBreachComponent` с привязкой к панели и комнате, мировым hit, радиусом, локальной точкой и нормалью; добавляются `DecompressionSourceComponent`, `RepairableComponent`, ссылка на mesh cut operation, `ActiveComponent`; пишется `WorldEventType::BreachOpened`, создаются decal и частицы. Если порог пробоины ещё не достигнут, система только отмечает визуальные стадии повреждения при пересечении 20%/50%. Это не тот же список `CompartmentComponent::breaches`, который использует водный `FloodingSystem`; для подводной течи нужен явный мост между типами повреждений. Иначе можно увидеть красивую дырку в панели без роста `waterVolume`. В космической модели `DecompressionSourceComponent` — подходящий вход для атмосферы, но проверяй конкретный код потребителя перед утверждением о результате.

8.4. Как добавить настоящий летящий снаряд

Проектное предложение. Создай `ProjectileComponent` с owner/weapon Entity, скоростью в м/с, направлением, временем жизни, радиусом столкновения, типом урона и уникальным shot ID. При выстреле выполняй всё то же `CanFire`/`CommitShot`, но вместо немедленного raycast создавай projectile Entity. На каждом **фиксированном физическом шаге** перемещай снаряд от `previousPosition` к `nextPosition=previousPosition+velocity*fixedDt`; делай swept ray/sphere cast вдоль **всего сегмента**, чтобы быстрая пуля не проскочила тонкую стену между кадрами. Игнорируй owner и собственный collider, как делает `IgnoreEntityRayCallback`. При первом контакте сформируй тот же `DamageEvent` и уничтожь/деактивируй снаряд после обработки. Жизненный цикл должен учитывать попадание, максимальную дальность, истёкшее время, выгрузку сцены и save/load политику: сохранять ли пули на лету или стирать их при загрузке. Для ракеты добавь поворот к цели и взрывной радиус; для энергетического шарика — расход энергии и визуальный след. Тестируй одну стену между стрелком и целью при 30/60/144 FPS и скорость, за шаг проходящую больше толщины стены. Только после такого теста можно говорить о реальных «снарядах» в коде.

9. Фоновые задачи и ассеты: когда результат готов к использованию

9.1. Две очереди `JobSystem`

`JobSystem` (`src/Core/JobSystem.h`) содержит очередь заданий worker threads и отдельную очередь завершений для главного потока. `Initialize(workerCount)` (`:31–59`) требует положительное число работников, останавливает прежние потоки, очищает флаг остановки и создаёт worker threads. В `ApplicationClass` вызывается `m_jobSystem.Initialize(4)` (`src/Core/ApplicationClass.cpp:558`). `SubmitJob` (`JobSystem.h:61–72`) под mutex отклоняет работу, если система остановлена, иначе копирует `std::function` в очередь и будит один поток. Начальное значение `stop=true` (`:158–161`) не даёт тихо поставить задачу до инициализации, когда некому её исполнить.

`WorkerLoop` (`:124–150`) ждёт задание или остановку, извлекает job, увеличивает `activeJobs`, отпускает mutex и исполняет callback. После выполнения уменьшает `activeJobs` и, если очередь пуста, будит ожидающий `Wait`. Важно: `job()` не обёрнут в `try/catch`; `try/catch` в `Initialize` защищает создание потоков, а не любое пользовательское задание. Необработанное исключение внутри worker не превращается в `AssetState::Error` автоматически и может завершить процесс. **Проектное предложение:** ловить исключение на границе worker, передавать диагностическое состояние на главный поток и гарантировать уменьшение счётчика активных заданий. Но при проектировании конкретной задачи лучше возвращать структурированный результат с `success/error`, как делает загрузчик ассетов, и не бросать из обычной ошибки файла.

`SubmitMainThread` (`:74–78`) складывает callback в другую очередь под отдельным mutex. `ExecuteMainThreadJobs` (`:80–94`) быстро меняет очередь на локальную, отпускает блокировку и выполняет jobs уже в вызывающем потоке; `ApplicationClass::Frame` делает это на `:1228`. Поэтому worker может читать и готовить CPU-данные, а создание D3D buffers и изменение ECS, зависящее от текущей сцены, нужно делать в главном потоке после передачи результата. `Wait` (`:96–102`) ждёт только worker-очередь и `activeJobs==0`; он **не выполняет** `ExecuteMainThreadJobs`. Если worker завершился и положил callback в главную очередь, `Wait()` может вернуться до видимого результата. Это частый источник ошибки «загрузка закончилась, но меша нет».

`Shutdown` (`:104–118`) выставляет `stop=true`, будит всех, присоединяет workers. Worker выходит лишь когда `stop && jobs.empty`, поэтому уже поставленные задания будут доработаны. В `ApplicationClass::Shutdown` вызывается `m_jobSystem.Shutdown()`, затем `ExecuteMainThreadJobs()`, а позже `m_assetLoadSystem.Shutdown()` (`ApplicationClass.cpp:598–639`): готовые callback успевают примениться, пока asset system жива. Если поменять порядок удаления объектов, lambda, захватившая `this` загрузчика, может обратиться к уже уничтоженному экземпляру. Для нового async-сервиса заранее запиши, кто владеет объектом, сколько живёт callback и как отменяется работа при `SceneManager::Clear`. `Clear` отдельно сбрасывает `m_asyncShipImport` и использует shared state, чтобы поздний результат не вошёл в новую сцену (`SceneManager.cpp:2128–2132`).

9.2. `AssetLoadSystem`: четыре перехода состояния

`AssetState` (`src/Assets/AssetsLoadSystem.h:15–22`) задаёт `Unloaded`, `Loading`, `CpuLoaded`, `Ready`, `Error`. `RequestMeshLoad(path)` (`:103–157`) первым делом ищет запись в `m_meshes`. Если она есть и не в Error, повторный запрос ничего не запускает; если была Error, старые GPU-буферы освобождаются, запись стирается и тот же путь можно попробовать снова после исправления файла. Новая запись получает `Loading` и tracker прогресса. Worker вызывает `LoadMeshFromFile`, а его результат передаёт в main-thread callback: при ошибке ставятся `Error` и текст, при успехе CPU-меш переносится в asset и состояние становится `CpuLoaded` (`:125–147`). Если `SubmitJob` отклонён, метод сразу ставит Error (`:150–155`). Такой

автомат состояний лучше не сокращать до одного bool `loaded`: различаются чтение файла, подготовка геометрии, создание GPU и ошибка.

`LoadMeshFromFile` (`:411-...`) использует Assimp с triangulation, generated normals, tangent calculation, объединением одинаковых вершин и `aiProcess_PreTransformVertices` (`:420-429`). Это CPU-этап: он читает формат модели и формирует массивы, которые можно безопасно подготовить вне D3D context. Флаг pretransform означает, что данный быстрый путь превращает узловые преобразования в вершины; для skinned иерархий нужен отдельный проверенный путь импорта, не следует переносить сюда анимацию без анализа костей. `AssetLoadSystem::Update(device)` (`:158-178`) на главном потоке ищет `CpuLoaded` и вызывает `CreateGpuMesh`. Тот требует device и непустые vertices/indices, делает D3D vertex и index buffers, а после успеха очищает CPU-массивы (`:354-409`). Если index buffer не удалось создать, освобождает уже созданный vertex buffer (`:396-400`), чтобы не оставить половину GPU-ресурса. Статус становится `Ready` или `Error`.

`BindMeshAsset` (`:255-...`) отказывается от ресурса, пока нет `Ready` и обоих GPU buffers. Это граница между «файл прочитан» и «можно рисовать». При медленной загрузке renderer не должен пытаться читать `cpuData.vertices` как D3D buffer. В `Renderer.cpp:705` этот путь привязывает готовый async mesh для shadow pass; у `SceneManager::RenderAllECS` есть проверка особого пути асинхронного меша, которая при наличии spot light возвращается к полному renderer, поскольку быстрый путь передаёт лишь point-light данные (`SceneManager.cpp:2467-2470`). Значит «готовый GPU buffer» ещё не означает «полностью функциональный материал во всех проходах»; проверяй каждый render pass.

Практический тест загрузчика: запросить существующий меш дважды и убедиться, что загрузка не дублируется; запросить отсутствующий путь, увидеть Error и текст; создать файл на том же пути и повторить запрос; получить Loading → CpuLoaded → Ready, где `Ready` наступает только после main-thread pump и GPU Update. Затем закрыть приложение при активной загрузке: порядок Shutdown должен завершить work и освободить ресурсы без позднего обращения к уничтоженным объектам. Поскольку результат асинхронен, проверка `Ready` сразу после `RequestMeshLoad` не является тестом ошибки — это нормальная задержка.

10. Как добавлять систему, чтобы она стала частью игры

10.1. Контракт системы до первой строки C++

Для новой механики составь короткую карточку: «какая сущность и компонент владеют настройкой», «какие данные вычисляются каждый кадр», «в каких единицах поля», «кто обновляет и когда», «кто читает результат», «что сохраняется», «что уничтожается», «что происходит при паузе и загрузке». Например для аварийной сирены авторские `volume` и `enabled` можно хранить в компоненте, а вычисляемое `isAudible` зависит от питания, повреждения, расстояния до игрока и активного режима тревоги. Если просто добавить `SirenSystem::Update` в конец `Frame`, она может прочесть старое питание, а звук будет обновляться после UI или после освобождения сцены. Правильное место определяется входами и потребителями, а не алфавитным названием функции.

Сначала найди ближайший существующий путь. Для оборудования это цепочка `GeneratorSystem` → `ShipPowerGridSystem` → конкретный consumer → `RoomEnvironmentSystem`/HUD. Для повреждения — `WeaponSystem::TryFire` → `DamageEvent` → `DamageSystem` → `HullBreachComponent`/decal/particle → ремонт. Для визуала — ECS `Transform` + mesh + material → culling → shader buffers → VS/PS. Для ассета — запрос → worker CPU → main thread → GPU Ready → render pass. Использование готового потока данных сокращает число новых соглашений. Но нельзя насильно использовать `PressureSystem` для вакуума: среда и единицы там другие.

10.2. Минимальный полный вертикальный срез

Проектное предложение. Возьми небольшую функцию «аварийная лампа у пробоины». На сцене должны быть корабль, панель, пробоина, лампа и источник питания. `DamageSystem` открывает пробоину; система тревоги читает активные `HullBreachComponent` и ставит сигнал; consumer лампы получает мощность; визуальный компонент меняет цвет/эмиссию; HUD показывает аварийную комнату; сохранение/загрузка возвращает авторское расположение и состояние пробоины, а runtime телеметрия пересчитывается заново. Тест из одного события проверяет всю цепь. Если лампа не горит, измерь на каждом звене: создан ли breach Entity, какой у него `room`, есть ли `ActiveComponent`, какой `requiredMw`, `suppliedMw`, `hasPower`, дошёл ли объект до `RenderAllECS`, что лежит в emissive constant buffer. Так можно объяснить конкретную причину, а не переписать систему целиком.

При добавлении компонента с `Entity`-ссылками обнови SaveScene и remap, при добавлении GPU ресурса — инициализацию, освобождение и состояние после потери/пересоздания устройства, при добавлении Bullet тела — порядок удаления constraints/body, при добавлении фоновой задачи — lifetime capture и main-thread commit. Если поле влияет на геймплей и рендер, не позволяй шейдеру быть единственным хранилищем факта: он должен показывать состояние, которое есть в ECS. Если поле выводится из других каждый кадр, обычно не надо сохранять его как авторскую истину; при загрузке сначала восстанови входы, затем пересчитай выход. Если порядок зависит от предыдущего результата, зафиксируй его рядом с вызовом в `Frame` или `UpdateAll`, чтобы будущий рефакторинг не переставил его «для красоты».

10.3. Набор проверок перед большим рефакторингом

Перед переписыванием системы сделай маленькую сцену с известными числами. Для матриц — одна вершина и один объект с scale/rotation/translation; для физики — одно тело с заданной массой и импульсом; для давления — глубина 0/10/20 м; для сети — две ступени приоритета и батарея с известными MW/MWh; для рендера — один меш без normal map, один с ней и один с неравномерным scale; для сохранения — объект без меша с ссылкой на другой Entity. На каждом опыте запиши вход и ожидаемый выход в единицах. Это не «тест ради теста», а способ зафиксировать смысл кода, который ты собираешься переучить. Особенно полезно сравнивать один и тот же сценарий при 30/60/144 FPS, Editor/Play и до/после Save/Load: эти три оси обнаруживают неверное время, скрытую зависимость от порядка и недосериализованные данные.

Для работы в перерывах достаточно читать один поток за раз: `Frame` показывает **когда**, компонент — **где лежит состояние**, система — **как меняется**, serializer — **что переживает загрузку**, renderer — **как это видно**. Пройдя этот путь для одного объекта от создания до удаления, ты узнаешь больше о движке, чем после заучивания названий всех классов. В следующем шаге выбери конкретную фичу космического режима и заполни эту же цепочку без пропусков.

11. Атмосфера комнат: уже работающая игровая модель и её границы

`RoomEnvironmentSystem::Update` (`src/Gameplay/Ship/RoomEnvironmentSystem.h:15–45`) уже реализует часть эффекта разгерметизации. Это важно отделить от проектного предложения о более физической модели газа. При `deltaTime<=0` метод ничего не меняет; положительное время ограничивает `0.10 c` за вызов. Для каждой активной `RoomComponent` он сбрасывает текущую `leakRate`, считает персонажей по `objectsInRoom` с `CharacterController` и уменьшает `room.oxygen` на `count*0.01*dt`, ограничивая результат `[0,1]`. После этого вызываются `ApplyBreaches` и `EqualizeConnectedRooms`. `RoomComponent::pressure` и `oxygen` здесь — **нормализованные доли 0..1**, а `temperature` по умолчанию `20` (`src/ECS/Components/Components.h:1012–1021`); не подставляй `room.pressure=1` как один паскаль в формулу `pV=nRT`.

`ApplyBreaches` (`RoomEnvironmentSystem.h:57–105`) перебирает `HullBreachComponent`, пропускает запечатанную пробоину или пробоину без комнаты. Для активной комнаты обеспечивает наличие `DecompressionSourceComponent`, в том числе у старого сохранения. Отношение радиуса к `0.25` ограничивается `[0.25,4]` и умножает `breach.leakRate`; затем `lostFraction=clamp(effectiveLeak*dt,0,0.95)`. Давление и кислород умножаются на `(1-lostFraction)`, температура приближается к `outsideTemperature`, а `room.leakRate` хранит текущую интенсивность. По прежнему давлению, радиусу действия и расстоянию `ApplyPlayerSuction` (`:107–160`) добавляет игроку внешнюю скорость в направлении пробоины через `CharacterExternalVelocitySystem`. В текущем коде этот suction применяется только к `PlayerGameTag` из `objectsInRoom`, предметы с `RigidBodyComponent` пока не затрагиваются; этот ограниченный эффект не следует описывать как физическое истечение газа, воздействующее на любые тела.

`EqualizeConnectedRooms` (`:162–255`) проходит пары из `connectedRooms` только один раз (`roomEntity >= connectedRoomEntity` пропускается), ищет дверь между комнатами и берёт её `GetEffectiveFlowRate`. При отсутствии двери для уже объявленной связи множитель равен 1. Для кислорода, давления и температуры используются собственные скорости `0.35`, `0.55`, `0.14`; `LimitedFlow` берёт разность значений, умножает на `rate*doorFlow*dt` и ограничивает половиной текущей разности. Сначала собираются `RoomTransfer`, потом изменения применяются к обеим комнатам симметрично. Это устраняет зависимость расчёта потока от порядка записей в `unordered_map`, хотя последующий clamp может влиять на точное сохранение суммы в крайних случаях. Объём комнаты в этих формулах не учитывается: маленькая и большая комнаты с одной и той же разностью `pressure` обмениваются одинаковой долей. Поэтому это игровая модель выравнивания нормализованных показателей, не сохранение массы газа по м^3. **Проектное предложение:** при переходе к физической атмосфере хранить количество газа/массу, объём и температуру, выводить давление из состояния, переносить массу между комнатами равными и противоположными приращениями, а наружный отток считать отдельно. На первой итерации сохраняй существующий интерфейс HUD и NPC как нормализованные производные величины, но источник правды сделай размерным.

Связь с выстрелом такова: `DamageSystem` создаёт `HullBreachComponent` и `DecompressionSourceComponent` (`src/Gameplay/Combat/DamageSystem.cpp:241–276`), затем в `SceneManager::UpdateAll` атмосфера видит пробоину и уменьшает показатели. Водный `FloodingSystem` читает другой тип — `CompartmentComponent::breaches` — и потому не получает её автоматически. Ремонт, который выставит `breach.sealed`, останавливает атмосферную утечку на следующем `ApplyBreaches`, но воду в отсеке откачивает отдельный насос. Это две разные цепочки, и именно на их границе удобнее всего строить осознанное переключение «космос / вода».

12. Секции корабля, масса и разрушаемые соединения

`SubmarineSystem::AddCompartmentToSubmarine` (`src/Gameplay/Ship/SubmarineSystem.h:13–79`) проверяет у корня `TransformComponent` и `SubmarineComponent`, создаёт Entity отсека, задаёт его Transform, геометрические `localMin/localMax`, `CompartmentComponent`, ссылку `SubmarineComponent::submarineRoot`, `ActiveComponent`, добавляет ID в `rootSub->allCompartments` и выставляет `needsPhysicsUpdate`. Здесь список отсеков — часть владения корня: если компонент есть, но ID не добавлен, `FloodingSystem` ещё может найти отсек через его связь, но суммарная масса/объём и сила корня будут неполными. У метода есть параметры `localOffset` и `compartmentMass`, но текущий код ставит `Transform.position` равной позиции корня, `localOffset` кладёт в `comp.localCenter`, а `compartmentMass` не использует. Не предполагай, что переданная масса и смещение сами превращаются в отдельное физическое тело отсека: это следует проверять визуально и в Bullet для конкретной сцены. **Проектное предложение:** если API обещает смещение и

массу, либо задействовать их последовательно, либо переименовать/убрать неработающие параметры и покрыть сценой с двумя отсеками на разных позициях.

`RecalculateSubmarineMass` (`:82–133`) суммирует `comp.volume` по `allCompartments`, добавляет `15 м³` условного корпуса, затем считает нейтральную массу как `fluidDensity*totalVolume` и выбирает `0.95` от неё для небольшого запаса плавучести. Результат пишет в `rootRb->mass`, `rootBuoy->volume`, `rootBuoy->lastCalculatedVolume` и кэши `rootSub`. Это **водная** модель и один из источников `lastCalculatedVolume`, из-за которого описанное выше двойное снижение плавучести при затоплении реально может активироваться. `RefreshSubmarinePhysics` (`:136–175`) сохраняет линейную и угловую скорости, удаляет прежнее тело из Bullet, пересчитывает массу, создаёт новое через `PhysicsSystem::CreateRigidBody`, возвращает скорости. Для динамического корабля восстановление скорости избегает остановки после редакторского изменения отсека, но момент такого пересоздания всё равно может влиять на constraints и контакты — проверяй их после перестройки.

`UpdateSubmarine` (`:201–223`) не пересчитывает всё каждый кадр. `NeedsPhysicsUpdate` проверяет явный флаг, изменение количества отсеков или их размеров через кэш (`:177–199`). После refresh флаг снимается, число отсеков и размеры кэшируются. Если новая система меняет физическую геометрию, она обязана выставить `needsPhysicsUpdate`; если меняется только уровень воды, геометрию пересоздавать каждый кадр не нужно. В `ApplicationClass::UpdatePhysics` метод вызывается только для корневых `SubmarineComponent` до `PhysicsSystem::Update` (`src/Core/ApplicationClass.cpp:2286–2295`). Такая позиция позволяет Bullet увидеть новую форму и массу при следующем шаге.

Есть два вида соединений. `JointConnectionSystem::Update` (`src/ECS/System/JointConnectionSystem.cpp:5–40`) находит `JointConnectionComponent`: если target выбран, joint отсутствует и он не сломан, создаёт связь; если target сброшен — удаляет; если Bullet отключил существующий joint — отмечает `isBroken` и снимает его. `CreateJoint` (`:42–82`) требует оба `RigidBodyComponent` и живые Bullet bodies, создаёт `btFixedConstraint` в локальных pivot A/B, задаёт порог импульса `breakForce`, 100 итераций solver, регистрирует constraint в мире с выбранной настройкой столкновений между связанными телами. Если world отсутствует, выделенный joint удаляется. Это пример полного пути «авторская ссылка → runtime constraint»; только сериализовать указатель `joint` нельзя.

`SubmarineJointSystem::CreateJoint` (`src/ECS/System/SubmarineJointSystem.cpp:5–85`) похож, но хранит constraint на отдельной ECS-сущности `SubmarineJointComponent`, передаёт владение компоненту до регистрации в Bullet, чтобы исключение при создании Entity не оставило неуправляемый constraint. Его `Update` вызывается **после** Bullet шага (`ApplicationClass.cpp:2297–2298`) и читает `joint->getAppliedImpulse()` как `currentStress`; если `!joint->isEnabled()` или импульс больше `breakForce`, вызывает `ResetJoint`, отмечает `isBroken` и `isDetached` у обоих отсеков (`SubmarineJointSystem.cpp:87–123`). `RemoveJointsForEntity` сначала собирает ID связей в vector, затем удаляет их вне обхода (`:135–148`) — ровно тот безопасный способ изменения ECS-пула, который нужен при удалении сцены. Когда пробоина или взрыв должны разрывать секцию, не удаляй тело прямо из callback контакта: создай событие, дай физическому шагу закончиться, затем удали constraint и только после этого тело.

Дополнительные системы DX Framework: от нажатия клавиши до события и эффекта

Предыдущие главы подробно разобрали кадр, ECS, NPC, физику и основные сети корабля. Здесь мы пройдём ещё шесть подсистем текущего репозитория. Для каждой важно одно и то же: где хранятся постоянные данные, где живёт временное состояние, в какой момент кадра оно меняется и какой результат увидит игрок. Путь к методу указан для поиска в исходнике; точный номер строки после правок может сдвинуться.

1. Ввод игрока: от физической клавиши к игровому действию

1.1. Почему ввод разделён на два слоя

`InputClass` знает про клавиши и кнопки мыши. `GameplayInputSystem` в `src/Gameplay/Player/GameplayInputSystem.cpp` знает, что клавиша W означает `MoveForward`, R — `Reload`, E — `Interact`, а V — `ToggleView`. Это различие даёт возможность поменять привязку без изменения кода движения. Движение не должно спрашивать `IsKeyDown('W')` напрямую: иначе новая настройка клавиш подействует только на часть механик. В конструкторе `GameplayInputSystem` вызывается `SetBindings` для каждой команды. У `Fire` сейчас две привязки: левая кнопка мыши и H; у `PrimaryMouse` тоже левая кнопка. Это не ошибка само по себе, но контекст Editor/Play должен решить, кому достанется нажатие.

`Sample(input, playMode, gameplayEnabled)` проходит все `GameAction`. Для каждого действия вычисляет `enabled = playMode && (gameplayEnabled || IsSessionAction(action))`. Значит обычные действия заблокированы вне Play и при отключённом gameplay, а команды сессии вроде паузы и вида могут продолжать работать. Затем цикл перебирает привязки и складывает их логическим ИЛИ: если нажата хотя бы одна, `down=true`. Когда добавишь переназначение клавиш, не забудь проверять конфликт двух действий на одной физической кнопке в одном контексте. Сейчас такая общая кнопка существует намеренно, поэтому простое «запретить любой дубль» было бы неверной валидацией.

1.2. `pressed`, `down`, `released` на трёх кадрах

Ключевые строки `state.pressed = down && !state.down; state.released = !down && state.down; state.down = down` сравнивают новое наблюдение со старым. На первом кадре без клавиши получаем `(false,false,false)`. При нажатии: новое `down=true`, старое `state.down=false`, значит `pressed=true`, `released=false`. Пока клавиша удерживается, старое `down=true`, новое тоже true: `pressed=false`, `down=true`. В кадре отпускания новое false и старое true: `released=true`. Это маленький конечный автомат, а не три независимых сигнала.

Для движения нужна непрерывная проверка `Down(MoveForward)`: персонаж идёт всё время удержания. Для переключения камеры нужен `Pressed(ToggleView)`: иначе камера будет менять режим каждый кадр и почти сразу вернётся назад. `UpdateSession` именно так обрабатывает `ToggleView`, `TogglePause`, ускорение и замедление времени. `UpdateInventory` по `Pressed(Slot1..Slot3)` выбирает слот и по `Pressed(DropWeapon)` передаёт позицию и направление камеры в `WeaponPickupSystem`. `static_assert` там связан с тремя слотами инвентаря: если расширишь его до четырёх, компилятор напомнит обновить список команд. Это пример полезной проверки архитектурной связи во время сборки.

Упражнение. Нарисуй таблицу для клавиши R на четырёх последовательных кадрах: не нажата, нажата, удерживается, отпущена. Затем проследи, почему `RequestReload` должен запускаться на `Pressed`, а автоматический огонь — использовать `Down` либо отдельный флаг спуска. При потере фокуса окна

проверь, что `InputClass` сбрасывает состояния: иначе старое `down=true` может оставить персонажа движущимся без руки на клавиатуре.

2. Триггерные зоны: геометрия, память пересечения и действие

2.1. Зона не является Bullet телом

`TriggerZoneSystem.h` прямо описывает point-based вычисление. `Update(registry, deltaTime, events)` сначала накапливает собственное время, затем собирает `TriggerZoneComponent + TransformComponent` в список зон. Вторым проходом собирает кандидатов актёров по Transform и классификации типа. Оба списка сортируются по `Entity`. Это даёт устойчивый порядок событий даже если внутренний порядок pools меняется. Проверка пересечения идёт по координате актёра и форме зоны, а не по контакту Bullet ghost object. Поэтому видимый меш, физический collider и зона могут не совпадать; debug отображение формы зоны обязательно для авторской сцены.

На бумаге возьми круг радиуса 2 м в точке `(5;0;5)` и игрока `(6;0;5)`. Разница `(1;0;0)`, квадрат расстояния 1, квадрат радиуса 4: игрок внутри. Для игрока `(8;0;5)` квадрат расстояния 9, он снаружи. Проверка квадрата избавляет от `sqrt` и сохраняет тот же ответ при положительном радиусе. Текущий `ContainsPoint` поддерживает условия формы из компонента; при изменении способа расчёта сравни граничные точки ровно на поверхности. Флаг «внутри» относится к одному актёру и одной зоне, а не ко всей сцене.

`TriggerZoneSystem` хранит прошлые пересечения, cooldown и one-shot состояние в своём `m_runtime`, а не в авторском компоненте. Поэтому `Enter` означает переход «в прошлом кадре снаружи, сейчас внутри», `Stay` — «внутри снова», `Exit` — «был внутри, теперь снаружи». `Reset` очищает эту память при начале нового эпизода. Если удалить зону, система удалит её временную запись без выдуманного Exit: исходной зоны уже нет. Память состояния важна: без неё каждое нахождение внутри выглядело бы новым входом, и дверь либо HUD срабатывали бы десятки раз за секунду.

2.2. От события к команде

`TriggerActionSystem.h` получает `EventQueue`, ищет в `GameEvent` вариант `TriggerZoneEvent` через `std::get_if` и читает `TriggerActionComponent` у зоны. Флаги `executeOnEnter`, `executeOnStay`, `executeOnExit` решают, на какой переход реагировать. Затем `Execute` выбирает цель: явный `action.target`, иначе `event.linkedEntity`. При выходе и `invertOnExit` логическое значение меняется на противоположное. Например, зона входа активирует свет, зона выхода деактивирует тот же свет без второго компонента.

Доступные действия в текущем коде: сообщение HUD, включение/выключение Entity через `ActiveComponent`, переключение активности и установка `DoorComponent::isOpen`. При `SetDoorOpen` эта система меняет желаемое состояние двери, но физический проход зависит от дальнейшего обновления DoorSystem, а питание и анимация могут препятствовать фактическому открытию. Событие не должно обещать, что collider уже освобождён. `TriggerActionFeedback` держит текст и оставшееся время для HUD; `Update` уменьшает таймер по `deltaTime`, а `GetActivationsThisFrame` помогает отладке.

Практика: сделай зону перед дверью, `SetDoorOpen=true` на Enter и `invertOnExit=true` на Exit. Выведи по кадрам положение игрока, состояние inside, тип события, `door.isOpen`, питание двери и положение collider. Пройди границу медленно, затем телепортируй игрока через неё, поставь паузу и перезагрузи сцену. Так обнаруживаются повторный Enter, неверный cooldown и оставшаяся после reload runtime память.

3. Инвентарь: одна Entity оружия и три слота

`InventorySystem.h` отделяет данные `InventoryComponent` от операций `PickupWeapon`, `SelectSlot`, `DropSlot`, `SwapSlots`, `Sanitize` и `SyncEquippedWeapon`. Слот хранит ссылку на Entity предмета; ссылка должна указывать на живой объект и на допустимого владельца. `EnsureInventory` создаёт компонент, если его ещё нет. `PickupWeapon` не должен просто нарисовать винтовку в руке: он должен занять слот, назначить владельца, убрать или изменить представление предмета в мире и при необходимости экипировать его. Если операция неуспешна из-за полного инвентаря, состояние владельца и world pickup должно остаться прежним.

`WeaponPickupSystem.h` связывает взаимодействие и инвентарь: `Update` обрабатывает запросы контакта/подбора, `DropActive` возвращает выбранное оружие в мир, при настройке может создать физику для брошенного предмета. `ConsumeInventoryChanged` возвращает и сбрасывает флаг изменения — это сигнал для HUD или другого потребителя, чтобы он не перечитывал всё без причины. `InventoryOperation` и статусы нужны не только программе: игроку полезно объяснить «слоты заняты» или «предмет уже у другого владельца», а тесту — сравнить конкретный отказ.

Инвариант: одна Entity оружия не должна одновременно быть предметом двух инвентарей. На сцене с игроком A и NPC B помести оружие W в слот A, попытайся поднять его B, затем выбрось A и подними B. На каждом шаге запиши владельца W, слот, активный предмет и world представление. Если после reload W встречается дважды, ищи ошибку сериализации ссылок или восстановления производного визуала. `SanitizeSlots` проверяет живость и валидность ссылок — полезная защита после удаления Entity, но она не заменяет правильный lifecycle в операции подбора.

4. Анимация: игровое время, поза и вид от первого лица

`AnimationSystem::Update` в `src/Animation/AnimationSystem.h` делает два прохода по `AnimationComponent`. Сначала обновляет мировых персонажей. Синхронизируемые first-person копии пропускаются, чтобы их не опередил произвольный порядок Entity в pool. Если `isPlaying=false`, нет меша, модели или клипов, текущая Entity пропускается. Для персонажа `AssimpModelClass::UpdateAnimation(animComp, deltaTime)` является единственным местом накопления времени клипа; двойное прибавление `dt` в системе ускорило бы анимацию. В редакторском socket preview пауза даёт `UpdateAnimation(...,0)`: поза сэмплируется в установленное время без хода часов.

Вторым проходом first-person представление получает `owner`, читает его анимацию и копирует нужные настройки скорости и фаз. Так руки в камере и мировой персонаж используют согласованную позу. Когда мир уже обновлён, fp-копия может брать его текущее время; если сделать наоборот, оружие в руках и ствол на мировой модели разойдутся на один кадр. Для NPC это критично: `NpcRangedCombatSystem` строит мировую позицию дула из кости и Transform. Если вызвать стрельбу до обновления костей, математически правильный `muzzlePosition` будет принадлежать прошлой позе. Случай «дуло у ног» часто является ошибкой пространства или порядка матриц, а не логики AI.

Анимация показывает действие, но не обязана быть владельцем игрового результата. `RepairSystem` хранит прогресс работы, а не кадр клипа ремонта. WeaponSystem списывает патрон и создаёт луч, а не цвет пикселя muzzle flash. Если добавить animation event, определи, что именно оно запускает и как исключить повтор события при зацикливании или пересэмплировании клипа в редакторе. Проверь: пауза игры останавливает gameplay действие; пауза preview меняет только редакторскую позу; изменение FPS не меняет длительность шага атаки.

5. Атмосфера комнат: простая игровая модель и физические пределы

Текущий `RoomEnvironmentSystem::Update` в `src/Gameplay/Ship/RoomEnvironmentSystem.h` уже уменьшает кислород, давление и температуру при пробоине. Это важно назвать точно: `PressureSystem` из другого каталога описывает внешнюю водную глубину, а `RoomEnvironmentSystem` — упрощённую атмосферу комнат. В начале update неположительный `dt` прекращает работу, положительный ограничивается 0,10 с. Затем `CharacterExternalVelocitySystem::BeginAtmosphereFrame` начинает новый набор внешних скоростей. Для активной комнаты считается число `CharacterController` среди `objectsInRoom`, после чего `oxygen -= count*0.01*dt` и ограничивается [0,1]. Это игровая доля, не масса кислорода в килограммах.

`ApplyBreaches` обходит незапечатанные `HullBreachComponent`, проверяет комнату и добавляет недостающий `DecompressionSourceComponent` для старых сохранений. `effectiveLeak = max(0,leakRate)*clamp(radius/0.25,0.25,4)`. За шаг `lostFraction=clamp(effectiveLeak*dt,0,0.95)`; давление и кислород умножаются на `1-lostFraction`. Если `effectiveLeak=0.2/c` и `dt=0.1c`, то доля потери 0.02, давление 1.0 станет 0.98; на следующем таком шаге 0.98 станет 0.9604, а не 0.96. Поэтому кривая похожа на экспоненциальный спад. За 10 шагов получится `0.98^10≈0.817`, если другие системы ничего не добавляют. Это численное правило текущей игры, не уравнение газодинамики через сечение отверстия в паскалях.

Температура сдвигается к `outsideTemperature` по `thermalStep`; давление, кислород и температура ограничиваются. `ApplyPlayerSuction` находит игрока в комнате, нормирует направление к пробоине, вычисляет спад `(1-distance/radius)^2` и передаёт внешнюю скорость через `CharacterExternalVelocitySystem`. При нулевой длине направления деление запрещено. Тяга сейчас применяется к игроку, не ко всем предметам. После пробоин `EqualizeConnectedRooms` считает обмен между связанными помещениями. Текст учебника может использовать закон сохранения массы как цель дальнейшей физической модели, но эти поля [0,1] и коэффициенты обмена сейчас являются игровым приближением. Для добавления реалистичной модели пришлось бы хранить объём комнаты, количество газа и температуру в единицах, затем проверять баланс вещества при каждой связи.

Проверка на одной сцене: две одинаковые комнаты, дверь закрыта, в первой давление 1, во второй 0.5. Зафиксируй показатели до и после 1 с, затем открой дверь и повтори. С пробоиной отдельно сравни 30/60/120 FPS по одинаковому игровому времени. Если разница растёт, проверь ограничение `dt=0.10`, порядок открытия двери и вызовы EnvironmentSystem. Не смешивай этот тест с `FloodingSystem`: тот считает воду и плавучесть.

6. Частицы: рождение, жизнь и отрисовка

`ParticleSystem.cpp` хранит emitters, частицы и render items в заранее зарезервированных векторах. `CreateEmitter` ограничивает число источников `MAX_EMITTERS`, выдаёт ненулевой ID, сохраняет описание и может сразу подготовить первое рождение через `spawnAccumulator`. Обычно скорость эмиссии выражается в частицах/с: если она 12/с и `dt=0.25c`, аккумулятор увеличится на 3 и можно создать три частицы. При 60 FPS каждый шаг даст 0.2, но после пяти шагов сумма снова равна 1. Без дробного аккумулятора эмиттер с малой скоростью на высоком FPS вообще ничего бы не порождал.

В `Render` есть защита от двойного обновления: Play уже вызывает update из `ApplicationClass`, а Editor может сделать запасной шаг для preview. Флаг `m_updatedSinceLastRender` решает, нужен ли этот шаг; после рендера он сбрасывается. `ParticleRenderer` создаётся при необходимости и только один раз пробует `Initialize(device)`; это граница CPU симуляции и GPU ресурсов. Если GPU инициализация не удалась, gameplay событие взрыва не должно превращаться в отсутствие урона. Текстуры и shader particles — способ показать событие, но его игровая причина хранится вне render items.

Для одной частицы с начальной позицией `(0;0;0)`, скоростью `(2;4;0)` м/с и временем жизни 1 с простое движение без сил даёт через 0.25 с позицию `(0.5;1;0)`. Если подключить гравитацию, положение зависит от выбранного интегратора. Цвет можно считать `lerp(startColor,endColor,t/lifetime)`; при половине жизни результат находится между цветами по каждой компоненте. `Lerp` и `LerpColor` есть в этом файле. Следи, чтобы визуальная альфа и физический урон не управляли друг другом: исчезновение дыма не должно «отменять» уже полученное повреждение.

7. Взрыв: видимый эффект и импульс для физических тел

`ExplosionSystem::Emit` в `src/Gameplay/Effects/ExplosionSystem.cpp` принимает позицию и `ExplosionSettings`. Сначала ограничивает radius снизу 0.05 м, impulse — нулём, и вызывает `particleSystem.EmitExplosion`. Это визуальная часть. Затем проходит `RigidBodyComponent`. Тела без Bullet pointer, статические и кинематические исключаются; масса через `getInvMass()` должна быть положительной. Это важно: от взрыва не должен двигаться неподвижный интерьер лишь потому, что у него есть видимый меш.

Для оставшегося тела система берёт центр масс и вектор от эпицентра. Если расстояние больше радиуса, ничего не происходит. Если почти равно нулю, вместо деления на ноль выбирается направление вверх. После добавления `upwardBias` направление нормируется. Сила толчка задаётся `impulseStrength*(1-distance/radius)^2`. При radius=10 м, impulseStrength=100 Н·с и distance=5 м получаем долю 0.5, квадрат 0.25, импульс 25 Н·с. Для тела массой 5 кг это могло бы дать изменение скорости 5 м/с в направлении импульса до влияния других контактов. У границы radius импульс ноль; рядом с центром почти полный. Применение `applyCentralImpulse` будит тело, а небольшой `applyTorqueImpulse` добавляет вращение. Возвращаемое число — количество затронутых физических тел, а не число получивших урон персонажей.

У взрыва здесь нет обещания прямого уменьшения здоровья: этот метод занимается частицами и физическим импульсом. Если нужен игровой урон по радиусу, создай отдельный путь через `DamageEvent` и `DamageSystem`, с проверкой стен и фракций по выбранным правилам. Не используй число затронутых Bullet тел как суррогат количества раненых: статический игрок или персонаж с отдельным character controller может быть опасно близко, но не попасть в этот список.

8. Тревога корабля: выбор самого серьёзного сообщения

`ShipAlertSystem::Update` в `src/Gameplay/Ship/ShipAlertSystem.h` каждый раз очищает `s_current`, находит комнату игрока по `objectsInRoom` и затем последовательно проверяет кислород, пробоину, power grid и генераторы. При кислороде ≤0.20 выставляется Critical, при ≤0.45 — Warning. Открытая пробоина именно в комнате игрока также даёт Critical. У энергосети brownout — Critical; работа от батареи без генерации — Warning; запас батареи ≤15% — Advisory. У генератора `isScrammed` даёт Critical, высокая температура — Warning, нехватка топлива — Advisory.

`SetAlert` заменяет сообщение только если новый `level` строго выше текущего. Если два события имеют уровень Critical, остаётся первое, встреченное в порядке обхода. Для UI это простой приоритет, но он скрывает остальные аварии. Например, пробоина и brownout одновременно покажут лишь одну надпись. Если игра должна показывать очередь тревог, потребуется список событий с устойчивым порядком и сроком действия, а не одно поле `s_current`. Обрати внимание и на область видимости: найденный `shipRoot` ограничивает проверку генераторов, если корень известен. Если комната игрока не найдена, логика работает с fallback `INVALID_ENTITY`; этот случай стоит испытать отдельно в сцене без разметки комнат.

Для проверки создай комнату с кислородом 0.44 и генератором без топлива: ожидается Warning, потому что оно выше Advisory. Добавь пробоину: ожидается Critical. Закрой пробоину и поднять oxygen до 0.8 — останется Advisory, если именно низкий запас топлива всё ещё виден системе. Это не «переключение текста» в HUD: оно показывает результат чтения нескольких ECS компонентов в определённом порядке.

9. Room culling: какие комнаты видит камера

`RoomCullingSystem.cpp` уменьшает число объектов, передаваемых рендеру. `FindRoomContainingPoint` проходит комнаты, берёт Transform и переводит мировую точку камеры в локальное пространство комнаты через обратную world matrix. Затем сравнивает её с `localMin/localMax` с небольшими допусками границ. Почему нужен inverse? Если отсек переместили или повернули, сравнение мировой координаты камеры с локальными min/max ошибочно. Обратная матрица возвращает точку в систему координат отсека, где прямоугольные границы снова имеют простой смысл.

Кэш `s_roomTransformCache` хранит position, rotation, scale и worldToLocal. При неизменном Transform повторно вычислять inverse не нужно. Это экономит работу при множестве объектов, но требует корректного сброса при загрузке сцены и изменении geometry bounds. `BuildVisibleRooms` начинает с комнаты камеры и использует связи дверей, чтобы сформировать список видимых помещений. Дверь может быть видна из любой из двух соседних комнат. Это уровень отсечения кандидатов, не доказательство фактической видимости каждого треугольника: от камеры до предмета ещё может стоять колонна.

Сложность. Если в корабле 100 комнат и 10 тысяч объектов, отправлять все объекты в каждый draw pass дорого. Отсечение по комнатам сокращает число кандидатов, но сама проверка принадлежности комнаты тоже имеет цену. Поэтому в коде есть кэши Entity→Room и debug статистика: число запросов, проверок и текущая комната камеры. При подозрении на исчезающий объект сначала сравни найденную комнату камеры, комнату объекта, состояние двери и итоговый visibleRooms. Не лечи culling отключением depth или изменением PBR shader — объект, который не попал в список draw candidates, GPU никогда не увидит.

10. Показатели игрока: перевод комнаты в здоровье и выносливость

`PlayerVitalsSystem::Update` в `src/Gameplay/Player/PlayerVitalsSystem.h` ищет `PlayerComponent` и комнату, содержащую Entity игрока. Доля `room.oxygen` в [0,1] превращается в целевой процент игрока умножением на 100. Текущее `player.oxygen` приближается к цели со скоростью 25 процентных пунктов в секунду. Если комната внезапно упала с 1.0 до 0.5, а у игрока пока 100, при `dt=0.1` показатель уменьшится до 97.5, а не мгновенно до 50. Такой лаг является правилом текущей системы; он может представлять запас дыхания/снаряжения, но код прямо не моделирует объём лёгких.

При нуле кислорода здоровье убывает на 10 единиц/с, если нет god mode или `InvulnerableComponent`. Значения ограничиваются [0,100]. `StaminaUpdate` уменьшает stamina на 20/с при `isRunning`, восстанавливает на 10/с, если `wantsRun=false`. Различие двух флагов существенно: игрок может хотеть бег, но фактически не бежать из-за иных условий; тогда обычное восстановление по этой ветке не происходит. Проверь границы: нулевой `dt`, потерю комнаты, паузу, смерть, переход между комнатами и смену флага бега. Система считывает атмосферу; она не должна самостоятельно менять давление комнаты, иначе двое игроков или NPC нарушат единый источник состояния среды.

Application ECS input scene и animation runtime

Эта глава закрывает связующий код движка. Методы Editor UI, gizmo, drag and drop и editor-only spawning намеренно не разбираются. В `ApplicationClass` объясняется только runtime-путь кадра и общие ресурсы. Импорт моделей и Studio уже подробно разобраны в предыдущих главах книги; здесь добавлен порядок исполнения конкретных методов.

1 ApplicationClass почему он сейчас слишком большой

`ApplicationClass` одновременно инициализирует сервисы, управляет режимами, вызывает gameplay systems и строит render frame. Это рабочая composition root, но уже не удобное место для новых механик.

Правило чтения файла:

1. Поля `unique_ptr` обычно означают владение.
2. Raw pointers на device resources требуют явного `Shutdown` или `Release`.
3. ECSRegistry принадлежит SceneManager.
4. Frame не должен становиться владельцем состояния NPC, оружия или комнаты; он только задаёт порядок.

2 ApplicationClass Initialize

2.1 InitializationGuard

1. Локальный guard получает `owner` и `committed = false`.
2. Если любой следующий шаг вернёт false, destructor guard вызовет `Shutdown`.
3. Только в самом конце успешной инициализации `committed` становится true.

Это транзакция ресурсов: частично созданное приложение не оставляет D3D и Bullet объекты.

2.2 Графический фундамент

1. Сохраняются width, height и HWND.
2. Создаётся `D3DClass` и Direct3D device.
3. Создаются camera и camera controller.
4. Подготавливаются shader wrappers, shadow map и render textures.
5. Default material и default texture обеспечивают fallback для неполного asset.
6. Physics debug и particle presentation получают device, но не владеют им.

2.3 Physics и ECS services

1. `PhysicsSystem::InitializeWorld()` создаёт Bullet singleton.
2. SceneManager получает device, context, default material и asset services.
3. GameplayController связывается с registry и input.
4. JobSystem запускает ограниченное число workers.
5. AssetLoadSystem получает main-thread upload context.

2.4 Scene и gameplay content

1. Выбирается стартовая scene или combat arena.
2. `LoadScene` создаёт ECS entities и запускает asset paths.
3. Player spawn выполняется только для режима, где игрок нужен. NPC-only симуляция не обязана иметь player.
4. Weapon visuals создаются после inventory и model availability.
5. Skybox загружается через отдельный SRV path.

3 ReloadSkyBox

1. Новый HDR или cube asset сначала загружается во временный SRV.
2. Если загрузка не удалась, старый sky остаётся рабочим.
3. Только успешный new SRV заменяет `m_skySRV`.
4. Старый SRV освобождается после замены.
5. Sky shader и PBR shader получают новую environment texture.

Это правильный commit pattern: пользователь не остаётся без неба из-за ошибочного нового файла.

4 ApplicationClass Frame runtime-порядок

Большой `Frame` лучше читать фазами.

4.1 Снимок режима

1. В начале читаются editor/play mode, pause, Studio и UI capture flags.
2. Edge flags F1, F2, F5, F6 сравниваются с прошлым кадром.
3. Переход режима выполняется один раз на нажатие.
4. ImGui capture влияет на input routing, но не должен останавливать сам render.

4.2 NPC Studio и свободная камера

1. Logic Studio может работать в живой сцене.
2. При входе сохраняется предыдущий camera/control state.
3. Если курсор над UI, ввод идёт интерфейсу.
4. Вне UI правой кнопкой или настроенным режимом камера получает look input.
5. Выход восстанавливает управление без телепорта gameplay entity.

4.3 Simulation gate

1. `simulationRunning` объединяет play mode, pause и step.
2. Системы решений, physics, timers и vitals получают `dt` только при разрешённой симуляции.
3. Render, UI и debug drawing работают даже на паузе.
4. Step пропускает ровно один фиксированный simulation slice, затем снова ставит паузу.

4.4 Gameplay и AI

1. Input snapshot создаётся до систем персонажа.
2. Weapon timers обновляются до попыток выстрела.
3. Damage queue прошлого producer-прохода обрабатывается в определённой фазе.
4. NPC command, utility, tactical, cover, navigation и ranged execution вызываются в зафиксированном порядке.
5. Group tactics могут быть выключены отдельным feature flag, не меняя одиночный AI.
6. Animation state читает уже принятое gameplay-решение.

4.5 Physics и transform

1. Character intent записывается до physics.
2. Bullet делает fixed substeps.
3. PhysicsSystem синхронизирует dynamic bodies в ECS transforms.
4. Character controller отдельно синхронизирует ghost transforms.
5. Animation attachments обновляются после итогового transform.

4.6 Render preparation

1. Room visibility и frustum candidates вычисляются по текущей camera.
2. Shadow pass использует light view-projection.
3. Opaque world pass рисует обычные meshes.
4. Transparent и effects идут позднее.
5. First-person pass использует отдельные правила depth/view model.
6. Debug primitives и UI завершают кадр.

4.7 Очистка одноразовых данных

1. Event queue очищается после всех consumers.
2. Input mouse delta сбрасывается после использования.
3. Per-frame debug buffers очищаются после render.
4. Step state возвращается в pause.

5 ApplicationClass Shutdown

1. `m_isShutdown` делает метод идемпотентным.
2. JobSystem останавливается до уничтожения объектов, которые могли использовать jobs.
3. Runtime systems отсоединяют Bullet constraints и bodies.
4. SceneManager очищает ECS-owned presentation resources.
5. GPU objects освобождаются до D3D device.
6. Device context очищает привязки.
7. Raw COM resources вызывают `Release` и зануляются.
8. D3DClass выключается последним среди графических объектов.

Если child resource освобождается после device, его Shutdown уже может обратиться к мёртвому context.

6 Weapon visual helpers

6.1 SetupTestRifleVisuals

1. Ищется local player и equipped weapon.
2. Старые view model entities удаляются, если owner или weapon изменился.
3. Grip profile загружается один раз и кешируется.
4. Animation bindings игрока получают нужные clips.
5. World visual создаётся для third-person/NPC view.
6. First-person visual создаётся отдельно и получает attachment component.
7. Arms и weapon visibility зависят от camera mode.

6.2 UpdateTestRifleVisuals

1. Каждый кадр проверяется актуальный equipped weapon.
2. Attachment bone и local grip transform обновляются.
3. Невалидный visual пропускается, а не разыменовывается.

6.3 UpdateNpcWeaponVisuals

1. Собираются живые NPC owners.
2. Inventory определяет реально экипированное оружие.
3. Профиль хвата загружается только при первом появлении weapon entity.
4. Visual создаётся или переиспользуется.
5. Bone attachment связывает его с рукой после animation update.

7 InputClass низкий уровень

7.1 Initialize

Массив клавиш, mouse position, delta и capture flags приводятся к нулевому состоянию. Иначе первый кадр может увидеть случайные нажатия.

7.2 Window events

1. `KeyDown` проверяет код меньше 256 и ставит true.
2. `KeyUp` ставит false.
3. `SetMousePosition` хранит координаты сообщения окна.
4. `AddMouseDelta` суммирует raw deltas нескольких сообщений.
5. `GetMouseDelta` читает накопление.
6. `ResetMouseDelta` очищает его один раз в конце кадра.

7.3 Captured mouse

1. `SetMouseCapture(true)` скрывает курсор и фиксирует режим.
2. `UpdateCapturedMouseDelta` сравнивает cursor с центром client area.
3. Разница добавляется в delta.
4. Cursor возвращается в центр, поэтому он не достигает края экрана.
5. Release возвращает нормальное положение и видимость.

8 InputManager высокий уровень

1. `Update` читает физическое состояние из InputClass.
2. Current и previous left button дают pressed/released edges.
3. Press запоминает стартовую позицию потенциального drag.
4. Пока расстояние меньше threshold, действие остаётся click candidate.
5. Превышение threshold включает dragging.
6. Drag callback получает текущую позицию каждый кадр.
7. Release вызывает drag end или обычный click.
8. При mouse capture UI click/drag path отключается.
9. `Reset` очищает переходные состояния при смене режима.

9 CameraController

9.1 Mode toggle

1. Требуются input manager и input.
2. Toggle edge вычисляется по текущему и прошлому состоянию.
3. UI block запрещает смену режима.
4. Free/follow mode переключается один раз.

9.2 Free look

1. Правая кнопка захватывает mouse.
2. Release снимает capture.
3. Delta умножается на sensitivity.
4. Yaw свободно накапливается.
5. Pitch ограничивается вверх и вниз.
6. Camera получает новые rotation values.

9.3 Free move

1. Из yaw вычисляются forward и right на плоскости XZ.
2. W/S двигают вдоль forward.
3. A/D вдоль right.
4. R/F меняют высоту.
5. Shift выбирает быстрый speed.
6. Все смещения умножаются на `dt`.

9.4 Follow и first person

1. Target transform задаёт anchor.
2. Third person camera отступает назад на distance и добавляет target offset.
3. First person использует eye height, right offset и forward offset.
4. Follow yaw/pitch меняются look input.
5. Socket Studio может временно заменить обычный follow orbit.

10 ECSRegistry

10.1 CreateEntity

1. `nextEntity` проверяется на invalid sentinel.
2. Новый id добавляется в live set.
3. Counter увеличивается.
4. Возвращается стабильный integer entity id.

Текущая схема не переиспользует id и не имеет generation. Для длительных сессий полезен entity+generation handle.

10.2 DestroyEntity

1. Entity удаляется из live set.
2. Каждый type-erased component pool получает `RemoveEntity`.
3. Компоненты всех типов исчезают независимо от того, знает ли registry их compile-time тип здесь.

Внешние Bullet и GPU ресурсы нельзя бездумно оставлять destructor компоненту, если они требуют world/context. Специализированная system cleanup должна выполняться раньше.

10.3 Add Get Has Remove

1. `AddComponent` получает или создаёт pool по `type_index`.
2. Lvalue копируется, rvalue перемещается.
3. `GetComponent` возвращает pointer или null.
4. `HasComponent` только проверяет наличие.
5. `RemoveComponent` ничего не делает при отсутствии pool.

10.4 ForEach одного типа

1. Получается pool T.
2. Отсутствующий pool завершает метод.
3. Итерация отдаёт entity и ссылку на component.

10.5 ForEach нескольких типов

1. Первый pool используется как источник entities.
2. Для каждой entity проверяются остальные components.
3. Callback вызывается только при полном наборе.

Оптимизация на будущее: выбирать самым маленьким не первый, а наименьший pool.

11 ComponentManager

11.1 Обычный компонент

1. `unordered_map<Entity,T>` хранит данные.
2. Has ищет ключ.
3. Get возвращает адрес value или null.
4. Add вставляет или заменяет component.
5. Remove стирает key.
6. begin/end позволяют systems итерироваться напрямую.

11.2 ActiveComponent специализация

1. Пустой tag не требует отдельного объекта на entity.
2. `unordered_set<Entity>` хранит только наличие.
3. Get возвращает адрес общего dummy, потому что данных у tag нет.
4. Add вставляет entity в set.

12 TransformSystem

1. Перебираются ECS transforms.
2. Entity связывается с legacy `GameObject` representation, если она существует.
3. Position, rotation и scale копируются в presentation object.
4. Система не должна копировать transform обратно в ECS в том же кадре, иначе появятся два владельца.

13 RenderSystem

1. Перебираются mesh renderer components.
2. Проверяются ActiveComponent, visibility, transform, model и material.
3. World matrix строится из transform.
4. Model привязывает buffers.
5. Material выбирает shader path.
6. Draw отправляется через shader manager/renderer.

Современный `Renderer` и `SceneManager` уже выполняют более сложные passes; старый RenderSystem следует считать простым reference path, а не вторым равноправным renderer.

14 SceneManager runtime ownership

14.1 SpawnObject

1. Создаётся entity.
2. Добавляется transform из descriptor.
3. Name и Active tags добавляются по настройкам.
4. MeshRenderer получает model и material.
5. Interaction component добавляется только при запросе.
6. RigidBody создаётся через PhysicsSystem.
7. Animated model получает AnimationComponent.
8. Entity возвращается caller.

14.2 RemoveObject

1. Проверяется, принадлежит ли entity scene.
2. Custom material освобождается, default material не удаляется.
3. Character controller и rigid body отсоединяются от Bullet.
4. Связанные presentation caches очищаются.
5. Registry уничтожает entity последним.

14.3 Clear

1. Сцена останавливает async work или отделяет его от старого generation.
2. Materials и models освобождаются один раз.
3. Physics objects удаляются до очистки registry.
4. Model cache очищается после entities.
5. Sprites и lights сбрасываются.

14.4 Render passes

1. `ShouldRenderInWorldPass` исключает first-person-only entities.
2. Opaque pass использует room visibility cache.
3. Async mesh path и обычный model path взаимоисключаются.
4. Transparent objects собираются отдельно.
5. First-person pass рисует только view models нужного owner.
6. Reflection pass может пропустить саму отражающую поверхность.

14.5 Save и Load

1. Save проходит по живым scene entities.
2. Runtime-only visuals, например NPC weapon visual, не сериализуются как самостоятельный gameplay object.
3. Компоненты записываются в JSON по версии schema.
4. Load сначала создаёт entity mapping old id -> new id.
5. Затем создаёт простые components.
6. Ссылки между entities разрешаются вторым проходом.
7. GPU/Bullet runtime handles пересоздаются, а не читаются из файла.

15 Асинхронный импорт SceneManager

1. `RequestShipImportAsync` отбрасывает пустой path, повторный импорт и отсутствие AssetLoadSystem.
2. Worker читает Assimp scene и собирает CPU descriptions.
3. Phase становится CpuReady только после полного snapshot.
4. Main thread загружает узлы небольшими порциями.
5. Panel batches создаются отдельной фазой.
6. Finalizing связывает rooms, doors, panels и membership.
7. Snapshot метода отдаёт progress UI без прямого доступа к мутирующему состоянию.

16 GameObject legacy wrapper

1. Constructor задаёт position/rotation нулём и scale единицей.
2. Initialize сохраняет non-owning model pointer и стартовую позицию.
3. `GetWorldMatrix` строит scale, затем rotation, затем translation в принятом соглашении.
4. Setters меняют локальные поля.

В ECS-сцене TransformComponent должен быть главным источником истины; GameObject полезен как legacy presentation wrapper.

17 PickingSystem

1. Camera создаёт world ray из mouse coordinates.
2. Для mesh ray переводится inverse world matrix в local space.
3. Сначала AABB быстро отбрасывает далёкий object.
4. Затем triangles дают точное попадание.
5. Local distance переводится обратно в сопоставимую world distance.
6. Lights проверяются сферами.
7. Sprites используют радиус из размера.
8. Ближайший hit выигрывает независимо от порядка перебора.
9. Equal-distance tie-breaker делает выбор стабильным.

Editor selection path в этой книге не рассматривается; сама математика picking остаётся runtime-полезной для interaction и shooting tools.

18 AnimationController

18.1 FindBinding

1. Перебирается список animation bindings.
2. Первое совпадение state возвращается pointer.
3. Отсутствие binding даёт null и позволяет fallback.

18.2 GetClipDurationSeconds

1. Duration ticks делится на ticks per second.
2. Нулевой ticks-per-second заменяется безопасным значением.
3. Нулевая duration не должна порождать деление или бесконечный loop.

18.3 Update state

1. Gameplay flags переводятся в желаемое animation state.
2. Приоритет одноразового action выше locomotion.
3. Смена state находит новый binding.
4. Clip index, time и loop policy обновляются.
5. Окончание non-loop clip возвращает locomotion/idle.
6. Controller не рассчитывает bone matrices; это делает AnimationSystem.

19 AnimationSystem

1. Перебираются entities с AnimationComponent и mesh model.
2. Невалидный model или отсутствие clips пропускается.
3. Playback time увеличивается на `deltaTime * speed`.
4. Loop clip использует modulo duration.
5. Non-loop clip clamp-ится к концу.
6. Model evaluates node hierarchy на текущем времени.
7. Для каждой bone вычисляется final matrix.
8. Matrices записываются в component palette.
9. Второй проход обновляет attachments после готовности всех owner palettes.

Если attachment выполнить в первом проходе до pose owner, визуал будет отставать на кадр.

20 TwoBoneIK

1. Root, joint и end positions задают длины двух сегментов.
2. Target distance clamp-ится между достижимым минимумом и максимумом.
3. Закон косинусов вычисляет угол локтя/колена.
4. Pole vector задаёт сторону сгиба.
5. Почти нулевой pole заменяется устойчивым перпендикуляром.
6. `RotationBetween` нормализует from и to.
7. Почти противоположные векторы требуют отдельной оси, потому что cross product почти нулевой.
8. Root и joint rotations комбинируются с исходной pose.

21 WeaponIKSystem

1. `GripMatrix` строит local transform из position и rotation профиля.
2. Attachment matrix оружия умножается на bone world matrix правой руки.
3. Позиция левой рукояти превращается в world target.
4. TwoBoneIK двигает левую руку к target.
5. Pole target удерживает локоть с правильной стороны.
6. Weight смешивает исходную animation pose и IK correction.
7. Отсутствующий bone или grip profile безопасно отключает IK.

22 MeleeActionJson и профили

1. JSON loader проверяет наличие полей до чтения.
2. Неизвестные поля игнорируются для forward compatibility.
3. Contact time clamp-ится внутри duration.
4. Animation name разрешается в clip позднее, когда model загружен.
5. Ошибка файла возвращает понятную диагностику и не стирает рабочий профиль.

23 Что разделить следующим рефакторингом

1. `ApplicationClass::Frame` разложить на `UpdateSession`, `UpdateSimulation`, `PrepareRender`, `RenderFrame` без перестановки вызовов.
2. Runtime render ownership вынести из SceneManager в Renderer, оставив SceneManager владельцем entities и serialization.
3. Legacy GameObject path пометить и постепенно удалить после миграции.
4. ECS ForEach выбирать минимальный component pool.
5. Async jobs привязать к scene generation token.
6. Animation evaluation и attachment pass закрепить разными именованными фазами.

Gameplay runtime построчный разбор

Эта глава читает gameplay-код сверху вниз. Под «строкой» здесь понимается значимая операция: проверка, чтение компонента, формула, изменение состояния или вызов другой подсистемы. Скобки, `#include` и одинаковые служебные перегрузки объединены, иначе объяснение стало бы длиннее исходника и хуже показывало бы мысль.

1 GameplayController роль класса

`GameplayController` — не ещё один игрок и не место хранения мира. Это координатор локального gameplay-кадра.

1. Конструктор сохраняет ссылки на registry, input, camera и physics services. Он не должен становиться владельцем объектов, созданных другими подсистемами.
2. `Initialize` создаёт или находит локального игрока, подготавливает input bindings и приводит runtime-флаги к известному состоянию.
3. `Update` сначала получает снимок ввода, затем обновляет сессию, движение, взгляд, взаимодействие, оружие и ремонт.
4. Порядок важен. Нажатие, полученное в этом кадре, должно попасть в системы до очистки event queue.
5. Результаты взаимодействий и урона обрабатываются после producer-систем, но до `events.Clear()`.
6. Камера получает итоговое положение уже после движения character controller, иначе изображение отстаёт на кадр.

Главная опасность: добавлять в Controller собственную копию состояния, которое уже принадлежит компоненту. Controller должен передавать данные, а не создавать второй источник истины.

2 GameplayInputSystem bindings

2.1 IsSessionAction

`IsSessionAction(GameAction action)` отделяет команды самой игровой сессии от действий персонажа.

1. `switch` сравнивает action с pause, сменой камеры и масштабом времени.
2. Возвращаемый `true` разрешает обрабатывать действие даже тогда, когда обычный gameplay-ввод заблокирован UI.
3. Остальные действия получают `false` и не должны двигать или стрелять при захваченном интерфейсом вводе.

2.2 IsBindingDown

1. Проверка `binding.device` выбирает keyboard или mouse path.
2. Для клавиатуры вызывается состояние конкретной клавиши.
3. Для мыши `switch` переводит абстрактную кнопку binding в API `InputClass`.
4. Неизвестная кнопка возвращает `false`; это безопаснее случайного срабатывания.

2.3 Конструктор

Последовательные `AddBinding` задают таблицу управления по умолчанию. Здесь нет gameplay-логики: конструктор только связывает `GameAction` с физической кнопкой. Поэтому переназначение клавиш позднее сможет заменить таблицу, не переписывая движение и оружие.

2.4 Sample

1. Метод получает `playMode` и `gameplayEnabled` отдельно. Первый говорит, запущена ли игра, второй — можно ли сейчас управлять персонажем.
2. Для каждого action вычисляется `down`.
3. `pressed` получается как текущий `down` при прошлом `false`.
4. `released` получается обратным переходом.
5. Session actions сохраняются при блокировке gameplay, остальные принудительно выключаются.
6. В конце прошлое состояние обновляется для следующего кадра.

Такой edge detection нужен, чтобы reload или pause происходили один раз, а движение продолжалось всё время удержания.

2.5 UpdateSession

1. `Pressed(ToggleView)` меняет режим камеры только на фронте нажатия.
2. `Pressed(TogglePause)` переключает `GameTimer`.
3. Slower и Faster изменяют time scale в допустимом диапазоне.
4. Session update не должен напрямую менять компоненты оружия или движения.

3 CharacterSpawnSystem

Spawn собирает сущность персонажа из ECS-компонентов и Bullet-объектов. Destroy делает обратную операцию.

3.1 Spawn порядок

1. Проверяются registry и physics world.
2. Создаётся entity и базовые компоненты: transform, active tag, player или NPC data.
3. Создаётся capsule shape. Радиус и высота берутся из настроек персонажа, а не из render mesh.
4. Ghost object получает transform и collision flags.
5. Kinematic character controller связывается с ghost object.
6. Ghost и controller добавляются в Bullet world с нужными collision masks.
7. Указатели записываются в `CharacterController` component.
8. ECS transform синхронизируется с фактической стартовой позицией Bullet.

Если операция оборвётся после пункта 4, уже созданный Bullet-объект нужно освободить. Поэтому spawn следует рассматривать как транзакцию.

3.2 Destroy

1. `GetComponent<CharacterController>` проверяет, действительно ли entity имеет физического персонажа.
2. Получается world.
3. Controller сначала удаляется из world, потому что он ссылается на ghost object.
4. Ghost object удаляется после controller.
5. Shapes и heap objects освобождаются только после detach.
6. Затем уничтожается ECS entity.

Именно такой обратный порядок предотвращает висячие ссылки Bullet.

4 CharacterMovementSystem

4.1 Поиск локального игрока

1. `INVALID_ENTITY` означает, что подходящий entity ещё не найден.
2. Перебор player tags выбирает локального управляемого персонажа.
3. Ранний `return`, если игрока нет, делает систему пригодной для NPC-only симуляции.

4.2 Направление ввода

1. Forward/back и left/right превращаются в две оси.
2. Вектор нормализуется только если длина больше единицы. Диагональ поэтому не быстрее прямого движения.
3. `cameraYaw` поворачивает локальное направление в world space.
4. `isMoving` вычисляется по длине, а не по отдельной клавише.

4.3 Ходьба бег и stamina

1. `wantsRun` отражает желание игрока, но ещё не гарантирует бег.
2. `hasEnoughStaminaToRun` проверяет ресурс.
3. `isRunning` становится true только при движении, удержании run и достаточной stamina.
4. Скорость выбирается между walk и run.
5. При нехватке stamina флаг бега снимается, но `wantsRun` сохраняет причину для UI.
6. Итоговая скорость записывается в movement component; character controller исполняет её отдельно.

Разделение желания и факта особенно полезно для NPC: AI может хотеть спринт, а stamina policy запретит его.

5 CharacterControllerSystem общий Update

Это один из самых насыщенных runtime-файлов. Его верхний `Update` не просто перемещает capsule: он связывает ECS, Bullet, automatic step, опору, толкание динамических тел и восстановление после смены настроек.

1. Невалидный или отрицательный `dt` приводится к безопасному значению.
2. Система перебирает entities с controller и transform.
3. Проверяется существование `controller`, `ghostObj` и `capsuleShape`.
4. Если параметры physics tuning изменились, `Reconfigure` перестраивает controller.
5. Movement velocity переводится из метры в секунду в displacement для текущего шага.
6. External velocity прибавляет атмосферный толчок, взрыв или другой независимый импульс.
7. Automatic step может временно заменить обычный walk direction траекторией подъёма.
8. Bullet controller получает итоговый direction.
9. После physics шага transform считывает world transform ghost object.
10. Ground support и idle lock обновляются по фактическому контакту.

5.1 Reconfigure

1. Старые controller и ghost отсоединяются от world.
2. Текущая позиция сохраняется.
3. Capsule пересоздаётся с новыми radius, height и step height.
4. Collision flags и masks восстанавливаются.
5. Новый controller возвращается в прежнюю позицию.
6. Runtime traversal state сбрасывается, потому что старая ступень могла зависеть от прежней геометрии capsule.

5.2 PushDynamicBody

1. Sweep или ray определяет объект перед персонажем.
2. Static, kinematic и нулевая inverse mass отбрасываются.
3. Surface normal нормализуется.
4. `approach` показывает, действительно ли персонаж движется в поверхность.
5. Target push speed ограничивается tuning-параметрами.
6. Из текущей скорости тела вычитается уже имеющаяся скорость вдоль толчка.
7. Оставшаяся скорость переводится в impulse с учётом массы.
8. Impulse ограничивается, body активируется и получает воздействие.

Это не «телепорт коробки», а попытка довести её скорость до целевой, поэтому контакт устойчивее.

5.3 Automatic step detection

1. Сначала проверяется, включена ли функция и есть ли горизонтальное движение.
2. Forward sweep ищет препятствие на уровне capsule.
3. Нормаль должна смотреть достаточно против движения; касание боком не считается ступенью.
4. Верхний sweep проверяет свободное место над препятствием.
5. Down sweep ищет поверхность, на которую можно поставить персонажа.
6. Высота кандидата должна попасть между minimum и maximum step height.
7. Slope normal сравнивается с допустимым углом пола.
8. Predictive режим может начать traversal до непосредственного упора.
9. При успехе создаётся runtime traversal с start, apex, end и progress.

5.4 Traversal

1. Progress увеличивается через `dt / duration`.
2. Горизонтальная часть движется к конечной точке.
3. Вертикальная часть использует сглаженную кривую, а не мгновенный скачок.
4. Проверки свободного пространства продолжаются во время движения.
5. Отсутствие прогресса увеличивает stuck timer.
6. При блокировке traversal отменяется и запускает cooldown.
7. При достижении конца state очищается, обычное управление возвращается.

6 CharacterExternalVelocitySystem

Эта система — маленький сумматор внешних воздействий.

1. `BeginAtmosphereFrame` очищает накопление прошлого кадра.
2. Producer вызывает Add для entity: suction, взрыв или scripted force складываются.
3. Ограничение длины не даёт нескольким источникам разогнать персонажа до бесконечности.
4. `ConsumeVelocity` возвращает итог и удаляет запись.
5. Отсутствующая запись даёт нулевой вектор.

Важно вызывать Begin один раз до всех producers, а Consume — один раз внутри character movement. Иначе порядок систем меняет результат.

7 CharacterFacingSystem

1. Система пропускает мёртвых и неактивных персонажей.
2. Направление берётся из движения или отдельного aim direction.
3. Почти нулевой вектор не меняет ориентацию, чтобы idle-персонаж не дрожал.
4. `atan2` переводит XZ-направление в yaw.
5. `NormalizeAngle` возвращает разницу в диапазон от минус pi до pi.
6. Максимальный поворот равен turn speed умножить на `deltaTime`.
7. Transform получает новый yaw постепенно.

Нормализация нужна для короткого пути через границу 180 градусов.

8 FirstPersonViewSystem

1. Отсутствующая camera или local player завершает метод.
2. Transform игрока даёт базовую world position.
3. Camera height добавляет положение глаз.
4. Look yaw и pitch формируют forward vector.
5. Pitch ограничивается, чтобы камера не переворачивалась.
6. Camera получает position и rotation.
7. First-person arms получают отдельный transform, не совпадающий с world body mesh.
8. `UpdateBoneAttachments` читает итоговые bone matrices после animation update.
9. `XMMatrixDecompose` извлекает scale, rotation и translation attachment bone.
10. Ошибка decomposition оставляет прошлую корректную позу вместо мусора.

Порядок кадра: animation вычисляет pose, затем attachments читают кости, затем render рисует оружие.

9 PlayerVitalsSystem

9.1 Oxygen

1. Невалидный player и неположительный `deltaTime` отбрасываются.
2. `PlayerComponent` является владельцем текущего oxygen.
3. Система ищет комнату, содержащую игрока.
4. Если комната найдена, её oxygen и pressure определяют target.
5. Вне комнаты используется внешний environment policy.
6. Если текущее значение ниже target, оно плавно восстанавливается.
7. Если выше — плавно уменьшается.
8. Итог ограничивается от нуля до максимума.
9. Нулевой oxygen запускает потерю health через damage path, а не тайное прямое удаление entity.

9.2 StaminaUpdate

1. Проверяются player и movement components.
2. Реальный `isRunning`, а не только кнопка, тратит stamina.
3. Когда игрок не пытается бежать, stamina восстанавливается.
4. Во время удержания run без возможности бега можно оставить задержку восстановления.
5. Clamp защищает диапазон от нуля до maximum.

10 WeaponSystem

10.1 GetEquippedWeaponEntity

1. Сначала читается `EquippedWeaponComponent` owner.
2. Entity проверяется на alive.
3. Затем подтверждается наличие `WeaponComponent`.
4. Любая ошибка возвращает `INVALID_ENTITY`.

Так каждая следующая операция начинает работу с валидного оружия.

10.2 RequestReload

1. Находится экипированное оружие.
2. Reload запрещается, если уже идёт, магазин полон или резерв пуст.
3. Флаг `isReloading` включается.
4. `reloadTimeRemaining` получает длительность из weapon data.
5. Патроны пока не переносятся: перенос произойдёт при завершении таймера.

10.3 GiveAmmo

1. Неположительное число сразу возвращает ноль.
2. Ищется оружие owner.
3. Вычисляется свободное место в reserve.
4. Добавляется минимум между requested и capacity.
5. Метод возвращает реально принятое количество, чтобы pickup мог оставить остаток.

10.4 UpdateTimers

1. `dt` ограничивается снизу нулём.
2. Для каждого weapon уменьшаются shot cooldown и muzzle flash timer.
3. Reload timer уменьшается только при активной перезарядке.
4. При достижении нуля вычисляется недостающее место магазина.
5. Из reserve переносится минимум между нехваткой и доступными патронами.
6. Reload flags очищаются.

10.5 TryFire

1. Проверяется weapon, cooldown, reload и ammo.
2. Пустой магазин может запустить reload request.
3. Ammo уменьшается ровно один раз после подтверждения возможности выстрела.
4. Cooldown получает задержку между выстрелами.
5. Muzzle flash state включается на короткий срок.
6. Ray начинается у muzzle, а не в центре NPC.
7. Направление получает spread, после чего нормализуется.
8. Bullet raycast игнорирует shooter и его собственную геометрию.
9. Debug ray записывается только если включён соответствующий флаг.
10. При попадании создаётся DamageEvent с source, target, point, normal и silent flag.
11. Hull panel может получить отдельный mesh damage path.
12. Particle system создаёт muzzle flash и impact effect.

Важный контракт: AI решает, можно ли стрелять; WeaponSystem ещё раз проверяет физическую возможность и выполняет один выстрел.

11 WeaponFiringRules

Этот модуль держит чистые правила, которые полезны игроку и NPC.

1. `CanFire` проверяет cooldown, reload и ammo без изменения состояния.
2. `BeginReload` меняет только reload state.
3. `FinishReload` переносит ammo арифметически и возвращает число перенесённых патронов.
4. `AdvanceTimers` уменьшает timers с clamp до нуля.
5. Spread строится в локальном базисе forward-right-up.
6. Recoil возвращает величину отдачи, но не обязан сам двигать camera.

Чистая функция легче тестируется: одинаковый вход всегда даёт одинаковый выход.

12 PlayerPunchSystem

1. В начале требуются player, animation, mesh, camera и world.
2. Наличие оружия запрещает punch path, если дизайн не допускает melee с оружием.
3. Удержание primary mouse запускает или продолжает атаку.
4. При старте выбирается weighted animation variant.
5. `punchTime` увеличивается каждый кадр.
6. До contact time система только проигрывает движение.
7. В contact window строится ray или короткий sweep из камеры.
8. Собственный body игрока игнорируется.
9. `punchHitApplied` гарантирует один урон за одну анимацию.
10. Hit point может корректировать IK кисти, чтобы контакт визуально совпал.
11. После duration flags очищаются и рука возвращается в idle.

13 DamageSystem

13.1 Очередь

1. Система читает `DamageEvent` по порядку создания.
2. INVALID target, неположительный damage и неактивная entity отбрасываются.
3. Source и hit data сохраняются для AI perception и effects.
4. Silent kill не создаёт обычный шумовой сигнал, но сам урон остаётся.

13.2 Invulnerable и типы цели

1. `InvulnerableComponent` блокирует изменение health, но отладочные hit facts могут сохраниться.
2. Player path изменяет player health.
3. NPC path изменяет NPC или общий Damageable health.
4. Hull panel path обновляет damage decals, cracks или repairable state.
5. Переход alive в dead создаёт событие смерти только один раз.
6. Repairable получает накопленный structural damage.

Для AI важно различать «попадание зарегистрировано» и «здоровье уменьшилось». На бессмертной цели тактика всё равно должна понимать, что по NPC попали.

13.3 Decals

1. Hit point переводится в локальные координаты поверхности.
2. Ищется ближайшая существующая трещина.
3. Близкая трещина усиливается вместо создания бесконечного числа элементов.
4. При достижении лимита заменяется наиболее подходящий старый элемент.
5. Normal и tangent сохраняют ориентацию decal.

14 InventorySystem

14.1 Validate owner

1. Owner должен быть alive.
2. Inventory component создаётся только там, где это разрешает вызывающая операция.
3. Любой публичный метод возвращает `InventoryResult`, а не только bool. Status объясняет причину отказа.

14.2 AddWeapon

1. Проверяется owner, weapon entity и `WeaponComponent`.
2. Если weapon уже находится в inventory, возвращается existing slot.
3. Destination может быть конкретным или `AnySlot`.
4. Конкретный slot проверяется по диапазону и занятости.
5. Для AnySlot ищется первое свободное место.
6. Entity записывается в slot.
7. Ownership components оружия обновляются.
8. При первом pickup или `equipOnPickup` активный slot переключается.
9. `SyncEquipment` приводит EquippedWeaponComponent и visibility к inventory.

14.3 Remove Drop Select Swap

1. Remove проверяет slot и сохраняет item до очистки.
2. Active slot после удаления нормализуется.
3. Drop дополнительно возвращает предмету world transform и pickup component.
4. Select запрещает пустой или несуществующий slot.
5. Swap меняет две slot records и корректирует active index.
6. После каждой структурной операции вызывается единый equipment sync.

14.4 SyncEquipment

1. Определяется active item.
2. Owner получает или очищает EquippedWeaponComponent.
3. У активного оружия выключается world pickup physics и включается attachment visibility.
4. У неактивных inventory items отключается world render.
5. Dropped item, напротив, получает world render и collision.

Без централизованного sync один и тот же предмет легко окажется одновременно в руке и на полу.

15 WeaponPickupSystem

1. Если interactions в этом кадре не обрабатываются, метод ничего не делает.
2. Проверяются local player и его alive state.
3. Из EventQueue выбираются interaction events.
4. Target должен иметь pickup и weapon components.
5. `InventorySystem::AddWeapon` выполняет реальную операцию.
6. Успех удаляет или деактивирует world pickup representation.
7. Ошибка сохраняет feedback text для HUD.
8. `ConsumeInventoryChanged` возвращает одноразовый флаг и сразу его сбрасывает.

16 EventQueue

1. `Emit(T&& event)` помещает типизированное событие в variant vector.
2. `GetEvents` отдаёт read-only список consumers.
3. `Clear` вызывается один раз после всех consumers текущего кадра.

Producer не должен очищать очередь. Consumer не должен хранить ссылку на event после Clear.

17 InteractionSystem

1. Отсутствие camera, registry или world завершает update.
2. Ray строится из camera position по forward на interaction distance.
3. Bullet callback игнорирует player body.
4. Hit object переводится обратно в ECS entity.
5. Entity без InteractionComponent и без NPC data не считается целью.
6. Disabled interaction отбрасывается.
7. Точная дистанция перепроверяется по transform, чтобы чужой collider не расширял интеракцию бесконечно.
8. Prompt выбирается из component, NPC name или default текста.
9. Hover data сохраняет entity, point, normal и тип цели.
10. Нажатие Interact создаёт событие или открывает NPC UI.
11. Debug ray рисуется только при включённой настройке.

18 TriggerZoneSystem

18.1 Сбор кандидатов

1. `deltaTime` принимается только если конечен и положителен.
2. Собираются активные trigger zones.
3. Отдельно собираются player, NPC и dynamic body actors.
4. Actor получает mask своего типа.
5. Сортировка по Entity делает результат стабильным между запусками.

18.2 Contains

1. Sphere сравнивает квадрат дистанции с квадратом radius.
2. Box переводит world point в local coordinates зоны.
3. Обратная матрица проверяется по determinant; невырожденность обязательна.
4. Local point сравнивается с half extents.

18.3 UpdateZone

1. Disabled zone очищает или корректно завершает прошлые overlaps.
2. Для каждого actor проверяется mask и contains.
3. Новое пересечение создаёт Enter event.
4. Уже активное пересечение может создать Stay event.
5. Исчезнувшее пересечение создаёт Exit event.
6. Cooldown и one-shot policy решают, можно ли активировать action.
7. Runtime map хранит overlap отдельно для каждого actor.

18.4 Reset Rearm query methods

`Reset` очищает все runtime states. `ResetZone` очищает одну зону. `RearmZone` возвращает one-shot зоне способность активироваться и при необходимости очищает cooldown. Query methods только читают состояние и не должны неожиданно менять симуляцию.

19 TriggerActionSystem

1. Система читает только trigger events из EventQueue.
2. Event type сопоставляется с разрешёнными action phases.
3. `switch(action.action)` выбирает конкретную команду: visibility, active state, door, spawn или другая операция.
4. До изменения проверяются target entity и нужный component.
5. Feedback хранит последнюю успешную или неуспешную активацию для отладки.
6. `m_activationsThisFrame` сбрасывается в начале update и увеличивается только после фактического действия.

20 DoorSystem

20.1 ProcessEvents

1. Из очереди выбираются interaction и trigger actions для дверей.
2. Locked door отклоняет toggle.
3. Целевой `openAmount` меняется между нулём и единицей.
4. События `DoorOpenedEvent` откладываются до завершения перебора, чтобы не менять очередь во время чтения.
5. После цикла новые события добавляются безопасно.

20.2 Update

1. `openAmount` движется к target с заданной speed.
2. Transform двери вычисляется из закрытой позы и оси движения.
3. Collision state обновляется при прохождении порога.
4. Связанные комнаты используют openAmount как коэффициент обмена среды.

21 Правильный порядок gameplay кадра

Рекомендуемый порядок текущих систем:

1. Input sample.
2. Session actions.
3. Character intent: movement, facing desire, weapon request.
4. Environment producers и external velocities.
5. Character controller execution.
6. Physics step.
7. Transform sync и first-person attachments.
8. Interaction и trigger detection.
9. Weapon, punch, repair producers.
10. Damage processing.
11. AI perception получает факты попаданий и звуки.
12. Pickup и trigger action consumers.
13. Vitals.
14. Particle update.
15. Event queue clear.

Если producer работает после consumer, событие будет обработано только на следующем кадре. Иногда это допустимо, но должно быть осознанно.

22 Что улучшать в gameplay runtime

1. Разделить CharacterControllerSystem на ground probe, traversal, pushing и execution helpers без изменения порядка.
2. Сделать единый `HitFact`, который существует даже при invulnerability.
3. Покрыть WeaponFiringRules unit-тестами без D3D и Bullet world.
4. Явно документировать owner каждого input flag.
5. Добавить transactional guard в CharacterSpawnSystem.
6. Убрать скрытые static maps external velocity при переходе на несколько worlds.
7. Зафиксировать EventQueue phases, чтобы producers и consumers нельзя было случайно переставить.
8. Отделить first-person presentation от gameplay оружия окончательно.

Мир энергия помещения ремонт и эффекты

Исторические названия `Ship`, `Submarine` и `Hull` в этой главе не означают, что игра снова обязана стать игрой про подлодку. Они описывают сохранившиеся runtime-модули. `FloodingSystem` намеренно исключён как рудимент. Системы давления, кислорода, помещений и энергии разобраны, потому что они по-прежнему полезны для станции, корабля, базы или герметичного здания.

1 PowerSystem простой распределитель

1. `Update` собирает активные generators и суммирует доступные мегаватты.
2. Неактивный, сломанный или остановленный generator не добавляет мощность.
3. Consumers собираются отдельно вместе с priority и requested power.
4. Перед новым распределением каждый consumer получает `powered = false` и нулевую delivered power. Это стирает результат прошлого кадра.
5. Сортировка ставит меньший номер приоритета раньше.
6. Цикл берёт первый consumer и сравнивает request с available power.
7. Полное питание отмечает consumer powered и уменьшает остаток.
8. Если частичное питание не разрешено, недостаточная мощность оставляет consumer выключенным.
9. Если частичное разрешено, delivered получает остаток, а коэффициент питания считается как delivered/requested.
10. Телеметрия сохраняет generation, demand, supplied и shortage.

Этот вариант подходит одному общему bus. Для нескольких независимых сетей используется `ShipPowerGridSystem`.

2 GeneratorSystem

2.1 Update

1. Отрицательный `deltaTime` заменяется нулём.
2. ECS-перебор вызывает `UpdateGenerator` для каждого generator component.
3. Система не распределяет электричество; она только вычисляет, сколько конкретный источник способен произвести.

2.2 MoveTowards

1. Если current меньше target, прибавляется не больше `maxDelta`.
2. Если current выше target, вычитается не больше `maxDelta`.
3. Иначе значение уже достигнуто.

Функция убирает overshoot и делает ramp симметричным.

2.3 UpdateGenerator

1. Requested output ограничивается паспортным максимумом.
2. Состояние enabled, fuel и scram определяет target output.
3. Output плавно идёт к target через ramp rate.
4. Расход fuel равен мощности умножить на время и коэффициент расхода.
5. Работа увеличивает heat, остановка охлаждает.
6. Достижение `scramTemperature` включает аварийное отключение.
7. Scrammed generator остаётся выключенным, пока heat не упадёт до `restartTemperature`.
8. Отсутствие fuel принудительно обнуляет target и фактический output постепенно.
9. Все ресурсы ограничиваются допустимыми диапазонами.

3 EmergencyBatterySystem

3.1 Discharge

1. Нулевой запрос, нулевой `deltaTime` или пустая батарея возвращает ноль.
2. Requested MW переводятся в MWh через время в часах.
3. Доступная энергия ограничивает фактическую мощность.
4. Charge уменьшается на реально отданную энергию.
5. Возвращается delivered MW, а не MWh.

3.2 Charge

1. Отрицательный surplus не заряжает батарею.
2. Свободная ёмкость вычисляется из capacity минус charge.
3. Charging efficiency уменьшает реально сохранённую энергию.
4. Charge ограничивается capacity.
5. Возвращаемая принятая мощность позволяет telemetry правильно посчитать остаток surplus.

MW — скорость передачи энергии, MWh — запас. Их нельзя складывать без умножения на время.

4 ShipPowerGridSystem

4.1 FindGridRoot

1. Если entity сама имеет `PowerGridComponent`, она и есть root.
2. Иначе `ShipPartComponent` указывает на родительскую конструкцию.
3. Невалидная связь возвращает `INVALID_ENTITY`.

4.2 Update сбор

1. Создаётся временная map `root -> WorkingGrid`.
2. Grid components регистрируют roots.
3. Каждый generator относится к root через `FindGridRoot`.
4. Каждый consumer добавляется в соответствующий рабочий grid.
5. Grid без валидного root не получает случайную энергию из другой сети.
6. После сбора `UpdateGrid` вызывается отдельно для каждой сети.

4.3 UpdateGrid

1. Generation суммируется из фактического output generators.
2. Demand суммирует запросы активных consumers.
3. При дефиците battery разряжается и добавляет available MW.
4. Consumers сортируются по priority и стабильному entity order.
5. Перед распределением старые powered flags очищаются.
6. Available power выдаётся последовательно.
7. При избытке остаток направляется в battery charge.
8. Telemetry получает generation, battery flow, demand, supplied и blackout state.

Сортировка по entity после priority нужна для воспроизводимости: одинаковая сцена не должна случайно питать разные устройства.

5 OxygenGeneratorSystem

1. Неположительный `deltaTime` завершает update.
2. Для каждого oxygen generator определяется grid root и доступность питания.
3. Disabled, broken или unpowered unit не производит кислород.
4. Output плавно зависит от power fraction.
5. Ищутся активные rooms, связанные с той же конструкцией.
6. При отсутствии rooms производство не исчезает в неизвестность: output telemetry обнуляется или остаётся как potential согласно component policy.
7. Produced oxygen делится между rooms.
8. Каждая комната ограничивает oxygen своим maximum.
9. Generator component сохраняет фактическую производительность.

6 RoomSystem

6.1 Зачем система нужна

`RoomSystem` определяет, какие динамические объекты находятся в каких объёмах. Это spatial membership, а не сама симуляция кислорода.

6.2 Synchronous path

1. Сначала у всех rooms очищаются `objectsInRoom`.
2. Перебираются trackable entities.
3. Character controller считается trackable всегда.
4. Обычный rigid body учитывается, только если он динамический.
5. World position переводится в local space каждой комнаты.
6. Point-in-box проверяет half extents.
7. Entity добавляется в первую или лучшую подходящую room согласно текущей политике.

6.3 Job path

1. `s_membershipJobPending.exchange(true)` не даёт запустить второй расчёт поверх первого.
2. Main thread создаёт snapshot rooms и entity positions. Worker не должен читать мутирующий ECS напрямую.
3. Пустой список комнат завершает задачу заранее.
4. Job вычисляет membership только на snapshot data.
5. Результат передаётся в main-thread queue.
6. Main thread ещё раз проверяет, что entities существуют, и применяет списки.
7. Pending flag снимается после применения или при ошибке submit.

Ссылка `&registry`, захваченная worker-задачей, безопасна только пока registry гарантированно жив дольше job. Лучше передавать в worker исключительно snapshot, а registry касаться лишь в main-thread callback.

7 RoomEnvironmentSystem

7.1 Update комнат

1. Нулевое время ничего не меняет.
2. Значения pressure и oxygen сначала ограничиваются физически допустимым диапазоном.
3. `ApplyBreaches` вычисляет утечки.
4. `EqualizeConnectedRooms` переносит среду через двери.
5. После transfer снова выполняется clamp, чтобы ошибки накопления не дали отрицательную атмосферу.

7.2 ApplyBreaches

1. Перебираются незапечатанные breach sources.
2. Source связывается с комнатой через membership или owner data.
3. Утечка зависит от размера отверстия, разницы давления и `deltaTime`.
4. Pressure и oxygen уменьшаются пропорционально, но не ниже нуля.
5. Suction включается только при ненулевых radius, strength и room pressure.
6. Для каждого объекта комнаты вычисляется направление к breach.
7. За пределами radius объект пропускается.
8. Falloff уменьшает воздействие с расстоянием.
9. Dynamic rigid body получает force или impulse.
10. Character не толкается как rigid body: скорость записывается через `CharacterExternalVelocitySystem`.

7.3 EqualizeConnectedRooms

1. Сначала строится список transfer, не изменяющий rooms на месте.
2. Для каждой пары связанных rooms читается door flow multiplier.
3. Разница давления определяет направление.
4. `LimitedFlow` не разрешает перенести больше половины разницы за один шаг.
5. Oxygen переносится вместе с долей атмосферы источника.
6. После сбора transfer применяются ко всем rooms.

Двухфазная схема исключает зависимость результата от порядка перебора дверей.

7.4 GetDoorFlowMultiplier

1. Система ищет дверь, соединяющую пару rooms в любом порядке.
2. Закрытая дверь даёт почти нулевой flow.
3. Открытая — полный или заданный коэффициент.
4. Частично открытая интерполирует поток через `openAmount`.

8 PressureSystem без затопления

Этот файл исторически принимает `waterLevel`, но рассматривается только как пример внешнего давления на корпус. Сам `FloodingSystem` в книгу не включён.

1. Система перебирает hull components.
2. Invulnerable entity пропускается.
3. Требуется rigid body и валидный Bullet body.
4. Depth вычисляется относительно уровня среды.
5. Объект выше уровня не получает внешнего гидростатического давления.
6. Pressure растёт с depth по заложенной формуле.
7. Если pressure выше `maxPressure`, создаётся damage или failure state.

Для космической игры этот модуль лучше обобщить до `ExternalPressureSystem`: входом будет давление снаружи, а не water level.

9 RoomCullingSystem

9.1 FindRoomContainingPoint

1. Перебираются active rooms.
2. World point переводится обратной матрицей transform комнаты.
3. Local coordinates сравниваются с bounds.
4. Возвращается room entity или INVALID.

9.2 BuildVisibleRooms

1. Начинаем с комнаты камеры.
2. Room добавляется в output только один раз.
3. Через открытые двери обходятся соседние rooms.
4. Depth или количество переходов ограничивается, чтобы граф с циклом не зациклил поиск.
5. Полностью закрытая дверь останавливает видимость.

9.3 Apply visibility

1. Пустой visible list означает fallback policy, а не обязательное скрытие всего мира.
2. Doors видимы, если относятся хотя бы к одной видимой комнате.
3. Специальные panel batches обрабатываются отдельно.
4. Для обычной entity ищется cached room.
5. Устаревший cache перепроверяется по transform.
6. Mesh renderer получает visible flag.
7. Debug stats считают tested, visible и culled entities.

Room culling не заменяет frustum culling. Сначала можно убрать невидимые комнаты, затем камерой отсечь объекты внутри видимых.

10 ShipAlertSystem

1. Каждый update начинает с нормального состояния.
2. Определяется комната игрока.
3. Низкие oxygen или pressure повышают уровень тревоги.
4. Breach в комнате формирует более конкретное сообщение.
5. Через ship root читается power grid.
6. Blackout или severe shortage повышает уровень.
7. `SetAlert` заменяет текущую тревогу только более серьёзной.
8. `GetCurrentAlert` отдаёт готовый snapshot HUD и звуку.

Это aggregator: он не чинит генератор и не наносит урон, а только превращает факты разных систем в одну понятную диагностику.

11 SubmarineJointSystem исторический модуль соединений

1. Система читает joint component и две связанные entities.
2. Невалидные или удалённые bodies приводят к безопасному удалению constraint.
3. При отсутствии runtime constraint он создаётся по anchor и axes.
4. Constraint добавляется в Bullet world после обоих bodies.
5. Нагрузка или относительное смещение сравниваются с break threshold.
6. При разрушении constraint сначала удаляется из world, затем освобождается.
7. Component получает broken state для gameplay и ремонта.

Название можно позже заменить на `StructuralJointSystem`, не меняя механику.

12 JointConnectionSystem

1. Система разрешает component-level ссылки на тела.
2. Локальные anchors переводятся в transforms Bullet.
3. Тип joint выбирает fixed, hinge или другой constraint.
4. Limits применяются после создания.
5. Existing constraint обновляется или пересоздаётся, когда configuration revision меняется.
6. Удаление entity вызывает detach constraint до body destruction.

13 RepairSystem Update

13.1 Начало кадра

1. Feedback и текущая progress display приводятся к состоянию этого кадра.
2. Неположительный `deltaTime` не накапливает ремонт.
3. Требуются camera и physics world.
4. Ищется local player и controller.
5. Если кнопка repair не удерживается, активная попытка сбрасывает progress согласно policy.

13.2 Поиск цели

1. Ray идёт из камеры на repair distance.
2. Bullet hit даёт ближайший collider entity.
3. Дополнительно проверяются runtime mesh damage marks, потому что одна entity может иметь несколько повреждений.
4. Кандидат должен иметь RepairableComponent и быть не repaired.
5. Distance и angle сравниваются с допустимыми значениями.
6. Лучший кандидат выбирается по ray distance, а не по порядку ECS.

13.3 Накопление

1. Смена target сбрасывает или переносит progress по явному правилу.
2. Repair rate может зависеть от инструмента.
3. Progress увеличивается на rate умножить `deltaTime`.
4. При достижении required time вызывается CompleteRepair.
5. HUD получает target, progress и причину блокировки.

13.4 CompleteRepair

1. Repairable отмечается repaired.
2. Breach source получает sealed state.
3. Damage decal или runtime hole удаляется через специализированный helper.
4. Structural health восстанавливается в пределах настройки.
5. Событие ремонта отправляется другим системам.
6. Active target и progress очищаются.

13.5 Runtime mesh repair

1. Damage record ищется по id.
2. Async cut/repair job не должен одновременно менять тот же mesh.
3. CPU geometry восстанавливается или перестраивается.
4. GPU buffers заменяются на main thread.
5. Старые buffers освобождаются после успешной замены.

14 ParticleSystem lifecycle

14.1 Initialize и renderer

1. Конструктор задаёт deterministic random seed и назначает active system.
2. `SetDevice` не создаёт renderer повторно.
3. Первая отрисовка лениво создаёт `ParticleRenderer`.
4. Флаг initialization attempted предотвращает бесконечные неудачные попытки каждый кадр.
5. Destructor снимает active pointer, только если он указывает на этот instance.

14.2 Emitters

1. `CreateEmitter` проверяет `MAX_EMITTERS`.
2. Новый id не может совпасть с invalid sentinel.
3. Descriptor копируется в runtime emitter.
4. Некоторые effect types получают готовые presets.
5. SetPosition, SetActive и UpdateEmitter сначала ищут id.
6. DestroyEmitter удаляет именно runtime emitter, но уже выпущенные частицы могут дожить.
7. `EmitBurst` создаёт заданное количество сразу без постоянного emitter.

14.3 Muzzle explosion impact presets

1. `EmitMuzzleFlash` нормализует direction и создаёт короткие яркие частицы.
2. Smoke получает больший lifetime и меньшую скорость.
3. `EmitExplosion` комбинирует flash, sparks, smoke и debris presets.
4. Metal impact использует hit normal для полусферы разлёта.
5. Hull dent и crack создают отдельные визуальные записи.
6. Breach может запустить постоянный emitter.

14.4 Update particles

1. Неположительный `deltaTime` пропускается.
2. Active emitter накапливает fractional spawn count.
3. Целая часть accumulator создаёт particles, дробная остаётся на следующий кадр.
4. Age каждой частицы увеличивается.
5. Position интегрируется через velocity умножить на time.
6. Gravity изменяет velocity.
7. Size и color интерполируются по нормализованному age.
8. Мёртвые particles удаляются компактным проходом.
9. Debug trails рисуются только по флагу.
10. Crack lifetimes обновляются отдельно.
11. `RebuildRenderItems` готовит presentation data после simulation.

14.5 SpawnParticle

1. Global maximum защищает память и draw cost.
2. Random unit vector формирует variation.
3. Cone смешивает forward с random vector.
4. Нулевое направление заменяется безопасным up.
5. Lifetime, speed, size и rotation выбираются в диапазонах descriptor.
6. Texture shape сохраняет path и atlas parameters.
7. Random start frame выбирается только при atlas frame count больше одного.

15 ParticleRenderer

1. Render items разделяются по texture и blend mode.
2. Для каждой particle строится camera-facing quad.
3. Right и up камеры масштабируются размером частицы.
4. Dynamic vertex buffer обновляется пачкой.
5. Alpha и additive states включаются явно для соответствующего batch.
6. Texture cache не загружает один файл каждый кадр.
7. После particles render state восстанавливается для следующего pass.

16 ExplosionSystem Emit

1. Radius ограничивается минимумом, impulse — нулём снизу.
2. Счётчик affected bodies начинается с нуля.
3. Visual explosion отправляется ParticleSystem независимо от физики.
4. Перебираются rigid bodies.
5. Null, static, kinematic и нулевая inverse mass отбрасываются.
6. Direction строится от центра взрыва к center of mass тела.
7. Тела дальше radius пропускаются.
8. Нулевая дистанция получает направление вверх.
9. `upwardBias` поднимает траекторию.
10. Falloff равен единице минус нормализованная дистанция.
11. Квадрат falloff делает край взрыва мягче.
12. Body активируется и получает central impulse.
13. Cross product создаёт torque axis для небольшого вращения.
14. Счётчик увеличивается только после фактического импульса.
15. Метод возвращает число затронутых тел для теста и debug UI.

17 HUD классы

`GameHud`, `GeneratorHud` и `OxygenGeneratorHud` должны только читать snapshots и рисовать.

1. UI получает player vitals или component telemetry.
2. Значения clamp-ятся для progress bars.
3. Цвет выбирается по thresholds.
4. Кнопка управления создаёт command/event, но не должна дублировать формулы GeneratorSystem.
5. Отсутствующий component показывает понятное состояние, а не разыменовывает null.

18 Что сохранить и что переименовать

Сохранить стоит room membership, atmosphere equalization, decompression, power grid, battery, repair, particles и alerts. Они не зависят от подлодки как жанра.

Переименовать со временем можно:

1. `ShipPowerGridSystem` в `PowerNetworkSystem`.
2. `SubmarineJointSystem` в `StructuralJointSystem`.
3. `HullPanelComponent` в более общий `DamageablePanelComponent`, если панели встречаются не только на корпусе.
4. `PressureSystem` в `ExternalPressureSystem` с нейтральным входом environment pressure.

Переименование следует делать отдельной механической миграцией, не одновременно с изменением поведения.

19 SubmarineSystem и WorldArea исторические helpers

`SubmarineSystem` оставлен в книге, потому что пользователь исключил затопление, а не весь сохранившийся structural runtime. Название историческое, поведение можно читать как составную конструкцию из отсеков.

19.1 AddCompartmentToSubmarine

1. Root обязан иметь TransformComponent и SubmarineComponent.
2. Ошибка возвращает INVALID_ENTITY до создания частичной сущности.
3. Создаётся entity отсека.
4. Transform получает позицию root; local offset хранится в geometry компонента.
5. Local min/max строятся как половины размера со знаками.
6. `localCenter` получает offset.
7. Volume пересчитывается, текущий размер кешируется.
8. Child SubmarineComponent ссылается на root.
9. Active tag делает entity участником scene/runtime.
10. Entity добавляется в `allCompartments` root.
11. `needsPhysicsUpdate` откладывает дорогой rebuild до update.

Параметр `compartmentMass` сейчас не используется — это явный долг API, а не скрытая физика.

19.2 RecalculateSubmarineMass

1. Требуются root rigid body, buoyancy и group component.
2. Volume суммируется по существующим compartments.
3. Историческая константа корпуса добавляет фиксированный объём.
4. Neutral mass вычисляется как density умножить на volume.
5. Коэффициент 0.95 делает массу чуть меньше нейтральной.
6. RigidBody mass и Buoyancy volume обновляются вместе.
7. Итоги кешируются в group component.

Для общего structural assembly формулу плавучести следует вынести из группировки, иначе космический корабль продолжает зависеть от плотности воды.

19.3 RefreshSubmarinePhysics

1. Старые linear и angular velocities сохраняются.
2. Body удаляется из Bullet world.
3. Motion state и body освобождаются.
4. Масса пересчитывается.
5. PhysicsSystem создаёт новый body.
6. Сохранённые velocities возвращаются, чтобы rebuild не останавливал объект.

19.4 NeedsPhysicsUpdate и UpdateSubmarine

1. Явный dirty flag даёт немедленный true.
2. Изменение количества compartments тоже требует rebuild.
3. Каждый compartment сообщает изменение размера относительно cache.
4. Update при false сразу выходит.
5. После refresh dirty flag снимается.
6. Count и размеры кешируются только после успешного обновления.

19.5 WorldAreaComponent

World area — data-only классификация области. Type сообщает смысл зоны, bounds/transform задают объём, enabled разрешает участие. Systems должны читать этот компонент как описание и не прятать внутри него pointers на Bullet или renderer resources.

Шейдеры с нуля: от одного треугольника до материала DX Framework

Шейдер — небольшая программа, которую GPU выполняет для элементов рисования. Начинающему полезно представить фабрику: CPU подготавливает геометрию, матрицы и ресурсы; vertex shader получает одну вершину и сообщает, где она окажется на экране и какие данные передать дальше; rasterizer определяет покрытые треугольником экранные фрагменты и интерполирует данные; pixel shader рассчитывает выходной цвет. Это не отдельный мир, который сам знает про Entity, здоровье NPC или двери. Он видит только переданные числа, buffers и textures. Официальное описание стадий и семантик есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources) и справочнике HLSL semantics (https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl-semantics).

1. Один треугольник до первого HLSL слова

Пусть в локальной плоскости есть вершины A=`(-1;-1;0)`, B=`(1;-1;0)`, C=`(0;1;0)`. Vertex buffer содержит их координаты, возможно цвет и UV; index buffer говорит, что треугольник составлен из индексов 0,1,2. Input layout сообщает D3D, в каких байтах каждой вершины находятся `POSITION`, `COLOR`, `TEXCOORD`. Ошибка смещения в layout означает, что HLSL прочтёт цвет как позицию или мусор, даже если текст шейдера безупречен. Для первого опыта используй один треугольник без освещения и текстуры: меньше стадий могут сломаться одновременно.

После vertex shader вершины проходят через world, view, projection. Для принятой в проекте конвенции с вектором-строкой: `clip = local * world * view * projection`. `world` ставит объект в мир, `view` описывает мир относительно камеры, `projection` создаёт перспективу. Результат содержит четыре координаты `(x;y;z;w)`. После деления на w получается нормализованная экранная позиция; если w некорректен или вершина вне видимого объёма, часть геометрии отсечётся. Не путай переменную `SV_POSITION` в pixel shader с мировой позицией: там уже экранная информация. Если нужен свет в мировом пространстве, передай отдельный `worldPos` и нормаль из VS.

2. Читаем простейший vertex shader почти по строкам

Файл `src/Render/Shader/Vs/color.vs` объявляет `cbuffer MatrixBuffer : register(b0)` с тремя матрицами. `b0` — слот буфера констант vertex stage, который CPU должен заполнить и привязать. `VertexInputType` получает `position : POSITION` и `color : COLOR`; `PixelInputType` отдаёт `position : SV_POSITION` и цвет. В `ColorVertexShader` сначала `input.position.w=1.0f`: это точка, на которую должен действовать перенос. Затем идут три `mul` в порядке world, view, projection. Последняя строка `output.color=input.color` переносит значение к следующей стадии. Если убрать эту запись, pixel shader, который ожидает COLOR, получит неопределённые для твоего алгоритма данные; если текущий pixel shader их не читает, картинка может случайно остаться прежней.

Минимальная учебная версия того же смысла:

```
cbuffer MatrixBuffer : register(b0) {
    float4x4 worldMatrix;
    float4x4 viewMatrix;
    float4x4 projectionMatrix;
};
struct VSIn { float4 position : POSITION; float4 color : COLOR; };
struct VSOut { float4 position : SV_POSITION; float4 color : COLOR; };
VSOut MainVS(VSIn input) {
    VSOut output;
    input.position.w = 1.0f;
    output.position = mul(input.position, worldMatrix);
    output.position = mul(output.position, viewMatrix);
    output.position = mul(output.position, projectionMatrix);
    output.color = input.color;
    return output;
}
```

Это учебный фрагмент для объяснения. Реальные entry point и buffer layout бери из файла проекта и соответствующего C++ класса. На CPU `ColorShaderClass::SetShaderParameters` в `src/Render/Shader/colorshaderclass.cpp` получает матрицы, делает `Map` dynamic buffer, записывает значения, `Unmap` и привязывает буфер через `VSSetConstantBuffers`. `InitializeShader` компилирует VS и PS через `D3DCompileFromFile`, создаёт shader objects и input layout. Если текст HLSL правильный, но CPU не привязал `b0`, шейдер не получит нужные матрицы. Если размер/упаковка C++ структуры не совпали с HLSL, одна из матриц или параметров будет считана неверно. Сначала сравни поля и их порядок, затем слоты и стадию привязки.

3. Что делает rasterizer между VS и PS

Vertex shader выполняется для вершин, но экран состоит из пикселей. Rasterizer превращает треугольник в покрытые им фрагменты. Для внутренней точки он вычисляет веса относительно трёх вершин. Если веса равны `(0.2;0.3;0.5)` и вершинные цвета красный `(1;0;0)`, зелёный `(0;1;0)`, синий `(0;0;1)`, интерполированный цвет равен `(0.2;0.3;0.5)`. Веса суммируются до 1. Для точек внутри треугольника они неотрицательны; если один вес отрицателен, точка лежит вне него в этой простой геометрической проверке. Для UV применяется схожая идея, но при перспективе GPU учитывает глубину, чтобы текстура не выглядела растянутой неправильно.

Rasterizer также учитывает лицевую сторону треугольника, режим culling, viewport и depth. Если порядок индексов поменять с (0,1,2) на (0,2,1), треугольник может стать обратной стороной и исчезнуть при back-face culling. Если он за другой поверхностью, depth test не позволит ему перекрасить ближний пиксель. Поэтому «pixel shader не работает» сначала проверяют одноцветным материалом, culling и depth, затем текстуру и свет. В документации Microsoft описаны pipeline stages и rasterization (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources); конкретные states выбирает renderer твоего движка.

4. Простейший pixel shader и ловушка текущего файла

`src/Render/Shader/Ps/color.ps` объявляет `ColorBuffer : register(b0)` и `ColorPixelShader(...):SV_TARGET`. `SV_TARGET` означает цвет, записываемый в текущий render target. Хотя структура входа содержит `color : COLOR`, текущая реализация возвращает `uniformColor`, полученный из constant buffer. Следовательно, разноцветные вершины из `color.vs` **не создадут градиент** с этим pixel shader. Это важный урок чтения пары VS/PS: надо проследить, что PS действительно использует из вывода VS. Для учебного эксперимента можно заменить возврат на `input.color`, но тогда потребуется согласовать C++

параметры и ожидаемое назначение `ColorShaderClass`; это изменение поведения проекта, а не описание текущего результата.

`register(b0)` в VS и `register(b0)` в PS не означают один общий буфер: это слоты двух разных стадий. C++ привязывает matrix buffer к VS и color buffer к PS. Если ошибочно привязать оба через `VSSetConstantBuffers`, PS не увидит цвет. Шейдер ничего не «ищет» по имени C++ переменной — совпадают слот, стадия и байтовая форма данных.

5. Текстура: UV, sampler и прозрачный вырез

В `src/Render/Shader/Vs/texture.vs` к позиции добавляется `float2 tex : TEXCOORD0`. Vertex shader вычисляет clip position, затем копирует UV в выход. `texture.ps` объявляет `Texture2D shaderTexture : register(t0)` и `SamplerState SampleType : register(s0)`. Выражение `shaderTexture.Sample(SampleType,input.tex)` возвращает цвет текстуры в интерполированной UV точке. Texture resource и sampler — разные вещи: первый хранит данные, второй задаёт способ выборки за границами, фильтрацию и другие правила. `TextureShaderClass::SetShaderParameters` передаёт texture SRV в pixel stage через `PSSetShaderResources`; без этой привязки `t0` не содержит нужную картинку.

Текущий `texture.ps` выполняет `clip(textureColor.a-0.01f)`. Если alpha меньше 0.01, выражение отрицательно и фрагмент отбрасывается. Это пороговый вырез, полезный для решётки или листвы: там нет полупрозрачного смешения у края. Если нужен дым или стекло с alpha 0.5, одного `clip` недостаточно; нужны соответствующие blend и depth правила. Пиксель с RGBA `(0.8;0.2;0.1;0.005)` отбрасывается, с alpha `0.02` остаётся. Не путай alpha cutout с «прозрачностью всего объекта».

Для проверки UV создай квадрат из двух треугольников и текстуру 2×2: красный, зелёный, синий и белый углы. Если цвета переставлены, проверь U/V и ориентацию изображения. Если виден один размазанный цвет, проверь UV в vertex buffer, input layout, передачу TEXCOORD из VS, sampler и привязку t0. Последовательная проверка каждого интерфейса быстрее случайной замены файла изображения.

6. Нормаль, Lambert и первая формула освещения

Нормаль N — единичное направление, перпендикулярное поверхности. Направление L от поверхности к источнику тоже нормируем. Простой диффузный свет пропорционален `max(dot(N,L),0)`. Если N и L совпадают, dot=1 и поверхность освещена сильнее. Если угол 60°, dot=cos60°=0.5. Если свет за поверхностью, dot отрицателен, `max` даёт 0. Зачем нормировать? При N длиной 2 и L длиной 3 dot может быть 6, хотя косинус никогда не больше 1. Тогда яркость ошибочно зависит от длины вектора.

Нормаль не является точкой: перенос world matrix не должен на неё влиять. При неравномерном масштабе простое умножение на world 3×3 может испортить перпендикулярность; нужна матрица, обратная транспонированной линейной части, затем нормировка. В проекте есть `normal_transform.hlsli`; сравни его с `pbr.vs` при диагностике растянутого меша. Проверка на кубе с scale `(2;1;1)`: выведи normal как RGB `(N*0.5+0.5)` и вращай камеру. Если цвет меняется от камеры, нормаль, вероятно, оказалась в другом пространстве, чем ожидалось. Если нормаль верна, переходи к light direction и shadow map.

7. От обычного света к PBR без заклинаний

`pbr.vs` и `pbr.ps` уже сложнее учебной пары. Вершинный этап может учитывать кости персонажа, преобразует позицию world/view/projection и передаёт сведения для освещения. Пиксельный этап

читает material textures и вычисляет цвет на основе normal, направления света и камеры, roughness и metallic. Сначала покажи на debug material каждую входную величину отдельно: albedo; normal; roughness; metallic; shadow factor. Затем объединяй их. Если итоговая картинка чёрная, один из входов может быть неверным; общая формула не объяснит, какой.

Albedo обычно представляет цвет и проходит через sRGB преобразование при чтении/выводе; normal, roughness и metallic — числовые данные, которые нельзя бездумно декодировать как цвет. Roughness влияет на ширину блика, metallic меняет баланс отражения и диффузной составляющей. PBR микрофасеточная модель использует распределение микронормалей, маскирование и Fresnel. Для новичка сначала достаточно понять энергетический вопрос: сколько света приходит, в какую сторону отражается и почему отражение меняется с углом взгляда. Параметры материала — входы модели, не независимые «эффекты красоты».

Тень обычно начинается отдельным проходом. `ShadowMap` хранит глубину от источника; основной pixel shader сравнивает текущую глубину с записанной и ослабляет прямой свет, если на пути была другая поверхность. Если тень оторвана от объекта, проверь bias, матрицу света и единицы глубины. Если поверхность исчезает лишь в теневом проходе, проверь culling и skinning обоих проходов. Световое PBR и теневая карта должны смотреть на одну и ту же геометрию текущего кадра.

8. Как добавить свой маленький шейдерный эффект в движок

Поставь конкретную задачу: аварийная панель плавно мигает, когда уровень опасности комнаты больше нуля. Gameplay система вычисляет `severity` в [0,1]. C++ передаёт её и время визуального эффекта в pixel constant buffer; PS рассчитывает `pulse=0.5+0.5*sin(time*frequency)` и смешивает базовый цвет с красным по `severity*pulse`. При `severity=0` панель остаётся исходной; при `severity=1` и `pulse=1` становится красной. Время можно брать из render clock, если мигание декоративное. Причина тревоги живёт в ECS, чтобы AI, сохранение и HUD видели то же состояние без GPU.

Сделай этот эффект по шагам. Сначала верни постоянный красный `float4(1,0,0,1)` и убедись, что нужный объект получает новый PS. Потом передай один `severity` через buffer, проверь значения 0, 0.5, 1. Затем добавь time и синус. После этого проверь обновление constant buffer (`Map`, запись, `Unmap`, `PSSetConstantBuffers`) и отсутствие конфликта слотов с другим материалом. Наконец проверь, что renderer сбрасывает ресурсы между draw calls: иначе следующая обычная панель может наследовать аварийный buffer.

Выполни четыре диагностические сцены: один треугольник с uniformColor; треугольник с vertex colors после явного изменения PS; квадрат с текстурой 2×2; куб с normal visualizer и одним источником Lambert. Каждый шаг добавляет только одну неизвестную. Когда они работают, читай `PBRShaderClass.cpp` как перечень CPU→GPU договоров: какие constant buffers созданы, в какие слоты привязаны, какие SRV поступают в PS, какой input layout ожидается и когда вызывается DrawIndexed. Эта цепь делает сложный шейдер проверяемым.

9. Буфер констант — точное соглашение о байтах

HLSL `cbuffer` не является C++ объектом, который GPU «находит» по имени. Это область байтов с правилами упаковки и привязкой к стадии и номеру регистра. Если в HLSL есть `float4 color; float severity;`, на стороне CPU нельзя без проверки положить подряд только 20 байт и предположить, что следующий buffer начнётся сразу после `severity`. Для D3D11 размер constant buffer должен соответствовать требованиям выравнивания, а поля внутри упаковываются по правилам HLSL. Проще начинать с группы из `float4` и матриц, сверять `sizeof` C++ структуры и отладочным шейдером выводить

каждое поле как цвет. Официальное объяснение constant buffers и их привязки есть в документации Microsoft (https://learn.microsoft.com/en-us/windows/win32/direct3dgetstarted/work-with-shaders-and-shader-resources).

Путь обновления выглядит так: `Map(buffer,WRITE_DISCARD)` даёт CPU доступ к новой памяти, код копирует структуру, `Unmap` завершает запись, `VSSetConstantBuffers` или `PSSetConstantBuffers` привязывает buffer к нужному stage. Для карты, которая меняется каждый кадр, такой путь должен соответствовать флагам её создания. Если `Map` не удался, не вызывай Draw со случайным старым содержимым, если игра ожидает новую матрицу. Если изменился HLSL layout, но C++ структура осталась прежней, ошибка может выглядеть как «всё тёмное» или «свет прыгает», хотя компиляция обеих сторон успешна: C++ компилятор не проверяет контракт с HLSL автоматически.

Матрицы требуют ещё одной осторожности. Транспонирование перед записью в shader buffer связано с соглашением размещения и выбранной формой `mul`, а не с желанием поменять геометрический порядок. В конкретном `PBRShaderClass.cpp` CPU использует `XMMatrixTranspose`; в `pbr.vs` HLSL умножает позицию слева на world, затем view, projection. При отладке запиши одну тестовую точку на бумаге: локальная `(1;0;0;1)`, перенос по X на 5, камера без поворота. После world ожидается X=6. Если шейдер выдаёт другое число, сначала проверь upload и соглашение вектора/матрицы, а не подбирай camera FOV.

10. Какие ещё бывают шейдеры и нужны ли они здесь

В этом курсе подробно рассматриваются vertex и pixel shader, потому что они составляют основной путь видимого mesh в текущем DirectX 11 рендере. В графическом pipeline могут быть и другие стадии: geometry shader работает с примитивами, hull/domain используются для tessellation, compute shader запускает общее параллельное вычисление вне обычного draw треугольника. Наличие этих стадий в API не означает, что проект уже применяет их в главном рендере. Добавляй стадию только под конкретную задачу с измерением стоимости и данных. Для обычной подсветки аварийной панели новый compute pass не нужен: достаточно параметра pixel shader.

Один и тот же pixel shader может запускаться для очень многих фрагментов. Если экран 1920×1080, это около 2.07 млн пикселей до учёта перекрытия объектов и нескольких render passes. Дополнительная операция на каждом пикселе может стоить гораздо больше одной операции на каждом из нескольких тысяч объектов CPU. Но нельзя оценивать производительность только числом строк HLSL: выборки текстур, ветвления, память, overdraw и кэш GPU имеют разные цены. Измеряй GPU время конкретного прохода, а при диагностике сначала сделай одноцветную версию, затем по одному возвращай normal map, shadow, IBL и эффекты повреждения.

11. Цвет, яркость и тональная коррекция

Цвет текстуры, число света и цвет монитора относятся к разным пространствам значений. Большинство привычных цветных изображений предназначены для просмотра в sRGB; расчёт освещения удобнее делать в линейном пространстве, где удвоение числа примерно соответствует удвоению светового вклада. Если просто сложить два закодированных sRGB цвета и ограничить каждый канал единицей, результат может быть неверно ярким или тёмным. Карты roughness, normal и metallic хранят данные; применять к ним такое же цветовое преобразование нельзя. В отладке отдельно показывай сырое значение sample и значение после необходимого декодирования.

HDR позволяет временно хранить значения яркости больше 1.0. Затем tone mapping переводит диапазон сцены к диапазону экрана, а output conversion готовит цвет для отображения. Если

светильник в HDR buffer даёт `(4;2;1)`, простой clamp сделал бы `(1;1;1)` и потерял цвет яркого источника; tone mapping может сохранить различие каналов. Но выбранная кривая — художественное и техническое решение, а не физический закон для всей игры. Не пытайся исправить отсутствующую normal map увеличением exposure: сначала отладь входы материала.

12. Задачи для самостоятельной проверки шейдеров

Задача A. VS получил локальную точку `(2;0;0;1)`, world переносит по X на 3, view и projection условно единичные. Какой X уйдёт из VS? Ответ: 5. Если `w=0`, перенос не должен подействовать; это уже направление, а не точка. Найди в `color.vs` строку, которая принудительно ставит `w=1`.

Задача B. В вершинах треугольника цвета `(1;0;0)`, `(0;1;0)`, `(0;0;1)` и веса внутренней точки `(0.25;0.25;0.5)`. Интерполированный цвет `(0.25;0.25;0.5)`. Почему текущая пара `color.vs` + `color.ps` не покажет этот цвет? Pixel shader возвращает `uniformColor`, а не `input.color`.

Задача C. Normal N=`(0;1;0)`, направление света L=`(0.6;0.8;0)`, оба единичные. `dot=0.8`, Lambert diffuse factor 0.8. Если L=`(0;-1;0)`, фактор `max(-1,0)=0`. Проверь, что N и L находятся в одном пространстве: world normal и view light direction нельзя складывать без преобразования.

Задача D. Texture sample вернул alpha 0.009. `texture.ps` считает `clip(0.009-0.01)=clip(-0.001)`, поэтому фрагмент отбрасывается. При 0.011 выражение положительно, фрагмент остаётся. Почему это не равнозначно полупрозрачности 1%? Потому что `clip` принимает двоичное решение оставить/отбросить, а смешение с фоном выполняется другой стадией при подходящем blend state.

Задача E. Добавь свой параметр `severity` в учебный pixel shader. Сначала выведи `float4(severity,severity,severity,1)` для значений 0, 0.5, 1. Если экран остаётся одинаковым, проверь C++ buffer, `PSSetConstantBuffers`, номер `register(b#)` и то, что Draw действительно использует этот PS. Только после прохода такого простого теста добавляй цветовое смешение и синусоидальный pulse.

Графический конвейер и HLSL от данных до пикселя

Шейдер не рисует модель в одиночку. Он является одной программируемой стадией большого контракта между CPU, Direct3D, видеодрайвером и GPU. Чтобы уверенно писать HLSL, надо одновременно понимать форму вершины в C++, input layout, регистры ресурсов, пространства координат и математический смысл результата.

1 Что происходит при отрисовке одного меша

Упрощённая последовательность Direct3D 11:

1. CPU выбирает меш, материал, шейдер и состояние конвейера.
2. В Input Assembler привязываются vertex buffer, index buffer и input layout.
3. Vertex Shader вызывается для каждой используемой вершины.
4. Rasterizer собирает треугольники, отсекает невидимые стороны и определяет покрытые пиксели.
5. Значения между вершинами интерполируются.
6. Pixel Shader вычисляет цвет каждого фрагмента.
7. Output Merger применяет depth test, stencil и blending и пишет итог в render target.

На GPU одновременно обрабатывается множество вершин и пикселей. У шейдера нет привычного доступа к Registry, файлам или соседнему пикселю. Входные данные должны быть заранее привязаны CPU-кодом.

2 Input Assembler и соглашение о вершине

Текущий `AssimpModelClass::VertexType` хранит:

- позицию;
- UV;
- нормаль;
- tangent и binormal;
- четыре индекса костей;
- четыре веса.

HLSL-структура должна описывать тот же поток семантиками:

```
struct VertexInputType
{
    float3 position  : POSITION;
    float2 tex       : TEXCOORD0;
    float3 normal    : NORMAL;
    float3 tangent   : TANGENT;
    float3 binormal  : BINORMAL;
    int4   boneIDs   : BLENDINDICES;
    float4 weights   : BLENDWEIGHT;
};
```

Семантика — имя канала, а не имя C++ поля. Input layout говорит Direct3D, в каком формате, смещении и слоте лежит каждая семантика. Если C++ добавил поле, но layout и HLSL не обновлены согласованно, GPU будет читать соседние байты под неверным смыслом.

Полезная проверка перед запуском:

- число компонентов совпадает;
- формат совпадает со знаком и разрядностью;
- byte offset соответствует реальной структуре;
- семантический индекс совпадает;
- stride равен размеру вершины;
- целочисленные bone ID не объявлены как нормализованный цвет.

3 Vertex Shader и пространства координат

Позиция модели проходит несколько пространств.

- Model space: координаты, экспортированные из редактора модели.
- World space: положение объекта в сцене.
- View space: мир относительно камеры.
- Clip space: координаты после projection, необходимые rasterizer.

В текущем соглашении используется строковый вектор:

```
float4 worldPosition = mul(float4(input.position, 1.0f), worldMatrix);
float4 viewPosition = mul(worldPosition, viewMatrix);
output.position = mul(viewPosition, projectionMatrix);
```

Читать надо слева направо. Сначала локальная точка становится мировой, затем камерной, затем clip-space.

3.1 Почему `w` равен 1 для точки и 0 для направления

Однородная координата позволяет матрице содержать перенос. У точки `w = 1`, поэтому строка переноса участвует. У направления `w = 0`, поэтому перенос не меняет нормаль или скорость.

```
float3 worldNormal = mul(float4(input.normal, 0.0f), worldMatrix).xyz;
```

Но эта упрощённая формула корректна только без неравномерного масштаба. В общем случае нормаль преобразуется обратной транспонированной матрицей верхнего блока world matrix. Если проект разрешит разный scale по осям у освещаемых моделей, стоит передавать normal matrix отдельно или запрещать такой scale контрактом.

3.2 Clip space и деление на W

Vertex Shader возвращает `SV_POSITION`. После него GPU делит x, y и z на w и получает normalized device coordinates. Perspective projection делает дальние предметы меньше именно благодаря зависимости w от глубины.

Не надо выполнять perspective divide вручную в обычном VS. Rasterizer должен получить clip-space позицию, чтобы правильно отсекать треугольники у границ камеры.

4 Интерполяция между вершинами

Pixel Shader не вызывается только в трёх вершинах. Rasterizer создаёт фрагменты внутри треугольника и интерполирует выходы VS.

Если в вершинах UV равны `(0,0)`, `(1,0)` и `(0,1)`, каждый фрагмент получит промежуточное значение. То же происходит с мировой позицией, нормалью и fog factor.

Целочисленные или дискретные данные нельзя интерполировать как обычный `float`. Для них применяется модификатор `nointerpolation` или иной способ передачи.

5 Pixel Shader и текстура

Минимальный пиксельный шейдер:

```
Texture2D albedoTexture : register(t0);
SamplerState linearSampler : register(s0);

float4 main(PixelInputType input) : SV_TARGET
{
    return albedoTexture.Sample(linearSampler, input.tex);
}
```

`Texture2D` хранит данные. `SamplerState` описывает, как вычислять значение между texel, что делать за границей UV и какие mip levels использовать. Один sampler может применяться к нескольким текстурам.

`register(t0)` — не выделение памяти. Это соглашение о слоте. CPU обязан вызвать `PSSetShaderResources` с тем же номером.

6 Constant buffer и правило 16 байт

Constant buffer обновляется CPU и читается многими shader invocation. Его поля упаковываются по 16-байтовым регистрам. Ошибка выравнивания часто даёт не crash, а бессмысленные числа.

```
cbuffer PbrParameters : register(b6)
{
    float metallic;
    float roughness;
    float2 padding;
};
```

Соответствующая C++ структура должна повторять порядок и размер:

```
struct PbrParametersBuffer
{
    float metallic;
    float roughness;
    DirectX::XMFLOAT2 padding;
};

static_assert(sizeof(PbrParametersBuffer) % 16 == 0);
```

Текущий `ShaderBaseClass::CreateConstantBuffer` округляет `ByteWidth` до кратного 16. Это защищает размер ресурса, но не исправляет неправильное расположение полей внутри структуры. Поэтому `static_assert`, явные padding-поля и единый заголовок контрактов всё равно нужны.

6.1 Обновление буфера

CPU обычно:

1. формирует C++ структуру;
2. отображает dynamic buffer через `Map`;
3. копирует данные;
4. вызывает `Unmap`;
5. привязывает буфер к VS или PS слоту.

Нельзя менять один общий dynamic buffer для сотен объектов заранее, а потом ожидать, что каждый draw запомнит свою версию. Данные должны обновляться перед соответствующим draw либо находиться в отдельном ресурсе.

7 Регистры текущего PBR шейдера

В текущем `pbr.ps` текстуры распределены так:

- `t0` albedo;
- `t1` normal map;
- `t2` metalness;
- `t3` roughness;
- `t6` cracks или damage mask;
- `t7` shadow map;
- `t8` environment map.

Используются обычный sampler `s0` и comparison sampler `s7` для теней. Буферы занимают, среди прочего, `b0`, `b1`, `b3`, `b4`, `b5`, `b6`, `b7`, `b9`; fog включается отдельным общим контрактом.

Такое ручное распределение работает, но с ростом числа эффектов становится источником коллизий. Следующий инженерный шаг — единая таблица shader bindings:

```
namespace ShaderSlots
{
    constexpr unsigned Albedo = 0;
    constexpr unsigned Normal = 1;
    constexpr unsigned Shadow = 7;
    constexpr unsigned BonePalette = 8;
}
```

HLSL не умеет напрямую включить C++ namespace, но номера можно генерировать из одного описания или хотя бы проверять shader reflection при загрузке.

8 Нормали tangent space и normal map

Normal map хранит направление поверхности в касательном пространстве. Обычно RGB из диапазона `[0,1]` преобразуется в `[-1,1]`:

```
float3 sampled = normalTexture.Sample(sampler0, uv).xyz;
float3 tangentNormal = normalize(sampled * 2.0f - 1.0f);
```

Чтобы вернуть его в world space, нужен базис TBN:

- T — tangent, направление роста U;
- B — bitangent или binormal, направление роста V;
- N — геометрическая нормаль.

```
float3x3 tbn = float3x3(T, B, N);
float3 worldNormal = normalize(mul(tangentNormal, tbn));
```

Знак и порядок зависят от соглашения матриц и от того, как Assimp преобразовал систему координат. Если normal map выглядит вывернутой, надо проверить не только текстуру, но и handedness, направление green channel и построение binormal.

9 Освещение от Lambert до PBR

Диффузный Lambert использует косинус угла между нормалью N и направлением к свету L:

```
diffuse = albedo * max(dot(N, L), 0)
```

PBR не означает «реальная физика полностью». Это набор приближений, которые устойчиво связывают свойства материала, свет и камеру.

Текущий шейдер использует Cook–Torrance microfacet BRDF. Зеркальная часть имеет форму:

```
specular = D * F * G / max(4 * dot(N,V) * dot(N,L), epsilon)
```

Здесь:

- D описывает распределение микрограней, в текущем коде GGX;
- F — Fresnel, приближение Schlick;
- G — геометрическое затенение Smith;
- V — направление к камере;
- L — направление к свету;
- H — нормализованная полусумма V и L.

9.1 Roughness

Малый roughness создаёт узкий яркий блик. Большой roughness распределяет отражение по широкой области. Roughness нельзя бездумно доводить до нуля: формулы содержат деления и степени. Шейдер должен clamp значение до малого положительного порога.

9.2 Metalness

У диэлектрика базовый Fresnel около 0.04, а основная энергия находится в diffuse. У металла отражение окрашивается albedo, а diffuse почти исчезает. Metalness обычно смешивает два случая, а не является произвольным коэффициентом блеска.

9.3 Сохранение энергии

Если отражённая зеркальная доля велика, диффузная должна уменьшиться. Иначе материал создаёт дополнительный свет. Упрощённо:

```
kS = Fresnel
kD = (1 - kS) * (1 - metallic)
```

10 Безопасная математика шейдера

GPU легко производит `NaN` и бесконечность. Визуально это может проявляться чёрными вспышками, цветными точками или исчезающим объектом.

Нужны защитные операции:

- нормализовать только вектор ненулевой длины;
- ограничивать знаменатель `max(value, epsilon)`;
- clamp аргумент `pow` и результат выборки;
- не доверять альфе и normal map без диапазона;
- очищать нечисловой цвет перед записью.

Текущий `pbr.ps` уже содержит `SafeNormalize`, `SafePow` и очистку цвета. Это не косметика, а часть контракта стабильного renderer.

11 Скиннинг на GPU

Скиннинг деформирует вершину взвешенной суммой bone transforms.

```
skinnedPosition = Σ weight[i] * transform[boneID[i]] * position
```

Текущий `pbr.vs` поддерживает до четырёх влияний и палитру до 128 костей в `b8`. Для каждого влияния он проверяет диапазон индекса и положительный вес, затем нормализует сумму весов.

Очень важен static fallback. Если вершина не имеет валидных костей, она должна использовать исходную позицию и нормаль. Нельзя читать старое содержимое `b8`: предыдущий анимированный draw мог оставить там любые матрицы.

11.1 Почему нормаль тоже скинится

Если меняется положение вершины, направление поверхности должно следовать за костью. Нормаль скинится матрицами без переноса. Tangent и binormal тоже должны следовать деформации, иначе normal map «отлипнет» от анимации.

12 Тени

Shadow mapping рендерит сцену из позиции света в текстуру глубины. При основном проходе мировая позиция проецируется в light clip space, и текущая глубина сравнивается с сохранённой.

Основные проблемы:

- shadow acne из-за ограниченной точности и самосравнения;
- peter panning из-за слишком большого bias;
- ступенчатые края;
- неправильная проекция координат за границами карты.

Текущий PBR шейдер использует небольшую 2×2 PCF-фильтрацию. Несколько сравнений вокруг UV усредняют жёсткую границу. Bias должен зависеть от масштаба сцены и угла поверхности, а не быть случайно большим числом.

13 Прозрачность и порядок отрисовки

Alpha blending не решается только строкой в pixel shader. Нужно:

- включить соответствующий blend state;
- решить, пишет ли объект глубину;
- обычно отсортировать прозрачные объекты от дальних к ближним;
- различать alpha test и полупрозрачность.

Alpha test отбрасывает пиксель полностью и хорошо подходит листьям или решёткам. Полупрозрачное стекло смешивает цвет и требует порядка. `AssimpModelClass::MeshPart` уже хранит признаки прозрачности, стекла и opacity, поэтому renderer должен выбирать проход на основании материала.

14 Fog как отдельный общий контракт

Fog factor можно вычислять по глубине в VS и интерполировать либо вычислять в PS по мировой/камерной позиции. Текущий `ShaderBaseClass` поддерживает fog buffer в `b2`.

Преимущество общего слота — разные материалы получают одинаковую атмосферу. Риск — новый шейдер случайно займёт `b2` другим буфером. Поэтому общие слоты должны быть документированы централизованно.

15 Что делает текущий ShaderBaseClass

`ShaderBaseClass` является инфраструктурным базовым классом. Его обязанности:

- скомпилировать VS или PS из файла через `D3DCompileFromFile`;
- вывести ошибки компиляции в `shader-error.txt`;
- создать input layout;
- создать sampler;
- создать выровненный constant buffer;
- привязать shader objects и layout к context;
- обновить общий fog buffer;
- освободить COM-ресурсы в `Shutdown`.

Логический путь `CompileShader`:

1. получает путь, entry point и shader model;
2. включает строгую компиляцию и debug-флаги в отладочной сборке;
3. вызывает compiler с обычным include handler;
4. если есть error blob, записывает его текст;
5. возвращает bytecode вызывающему классу;
6. вызывающий создаёт `ID3D11VertexShader` или `ID3D11PixelShader`.

Bytecode VS нужен также для создания input layout: драйвер проверяет, что layout согласован с входной сигнатурой.

16 ShaderManager

Текущий `ShaderManager` — singleton с картой `name → unique_ptr<ShaderBaseClass>`. Он умеет загружать конкретный тип шаблонным `Load<T>`, возвращать базовый или типизированный указатель, перечислять имена и освобождать набор.

Плюсы:

- один экземпляр каждого именованного шейдера;
- явное владение;
- редактор может перечислять доступные имена;
- материал хранит лёгкую ссылку по имени или типу.

Ограничения:

- singleton скрывает зависимость от глобального состояния;
- строковое имя допускает опечатку только во время запуска;
- загрузка и hot reload требуют аккуратно сохранить старый рабочий shader при ошибке новой компиляции;
- часть комментариев исходника сейчас имеет повреждённую кодировку. Исходники стоит сохранить в UTF-8, потому что нечитаемые комментарии разрушают документацию, даже если бинарное поведение не меняется.

17 Как добавить новый шейдер в этот движок

Рассмотрим эффект подсветки выбранного объекта.

Шаг 1 Сформулировать входы

Нужно решить, использует ли эффект существующую геометрию, требует ли normal, какие текстуры читает и какие параметры меняются на объект или на кадр.

Пример параметров:

```
struct SelectionBuffer
{
    DirectX::XMFLOAT4 color;
    float intensity;
    DirectX::XMFLOAT3 padding;
};
```

Шаг 2 Зарезервировать слоты

Проверить все include и текущие PBR bindings. Выбрать свободный `b` слот либо добавить поле в существующий буфер, если жизненный цикл совпадает. Нельзя занимать номер только потому, что он свободен в одном файле.

Шаг 3 Написать HLSL

Сначала сделать минимальный shader, возвращающий постоянный цвет. Затем добавить world position, маску и освещение. Один новый фактор за раз облегчает диагностику.

Шаг 4 Создать C++ класс

Наследник `ShaderBaseClass` создаёт shader objects, input layout и собственные buffers. `Render` или метод параметров обновляет ресурсы перед draw.

Шаг 5 Зарегистрировать

Добавить загрузку в `ShaderManager` при инициализации renderer. Не забыть понятное имя, обработку ошибки и `Shutdown`.

Шаг 6 Подключить материал или проход

Renderer должен реально выбрать новый класс и вызвать обновление параметров. Наличие HLSL на диске ничего не меняет.

Шаг 7 Добавить диагностику

В редакторе показать активное имя шейдера, доступные слоты и последнюю ошибку компиляции. Для hot reload сначала компилировать во временный объект; заменять рабочий shader только после полного успеха.

Шаг 8 Проверить эталонную сцену

Проверки: объект перед камерой, сзади, в тени, с отрицательным scale, со skinned mesh, без текстуры и с несколькими источниками света.

18 Shader reflection как следующий шаг

Direct3D shader reflection может прочитать из bytecode:

- имена и слоты constant buffers;
- размеры и поля;
- texture и sampler bindings;
- входную сигнатуру;
- число инструкций и параметры shader model.

Это позволяет валидировать контракт CPU/GPU при загрузке и вывести понятную ошибку вроде «PbrParameters ожидает 32 байта, C++ передаёт 28». Reflection не заменяет архитектуру, но превращает молчаливое искажение в диагностируемый отказ.

19 Compute shader и когда он нужен

Compute Shader исполняет общие группы потоков без rasterizer. Он полезен для частиц, постобработки, culling, генерации данных и некоторых симуляций.

Он не является автоматически быстрее CPU. Нужно учитывать передачу данных, синхронизацию ресурсов и характер задачи. Для небольшого числа NPC utility AI на CPU проще и дешевле. Для миллионов независимых частиц GPU естественнее.

20 Отладка шейдера

Надёжная последовательность:

1. Убедиться, что shader скомпилирован и действительно привязан.
2. Вернуть постоянный яркий цвет.
3. Показать UV как `float4(uv, 0, 1)`.
4. Показать normal как `normal * 0.5 + 0.5`.
5. По одному включать texture, light и shadow.
6. Проверить CPU buffer и слот.
7. Захватить кадр RenderDoc или Graphics Diagnostics.

Цветовые диагностические режимы полезнее случайного изменения формулы. Они превращают скрытые векторы в видимые данные.

21 Практические задания

Задание 1 Контракт вершины

Найди `VertexType`, input layout PBR-класса и `VertexInputType` в HLSL. Составь таблицу смещений и форматов. Объясни, какое поле сломается первым, если забыть bone IDs.

Задание 2 Пространства

На бумаге возьми точку `(1,0,0)`, объект с переносом `(3,0,0)` и камеру без поворота. Последовательно вычисли world и view position. Потом объясни, почему normal не должна получить перенос.

Задание 3 Материал

Предскажи вид четырёх материалов: roughness 0.1/0.9 и metallic 0/1. После этого найди в `pbr.ps`, где меняются `kD`, Fresnel и GGX.

Задание 4 Buffer

Спроектируй cbuffer из `float3 color`, `float intensity`, `bool enabled` и `float2 scale`. Нарисуй 16-байтовые строки и затем напиши безопасную C++ структуру без использования C++ `bool`.

Задание 5 Новый режим диагностики

Без кода опиши переключатель, который показывает tangent, normal, roughness или shadow factor. Укажи владельца настройки, слот данных и место в EditorUI.

22 Критерий понимания шейдера

Ты готов уверенно менять HLSL, если можешь для любого поля ответить:

- кто создал это значение;
- в каком пространстве оно находится;
- какой C++ код привязал ресурс;
- какой регистр используется;
- интерполируется ли значение;
- что увидит экран при неверном значении;
- каким одноцветным debug-режимом проверить гипотезу.

Тогда шейдер перестаёт быть магией и становится обычной программой с очень строгим вводом и массовым параллельным исполнением.

Построчный разбор Direct3D ресурсов моделей и render проходов

Предыдущие главы объясняют общий renderer и HLSL. Здесь разобраны вспомогательные runtime классы, которые создают Direct3D 11 устройство, текстуры, render targets, тени, 2D geometry, grass, text и конкретные shader wrappers.

1 D3DClass Initialize validation

Первая проверка отвергает null `HWND`, неположительные размеры, near plane не дальше нуля и far plane не дальше near. Без неё последующие DXGI calls могли бы вернуть менее понятную ошибку.

`Shutdown()` очищает предыдущий device. Локальный `InitializationGuard` вызовет повторный shutdown при любом раннем `return false`. Флаг `committed` ставится только в самом конце.

Локальные DXGI объекты используют `ComPtr`, поэтому временные factory, adapter, output и back buffer освобождаются автоматически. Основные долгоживущие поля класса пока остаются raw COM pointers.

2 Получение adapter и display mode

`CreateDXGIFactory` создаёт точку перечисления видеоустройств.

`EnumAdapters(0)` выбирает первый adapter. Это не всегда дискретная GPU; для расширения нужен явный выбор по памяти или preference.

`EnumOutputs(0)` выбирает первый monitor данного adapter.

Первый `GetDisplayModeList` с null array получает количество режимов. Vector изменяет размер. Второй вызов заполняет descriptors.

Цикл ищет mode с точными width и height и сохраняет refresh numerator/denominator. Последнее совпадение побеждает; при нескольких частотах лучше выбирать максимальную или текущую.

`GetDesc` возвращает dedicated video memory и UTF-16 имя. Память переводится в мегабайты, имя — в узкую строку через `wcstombs_s`.

3 Swap chain descriptor

Структура обнуляется, чтобы неинициализированные flags не меняли поведение.

`BufferCount = 1` создаёт старую discard swap chain с одним back buffer. Width, height и `R8G8B8A8_UNORM` задают изображение.

При vsync найденная refresh rate записывается в descriptor. Без vsync numerator 0 и denominator 1 означают unspecified.

`DXGI_USAGE_RENDER_TARGET_OUTPUT` разрешает использовать back buffer как RTV. `OutputWindow` связывает его с `HWND`.

MSAA отключён: sample count 1. `Windowed = !fullscreen` инвертирует пользовательский флаг.

`DXGI_SWAP_EFFECT_DISCARD` соответствует старому swap model. Более современный flip model уменьшает стоимость present, но требует отдельной миграции.

Feature level фиксируется на 11.0.

4 Device creation

`D3D11_CREATE_DEVICE_BGRA_SUPPORT` нужен для совместимости Direct2D с swap-chain surface.

Debug build добавляет `D3D11_CREATE_DEVICE_DEBUG`. Если debug layer не установлен, первый вызов может вернуть `DXGI_ERROR_SDK_COMPONENT_MISSING`.

Fallback удаляет debug flag и повторяет создание. Поэтому debug executable запускается и на машине без Graphics Tools.

`D3D11CreateDeviceAndSwapChain` заполняет четыре долгоживущих поля: swap chain, device, feature-level output и immediate context.

`DX_CHECK` прекращает инициализацию при failure. Guard очистит уже созданные поля.

5 Back buffer и depth buffer

`GetBuffer(0)` получает texture interface back buffer. Временный `ComPtr` удерживает его до создания RTV.

`CreateRenderTargetView` создаёт представление, которое Output Merger может привязать как цветовую цель.

Depth texture имеет те же размеры, один mip, один array slice, формат D24S8, sample count 1 и bind `DEPTH_STENCIL`.

Texture сама хранит память. Depth stencil view определяет, как pipeline интерпретирует эту память.

6 Depth states

3D state включает test и запись depth. `LESS` пропускает fragment, если его глубина ближе уже записанной.

Stencil включён и получает read/write mask 0xFF. Front/back operations увеличивают или уменьшают stencil при depth failure. Если stencil не используется осознанным алгоритмом, это лишняя работа; состояние стоит упростить.

Второй state выключает depth test для 2D. Поле write mask остаётся ALL, но при `DepthEnable = false` depth не участвует. Stencil всё ещё включён; это тоже можно пересмотреть.

`OMSetDepthStencilState` устанавливает 3D state по умолчанию.

Depth stencil view получает typed D24S8 format поверх texture.

`OMSetRenderTargets` одновременно привязывает color RTV и DSV.

7 Rasterizer blend viewport

Rasterizer использует solid fill, no culling, depth clipping и без MSAA/scissor. `CULL_NONE` рисует обе стороны треугольника и может скрывать ошибки winding, а также увеличивает нагрузку.

Alpha blend state использует стандартную формулу `src * alpha + dst * (1-alpha)` для RGB. Alpha channel берётся из source единицей и игнорирует destination alpha.

Viewport покрывает всё окно и диапазон depth `[0,1]`.

8 Матрицы D3DClass

Класс сохраняет размеры, near, far и FOV 75 градусов.

`XMMatrixPerspectiveFovLH` создаёт left-handed perspective matrix.

World matrix по умолчанию identity.

Ortho matrix строится off-center с origin в верхнем левом углу: x растёт вправо, y вниз. Это удобно для pixel coordinates UI.

Guard получает `committed = true`, поэтому destructor больше не вызывает cleanup.

9 D3DClass Shutdown

Immediate context сначала `ClearState` и `Flush`. Это отвязывает resources и отправляет накопленные команды.

RTV и DSV освобождаются раньше underlying depth texture и device.

Rasterizer, depth states и blend state release и обнуляются.

Swap chain принудительно переводится в windowed state перед release.

Context освобождается перед device. В debug build device запрашивает `ID3D11Debug` и печатает live objects; это показывает утечки COM ресурсов.

Повторный вызов безопасен за счёт null checks.

10 BeginScene EndScene и переключатели

`BeginScene` собирает четыре компонента clear color, очищает back buffer и depth до 1. Stencil не очищается, потому что flag содержит только `D3D11_CLEAR_DEPTH`. Если stencil реально используется между кадрами, требуется добавить clear stencil.

`EndScene` вызывает `Present(1,0)` при vsync и `Present(0,0)` без него.

`SetRenderDistance` отвергает NaN/Infinity, ограничивает far между near и 10000, не пересоздаёт матрицы при почти неизменном числе и обновляет perspective/ortho.

`SetBackBufferRenderTarget` восстанавливает main RTV/DSV после offscreen pass. `ResetViewport` восстанавливает размер окна.

`TurnZBufferOn/Off` меняют depth state. `TurnAlphaBlendingOn` привязывает созданный blend state, Off передаёт null и возвращает default no blending.

11 TextureClass

11.1 Initialize

`Shutdown` позволяет повторно загрузить объект.

`LoadImage` вызывает `stbi_load` с requested channels 4. Независимо от исходных channels возвращается RGBA memory. Отдельное output channels сообщает исходное число, но row pitch всегда использует четыре.

Ширина, высота и channels сохраняются.

Texture имеет `MipLevels = 0`, bind как shader resource и render target, misc `GENERATE_MIPS`. Render-target bind требуется Direct3D для автоматической генерации mip chain.

Format выбирается SRGB для color texture и UNORM для data texture. Metalness, roughness и normal не должны получать SRGB decoding.

`CreateTexture2D` создаёт пустую mip chain. При failure CPU image освобождается.

`UpdateSubresource` загружает mip 0. Row pitch равен width × 4 bytes.

SRV с `MipLevels = -1` охватывает всю chain. После создания `GenerateMips` заполняет нижние уровни.

CPU image освобождается ровно после upload.

11.2 Lifetime

Shutdown освобождает SRV раньше texture. GetTexture возвращает невладеющий pointer.

Copy constructor сейчас пустой. Это опасный контракт: копия становится объектом с неопределёнными полями. Копирование надо удалить либо реализовать shared resource semantics.

12 RenderTextureClass

Constructor обнуляет пять COM pointers. Destructor вызывает Shutdown.

Initialize выбирает format, сохраняет размеры и создаёт texture с двумя bind flags: RTV и SRV. Одна память сначала принимает render output, потом читается shader.

RTV указывает на mip 0. SRV также читает один mip.

Отдельная D24S8 texture и DSV дают корректную 3D глубину в offscreen pass.

Viewport соответствует texture, а не окну. Projection использует aspect texture, ortho — её physical size.

Текущий код не вызывает `Shutdown` в начале и не имеет rollback guard: failure после первого resource оставляет частичный объект до destructor. Улучшение — транзакционная инициализация как в `D3DClass`.

`SetRenderTarget` меняет Output Merger и viewport. После pass вызывающий обязан вернуть back buffer и main viewport.

`ClearRenderTarget` очищает color и depth. Getters возвращают SRV, matrices и размеры.

Shutdown освобождает view перед underlying texture.

13 ShadowMap

Depth texture создаётся typeless `R24G8`, потому что одна память имеет две интерпретации.

DSV видит её как `D24_UNORM_S8_UINT` для depth writing.

SRV видит depth bits как `R24_UNORM_X8_TYPELESS` для sample в pixel shader; stencil bits игнорируются.

Bind flags одновременно разрешают depth-stencil и shader resource.

Viewport соответствует shadow resolution.

`SetRenderTarget` передаёт ноль color targets и только DSV. Shadow pass пишет глубину без цвета.

`ClearRenderTarget` возвращает depth к 1.

`SetLightViewProjection` хранит две matrices для последующей передачи shader.

Перед привязкой SRV в main pass DSV должен быть отвязан, иначе Direct3D выдаст resource hazard.

14 ShadowShaderClass

Initialize компилирует shadow vertex/pixel файлы, создаёт shader objects, input layout и matrix buffer.

Хотя depth-only pass может обходиться без pixel shader, текущий wrapper сохраняет общий путь.

`Render` сначала вызывает `SetShaderParameters`; failure не продолжает draw.

Matrix buffer получает world, light view и light projection. Матрицы транспонируются или передаются в соответствии с HLSL соглашением класса.

`VSSetConstantBuffers` привязывает buffer к vertex stage.

`RenderShader` устанавливает input layout, VS/PS и вызывает `DrawIndexed(indexCount,0,0)`.

Shutdown освобождает собственный buffer и shader resources. Базовые ресурсы не должны освобождаться дважды.

15 BitmapClass

Класс представляет screen-space quad с динамическим vertex buffer.

Initialize сохраняет screen и bitmap sizes, создаёт buffers и при texture overload загружает texture.

Index buffer статичен: два треугольника всегда используют одинаковые индексы.

`SetRenderLocation` меняет screen x/y. `UpdateBuffers` вычисляет left/right/top/bottom относительно screen center, формирует шесть vertices и обновляет dynamic buffer только если позиция изменилась.

`Map(D3D11_MAP_WRITE_DISCARD)` отдаёт новую область для полного переписывания. `memcpy` копирует vertices, `Unmap` публикует их GPU.

`RenderBuffers` привязывает buffers и triangle list. Сам shader вызывается внешним кодом.

Shutdown освобождает texture, index и vertex buffers.

16 WorldSpriteClass

В отличие от Bitmap, sprite живёт в 3D мире и billboard-подобно строит world matrix.

InitializeBuffers создаёт quad заданной ширины/высоты и статический index buffer.

Position, scale и rotation setters помечают matrix dirty. Это избегает повторной математики, если transform не менялся.

`SetTransform` меняет все параметры одной операцией.

Render принимает view/projection, texture shader и color. Он вычисляет world matrix, привязывает geometry и вызывает shader.

Если sprite должен всегда смотреть на camera, rotation basis строится из camera direction. Если это обычная world plane, используется сохранённый rotation.

Класс содержит повреждённые комментарии кодировки; runtime поведение не страдает, но исходник надо перевести в UTF-8.

17 ModelClass legacy

`ModelClass` создаёт жёстко заданную или внутренне построенную geometry, загружает две textures и предоставляет index count.

`Initialize` вызывает InitializeBuffers, затем LoadTextures. Если textures не загрузились, buffers должны быть очищены.

`Render` только привязывает buffers. Lighting/material logic находится снаружи.

Две texture slots соответствуют старому multi-texture/light-map pipeline. Для GLB и PBR используется `AssimpModelClass`.

Класс является legacy path, но пока его вызывает renderer или тестовая geometry, его lifetime contract остаётся важным.

18 ModelManager

`LoadModel` сначала ищет string key. Повторная загрузка возвращает существующий pointer.

Новая модель создаётся, инициализируется device/context и textures, затем помещается в map.

При failure локальный объект уничтожается, а map не получает broken entry.

`GetModel` возвращает невладеющий pointer либо null.

`Shutdown` проходит map, вызывает Shutdown моделей, удаляет владельцев и очищает map. Если используются raw pointers, копирование Manager должно быть запрещено.

19 TinyGltfModelClass

Текущий `.cpp` почти пуст и не реализует самостоятельный runtime loader. Наличие файла не означает существующую систему.

Не следует описывать его как альтернативу Assimp, пока нет parse, buffer creation, materials и lifetime. Лучшее действие — либо удалить заготовку, либо дать ей явный experimental status.

20 GrassRenderer data

Grass renderer хранит shared quad geometry, instance buffers, shaders, constant buffers, sampler, blend/raster states и batches по texture/material.

`GrassInstance` содержит world position, rotation и scale. `GrassBatch` объединяет instances одной texture, GPU instance buffer и visible count.

Это instanced rendering: quad vertices не копируются для каждой травинки. Один draw использует общий mesh и множество instance records.

21 GrassRenderer Initialize

Null device/context отвергаются. Повторная инициализация очищает старые resources.

`InitializeShaders` компилирует VS/PS, создаёт input layout с per-vertex и per-instance slots, constant buffers, sampler и pipeline states.

Ошибка любой стадии вызывает Shutdown, чтобы не оставить половину renderer.

`InitializeBatch` загружает texture и создаёт dynamic instance buffer подходящей capacity.

22 Grass generation

`GenerateAreas` очищает batches и вызывает `AppendArea` для каждого `WorldAreaComponent` с grass settings.

Количество instances выводится из area size и density.

`std::mt19937 gen(area.seed)` делает размещение детерминированным. Одна сцена создаёт одинаковую траву между запусками.

Distributions выбирают x/z offset, rotation, scale и batch texture.

Max attempts больше target count, потому что не каждая случайная точка попадает на terrain.

`TryProjectToTerrain` посылает вертикальный Bullet ray в пределах area height. Hit point задаёт y и surface normal.

Фильтры могут отклонить слишком крутой slope или неподходящий collision object.

Принятая instance добавляется в CPU batch. После generation создаётся или обновляется GPU buffer.

23 Grass visibility и render

`UpdateVisibleInstances` проверяет distance/frustum и собирает только видимые records.

Dynamic instance buffer отображается с write discard и получает visible array.

`UpdateFrameBuffers` обновляет view/projection, camera, wind/time, light и ambient.

`Render` устанавливает shaders, input layout, sampler, blend/raster states и для каждого batch вызывает instanced draw.

`RenderBatch` привязывает vertex buffer в slot 0 и instance buffer в slot 1, texture SRV и выполняет `DrawIndexedInstanced`.

После прохода изменённые pipeline states и SRV должны быть восстановлены/отвязаны, чтобы grass не влиял на следующий material.

24 DWriteTextSystem

24.1 Factory и formats

`DWriteCreateFactory` создаёт shared DirectWrite factory.

`CreateTextFormat` задаёт family, weight, style, stretch, size и locale. Alignment и paragraph alignment настраивают baseline текста.

Основной и small format существуют отдельно.

24.2 Render target

`InitializeRenderTarget` создаёт Direct2D factory, получает DXGI surface swap chain и создаёт D2D render target с premultiplied alpha.

Brushes создаются на конкретном render target и должны пересоздаваться вместе с ним после device loss.

24.3 MeasureText

`CreateTextLayout` строит layout в заданном max box. Metrics возвращают width/height уже после font shaping и wrapping.

24.4 Draw calls

`DrawTextLineColor` меняет brush color и вызывает DrawText с нужным format.

Filled rect и outline используют D2D geometry primitives.

Bitmap loader создаёт WIC factory/decoder/frame/converter и D2D bitmap. Каждый промежуточный COM object имеет отдельное владение.

`BeginBatch` вызывает BeginDraw один раз перед серией HUD элементов. `EndBatch` вызывает EndDraw и обрабатывает `D2DERR_RECREATE_TARGET` как сигнал пересоздать target resources.

25 ParticleRenderer pipeline

ParticleRenderer получает уже подготовленные `ParticleRenderItem`, а не симулирует particles.

Initialize создаёт shaders и pipeline states. Geometry строится CPU как billboards и отправляется динамическими buffers.

Render вычисляет camera right/up из inverse view, группирует items по blend mode/texture, сортирует alpha particles по расстоянию и вызывает material batches.

`AppendBillboard` создаёт четыре world-space corners вокруг center, применяет rotation и size, записывает color/UV и шесть indices.

`ResolveTexture` кэширует SRV по path. Отсутствующая texture использует fallback white resource.

`RenderBatch` обновляет buffers, matrices, texture, sampler, blend/depth/raster states и вызывает draw indexed.

Additive particles не обязаны сортироваться так строго, alpha particles сортируются back-to-front.

Shutdown очищает texture cache, states, shaders, layouts и buffers.

26 Shader wrapper общий шаблон

Почти каждый старый shader class повторяет шесть фаз.

1. `Initialize` выбирает HLSL files.
2. `InitializeShader` компилирует blobs и создаёт VS/PS.
3. По VS blob создаётся input layout.
4. Создаются constant buffers и sampler.
5. `SetShaderParameters` map/unmap buffers и привязывает SRV.
6. `RenderShader` устанавливает layout/shaders и вызывает DrawIndexed.

При чтении конкретного wrapper надо искать только различия в данных и slots.

27 ColorShaderClass

Matrix buffer содержит world/view/projection. Color buffer содержит override color.

Перегрузка Render без color использует default, перегрузка с color передаёт выбранный цвет.

Это минимальный shader для debug geometry и одноцветных объектов. Texture SRV не нужен.

Метод называется `ShutDown`, тогда как base использует `Shutdown`. Из-за несовпадения это не override. Имена стоит унифицировать.

28 TextureShaderClass

Matrix buffer отправляется vertex stage. Texture SRV привязывается pixel stage t0. Sampler берётся из класса.

Wrapper не вычисляет lighting. Он выводит sampled texture.

После draw SRV полезно отвязывать перед использованием той же texture как render target.

29 LightShaderClass

Класс поддерживает arrays до `NUM_LIGHTS`, positions/colors, spot parameters, directional light, selected factor и shadow map.

`PrepareBeforeRender` обновляет общие light buffers один раз перед серией objects.

`SetShaderParameters` обновляет per-object matrices и texture.

`SetSelectedFactor` выделяет выбранный объект множителем.

`SetDirectionalLight` заполняет direction/color/ambient/intensity buffer.

`SetShadowMap` привязывает SRV, comparison sampler и light view-projection.

Публичный Render с большим числом parameters является признаком, что per-frame и per-object данные стоит разделить структурами.

30 PBRShaderClass

Initialize создаёт основной PBR pipeline и default 1×1 textures для отсутствующих normal, metalness и roughness.

`SetMatrixBuffers` обновляет b0.

Render получает mesh textures, material scalars, arrays lights, selected factor и bone buffer/model state. Missing textures заменяются default SRV.

`SetDirectionalLight`, `SetShadowMap` и `SetDamageDecals` обновляют независимые buffers/resources.

Damage buffer передаёт до ограниченного числа decals или cracks; crack texture хранится отдельно.

Environment map привязывается для отражения/ambient.

После skinned draw bone constant buffer b8 должен быть корректен или vertex shader должен выбрать static fallback.

31 NormalMapShaderClass

Input layout требует tangent и binormal кроме position/UV/normal.

SetShaderParameters передаёт matrix, diffuse/normal textures, light direction/color и camera-related данные.

Pixel shader восстанавливает world normal через TBN. Модель без tangents должна использовать fallback shader или importer-generated tangent space.

32 LightMap и MultiTexture

`LightMapShaderClass` привязывает base texture и light map в разные texture slots. Результат обычно перемножает albedo и baked light.

`MultiTextureShaderClass` привязывает несколько color textures и смешивает их по фиксированному правилу shader.

Оба используют общий matrix buffer и старый material path. Метод `ShutDown` снова не совпадает с base virtual name.

33 FogShaderClass

Matrix buffer трансформирует geometry. Fog buffer хранит start/end или другие параметры эффекта.

Vertex shader вычисляет factor по глубине, pixel shader смешивает texture и fog color.

Этот класс частично дублирует общий fog contract `ShaderBaseClass`. Стоит выбрать один источник buffer slots.

34 ClipPlaneShaderClass

Clip plane buffer содержит `float4 plane`.

VS вычисляет signed distance world point до plane и передаёт `SV_ClipDistance` либо pixel shader отбрасывает сторону.

`SetClipPlane` сохраняет plane для последующих draws. Render может принять plane явно.

Эффект нужен reflection/refraction passes, чтобы не рисовать geometry с неправильной стороны воды.

35 ReflectionShaderClass

Matrix buffer дополнен reflection matrix. Отдельные buffers передают blend/translation и reflect color.

Shader sample base texture и reflection texture, полученную offscreen pass.

Reflection matrix переводит world position в projective UV отражённой camera.

Класс зависит от правильного порядка: сначала reflection render target, затем main object.

36 WaterShaderClass

Water wrapper принимает reflection/refraction/normal textures, water translation, reflect/refract scale, tint и camera position.

Matrix buffer обрабатывает geometry plane. Reflection buffer даёт отражённую camera transform. Water buffer управляет distortion.

Хотя FloodingSystem исключён как исторический gameplay рудимент, visual water shader остаётся отдельным render effect и поэтому разобран.

Если вода больше не нужна направлению игры, класс можно пометить optional asset path, а не оставлять обязательной renderer зависимостью.

37 GlassShaderClass

Glass buffer хранит refraction scale, tint/opacity и padding. Render получает base texture, normal map и scene/refraction texture.

Pipeline должен включить alpha blending и решить depth write policy. Wrapper задаёт shader parameters, но global state находится в renderer.

Glass objects требуют сортировки после opaque pass.

38 ProjectionShaderClass

Matrix buffer трансформирует object. Light buffer даёт освещение. Дополнительная projected texture использует projector view/projection coordinates.

SetShaderParameters привязывает base и projected resources и matrices.

Это decal/projector-like путь. Не путать с обычной camera projection matrix.

39 TerrainBlendShaderClass

Terrain buffer хранит параметры высотных или slope bands. Несколько terrain textures и normal maps привязываются по slots.

Render также поддерживает shadow map и directional light.

Pixel shader вычисляет weights по высоте/slope и нормализует сумму, чтобы яркость не росла от наложения слоёв.

40 SkyShaderClass

InitializeGeometry создаёт fullscreen triangle или cube-like geometry.

Render получает inverse view-projection. Vertex/pixel shader восстанавливает direction луча для каждого screen pixel и sample sky texture.

Camera translation из view должна быть удалена, чтобы sky не двигался относительно мира.

Depth state обычно рисует sky позади geometry, а culling/raster state отличается от обычного object pass.

41 ShaderManager

`Load<T>` проверяет существующее имя, создаёт `unique_ptr<T>`, вызывает Initialize и только после успеха кладёт его в map.

`Get` возвращает base pointer. `GetAs<T>` использует dynamic/static проверку в зависимости реализации и позволяет вызвать специфичные методы.

`GetNames` перечисляет keys для UI/material selector.

`ForEach` даёт callback по загруженным shaders.

`Shutdown` вызывает virtual shutdown и очищает владельцев.

Singleton удобен, но renderer dependency скрыта. Следующий шаг — передавать manager ссылкой в systems, сохранив один owner в Application.

42 RenderCandidateSystem и FrustumCulling

Candidate system перебирает entities с transform и mesh, проверяет active/visibility flags, строит world bounds и формирует компактный список для renderer.

Frustum extraction получает шесть planes из view-projection. AABB тест отклоняет object только если он полностью снаружи хотя бы одной plane.

Skinned mesh bounds могут изменяться. Текущий static bound должен иметь запас либо обновляться.

Room culling и frustum culling являются независимыми фильтрами: object должен пройти оба.

43 Порядок render ресурсов

Полный путь одного opaque object:

1. Candidate system подтверждает видимость.
2. Model привязывает vertex/index buffers.
3. Material выбирает shader и textures.
4. Renderer обновляет per-frame lights при необходимости.
5. Shader wrapper обновляет per-object matrices/material.
6. Context получает topology, layout, VS, PS, buffers, SRV и samplers.
7. DrawIndexed отправляет command.
8. Следующий pass явно меняет необходимые states.

44 Что исправлять первым в render runtime

1. Удалить пустой copy constructor `TextureClass`.
2. Перевести raw COM fields на `ComPtr` постепенно.
3. Добавить rollback guards в RenderTexture и ShadowMap.
4. Централизовать shader slots и reflection validation.
5. Унифицировать `Shutdown` и `ShutDown`.
6. Сделать state ownership renderer явным, чтобы wrapper не оставлял blend/depth state.
7. Пометить ModelClass и TinyGltf path как legacy/experimental.
8. Выбрать adapter/output осознанно.
9. Перейти к flip swap effect отдельным проверяемым этапом.
10. Исправить кодировку комментариев без изменения поведения.

45 Renderer главный кадр

45.1 InspectSceneRenderFeatures

1. Null SceneManager даёт пустой набор features.
2. Mesh pool перебирается без рисования.
3. Material и shader type определяют, нужны ли reflection, refraction, water или preview passes.
4. Результат позволяет не выполнять дорогой offscreen pass, если в сцене нет его потребителя.

45.2 RenderFrame

1. Проверяются context, camera и SceneManager.
2. Scene features вычисляются один раз.
3. Directional light и shadow matrices обновляются до passes.
4. Shadow pass рисует depth с точки зрения света.
5. Reflection texture строится только при `needsReflectionPass`.
6. Refraction texture строится только при `needsRefractionPass`.
7. Main back buffer снова назначается render target.
8. Sky рисуется с удалённым translation view, чтобы оставаться бесконечно далёким.
9. Opaque scene получает shadow/environment resources.
10. Water и transparent surfaces используют уже готовые offscreen textures.
11. Grass, particles и debug geometry добавляются после непрозрачного мира.
12. First-person presentation рисуется отдельным pass.
13. SRV slots очищаются перед тем, как те же textures станут render targets следующего кадра.

45.3 Shadow helpers

1. `UpdateShadowLightViewProjection` нормализует направление света.
2. Почти нулевой direction заменяется безопасным.
3. Up vector меняется, если свет почти параллелен world up.
4. Shadow distance ограничивается разумным минимумом.
5. Light view и orthographic projection сохраняются для всех shadow-aware shaders.
6. `BindShadowState` связывает shadow SRV в одинаковый slot Light и PBR shaders.

45.4 ClearSRVSlots

1. Создаётся массив null SRV.
2. Он привязывается к заданному диапазону pixel shader slots.
3. D3D reference на старые SRV снимается.

Без этого Direct3D выдаёт conflict warning при попытке назначить texture одновременно input и render target.

46 ShaderBaseClass

1. Constructor зануляет shader, layout, sampler и common buffers.
2. Destructor вызывает `Shutdown`, поэтому повторный Shutdown обязан быть безопасен.
3. Compile helper вызывает D3DCompileFromFile с entry point и profile.
4. Error blob записывается в debug output или файл диагностики.
5. Успешный blob используется для CreateVertexShader или CreatePixelShader.
6. Input layout создаётся из vertex shader signature.
7. `BindShader` связывает VS, PS, layout и default sampler.
8. `CreateConstantBuffer` выравнивает byte width по требованию 16 байт.
9. `UpdateFog` map-ит dynamic buffer с WRITE_DISCARD, копирует settings, unmap-ит и связывает slot.
10. Shutdown освобождает buffers раньше shader objects и зануляет pointers.

47 MaterialClass

1. `Initialize` сохраняет name и подготавливает arrays texture slots.
2. `SetTexture` записывает non-owning pointer; вызывающий код сохраняет lifetime.
3. `AdoptTexture` принимает `unique_ptr` и одновременно обновляет удобный raw view.
4. `SetTexturePath` хранит сериализуемый путь отдельно от runtime SRV.
5. `Bind` собирает SRV всех занятых slots и передаёт их pixel shader.
6. Пустой texture slot получает null, а не случайный старый SRV.
7. `Unbind` очищает те же slots.
8. `Shutdown` очищает owned textures, но не удаляет borrowed textures.

Различие Set и Adopt является контрактом владения. Его нельзя скрывать за одним неоднозначным setter.

48 AssimpModelClass краткий порядок исполнения

Полный импорт и animation pipeline разобраны в специальной главе. Здесь порядок конкретного runtime-класса:

1. Importer читает scene с выбранными post-process flags.
2. Mesh vertices получают position, UV, normal, tangent и bone weights.
3. Materials загружают embedded или внешние textures, а отсутствующие карты получают 1x1 fallback.
4. Texture cache делает AddRef для совместно используемых SRV.
5. CPU arrays сохраняются для physics, picking и runtime damage.
6. GPU vertex/index buffers создаются отдельно для mesh parts.
7. `LoadBones` строит bone name -> index/offset.
8. `LoadAnimations` копирует channels и keys из Assimp во внутренние structs.
9. `UpdateAnimation` продвигает time и обходит node hierarchy.
10. InterpolatePosition использует lerp, InterpolateRotation — quaternion slerp, InterpolateScale — lerp.
11. Final bone matrix объединяет global inverse, node transform и offset в принятом в shader порядке.
12. `SetBoneMatricesFromComponent` загружает palette в constant/structured buffer.
13. `Render` привязывает buffers одного part за раз и вызывает его shader/material path.
14. `Shutdown` освобождает per-part buffers/SRV, owned textures и bone buffer ровно один раз.

Импорт моделей анимация и инструменты Studio

Файл модели не превращается в игрового персонажа одной функцией `Load`. Между диском и экраном существует длинный конвейер: формат файла, Assimp scene, CPU-геометрия, GPU buffers, материалы, скелет, клипы, runtime-компонент, контроллер состояния, IK и редакторские поправки. Ошибка на ранней стадии часто проявляется только в конце — например, растянутой рукой или чёрным материалом.

1 Что находится внутри GLB

GLB — бинарная упаковка glTF. Внутри могут находиться:

- сцена и иерархия узлов;
- meshes и primitives;
- позиции, нормали, UV, tangent, индексы;
- материалы и ссылки на текстуры;
- skin: joints и inverse bind matrices;
- animation clips с каналами translation, rotation и scale;
- камеры и дополнительные свойства.

Формат описывает данные, но не игровой смысл. Узел с именем `mixamorig:RightHand` ещё не знает, что к нему надо прикрепить винтовку. Клип `Kevin_Melee_Jab_L` ещё не знает, когда наносить урон. Эти связи добавляет движок и профиль Animation Studio.

2 Что делает Assimp

Assimp читает множество форматов и преобразует их в общий `aiScene`. Текущий `AssimpModelClass::Initialize` вызывает `ReadFile` с флагами:

- `aiProcess_Triangulate` — превращает полигоны в треугольники;
- `aiProcess_ConvertToLeftHanded` — согласует систему координат с DirectX;
- `aiProcess_CalcTangentSpace` — строит tangent/bitangent для normal map;
- `aiProcess_GenSmoothNormals` — создаёт сглаженные нормали при необходимости;
- `aiProcess_LimitBoneWeights` — ограничивает число влияний.

Флаг удобен, но его эффект должен соответствовать контракту проекта. Например, смена handedness влияет не только на позицию, но и на порядок обхода треугольника, tangent basis и анимационные преобразования.

2.1 Время жизни `aiScene`

`aiScene*` принадлежит `Assimp::Importer`. Его данные нельзя использовать после уничтожения importer или новой загрузки. Текущий класс хранит importer как поле, а нужные runtime-данные копирует в собственные контейнеры.

3 Пошаговый путь текущего импорта

`AssimpModelClass::Initialize` можно читать как последовательность транзакции.

1. `Shutdown` очищает предыдущую модель.
2. `ReadFile` создаёт Assimp scene.
3. Проверяются scene, root node и признаки ошибки.
4. Очищаются bone map, clips, mesh parts и CPU-копии.
5. `LoadBones` строит палитру деформирующих костей.
6. `LoadAnimations` копирует клипы и ключи.
7. Для анимированной модели создаётся dynamic constant buffer на 128 матриц.
8. Каждый mesh преобразуется в `VertexType` и index array.
9. Узлы с префиксом `REF_` пропускаются как авторские ориентиры.
10. Загружаются PBR-текстуры и scalar fallback материала.
11. Создаются vertex/index buffers.
12. Вычисляются границы для culling, picking и редактора.

Если любой обязательный этап не удался, объект не должен оставаться «наполовину загруженным». Поэтому очистка в начале и единая ветка отказа важны.

4 Вершины и индексы

Vertex buffer хранит уникальные наборы атрибутов. Index buffer говорит, какие вершины образуют треугольник.

```
indices: 0 1 2  2 1 3
```

Так квадрат использует четыре вершины вместо шести. Но вершина может быть разделена, если один и тот же position имеет разные UV или normal на шве.

Текущий importer создаёт значения по умолчанию, если канал отсутствует. Это лучше чтения несуществующей памяти, но не всегда даёт качественный материал. Модель без UV не сможет корректно показать albedo. Модель без tangent не даст правильный normal mapping. Asset validator должен сообщать эти ограничения до игрового кадра.

5 Материал и текстуры

Assimp material содержит свойства и пути, но renderer использует собственный PBR-контракт. Текущий `MeshPart` хранит SRV для albedo, normal, metalness и roughness, а также opacity и признаки transparency/glass.

У каждого канала нужен fallback:

- albedo: белый цвет или scalar base color;
- normal: `(0.5, 0.5, 1)` в texture space;
- metallic: обычно 0;
- roughness: среднее безопасное значение;
- opacity: 1.

Без fallback разные ветви shader будут зависеть от мусора в старом слоте. Перед draw отсутствующие SRV надо либо заменить стандартной текстурой, либо явно отвязать и сообщить shader флагом.

5.1 Путь и кэш

Текстурный путь в модели может быть относительным, содержать обратные слэши или ссылаться на embedded texture. Перед кэшированием путь нормализуется. Ключ кэша должен учитывать цветовое пространство: одна и та же картинка как albedo обычно читается в sRGB, а как roughness — в linear.

6 Скелет без мистики

Скелет — дерево преобразований. У каждого узла есть local transform относительно родителя. Model transform получается умножением вдоль иерархии.

```
Hips
└── Spine
    └── Shoulder
        └── Arm
            └── ForeArm
                └── Hand
```

Если повернуть shoulder, все потомки двигаются вместе. Если напрямую перенести Hand, можно искусственно изменить длину цепи и растянуть mesh. Поэтому Animation Studio использует виртуальные IK-контролы для рук, а не разрешает свободный перенос каждой deform bone.

7 Bind pose inverse bind и final matrix

В bind pose вершины уже находятся около своих костей. Чтобы применить текущую позу, вершину сначала переводят из model space в локальную систему кости с помощью inverse bind или offset matrix, а затем возвращают текущим global transform кости.

Упрощённо:

```
finalBone = inverseBind * currentBoneGlobal * globalCorrection
```

Точный порядок зависит от строковых или столбцовых векторов и того, что вернул Assimp. Нельзя переносить формулу из другого движка без проверки соглашения.

Главный тест: при проигрывании bind pose все final matrices должны оставить mesh в исходной форме. Если модель взрывается, сначала проверяют matrix order, root correction и transpose, а не веса.

8 До четырёх весов на вершину

Assimp хранит список влияний по костям. GPU-вершина текущего проекта хранит четыре слота. При импорте система:

1. находит свободный слот;
2. если все заняты, заменяет наименьший вес более значимым;
3. нормализует сохранённые веса;
4. оставляет `boneIDs = -1` и нулевые веса у статической вершины.

Нормализация необходима:

```
w0 + w1 + w2 + w3 = 1
```

Если сумма меньше, вершина тянется к нулю. Если больше, масштабируется. Нулевой набор нельзя делить на ноль — для него работает static fallback VS.

9 Клип и каналы

`AnimationClip` хранит имя, duration, ticks per second и набор каналов. Канал привязан к node name и содержит три независимые дорожки:

- position keys;
- quaternion rotation keys;
- scale keys.

Не каждый канал имеет ключ каждого типа. Тогда используется исходный local transform или разумный default.

9.1 Ticks и секунды

Assimp duration обычно выражен в ticks. Runtime время удобно хранить в секундах:

```
durationSeconds = durationTicks / ticksPerSecond
timeTicks = timeSeconds * ticksPerSecond
```

Текущий importer использует fallback 24 ticks/s, если файл не задал скорость. `UpdateAnimation` накапливает секунды с учётом `clipSpeeds`, зацикливает по duration и только затем переводит в ticks.

Нельзя прибавлять `deltaTime` и в `AnimationSystem`, и внутри `AssimpModelClass::UpdateAnimation`. Комментарий текущего кода прямо защищает от двойного ускорения.

10 Интерполяция ключей

Позиция и scale линейно интерполируются:

```
value = a + (b - a) * alpha
```

Поворот хранится quaternion и интерполируется slerp или нормализованным lerp. Обычный линейный lerp углов Эйлера может выбрать длинный путь через 360 градусов и даёт неравномерную скорость.

Для редакторских additive rotation, которые хранятся градусами, текущие helpers используют shortest-angle interpolation: разность приводится к диапазону около `[-180, 180]`.

11 Обход иерархии

Сэмплирование клипа идёт по дереву узлов.

Для каждого узла:

1. берётся исходный local transform;
2. если есть канал, интерполируются position/rotation/scale;
3. применяются редакторские pose keys;
4. local умножается на parent global;
5. если узел является deform bone, вычисляется final matrix;
6. рекурсивно обрабатываются дети.

Имена нормализуются, потому что экспортёр может добавить суффикс вроде `_scaleCompensation` или цифровой хвост. Сопоставление должно быть предсказуемым и диагностируемым: два разных узла не должны случайно стать одним.

12 Основной и верхний слой

Персонаж может идти ногами и одновременно целиться или бить верхней частью тела. Текущий `AnimationComponent` хранит независимые:

- `animationIndex` и `currentTime` для базы;
- `upperBodyAnimationIndex` и `upperBodyTime` для слоя;
- `upperBodyRoot`, обычно spine.

При обходе иерархии ниже выбранного root используется верхний клип, а остальная часть берётся из базы. Для синхронизации разных длительностей используется нормализованная фаза:

```
phase = baseTime / baseDuration
upperTime = phase * upperDuration
```

Синхронизация полезна для согласованных циклов, но не всегда нужна. Выстрел или одноразовый удар может иметь собственную фазу и завершение.

13 AnimationComponent как runtime контракт

Компонент хранит не сами импортированные клипы, а выбор и состояние воспроизведения:

- `AnimationState` и bindings имён;
- базовый и верхний индексы;
- времена;
- one-shot запрос;
- crouch pose;
- состояние поисковых клипов;
- pose/fist keys;
- speed per clip;
- punch variants;
- временное состояние Animation Studio.

Это важное разделение. `AssimpModelClass` владеет общими данными asset. `AnimationComponent` владеет состоянием конкретной сущности. Если две сущности используют одну модель, они не должны делить один playhead.

14 Одноразовые действия

Idle и locomotion переоцениваются каждый кадр. Удар, перезарядка и реакция на попадание должны удерживаться до завершения. Для этого компонент имеет `oneShotRequested`, `oneShotState` и оставшееся время.

Контроллер должен принять запрос один раз, найти клип, запустить его и не дать обычному walk-состоянию перезаписать результат в следующем кадре. После завершения управление возвращается базовой locomotion.

Следующий шаг для масштабирования — обобщённая очередь animation actions с приоритетом, blend-in, blend-out и флагом interruptibility.

15 Two bone IK

Двухкостный IK решает положение цепи shoulder → elbow → hand. Известны длины двух сегментов, target и направление сгиба.

По закону косинусов для расстояния `d` до цели:

```
cos(angleShoulder) = (a² + d² - b²) / (2ad)
cos(angleElbow)    = (a² + b² - d²) / (2ab)
```

`d` ограничивается диапазоном, который цепь может достичь. Pole vector определяет плоскость локтя. Без него локоть может внезапно переворачиваться при почти прямой руке.

Текущий `WeaponIKSystem`:

1. находит right hand transform;
2. строит weapon model matrix из grip transform;
3. вычисляет target левой руки из left-hand grip;
4. вызывает `ApplyTwoBoneIK` для левой цепи;
5. записывает диагностический флаг и ошибку.

IK применяется после базового сэмплирования, потому что он корректирует уже полученную позу.

16 Виртуальные контролы Animation Studio

Вместо свободного переноса deform bone используются имена, которых нет в skeleton:

- `@CTRL_Hand_IK.L` и `.R`;
- `@CTRL_Elbow_Pole.L` и `.R`;
- `@CTRL_Head_Look`;
- виртуальные fist controls.

Hand target задаёт достижимую цель. Elbow pole задаёт направление сгиба. Head target управляет взглядом. Fist control хранит scalar от открытой до закрытой кисти и применяет заранее известные вращения пальцев.

Это хороший инструментальный дизайн: UI оперирует понятным контролом, а runtime сохраняет длины костей и корректную деформацию.

17 Первая и третья персона

Мировая модель и руки от первого лица могут иметь разные GLB и клипы. Они должны совпадать по смыслу и фазе, но не обязаны иметь одинаковую геометрию или кадрирование.

`AnimationSystem` обновляет world characters первым проходом. FP-копии с `syncAnimation` обрабатываются вторым проходом, чтобы порядок итерации не сдвинул их на один кадр.

Для удара:

- world component выбирает punch variant;
- FP component находит `firstPersonClip` того же варианта;
- нормализованная фаза world clip переносится на длительность FP clip;
- FP использует собственные pose keys и framing offset.

Для NPC authoring намеренно оставлен только world/third-person режим. Это уменьшает число бессмысленных комбинаций в инструменте.

18 Что такое Animation Studio в текущем проекте

Animation Studio — runtime-редактор профиля поверх реально запущенной сцены. Он не создаёт отдельную копию персонажа. Изменения влияют на выбранную сущность, а профиль сохраняется в JSON.

Главные функции:

- выбор Player или NPC;
- переключение third-person/first-person для игрока;
- выбор base clip и optional upper-body clip;
- play/pause и точный playhead;
- выбор control или bone;
- добавление additive position/rotation key;
- hand/elbow/head IK controls;
- fist curve;
- punch variants и contact settings;
- clip speed и first-person framing;
- сохранение и reload профиля.

18.1 Почему preview state находится в AnimationComponent

`socketPreviewEnabled`, `socketPreviewClip`, preview time и другие поля являются runtime-only. Благодаря этому обычный `AnimationSystem` использует тот же путь сэмплирования, что игра. Studio не рисует фальшивую позу отдельным кодом.

Риск — смешать авторские данные и временное состояние. При сериализации сцены preview flags не должны сохраняться. Долговременно сохраняются pose keys, fist keys, clip speeds и profile name.

19 Сохранение профиля

`SaveAnimationStudio` создаёт или дополняет JSON версии 2. В него входят bindings, melee action, punch variants, pose keys, fist keys и данные первой персоны, если она применима.

Важная деталь: функция сначала читает существующий профиль и объединяет данные. Если FP arms временно отсутствуют, сохранение third-person не должно стереть ранее настроенные FP keys.

Для NPC используется `worldOnly`:

- поле view становится third person;
- FP clip удаляется из action/variant;
- FP offsets и FP keys не записываются.

Это пример миграционно устойчивого формата. Версия файла позволяет в будущем преобразовать старую схему явно.

19.1 Pose key и variant fallback

Ключ имеет clip, bone, variant, time, position и rotation. `variant = -1` означает общую дорожку. Если конкретный вариант удара получил собственную дорожку, при первом ключе общая кривая клонируется. Иначе создание одного variant key могло бы обнулить остальную анимацию.

20 Gizmo и преобразование обратно в local key

Редактор показывает gizmo в viewport в мировом или model space. Сохранить надо стабильную поправку относительно родителя и исходной анимации.

Концептуально:

```
desiredLocal = desiredWorld * inverse(parentWorld)
correction = desiredLocal * inverse(sampledLocal)
```

Порядок снова зависит от соглашения строковых матриц. Для IK control сохраняется цель, а не прямой transform deform bone. Для root можно разрешить translation; для limb bones обычно только rotation.

Snap применяется до записи ключа. После изменения Studio пересэмплирует позу тем же runtime-путём, чтобы gizmo и mesh не расходились.

21 Камера и ввод Studio

Текущий editor различает UI под курсором и свободный обзор. Когда ImGui захватывает mouse, события идут элементам. Вне UI RMB управляет обзором, WASD и R/F — полётом.

`CameraController` хранит отдельные studio yaw, pitch, distance и target offset. Режим free fly отделён от orbit вокруг персонажа. Это лучше постоянного переключения глобального input mode: решение зависит от контекста кадра и `WantCaptureMouse`.

22 NPC Logic Studio

NPC Logic Studio не является AI-системой. Это диагностический наблюдатель и генератор контролируемых тестовых событий.

Он умеет:

- включить паузу;
- выполнить один штатный тик;
- изменить time scale;
- выбрать инспектируемого NPC;
- сохранить позиции тестовой цели;
- отправить обычный DamageEvent или near miss;
- заморозить симуляцию после decision, damage или warning;
- показать Inspector;
- хранить до 2000 событий;
- экспортировать text или JSON на двух языках.

22.1 Snapshot и обнаружение изменений

`NpcSimulationStudio::Capture` строит небольшой снимок каждого NPC:

- state и command revision;
- target и visibility;
- health;
- tactical goal, cover и style;
- status, transition и blocked reason;
- door route progress;
- no progress time.

Затем он сравнивает снимок с предыдущим. Событие создаётся только при изменении. Это устраняет шум «200 одинаковых строк в секунду».

При первом наблюдении создаётся baseline event. Раньше первый снимок был молчаливым, и на паузе журнал выглядел сломанным. Этот пример показывает ценность начального диагностического события.

22.2 Почему Studio не должна менять решение

Studio может поставить DamageEvent в очередь, телепортировать тестовую цель по явной команде или изменить авторскую настройку. Но она не должна записывать `brain.state = Attacking`, чтобы «проверить атаку». Иначе тестируется UI-обход, а не настоящий pipeline.

Правильный тест:

```
Studio input → обычное игровое событие → штатные системы → Snapshot → журнал
```

Так bug воспроизводится и без Studio.

22.3 Монотонная последовательность

Каждое событие имеет `sequence` и игровое время. Время может остановиться или замедлиться, но sequence сохраняет точный порядок. Для детерминированного replay позже надо добавить seed случайности, scene revision и входные команды.

23 Как добавить поле в NPC Studio

Не надо сразу писать ImGui.

1. Определи, какой вопрос должен решить разработчик.
2. Найди владельца истины.
3. Добавь поле в `Snapshot`, если нужно отслеживать изменение.
4. Создай событие с причиной, а не только числом.
5. Добавь Inspector-представление.
6. Если требуется активный тест, отправляй штатное событие.
7. Добавь экспорт, если поле важно для анализа после запуска.

Пример для поиска оружия:

- `goal = AcquireWeapon`;
- выбранный weapon entity;
- score;
- reservation owner;
- path result;
- причина отказа.

24 Как связать Animation Studio и NPC Studio

Они решают разные задачи.

- NPC Studio объясняет, почему NPC выбрал действие.
- Animation Studio настраивает, как это действие выглядит.

Связь должна проходить через semantic action, например `PeekLeft`, `Surrender`, `PickupWeapon`, а не через прямой вызов UI. AI создаёт action request. Animation controller сопоставляет его с clip binding. NPC Studio показывает request и phase. Animation Studio редактирует binding и pose profile.

Тогда отсутствие клипа не ломает логику: executor может выполнить действие с fallback-позой и записать warning.

25 Asset validation

До запуска сцены полезно проверять:

- пустые meshes;
- отсутствие normal/UV/tangent при материале, которому они нужны;
- более четырёх значимых весов;
- сумма весов около единицы;
- bone index больше 127;
- отсутствующая bone в animation channel;
- clip с нулевой duration;
- дублированные нормализованные имена;
- texture path, который не существует;
- несовместимый upper-body root;
- punch contact phase вне `[0,1]`.

Validator должен выдавать asset path, mesh/clip/bone и рекомендацию. Красная модель на экране сообщает слишком поздно и слишком мало.

26 Hot reload без потери рабочего состояния

Безопасный hot reload является транзакцией.

1. Новый файл загружается во временный `AssimpModelClass`.
2. Проходит validation.
3. Создаются все GPU resources.
4. Профили перепривязываются по semantic names.
5. Только после успеха старая модель заменяется.
6. При ошибке старый asset остаётся активным.

Playhead можно перенести нормализованной фазой, если клип с тем же semantic name сохранился.

27 Практические задания

Задание 1 Путь одной вершины

Выбери одну skinned вершину. Опиши её путь: accessor GLB → aiMesh → `VertexType` → vertex buffer → `pbr.vs` → четыре bone matrices → world matrix → rasterizer.

Задание 2 Кость

Нарисуй цепь плечо–локоть–кисть. Объясни, почему перенос кисти портит длину, а IK target нет. Отметь роль pole vector.

Задание 3 Два времени

Возьми world clip 0.8 с и FP clip 1.2 с. Рассчитай FP time при world time 0.3 с. Объясни, почему копировать секунды напрямую неверно.

Задание 4 Studio event

Спроектируй событие «анимация не найдена». Укажи entity, semantic action, requested clip, fallback и категорию. Реши, должно ли оно автоматически ставить симуляцию на паузу.

Задание 5 Формат профиля

Придумай version 3 с blend-in/out для action. Опиши миграцию: старый профиль получает значения по умолчанию, но не теряет keys.

28 Критерий понимания конвейера

Ты понимаешь импорт и анимацию, если можешь диагностировать проблему по слоям:

- файл содержит данные;
- Assimp прочитал их;
- importer перенёс их в формат движка;
- GPU buffers и bone palette корректны;
- component выбрал клип и время;
- hierarchy выдала позу;
- IK изменил её после сэмплирования;
- Studio сохранила только авторские данные;
- renderer получил финальные матрицы.

Такой разбор заменяет случайное вращение всех настроек одной проверяемой цепочкой.

Bullet Physics в DX Framework

Эта часть объясняет Bullet не как набор загадочных классов, а как отдельную симуляцию, которую движок связывает с ECS. В проекте Bullet отвечает за столкновения, rigid body, character controller, raycast, convex sweep, импульсы взрывов и constraints подлодки. ECS хранит игровые данные, DirectX рисует результат, а Bullet вычисляет физическое состояние.

1 Ментальная модель Bullet

У физического объекта есть несколько разных представлений.

Представление	Пример	Ответственность
Игровая сущность	`Entity 42`	связывает компоненты и правила игры
Положение игры	`TransformComponent`	позиция для сцены редактора и рендера
Геометрия столкновения	`btCollisionShape`	отвечает где объект физически занимает место
Физическое состояние	`btRigidBody`	масса скорость силы трение сон
Регистрация в мире	`btDynamicsWorld`	делает объект участником запросов и симуляции
Визуальный меш	`MeshRendererComponent`	рисует форму но сам не сталкивается

Главный инвариант: render mesh, collision shape и `TransformComponent` могут описывать один предмет, но это не один объект памяти. Изменение масштаба модели не перестраивает коллайдер автоматически. Перенос `TransformComponent` не обязан переносить dynamic body, если система синхронизации не подтверждает внешний teleport.

2 Слои вычисления столкновения

Bullet не сравнивает каждый треугольник со всеми остальными треугольниками.

1. Broadphase хранит грубые AABB и находит потенциальные пары.
2. Narrowphase проверяет реальные формы потенциальной пары.
3. Dispatcher выбирает алгоритм столкновения.
4. Solver вычисляет импульсы, которые исправляют скорость и проникновение.
5. Dynamics world интегрирует движение и обслуживает constraints.

В текущем мире используется `btDbvtBroadphase`. DBVT означает динамическое дерево ограничивающих объёмов. Когда объект переносится вручную, его AABB должен быть обновлён, иначе broadphase некоторое время знает старое место.

3 Создание физического мира построчно

Файл `src/Core/PhysicsWorld.h` создаёт Bullet зависимости во временных `unique_ptr` и только после успеха передаёт их полям singleton.

```
auto collisionConfig = std::make_unique<btDefaultCollisionConfiguration>();
auto dispatcher = std::make_unique<btCollisionDispatcher>(collisionConfig.get());
auto broadphase = std::make_unique<btDbvtBroadphase>();
auto solver = std::make_unique<btSequentialImpulseConstraintSolver>();
auto ghostCallback = std::make_unique<btGhostPairCallback>();
auto dynamicsWorld = std::make_unique<btDiscreteDynamicsWorld>(
    dispatcher.get(), broadphase.get(), solver.get(), collisionConfig.get());
```

Строка	Что создаётся	Почему порядок важен
`collisionConfig`	настройки и фабрики narrowphase	dispatcher хранит на неё не владеющий указатель
`dispatcher`	выбор алгоритма для пары форм	мир вызывает его при обработке потенциальных пар
`broadphase`	DBVT для грубого отсева	без него мир не знает какие пары проверять
`solver`	последовательный импульсный решатель	исправляет скорости контактов и constraints
`ghostCallback`	поддержка overlap пар ghost object	нужен character controller
`dynamicsWorld`	объект который соединяет подсистемы	не владеет всеми переданными зависимостями

Временные `unique_ptr` защищают от частично созданного мира. Если конструктор одного следующего объекта бросит исключение, уже созданные зависимости будут уничтожены автоматически. После успешной настройки они перемещаются в поля. Адрес самих Bullet объектов при перемещении `unique_ptr` не меняется.

4 Гравитация ghost callback и GImpact

```
dynamicsWorld->setGravity(btVector3(0, -9.81f, 0));
dynamicsWorld->getPairCache()->setInternalGhostPairCallback(ghostCallback.get());
btGImpactCollisionAlgorithm::registerAlgorithm(dispatcher.get());
```

Первая строка задаёт ускорение мира в метрах на секунду в квадрате. Вторая разрешает pair cache отслеживать пересечения `btPairCachingGhostObject`. Третья регистрирует narrowphase алгоритм для GImpact. Создать `btGImpactMeshShape` недостаточно: dispatcher должен знать, каким алгоритмом обрабатывать такую пару.

5 Настройка solver

В текущем проекте число итераций увеличено до 50, включён split impulse и настроен ERP.

Параметр	Что делает	Симптом слишком малого или большого значения
`m_numIterations`	сколько раз solver уточняет ограничения	мало даёт мягкие дрожащие joints много повышает CPU стоимость
`m_splitImpulse`	отделяет коррекцию проникновения от обычной скорости	без него глубокое проникновение может подбросить тело
`m_splitImpulsePenetrationThreshold`	глубина для split correction	неверный порог меняет характер исправления контакта

Параметр	Что делает	Симптом слишком малого или большого значения
`m_erp`	доля позиционной ошибки исправляемая за шаг	слишком высоко даёт жёсткие рывки слишком низко оставляет ошибку

Эти параметры не заменяют правильную начальную расстановку тел. Если два тяжёлых отсека создаются глубоко друг в друге, solver всё равно вынужден бороться с большой ошибкой.

6 Fixed timestep и аккумулятор

`PhysicsWorld::Update` не передаёт произвольный кадровый `dt` напрямую в физику.

```
constexpr float timestep = 1.0f / 60.0f;
m_accumulator += (std::max)(0.0f, deltaTime);
if (m_accumulator > 0.25f)
    m_accumulator = 0.25f;
while (m_accumulator >= timestep)
{
    m_dynamicsWorld->stepSimulation(timestep, 1, timestep);
    m_accumulator -= timestep;
    ++m_stepIndex;
}
m_interpolationAlpha = std::clamp(m_accumulator / timestep, 0.0f, 1.0f);
```

Строка	Смысл
`1 / 60`	один физический шаг всегда длится одну шестидесятую секунды
прибавление `deltaTime`	кадр отдаёт накопителю прошедшее реальное время
clamp до `0.25`	предотвращает spiral of death после долгого зависания
`while`	выполняет столько полных шагов сколько накоплено
`stepIndex`	сообщает синхронизации что появился новый физический результат
`interpolationAlpha`	показывает долю пути от прошлого fixed state к текущему

Если рендер работает на 144 Hz, а Bullet на 60 Hz, несколько кадров могут видеть одинаковый физический state. Интерполяция делает визуальное движение плавным, но не должна возвращаться в Bullet как новая физическая позиция.

7 Static dynamic и kinematic

Тип	Масса	Кто задаёт Transform	Реагирует на силы	Типичный объект
Static	`0`	создаётся один раз	нет	пол стена
Dynamic	больше `0`	Bullet после шага	да	ящик обломок отсек
Kinematic	обычно `0` и флаг	ECS перед шагом	не как обычное dynamic body	управляемая дверь платформа

Нельзя определять тип только по массе. Kinematic body имеет специальный collision flag. В `PhysicsSystem::Update` проект сначала проверяет этот флаг, затем массу.

8 Владелец Transform и направление синхронизации

Текущий `PhysicsSystem::Update` разделяет синхронизацию на две стороны.

```
до fixed step
    kinematic        ECS -> Bullet каждый кадр
    dynamic teleport ECS -> Bullet только при внешнем изменении
    static           обычно не трогаем

после fixed step
    dynamic          Bullet -> ECS
    character        ghost object -> визуальный Transform с интерполяцией
```

Это один из главных контрактов физики. Если каждый кадр писать Transform dynamic body обратно в Bullet, физика не владеет движением: скорость и solver результат постоянно перезаписываются. Если никогда не отправлять внешний teleport, перемещение в редакторе останется только визуальным.

9 Проверка внешнего изменения

`TransformChangedExternally` сравнивает текущий Transform с `lastSyncedPosition`, `lastSyncedRotation` и `lastSyncedScale`. Равенство допустимо здесь потому, что обе стороны сравнения были записаны самим синхронизатором. Это не общий рецепт сравнения любых вычисленных float.

При внешнем изменении dynamic body проект:

1. строит новый `btTransform`;
2. записывает world transform тела и motion state;
3. обновляет interpolation transform;
4. обнуляет скорости и силы;
5. вызывает `updateSingleAabb`;
6. активирует тело;
7. сохраняет последние синхронизированные значения.

Обнуление скорости превращает операцию в teleport, а не в продолжение прежнего полёта. Для игровой механики броска с переносом понадобится отдельный контракт, который сохраняет или назначает скорость явно.

10 Деактивация ECS сущности

Неактивный объект не удаляется из Bullet, но временно переводится в состояние без контактной реакции и симуляции.

```
rb.body->setCollisionFlags(
    rb.collisionFlagsBeforeSuspend |
    btCollisionObject::CF_NO_CONTACT_RESPONSE);
rb.body->setLinearVelocity(btVector3(0, 0, 0));
rb.body->setAngularVelocity(btVector3(0, 0, 0));
rb.body->clearForces();
rb.body->forceActivationState(DISABLE_SIMULATION);
```

Перед этим сохраняются прежние flags и activation state. При возвращении `ActiveComponent` они восстанавливаются. Иначе временная деактивация навсегда превращала бы body в trigger или спящий объект.

11 Collision shape и масштаб

Bullet формы получают размеры при создании. Текущий проект поддерживает box, sphere, cylinder, capsule, convex hull, triangle mesh, GImpact и compound shape.

Shape	Использование	Ограничение
`btBoxShape`	стены ящики простые предметы	передаются полуразмеры
`btSphereShape`	снаряд область простая динамика	не зависит от вращения
`btCylinderShape`	цилиндрические объекты	ось формы должна совпасть с моделью
`btCapsuleShape`	персонаж	height относится к цилиндрической части Bullet API
`btConvexHullShape`	динамический сложный объект	должен оставаться выпуклым
`btBvhTriangleMeshShape`	точная статическая геометрия	не предназначен для обычного dynamic body
`btGImpactMeshShape`	динамический вогнутый меш	дороже простых и convex форм
`btCompoundShape`	несколько простых частей	нужно отдельно уничтожать child shapes

Нулевой и отрицательный scale недопустимы. В проекте они заменяются на единицу. Когда scale меняется после создания, `RefreshRigidBody` полностью перестраивает shape, потому что одного изменения визуального Transform недостаточно.

12 Почему box принимает полуразмеры

Для полного размера `(2, 4, 6)` в Bullet передаётся `(1, 2, 3)`. Эти числа означают расстояние от центра формы до каждой грани. Ошибка в два раза создаёт коллайдер, который выглядит смещённым или чрезмерно большим, хотя его центр математически правильный.

13 Triangle mesh и владение памятью

Текущий `CreateTriangleMeshShape` создаёт два связанных объекта: `btTriangleMesh` хранит треугольники, а `btBvhTriangleMeshShape` использует эту геометрию для collision query. Поэтому `RigidBodyComponent` хранит указатель `triangleMesh` отдельно. Удалить backing mesh раньше shape нельзя.

Путь каждой тройки индексов:

```
const auto& p0 = vertices[ia].position;
const auto& p1 = vertices[ib].position;
const auto& p2 = vertices[ic].position;
triangleMesh->addTriangle(
    btVector3(p0.x * scale.x, p0.y * scale.y, p0.z * scale.z),
    btVector3(p1.x * scale.x, p1.y * scale.y, p1.z * scale.z),
    btVector3(p2.x * scale.x, p2.y * scale.y, p2.z * scale.z),
    true);
```

Индексы проверяются до обращения к vertices. Масштаб применяется к каждой координате. Флаг `true` разрешает удаление дублирующихся вершин внутри `btTriangleMesh` по его правилам.

14 GImpact и цена точности

GImpact позволяет dynamic concave mesh, но его нельзя делать формой по умолчанию для каждого предмета. Простые формы и convex hull дешевле, устойчивее и дают solver более понятные контакты.

GImpact оправдан, когда игровая геометрия действительно вогнута и должна двигаться как единое тело.

После создания текущий код вызывает `updateBound`. Без обновлённой границы broadphase не имеет корректного AABB формы.

15 Compound shape

Compound shape хранит дочерние формы и локальный transform каждой части. Это позволяет собрать коллайдер корабля из нескольких box и capsule без одного тяжёлого triangle mesh.

```
world transform body
    * local transform child 0 -> box
    * local transform child 1 -> cylinder
    * local transform child 2 -> convex hull
```

При удалении compound текущий код проходит child shapes, удаляет их из compound и освобождает память. Одного `delete compound` недостаточно, если проект сам владеет созданными через `new` детьми.

16 Создание rigid body

Минимальная последовательность Bullet выглядит так.

```
btVector3 inertia(0, 0, 0);
if (mass > 0.0f)
    shape->calculateLocalInertia(mass, inertia);
auto* motionState = new btDefaultMotionState(startTransform);
btRigidBody::btRigidBodyConstructionInfo info(
    mass, motionState, shape, inertia);
auto* body = new btRigidBody(info);
world->addRigidBody(body, group, mask);
```

Строка	Контракт
нулевая inertia	static body не нуждается в динамическом моменте инерции
`calculateLocalInertia`	shape и mass определяют сопротивление вращению
motion state	канал между body и внешним представлением transform
construction info	собирает массу форму motion state и inertia
`addRigidBody`	только здесь body начинает участвовать в мире

Созданный объект до `addRigidBody` существует в памяти, но raycast мира его не увидит.

17 Collision groups и masks

Group отвечает на вопрос кто я. Mask отвечает на вопрос кого я хочу проверять. Пара рассматривается, когда оба битовых условия разрешают друг друга.

```
(groupA & maskB) != 0
and
(groupB & maskA) != 0
```

В `PhysicsSystem.h` группы оформлены как биты: default, dynamic, submarine exterior, submarine interior, player, projectile, trigger. Такое разделение позволяет игроку не сталкиваться со служебной внутренней формой подлодки, а внешнему корпусу не получать контакты с персонажами внутри.

При добавлении новой группы выпиши таблицу взаимодействий до кода. Ошибка маски проявляется как объект, который виден, но не сталкивается, либо как trigger, который никогда не получает overlap.

18 Entity внутри Bullet объекта

Проект связывает collision object с ECS через `setUserIndex` и читает его через `getUserIndex`. Это удобно для raycast, но имеет контракт: `INVALID_ENTITY` и диапазон типа должны обрабатываться одинаково во всех местах. После уничтожения Entity зарегистрированный Bullet object не должен остаться в мире с устаревшим index.

19 Raycast как запрос мира

Луч задаётся двумя точками, а не только направлением.

```
start = origin
end = origin + normalize(direction) * distance
```

`PhysicsRaycast::CastRay` сначала очищает `outHit`, проверяет мир и дистанцию, нормализует направление, отвергает нечисловой результат и только после этого вызывает `world->rayTest`.

```
IgnoreEntityRayCallback callback(
    outHit.rayStart, outHit.rayEnd,
    ignoredObject, ignoredEntity);
world->rayTest(outHit.rayStart, outHit.rayEnd, callback);
```

Raycast не двигает тела и не создаёт урон. Он только отвечает на геометрический вопрос о первом разрешённом контакте.

20 Пользовательский callback луча

`IgnoreEntityRayCallback` наследует `ClosestRayResultCallback` и переопределяет `addSingleResult`.

```
if (hitObject == m_ignoredObject ||
    static_cast<Entity>(hitObject->getUserIndex()) == m_ignoredEntity ||
    hitObject->getActivationState() == DISABLE_SIMULATION)
{
    return 1.0f;
}
return ClosestRayResultCallback::addSingleResult(
    rayResult, normalInWorldSpace);
```

Возврат `1.0f` сообщает callback не принимать этот кандидат как ближайшее попадание. Сравнение делается и по указателю объекта, и по Entity, потому что у одной сущности могут быть дополнительные collision objects. Неактивный объект также пропускается.

21 Closest и all hits

Callback	Результат	Пример
`ClosestRayResultCallback`	ближайший разрешённый hit	оружие взгляд взаимодействие
`AllHitsRayResultCallback`	массив всех hit	прострел нескольких слоёв анализ материалов
custom closest	ближайший после собственных фильтров	игнорирование стрелка и disabled objects

Для обычного оружия нужен closest hit. Иначе можно случайно нанести урон выбранной AI цели за стеной, проигнорировав реальный первый контакт.

22 От raycast к выстрелу

Текущее оружие является hitscan, а не летящим физическим снарядом.

```
нажатие или решение NPC
    -> WeaponSystem TryFire
    -> проверка магазина кулдауна и луча
    -> разброс и recoil
    -> PhysicsRaycast ближайший контакт
    -> DamageEvent
    -> DamageSystem в своей стадии кадра
```

Bullet здесь отвечает только за геометрическое подтверждение. Патроны, перезарядка, урон и фракции являются gameplay правилами.

23 Convex sweep

Raycast проверяет бесконечно тонкий отрезок. Персонаж имеет объём, поэтому для движения и step-up нужен sweep выпуклой формы.

```
world.convexSweepTest(
    &shape,
    fromTransform,
    toTransform,
    callback);
```

Sweep отвечает: как далеко форма может пройти по траектории до контакта. `m_closestHitFraction` находится в диапазоне от нуля до единицы. Позиция контакта вдоль перемещения вычисляется как `from + delta * fraction`.

24 Нормаль поверхности и допустимый склон

Character callbacks фильтруют поверхность по скалярному произведению нормали с мировой осью вверх.

```
upDot = normal dot (0, 1, 0)
```

Горизонтальный пол имеет normal примерно `(0, 1, 0)` и `upDot` около единицы. Вертикальная стена имеет значение около нуля. Порог можно получить как `cos(maxSlopeAngle)`. Чем больше допустимый угол склона, тем меньше порог.

25 Ghost object и character controller

`CharacterSpawnSystem` создаёт capsule shape, `btPairCachingGhostObject` и `btKinematicCharacterController`. Ghost object хранит форму и overlap пары, но сам не решает динамический контакт как rigid body. Character controller выполняет специальную логику ходьбы, step up, step down и penetration recovery.

Порядок регистрации:

1. создать capsule;
2. создать ghost object;
3. назначить transform и collision shape;
4. добавить `CF_CHARACTER_OBJECT`;
5. создать controller;
6. настроить gravity, slope и step height;
7. добавить collision object с group и mask;
8. добавить controller как action мира.

При уничтожении порядок обратный: удалить action, удалить collision object, затем освободить controller, ghost object и shape.

26 Почему центр капсулы выше ног

`TransformComponent` персонажа хранит игровую позицию у основания, а Bullet transform ghost object относится к центру формы. Поэтому используется `capsuleCenterOffsetY`.

```
bulletCenterY = feetY + capsuleCenterOffsetY
feetY = bulletCenterY - capsuleCenterOffsetY
```

Если вычесть offset дважды, модель провалится. Если не вычесть после чтения Bullet, она будет висеть над поверхностью.

27 Ground probe и step up

Ground detection не должен полагаться только на один флаг controller. Текущий код использует ray и convex sweep с фильтром поверхности. Для step up обычно проверяются три этапа.

1. Горизонтальное движение встречает препятствие.
2. Поднятая капсула имеет свободное место.
3. Из поднятой и сдвинутой позиции можно опуститься на допустимую поверхность.

Каждый этап требует отдельного наблюдаемого debug луча или sweep. Иначе баг выглядит как общее застревание, хотя причина может быть в потолке, слишком высокой ступени или неверной нормали посадки.

28 Контакты и manifolds

После шага dispatcher хранит contact manifolds. Каждый manifold относится к паре collision objects и содержит contact points. У точки доступны глубина, normal, позиции на A и B, а также применённый импульс.

Схема чтения:

```
const int count = dispatcher->getNumManifolds();
for (int i = 0; i < count; ++i)
{
    btPersistentManifold* manifold = dispatcher->getManifoldByIndexInternal(i);
    for (int j = 0; j < manifold->getNumContacts(); ++j)
    {
        const btManifoldPoint& point = manifold->getContactPoint(j);
        if (point.getDistance() <= 0.0f)
        {
            // подтверждённый контакт
        }
    }
}
```

Не создавай gameplay событие каждый fixed step без памяти пары. У устойчивого контакта manifold живёт много шагов, поэтому звук удара или damage on enter требует отслеживания перехода пары из отсутствующей в активную.

29 Trigger и no contact response

Trigger использует collision object с `CF_NO_CONTACT_RESPONSE`. Broadphase и overlap продолжают работать, но solver не выталкивает тела. Это отличается от mask, которая полностью запрещает образование пары.

Требование	Механизм
видеть вход без физического толчка	trigger и overlap pairs
полностью игнорировать категорию	group и mask
временно выключить объект	удалить из мира или согласованно отключить simulation

30 Force impulse velocity и teleport

Операция	Единица и смысл	Когда использовать
`applyCentralForce`	ньютон действует в течение шага	двигатель плавучесть постоянная тяга
`applyCentralImpulse`	изменение импульса мгновенно	взрыв удар прыжок dynamic body
`setLinearVelocity`	прямая замена скорости	специальный контролируемый режим
`setWorldTransform`	мгновенная смена положения	teleport редактор восстановление save

Сила должна учитывать время через интегратор Bullet. Импульс уже описывает мгновенное изменение количества движения и не умножается вручную на `deltaTime`.

31 Импульс взрыва построчно

`ExplosionSystem::Emit` перебирает rigid bodies, исключает static и kinematic, считает направление от центра взрыва и применяет квадратичный falloff.

```
const float normalizedDistance =
    (std::min)(distance / radius, 1.0f);
const float falloff = 1.0f - normalizedDistance;
const float finalImpulse = impulseStrength * falloff * falloff;
body->activate(true);
body->applyCentralImpulse(direction * finalImpulse);
```

Строка	Результат
`distance / radius`	перевод расстояния в диапазон области взрыва
`1 - normalized`	максимум в центре и ноль на границе
квадрат	быстро ослабляет дальнюю часть импульса
`activate`	будит спящее тело до применения
central impulse	меняет линейную скорость без точки приложения

Дополнительный torque impulse создаёт вращение. Для физически более точной модели можно применять обычный impulse в точке `applyImpulse`, тогда плечо само создаст момент.

32 Масса и обратная масса

Bullet часто хранит inverse mass. Для dynamic body `invMass = 1 / mass`. Для static и kinematic inverse mass равна нулю. Поэтому проверка `getInvMass() <= 0` быстро исключает тела, которые нельзя разогнать обычным импульсом.

Одинаковый impulse сильнее ускоряет лёгкое тело, потому что изменение скорости пропорционально inverse mass.

33 Трение restitution и damping

Свойство	Что регулирует	Ошибка настройки
friction	сопротивление скольжению в контакте	предмет едет как по льду или цепляется
restitution	доля упругого отскока	дребезг при слишком большом значении
linear damping	потеря линейной скорости без контакта	объект зависает или движется бесконечно
angular damping	потеря вращения	обломок никогда не перестаёт крутиться

Damping не заменяет сопротивление среды, когда нужна зависимость от скорости, глубины и направления. В физике подлодки проект дополнительно прикладывает drag force.

34 Плавучесть текущего проекта

Плавучесть следует идее Архимеда.

```
F buoyancy = fluidDensity * gravity * displacedVolume * submersionRatio
```

Вода внутри не уменьшает внешний объём корпуса. Она добавляет вес. Поэтому текущий код оставляет buoyancy volume прежним и вычитает дополнительный `waterWeight` из направленной вверх силы. Затем ограничивает экстремальное значение, добавляет drag и angular damping.

Этот раздел относится к gameplay approximation. Для точной плавучести нужны несколько sample points или погружённый объём сложной формы, центр приложения силы и torque от несовпадения центра плавучести с центром масс.

35 Constraints и joints

`btFixedConstraint` связывает два rigid body через два локальных frame. Frame A задан в локальном пространстве тела A, frame B в локальном пространстве тела B. Они описывают точки и ориентации, которые solver старается совместить.

В `SubmarineJointSystem::CreateJoint` сначала проверяются оба body, создаются локальные frames, затем constraint помещается в ECS компонент и только после этого регистрируется в Bullet.

```
jointComp.joint = new btFixedConstraint(
    *rbA->body, *rbB->body, frameA, frameB);
jointComp.joint->setBreakingImpulseThreshold(breakForce);
jointComp.joint->setOverrideNumSolverIterations(100);
```

Порог относится к solver impulse, а не непосредственно к силе в ньютонах. Название `breakForce` удобно для gameplay, но при балансировке важно помнить фактическую величину Bullet.

36 Владение constraint

Проект сначала передаёт constraint в ECS компонент, потом вызывает `world->addConstraint`. Это exception safe порядок: если добавление компонента не удалось, локальный владелец может освободить ресурс, и в мире не останется не владеющий dangling pointer.

При удалении constraint сначала отсоединяется от мира, затем уничтожается. `PhysicsWorld::Shutdown` также проходит все constraints и collision objects и снимает регистрации до разрушения зависимостей мира.

37 Сон и активация

Bullet усыпляет спокойные dynamic bodies для экономии CPU. Сила, impulse, значимое изменение скорости или `activate(true)` будит тело. `DISABLE_DEACTIVATION` запрещает обычный сон и нужен только для объектов, которые действительно должны постоянно обновляться, например некоторых управляемых механизмов.

Слишком широкое использование `DISABLE_DEACTIVATION` повышает стоимость мира и может скрыть ошибку: объект постоянно получает микросилу или дрожит в контакте.

38 CCD и быстрые тела

Discrete collision detection проверяет состояния по шагам. Очень быстрый маленький объект может оказаться по разные стороны тонкой стены на соседних шагах. Это tunneling.

Для dynamic projectile можно настроить CCD motion threshold и swept sphere radius. Но текущее стрелковое оружие hitscan и использует raycast, поэтому CCD ему не нужен. CCD понадобится реальному летящему снаряду, обломку высокой скорости или торпеде с маленьким коллайдером.

39 Настоящий физический снаряд

Чтобы заменить hitscan летящим projectile, нужны отдельные данные и жизненный цикл.

```
WeaponSystem
    -> создаёт projectile Entity
    -> Transform и ProjectileComponent
    -> sphere или capsule rigid body
    -> initial velocity

ProjectileSystem
    -> lifetime
    -> contact enter
    -> DamageEvent
    -> удаление body из Bullet
    -> DestroyEntity
```

Такой снаряд не должен оставлять body в мире после уничтожения Entity. Также нужен запрет попадания в стрелка на первых шагах через group mask, ignored owner или отдельный collision rule.

40 Потокобезопасность

Текущий Bullet world следует считать ресурсом главного потока. Фоновая задача может готовить CPU геометрию, но не должна одновременно добавлять body, менять shape или выполнять запрос мира, пока основной поток делает `stepSimulation`.

Безопасная схема асинхронной подготовки:

1. worker вычисляет чистые массивы вершин или decomposition;
2. результат помещается в main thread queue;
3. главный поток создаёт Bullet shape и регистрирует body;
4. ECS получает готовый runtime resource.

41 Отладочная визуализация

Для физического бага полезно одновременно видеть:

- collision shapes;
- AABB;
- contact points и normals;
- velocity vectors;
- raycast и sweep paths;
- center of mass;
- group и mask выбранного объекта;
- active state и collision flags;
- источник последней синхронизации Transform.

Один цвет должен иметь одно значение. Например зелёный query дошёл без hit, красный hit, жёлтый отфильтрованный объект, синий нормаль поверхности.

42 Диагностика по симптомам

Симптом	Первая проверка	Вероятная причина
видимый объект не блокирует луч	зарегистрирован ли body и mask луча	нет shape body не в мире неверная группа
dynamic body замер	mass invMass activation state	тело static или спит
объект дрожит	кто пишет Transform до и после шага	два владельца или проникновение
коллайдер остался на старом месте	AABB и rebuild shape	scale или teleport без updateSingleAabb
персонаж застрял на ступени	три sweep этапа и slope normal	нет места сверху плохой offset высокий step
raycast попадает в стрелка	ignored object и userIndex	не тот collision object или Entity
constraint рвётся сразу	начальные frames и penetration	большая стартовая ошибка
объект проходит через стену	скорость размер fixed step CCD	tunneling или неправильный mask
после load физика отличается	порядок создания и runtime поля	сохранён указатель или не пересобран body
утечка после смены сцены	remove из world до delete	нарушен жизненный цикл

43 Чек лист создания физического объекта

1. Определи источник истины Transform.
2. Выбери самый простой подходящий shape.
3. Запиши единицы размера и массы.
4. Реши static dynamic или kinematic.
5. Рассчитай inertia только для dynamic body.
6. Назначь group и mask по таблице взаимодействий.
7. Запиши Entity в userIndex.
8. Зарегистрируй body в мире только после готовности владельца.
9. Добавь debug draw формы и AABB.
10. Определи обратный порядок удаления.
11. Проверь save и повторную загрузку.
12. Проверь объект при 30, 60 и 144 кадрах рендера.

44 Практические задания по Bullet

Задание 1 Луч

Нарисуй start, end, нормаль и Entity первого hit. Добавь объект между оружием и мишенью и предскажи результат до запуска.

Задание 2 Фильтры

Составь таблицу для player, world, trigger и projectile. Для каждой пары вычисли оба битовых условия вручную.

Задание 3 Тип тела

Создай одинаковую коробку как static, dynamic и kinematic. Запиши кто меняет её Transform и как она реагирует на impulse.

Задание 4 Sweep

Покажи debug линии трёх этапов step up. Сделай препятствие немного ниже и немного выше порога.

Задание 5 Constraint

Соедини два простых dynamic box, задай небольшой threshold и выведи applied impulse. Затем измени только начальное проникновение и сравни результат.

Задание 6 Жизненный цикл

Десять раз создай и удали тестовый body. После каждого цикла проверь число collision objects мира. Оно должно возвращаться к исходному значению.

45 Контрольные вопросы

1. Почему render mesh не является collision shape.
2. Кто владеет Transform dynamic body после fixed step.
3. Зачем нужен broadphase и когда обновлять AABB.
4. Чем raycast отличается от convex sweep.
5. Почему trigger использует no contact response, а не пустую mask.
6. Зачем ghost pair callback character controller.
7. Чем force отличается от impulse.
8. Почему triangle mesh backing storage живёт не меньше shape.
9. Почему hitscan оружию не нужен CCD.
10. В каком порядке удалять constraint, body, motion state и shape.

Если на каждый вопрос можешь ответить через конкретный путь текущего кода, Bullet перестал быть внешней чёрной коробкой и стал понятным слоем движка.

Physics geometry navigation и AI построчный runtime

Эта глава разбирает исполнение физики, изменение mesh, навигацию и NPC. Теория поведения из предыдущих глав здесь не повторяется: внимание на том, что делает код сверху вниз и кто имеет право менять состояние.

1 PhysicsSystem InitializeWorld

1. Метод проверяет, не создан ли world раньше.
2. PhysicsWorld или совместимый singleton создаёт collision configuration.
3. Dispatcher получает configuration.
4. Broadphase отвечает за грубый поиск потенциальных пар.
5. Solver разрешает контакты и constraints.
6. Dynamics world связывает эти части.
7. Gravity устанавливается один раз из engine settings.
8. Ошибка любой стадии вызывает cleanup уже созданных частей.

2 Создание collision shapes

2.1 Primitive shapes

1. Radius, half extents и height ограничиваются положительными минимумами.
2. Scale применяется согласованно: sphere использует консервативный максимальный scale, box — покомпонентный.
3. Capsule отдельно учитывает радиус и цилиндрическую высоту.
4. Новая shape возвращается вызывающему коду, который обязан определить ownership.

2.2 Convex hull

1. CPU vertices читаются из model.
2. Пустая геометрия завершает создание.
3. Каждая вершина умножается на scale.
4. Точки добавляются в `btConvexHullShape`.
5. Hull recalculates local AABB.
6. Margin задаётся после построения.

2.3 Triangle mesh

1. Требуются vertices и минимум один triangle.
2. Индексы проверяются по диапазону.
3. Triangles добавляются в `btTriangleMesh`.
4. Static body может использовать BvhTriangleMeshShape.
5. Dynamic concave body запрещён или заменяется convex approximation, потому что Bullet не рассчитан на обычное динамическое BVH mesh.
6. `triangleMesh` хранится рядом с shape и живёт не меньше неё.

2.4 Compound

1. Создаётся `btCompoundShape`.
2. Каждый element преобразуется в primitive или mesh child.
3. Local position/rotation формируют child transform.
4. Child добавляется в compound.
5. Ownership child shapes сохраняется для корректного удаления.

3 PhysicsSystem CreateRigidBody

1. Требуются TransformComponent и RigidBodyComponent.
2. Physics model выбирается между отдельной collision model и visual model.
3. NaN position отклоняется до попадания в Bullet.
4. Неположительный scale исправляется или приводит к отказу.
5. Interior collider исторической конструкции создаётся отдельным path, если это включено.
6. Compound path перебирает elements.
7. Обычный path выбирает shape через `shapeType`.
8. Effective mass становится нулевой для static/kinematic body.
9. Dynamic body вычисляет local inertia.
10. NaN inertia не передаётся Bullet.
11. Motion state создаётся с world transform.
12. Construction info получает mass, shape, inertia, friction и restitution.
13. Body создаётся и записывается в component.
14. CCD threshold и swept sphere radius включаются для быстрых объектов.
15. Static, kinematic и dynamic flags устанавливаются взаимоисключающе.
16. Collision group и mask определяют, с кем body сталкивается.
17. Неактивная ECS entity не добавляется в world до активации.

4 RefreshRigidBody

1. Старый body удаляется из world.
2. Motion state освобождается.
3. Body удаляется до shape.
4. Old shape и triangle mesh освобождаются.
5. Runtime pointers зануляются.
6. `CreateRigidBody` вызывается с текущими component settings.

Refresh нельзя вызывать внутри Bullet callback, когда world итерирует collision objects.

5 PhysicsSystem Update до шага

1. Character controllers синхронизируются с внешне изменёнными transforms.
2. Для каждого rigid body проверяется ActiveComponent.
3. Деактивация удаляет body из world и ставит `suspendedByActiveState`.
4. Повторная активация возвращает body с корректной transform.
5. Kinematic body каждый кадр получает ECS transform.
6. Dynamic body не перезаписывается transform без явного teleport, иначе физика потеряет собственное движение.
7. `ScaleChangedExternally` запускает refresh shape.
8. `TransformChangedExternally` для static body обновляет Bullet transform.
9. Forces, buoyancy и другие producers применяются до `stepSimulation`.

6 Историческая buoyancy часть

Она остаётся в текущем PhysicsSystem, хотя flooding исключён из учебника.

1. Проверяются active entity, rigid body и transform.
2. Bounds тела сравниваются с effective water level.
3. Полностью выше — submersion zero.
4. Полностью ниже — submersion one.
5. Пересечение интерполирует долю.
6. Buoyant force зависит от вытесненного объёма и submersion.
7. Drag уменьшает линейную и угловую скорость.

Если вода окончательно не нужна, этот блок лучше удалить из общего PhysicsSystem или превратить в подключаемый force module.

7 PhysicsSystem Update после шага

1. World делает фиксированные substeps через PhysicsWorld.
2. Dynamic body world transform читается из motion state.
3. NaN и слишком далёкие координаты отбрасываются как аварийная защита.
4. Position и rotation записываются в ECS TransformComponent.
5. Scale остаётся ECS-данными и не читается из Bullet.
6. Character ghost transforms синхронизируются отдельным helper.
7. Последние applied transforms сохраняются для обнаружения внешнего teleport в следующем кадре.

8 PhysicsRaycast

1. Start и end переводятся в `btVector3`.
2. Callback хранит collision mask.
3. Ignored entities или collision objects фильтруются.
4. `rayTest` возвращает ближайшее допустимое попадание.
5. Collision object user pointer переводится в ECS entity.
6. Hit point, normal и fraction копируются в engine struct.
7. Отсутствие world или hit возвращает пустой результат.

Один общий wrapper не даёт оружию, interaction и cover query по-разному трактовать Bullet.

9 MeshCutting math helpers

9.1 Vector helpers

`Length`, `Subtract`, `Normalize`, `Cross` и `Dot` делают операции над XMFLOAT3 без хранения SIMD-ссылок. `Normalize` проверяет почти нулевую длину, чтобы не получить NaN.

9.2 LerpPolyVertex

1. Position интерполируется по `t`.
2. UV интерполируется тем же `t`.
3. Normal, tangent и binormal интерполируются.
4. Направляющие векторы затем нормализуются.

Одинаковый `t` сохраняет непрерывность атрибутов на новой границе triangle.

9.3 2D geometry

1. Cut plane задаёт локальные оси, проецирующие triangle в 2D.
2. `SignedDistanceToEdge` определяет сторону линии.
3. `DistancePointSegmentSq` clamp-ит projection parameter от нуля до единицы.
4. `PointInTriangle2D` проверяет ориентации рёбер.
5. `CircleOverlapsTriangle2D` проверяет вершины, центр в triangle и расстояние до рёбер.

10 Polygon clipping и triangulation

1. Triangle превращается в polygon из трёх PolyVertex.
2. Каждая граница cut profile последовательно режет polygon.
3. Переход inside/outside создаёт интерполированную вершину.
4. Inside vertices сохраняются для удаляемой области.
5. Outside pieces сохраняются как будущая поверхность.
6. Polygon меньше трёх вершин отбрасывается.
7. Выпуклый polygon триангулируется веером.
8. Ориентация triangles сохраняется, чтобы normals не перевернулись.

11 RayModelHit

1. World ray переводится inverse world matrix в local coordinates model.
2. Local direction нормализуется.
3. AABB test быстро отбрасывает model.
4. Индексы читаются тройками.
5. Невалидный индекс пропускает triangle.
6. Ray-triangle test вычисляет local hit.
7. Hit переводится обратно в world.
8. World distance сравнивается с текущим ближайшим.
9. Normal разворачивается против луча, если triangle orientation смотрит внутрь.
10. Результат хранит entity, model, hit point, normal и distance.

12 Cut radius и глубина

1. Radius берётся относительно bounds model.
2. Минимум не даёт отверстию исчезнуть из-за масштаба.
3. Максимум не позволяет одному выстрелу удалить половину объекта.
4. World radius переводится в local radius с учётом scale.
5. Depth определяет, режется только передняя поверхность или сквозной слой.

13 Pending lock и damage history

1. `TryMarkCutPending(model)` проверяет global set под mutex.
2. Уже обрабатываемый model получает false.
3. Новый model добавляется и получает true.
4. `ClearCutPending` вызывается при любом завершении, включая ошибку submit.
5. RuntimeMeshDamageComponent хранит baseline vertices/indices и список cut operations.
6. Baseline записывается один раз до первого изменения.
7. Operation id позволяет позже отменить конкретный cut.

14 Detached islands

1. Welded positions квантуют координаты, объединяя почти одинаковые вершины.
2. Triangle adjacency строится по общим welded positions.
3. DFS/BFS находит connected components triangles.
4. Площадь каждой island суммируется.
5. Самая большая island считается основным объектом.
6. Маленькие detached islands помечаются на удаление.
7. Оставшиеся indices compact-ятся.
8. Неиспользуемые vertices удаляются через remap table.

Это геометрическая связность, не Bullet islands.

15 Apply cut CPU и GPU

1. Проверяется размер исходных buffers и защитный maximum indices.
2. Каждый triangle тестируется против cut volume.
3. Незатронутые triangles копируются как есть.
4. Затронутые режутся polygon algorithm.
5. Если изменений нет, операция возвращает false.
6. При необходимости удаляются detached fragments.
7. Пустая итоговая mesh отклоняется.
8. Новые immutable/dynamic D3D buffers создаются во временные pointers.
9. Только после успеха старые vertex/index buffers освобождаются.
10. CPU arrays заменяются тем же итогом.
11. Bounds пересчитываются.

16 Async cut

1. Main thread выполняет ray query и собирает immutable input snapshot.
2. Pending lock резервирует models.
3. Worker изменяет только CPU arrays.
4. Worker не вызывает D3D device context и не трогает registry.
5. Result помещается в main-thread queue.
6. Main thread проверяет жизнь entity и актуальность operation generation.
7. GPU buffers заменяются.
8. PhysicsSystem refresh-ит collision mesh, если он тоже был изменён.
9. Pending locks снимаются на всех paths.

17 RepairCut

1. Нулевой operation id отклоняется.
2. Ищется damage track, содержащий operation.
3. Если model pending, ремонт не начинается поверх cut.
4. Geometry восстанавливается из baseline.
5. Все оставшиеся cut operations повторно применяются кроме удаляемой.
6. Полученный mesh заменяет CPU/GPU buffers.
7. Collision body обновляется.
8. Operation удаляется из history только после успешного commit.

Таким образом repair не пытается «заклеить triangles назад» локально, а детерминированно воспроизводит оставшиеся повреждения.

18 NpcCommandSystem

18.1 Begin

1. Command revision увеличивается.
2. Type и active flag записываются.
3. Ошибки прошлой команды очищаются.
4. Navigation runtime сбрасывается.

18.2 ResumeAutomatic

1. Ручная команда очищается.
2. Brain возвращается automatic policy.
3. Route dirty заставляет navigation пересчитать путь.

18.3 Stop Wander Move Follow Repair

Каждый публичный метод сначала вызывает Begin с нужным type, затем записывает только свои параметры: target position, room или entity. Он не двигает NPC сам.

18.4 ForceRepath и ResetNavigation

1. Route очищается.
2. Index ставится в ноль.
3. `routeDirty = true`.
4. Stuck/no-progress state сбрасывается.

19 NpcUtilitySystem Update

1. Перебираются NPCComponent и NpcBrainComponent.
2. Читаются crew, combat и faction.
3. Faction combatant исключается из старой repair utility, даже если legacy `isHostile` false.
4. Inactive и off-duty NPC тоже недоступны.
5. Если такой NPC выполнял автоматический repair state, target очищается и state возвращается Idle.
6. Manual command не стирается.
7. Доступный NPC передаётся `EvaluateNpc`.

20 NpcUtilitySystem EvaluateNpc

1. Idle score задаёт небольшой baseline.
2. Best repair target начинается INVALID, score нулём.
3. Все breaches оцениваются, но claim другого NPC исключает цель.
4. Все damageable equipment оцениваются отдельно.
5. Entity, уже являющаяся breach, не оценивается второй раз.
6. Лучший score и target сохраняются.
7. Debug `repairScore` обновляется даже при manual command.
8. Активная manual command завершает automatic arbitration.
9. Уже ремонтирующий NPC сохраняет валидную цель, уменьшая дёрганье.
10. Смена target ставит `routeDirty`.
11. Repair побеждает Idle только при валидной цели и большем score.
12. Иначе target очищается и state становится Idle.

20.1 Score ремонта

1. Severity растёт с недостающим health, размером breach или потерей функции.
2. Distance уменьшает score.
3. Role affinity повышает задачи своей профессии.
4. Danger или combat pressure может понизить обычный ремонт.
5. Claim исключает дублирование.
6. Clamp удерживает сравнимый диапазон.

21 CrewTaskCoordinator

1. Update сначала удаляет невалидные claims.
2. Затем собирает repair candidates.
3. Crew list содержит active, alive и on-duty членов.
4. Hostile/combat NPC и manual commands пропускаются.
5. Валидное прежнее assignment сохраняется, если claim всё ещё принадлежит NPC.
6. Для свободного NPC каждый target получает score по distance, role и urgency.
7. Лучший target получает `CrewTaskClaimComponent` с assignee.
8. Brain получает repairTarget и MoveToRepair.
9. Snapshot обновляет counts для UI.

Claim — reservation. Utility score отвечает «что выгодно», Coordinator — «кто именно имеет право это делать».

22 NpcCombatSystem поиск цели

1. Внешний цикл перебирает NPCCombatComponent: это observer/attacker.
2. Observer получает NPC, brain, transform и faction.
3. Мёртвый, inactive или manual-command NPC пропускается согласно policy.
4. Внутренний цикл перебирает faction candidates.
5. `candidateEntity == npcEntity` исключает самого себя.
6. `AreEnemies(observerFaction, candidateFaction)` определяет отношение симметрично или через таблицу.
7. Candidate должен быть живым NPC или живым PlayerComponent, но не обязан иметь оба компонента.
8. Distance сравнивается с detection range.
9. FOV проверяет угол между forward и направлением на кандидата.
10. Physics line of sight подтверждает контакт.
11. Лучший target выбирается по score/distance, а не «последний в ForEach».
12. Target entity и last known position обновляются только при подтверждённом восприятии.
13. Потеря видимости не телепатически обновляет позицию.

23 NpcTacticalDecisionSystem

1. Threat distance sample сравнивается с прошлым и делится на `deltaTime`.
2. Отрицательная скорость расстояния означает, что враг приближается.
3. Manual selected style сразу назначает active style.
4. Auto style соблюдает `tacticCommitTimer`, чтобы не переключаться каждый кадр.
5. Dangerously close выбирает defensive/withdraw response.
6. Cautious NPC раньше реагирует на близкую угрозу.
7. Enemy pushing и under fire усиливают оборонительный выбор.
8. Высокая aggression склоняет к assault.
9. Низкая aggression — к defensive.
10. Style меняет desired distance, cover preference и search pressure, но не двигает NPC напрямую.

24 NpcCoverQuery статические точки

1. Перебираются cover point entities.
2. Rejected cover и чужая reservation исключаются.
3. Distance от NPC ограничивает стоимость пути.
4. Ray от target к нескольким высотам cover position проверяет реальную защиту.
5. Hit самого target не считается стеной.
6. Cover point должен лежать на доступной navigation area.
7. Для левой и правой стороны строятся peek candidates.
8. Peek должен видеть target и иметь место для capsule.
9. Score учитывает protection, path distance, preferred combat range, angle и style.
10. Лучший result хранит cover, hide, peek и score.

25 NpcCoverQuery dynamic cover

1. Если статическая точка не найдена и `allowDynamicCover`, начинается geometry query.
2. Bullet obstacles рассматриваются как потенциальные стены.
3. Candidate hide position строится с противоположной от target стороны obstacle.
4. Высоты 1.0, 1.6 и 0.55 проверяют разные уровни защиты.
5. Navmesh проецирует точку на достижимую поверхность.
6. Левый и правый peek строятся вдоль tangent obstacle.
7. Dynamic result получает virtual id/state, а не постоянную ECS cover point.
8. Debug rays сохраняют причины rejection.

26 NpcCoverSystem Update

1. Null world, NaN или неположительное время завершают update.
2. Перебираются combat NPC с tactical, brain, transform и health.
3. Timers уменьшаются с clamp до нуля.
4. Damage reaction превращается в under-fire memory даже при invulnerability.
5. Невалидный target освобождает cover state.
6. Existing reserved cover повторно проверяется против текущей target position.
7. `coverChangeRequested` освобождает reservation, временно отвергает старую точку и обнуляет decision timer.
8. NPC без cover вызывает CoverQuery только когда cooldown разрешает.
9. Успешный result резервируется и создаёт move target hide position.
10. При достижении hide position state становится InCover и начинается hide timer.
11. По окончании hide timer NPC идёт к peek position.
12. В peek state `exposed = true`, запускается exposure timer и разрешается атака.
13. Истечение exposure или попадание возвращает за cover.
14. Failed peek увеличивается только при действительно неудачной экспозиции.
15. Достижение лимита просит сменить cover.
16. Reload может удерживать NPC за cover дольше.

27 NpcCombatDecisionRules

1. `CanUseMovingFire` разрешает стрельбу только определённым movement states и видимой цели.
2. TakingCover может стрелять, если не находится в скрытой фазе.
3. Chasing разрешает огонь при сохранённом контакте.
4. `ShouldRelocateAfterFailedPeeks` сравнивает count с limit.
5. `NeedsNewAngle` требует новый угол при невидимой цели или подавлении согласно policy.
6. Чистые inline rules не меняют components, поэтому легко тестируются.

28 NpcRangedCombatSystem

1. Система получает combat target и WeaponComponent.
2. Weapon timers уже должны быть обновлены.
3. Target visibility проверяется в этот кадр.
4. Muzzle ray подтверждает, что ствол не упирается в стену.
5. Fire lane ray ищет союзника между muzzle и target.
6. Союз определяется фракцией, а не legacy `isHostile`.
7. Aim time накапливается только при устойчивой возможности стрелять.
8. Movement увеличивает spread или требует дополнительного aim time.
9. Reload request создаётся при пустом магазине или тактической возможности.
10. `WeaponSystem::TryFire` выполняет выстрел и урон.
11. Ranged status по-русски объясняет hold fire, reload, aim или fire.

29 NpcNavMeshSystem build

1. Geometry собирается из отмеченных surfaces, rigid bodies и static colliders.
2. Doors исключаются или учитываются отдельным connectivity rule.
3. Duplicate Bullet bodies удаляются set-ом.
4. World bounds определяют Recast grid.
5. Cell count ограничивается, защищая память.
6. Triangles rasterize-ятся в heightfield.
7. Walkable slope и agent height фильтруют поверхность.
8. Compact heightfield строит свободные spans.
9. Regions и contours выделяют связные области.
10. Poly mesh и detail mesh создают navigation polygons.
11. Detour data превращается в `dtNavMesh` и `dtNavMeshQuery`.
12. Ошибка любой стадии освобождает промежуточные Recast structures.

30 Navmesh cache и fingerprint

1. Geometry fingerprint хеширует bodies, shapes и quantized transforms.
2. До `nextGeometryCheck` старый mesh переиспользуется.
3. Совпавший fingerprint продлевает cache без rebuild.
4. Изменение fingerprint пересобирает room/world mesh.
5. Paths, относящиеся к перестроенной области, инвалидируются.

31 BuildPath и движение

1. Start и target проецируются на ближайшие polygons.
2. Если target не проецируется, fallback пытается использовать допустимую ближайшую точку.
3. `findPath` строит corridor polygon refs.
4. Неполный corridor получает ближайший достижимый endpoint.
5. `findStraightPath` создаёт world waypoints.
6. Cache хранит points, next index, room и target.
7. Достигнутые points пропускаются по threshold.
8. Финальная точка завершает path и сообщает arrival.

32 Dynamic avoidance

1. Nearby rigid bodies фильтруются по высоте персонажа.
2. Для obstacle вычисляются forward distance и lateral clearance.
3. Объект позади или достаточно сбоку не мешает.
4. Лучший blocking obstacle запоминается.
5. Side memory удерживает выбранную сторону некоторое время, предотвращая метание влево-вправо.
6. Candidate avoidance point проецируется на navmesh.
7. Слишком близкая точка отклоняется.
8. При отсутствии точки используется основной path direction.

33 NpcCrowdSpacing

1. Для пары NPC вычисляется XZ difference.
2. Почти нулевая дистанция получает стабильное запасное направление.
3. Внутри personal radius сила отталкивания растёт.
4. Result суммируется по соседям.
5. Максимальная correction ограничивается.
6. Spacing корректирует steering, но не меняет tactical goal.

34 NpcSquadCombatSystem

Group tactics рассматривается как отдельный необязательный слой.

1. Сначала собираются участники одной стороны.
2. Shared contact создаётся только после подтверждения наблюдателем.
3. Получатель данных не считает target видимой автоматически; он должен подтвердить контакт самостоятельно.
4. Roles назначаются с commit timer.
5. Reservations не дают двум союзникам занять одну cover point.
6. Assault, support, flank и defensive goals пишутся как предложения tactical layer.
7. Individual safety rules имеют право отклонить предложение.
8. Feature flag может отключить весь слой, сохранив одиночный AI.

35 NpcSimulationStudio

1. `Capture` перебирает NPC и строит compact snapshot.
2. Health читается из общего damageable или NPC data.
3. Combat snapshot сохраняет target, visibility, ammo и reload.
4. Tactical snapshot сохраняет order, reason, cover, timers и failed peeks.
5. Первый snapshot создаёт baseline без ложных событий.
6. Следующие сравниваются с предыдущим.
7. Изменение state создаёт decision event.
8. Command revision создаёт command event.
9. Target/visibility создаёт perception event.
10. Уменьшение health создаёт damage event.
11. Route change и no-progress threshold создают navigation events.
12. Deque ограничивается `MaxEvents`, удаляя самые старые записи.

35.1 QueueDamage

1. UI записывает request в отдельный buffer.
2. Pending flag сообщает runtime, что запрос есть.
3. `ConsumeDamageRequest` атомарно отдаёт один запрос и снимает pending.
4. Damage проходит обычный DamageSystem, а не тайно меняет health из UI.

35.2 ExportHistory

1. Файл открывается до изменения UI state.
2. Text export пишет одну запись на строку.
3. JSON escape обрабатывает кавычки, slash и control characters.
4. Russian translation применяется только к отображению, не к enum storage.
5. Ошибка открытия возвращает false.

36 AI система без конфликтов

Для каждого поля нужен один writer:

1. Perception пишет visible и last known facts.
2. Utility/goal layer выбирает намерение.
3. Tactical layer выбирает позиционный способ.
4. Navigation пишет путь и steering.
5. Character controller исполняет движение.
6. WeaponSystem владеет ammo, reload и shot cooldown.
7. Animation читает факты и пишет pose.
8. Studio только наблюдает и создаёт команды через обычные очереди.

Если две системы пишут `brain.state`, вводится priority/arbitration или разделяются поля `desiredState` и `executedState`.

37 Что рефакторить в этом блоке

1. Выделить buoyancy из PhysicsSystem как подключаемый force module.
2. Заменить global pending maps MeshCutting на scene-owned service.
3. Ввести scene generation для async cut/repair results.
4. Разделить NpcCoverSystem на evaluator и phase executor.
5. Сохранить pure decision rules без ECS для unit-тестов.
6. Вынести Recast build, cache и steering в три класса.
7. Сделать squad system producer-ом предложений, а не прямым владельцем individual state.

38 StateSystem исполнитель выбранного поведения

`StateSystem` отвечает на вопрос «как выполнить уже выбранную задачу». Utility, command и tactical systems выбирают намерение; StateSystem превращает его в movement, navigation и repair execution.

38.1 WorkZone helpers

1. `GetClosestPointInWorkZone` строит inverse world matrix зоны.
2. World position NPC переводится в local coordinates.
3. Каждая координата clamp-ится внутри куба от минус единицы до единицы.
4. Local point переводится обратно в world.
5. `TransformWorkZoneLocalPoint` выполняет только обратное преобразование для заранее выбранного sample.
6. `TryPickReachableWorkZonePoint` собирает ближайшую точку, центр, углы и середины граней.
7. Каждый candidate проецируется NavMeshSystem.
8. Непроецируемая точка в известной комнате отбрасывается.
9. Побеждает ближайшая достижимая XZ-позиция.

38.2 Update общий проход

1. Неположительный `deltaTime` запрещает движение и прогресс ремонта.
2. Перебираются brain и transform.
3. Inactive NPC получает нулевое movement и нулевой Bullet controller velocity.
4. Конфликтующий RigidBodyComponent удаляется: capsule controller остаётся единственным physics owner.
5. `EnsureComponent` создаёт недостающие movement/controller runtime components.
6. Полностью пустой controller создаётся через CharacterSpawnSystem.
7. Частично созданный controller не перестраивается поверх старых pointers.
8. NPC speeds копируются в общий CharacterMovement.
9. Animated mesh без AnimationComponent получает default component.
10. Current room определяется с предпочтением прошлой комнаты на дверном пороге.
11. `UpdateNpc` исполняет command или state.
12. Pair separation корректирует направление, но не телепортирует transform.
13. Portal collision bypass выключается, если в этом кадре его никто не запросил.
14. `UpdateStuckStatus` сравнивает фактический прогресс с желаемым движением.

38.3 EnsureComponent

1. Сначала запрашивается существующий T.
2. При null добавляется default T.
3. Pointer запрашивается снова, потому что storage мог перераспределиться.
4. Возвращается актуальный адрес.

38.4 UpdateNpc arbitration

1. Active manual command имеет абсолютный приоритет и завершает метод через return.
2. Устаревшие combat states без target и memory переводятся в search/wander policy.
3. Switch по brain state вызывает узкий executor: idle, wandering, chasing, taking cover, attacking, repair или waiting door.
4. Executor меняет movement intent и status, но Bullet transform меняется позже controller system.
5. Неизвестное state безопасно останавливает движение и пишет диагностику.

38.5 Manual commands

1. Stop обнуляет движение.
2. Wander использует тот же устойчивый wander runtime, что automatic search.
3. MoveToPosition вызывает NavigateTo target position.
4. MoveToRoom сначала строит межкомнатный route.
5. FollowEntity каждый кадр читает актуальную target position, но держит follow distance.
6. RepairEntity подтверждает валидность repair target до движения.
7. Завершение или ошибка записывает command result и возвращает automatic control только по явному правилу.

38.6 Межкомнатный BFS

1. Start и goal rooms являются узлами графа.
2. Doors дают рёбра только при валидных связях.
3. Queue обходит комнаты по слоям.
4. `RouteNode` хранит previous room и via door.
5. Goal раскручивается назад в список steps.
6. Список разворачивается в прямой порядок.
7. Закрытая дверь переводит NPC в WaitingForDoor и создаёт interaction/open request.
8. После прохода порога current route index увеличивается.

38.7 NavigateTo

1. Для текущей room запрашивается NavMesh path или reachable point.
2. Следующий waypoint превращается в XZ direction.
3. Arrival threshold возвращает Arrived.
4. Door wait возвращает Waiting.
5. Ошибка path и невозможность fallback возвращают Blocked.
6. Обычное движение возвращает Moving.
7. Desired speed выбирает walk/run по distance, stamina и tactical urgency.

38.8 Repair execution

1. Repair target повторно проверяется каждый кадр.
2. Work zone или ближайшая reachable point задаёт позицию выполнения.
3. Пока NPC далеко, StateSystem только движет его.
4. В радиусе движения останавливается и начинается progress.
5. JobSystem получает тяжёлую mesh repair работу, если она нужна.
6. Claim остаётся за NPC до успеха, отмены или невалидной цели.
7. Успех очищает target, state и reservation; Utility выберет следующую задачу позднее.

38.9 Stuck status

1. Если NPC хочет двигаться, сравнивается текущая позиция с последним sample.
2. Достаточный прогресс сбрасывает no-progress timer.
3. Недостаточный прогресс накапливает timer.
4. Первый threshold просит repath.
5. Повторный длительный stuck меняет status/blocked reason и разрешает higher layer выбрать другую цель.
6. Когда движения нет по замыслу, stuck timer не растёт.

NPC в DX Framework: от данных ECS до ремонта, поиска и выстрела

Эта глава написана по исходникам проекта на 29 октября 2026 года. Её цель — дать возможность восстановить в голове причинно следственную цепочку: **какой проход читает компонент, какое решение принимает, что записывает и какой следующий проход использует эту запись**. Здесь нет абстрактного «обычного игрового AI», выданного за уже реализованный код. Формулировка «сейчас» означает поведение проверенных исходников. Формулировка «предложение» означает изменение, которое ещё нужно реализовать.

Соглашения для чтения: `Entity` — идентификатор, а не адрес объекта; `INVALID_ENTITY` — отсутствие выбранной сущности; `dt` или `deltaTime` — секунды одного шага симуляции; `TransformComponent::position` — мировая позиция; координаты `x,z` задают горизонтальную палубу, `y` — высоту. Ссылки вида StateSystem.h:1629 (../StateSystem.h#L1629) ведут к проверяемым строкам. После изменения кода номера строк могут сдвинуться, поэтому ищите также названия функций.

1. Сначала поймём один игровой кадр

NPC не живёт внутри одного класса `Npc`. Его поведение составлено из ECS компонентов и систем, проходящих по ним. Это ключ к переучиванию кода: **компонент хранит факт или намерение; система превращает входы в новые факты или намерения**. Например, `NpcUtilitySystem` может записать `brain.repairTarget`, но не передвигает капсулу и не чинит объект. `StateSystem` превращает эту цель в направление движения или прогресс ремонта. `CharacterControllerSystem` передаёт направление в Bullet. `RepairSystem` завершает изменение игрового мира.

В ApplicationClass.cpp:1448–1551 (../src/Core/ApplicationClass.cpp#L1448) при `isPlayMode && simulationRunning` цепочка начинается с ввода игрока. Дальше порядок таков:

1. `WeaponSystem::UpdateTimers` один раз уменьшает таймеры всех `WeaponComponent` и сбрасывает `triggerHeld`.
2. `m_npcUtilitySystem.Update` считает локальную пользу ремонта для каждого подходящего мирного NPC.
3. `CrewTaskCoordinator::Update` очищает старые заявки, назначает ремонтные цели и ставит `CrewTaskClaimComponent` на саму цель.
4. `NpcCombatSystem::Update` выбирает возможного врага, проверяет сектор и физический луч, обновляет личную память наблюдения.
5. `NpcCoverSystem::Update` выбирает или проверяет укрытие, выставляет тактический `moveTarget` и состояние `TakingCover`, когда это требуется.
6. `StateSystem::Update` исполняет ручную команду, ремонт, погоню, поиск или тактическое движение. Он пишет намерение в `CharacterMovement`.
7. `CharacterFacingSystem`, `CharacterControllerSystem`, `AnimationController` обрабатывают разворот, физическое движение и позу. `DamageSystem::Update` в районе 1549 (../src/Core/ApplicationClass.cpp#L1549) применяет **накопленную до этого момента** очередь событий урона.
8. После обновления позы и физики ApplicationClass.cpp:1757–1769 (../src/Core/ApplicationClass.cpp#L1757) выполняются выстрел игрока и `NpcRangedCombatSystem::Update`. Их `WeaponSystem::TryFire` добавляет событие в очередь `DamageSystem`.

Следствие пункта 8: событие от выстрела в конце данного кадра обычно будет применено следующим вызовом `DamageSystem::Update`, то есть на следующем шаге симуляции. Это **очередь событий**, а не немедленное `target.health -= damage`. В том же конце кадра уже виден результат трассировки луча и оружейный кулдаун. Реакция здоровья и `underFireTime` наступает при обработке очереди. WeaponSystem.h:119–155 (../src/Gameplay/Combat/WeaponSystem.h#L119), DamageSystem.cpp:104–113 (../src/Gameplay/Combat/DamageSystem.cpp#L104).

В CrewTaskCoordinator.h:107–109 (../src/Gameplay/AI/CrewTaskCoordinator.h#L107) сохранился комментарий «вызывать до Utility/State». Реальный вызов в `ApplicationClass` ставит координатор **после Utility и до State**. Читая код, доверяйте месту вызова, а комментарий считайте устаревшей памяткой. Важная практическая причина: `Utility` даёт предварительный лучший `repairTarget`, но координатор может заменить его окончательным назначением, и `StateSystem` в том же кадре пойдёт уже к нему.

В ApplicationClass.cpp:1494–1504 (../src/Core/ApplicationClass.cpp#L1494) вызов `NpcSquadCombatSystem::Update` закомментирован. Поля `squadId`, `NpcSquadRole` и связанные типы существуют, но текущий запущенный боевой цикл **не назначает роли прикрывающего и обходящего через этот проход**. Индивидуальные укрытия, короткие очереди и поиск работают через другие системы. Не приписывайте им координацию отряда только потому, что поля отряда есть в компоненте.

1.1. Как читать ECS вызов `ForEach`

В ECSRegistry.h:45–59 (../src/ECS/ECSRegistry.h#L45) `ForEach<T1,T2>` итерирует пул первого типа и для каждой сущности проверяет наличие второго. Он не гарантирует порядок ID. `GetComponent<T>(entity)` возвращает указатель либо `nullptr`; `HasComponent` отвечает только на наличие; `IsAlive` проверяет, что ID ещё зарегистрирован как живой. `ActiveComponent` в этом проекте служит отдельным игровым разрешением участвовать в симуляции. Живая `Entity` может быть неактивной. Поэтому типичная

охрана имеет несколько проверок: ID существует, компоненты есть, Entity активна и здоровье больше нуля.

Ссылки из `GetComponent` и параметров `ForEach` привязаны к хранилищу. После `RemoveComponent<T>` или `DestroyEntity` соответствующая ссылка недействительна. Добавление компонента в **тот же** пул во время его обхода тоже опасно: `ComponentManager` может перераспределить контейнер. Координатор сначала собирает ID экипажа в `vector`, и уже затем в отдельном цикле добавляет отсутствующий `CrewMemberComponent`: это правильная граница между чтением и структурной мутацией. CrewTaskCoordinator.h:130–151 (../src/Gameplay/AI/CrewTaskCoordinator.h#L130). В ремонте пробоина уничтожается при завершении, поэтому `StateSystem` после `ApplyRepairProgress` не продолжает читать старый `RepairableComponent*`, а обрабатывает результат и очищает свои ID. StateSystem.h:1909–1932 (../StateSystem.h#L1909).

1.2. Два вида времени

`deltaTime` участвует в движении, прицеливании, таймерах и прогрессе ремонта. Скорость 2 единицы/с при `dt=0.1` означает желаемый путь порядка 0.2 единицы за кадр до коллизий. `worldTime` в `CrewTaskCoordinator` используется как временная метка назначения, а не как множитель скорости. Поэтому `assignedAtSeconds` не надо увеличивать каждый кадр. CrewTaskCoordinator.h:104–110 (../src/Gameplay/AI/CrewTaskCoordinator.h#L104). `StateSystem::Update` сразу возвращается при `deltaTime <= 0`, так что пауза не увеличивает ремонт. StateSystem.h:212–217 (../StateSystem.h#L212).

2. Карта данных: кому принадлежит какое состояние

2.1. Сущность, здоровье, фракция

`NPCComponent` хранит имя, описание, `health`, `maxHealth` и старый флаг `isHostile`. `NpcBrainComponent` хранит решение и память маршрута. Разделение видно в NPCComponents.h:601–650 (../NPCComponents.h#L601): здоровье — свойство персонажа, а состояние маршрута — рабочее состояние AI. Если заменить `NpcBrainComponent`, не следует терять здоровье. Если NPC получил урон, не нужно менять маршрут там же, если только тактическая система не получила специальный сигнал.

Боевой выбор фракций находится в NPCComponents.h:51–89 (../NPCComponents.h#L51). `GetFactionRelation(a,b)` даёт `Friendly` для одинаковых `FactionId`, `Neutral`, если хотя бы одна сторона `Neutral`, и `Hostile` для прочих разных фракций. `AreEnemies` — короткая проверка именно `Hostile`. Старый `isHostile` ещё исключает NPC из мирной repair utility и координатора, но боевой поиск и стрельба проверяют `FactionComponent` пары стрелок–цель. Поэтому для нового противника нужно настроить фракцию, а не надеяться только на `isHostile=true`.

Есть две схемы здоровья цели. Игрок имеет `PlayerComponent::health`, обычный NPC — `NPCComponent::health`; устройство — `DamageableComponent`. `DamageSystem` отдельно обрабатывает игрока, NPC без `DamageableComponent`, затем устройство. Если на NPC одновременно есть `NPCComponent` и `DamageableComponent`, в конце обработка общего `Damageable` синхронизирует `npc.health` с ним. DamageSystem.cpp:196–211 (../src/Gameplay/Combat/DamageSystem.cpp#L196), DamageSystem.cpp:331–339 (../src/Gameplay/Combat/DamageSystem.cpp#L331). При добавлении нового типа персонажа это место придётся осознанно расширять: иначе восприятие не сочтёт его живой целью, даже если на нём есть фракция и Transform.

2.2. `NpcState` — итог текущего шага; `NpcCommand` — внешний приоритет

Перечисление NPCComponents.h:92–109 (../NPCComponents.h#L92) содержит `Idle`, `Wandering`, `MoveToRepair`, `Repairing`, `WaitingForDoor`, `Chasing`, `Searching`, `Attacking`, `TakingCover`, `InCover` и другие состояния. Это не очередь работ. `state` означает, какую ветку `StateSystem::UpdateNpc` выполнит сейчас; несколько систем вправе переписать его за один кадр по описанному порядку. Например, `Utility` пишет `MoveToRepair`, но `NpcCombatSystem` при подтверждённом контакте может записать `Chasing`. Для задач экипажа важно понимать, что координатор успевает поставить `repairTarget` до восприятия боя, но текущий боевой NPC обычно вообще исключён из мирной utility по фракционному признаку.

`NpcCommand::type` — другой слой. Значение `None` означает автономию; `Stop`, `Wander`, `MoveToPosition`, `MoveToRoom`, `FollowEntity`, `RepairEntity` задают ручной приказ. NPCComponents.h:117–126 (../NPCComponents.h#L117). `IsActive()` проверяет `type != None`; `Clear()` стирает целевые ID и повышает `revision`. NPCComponents.h:167–200 (../NPCComponents.h#L167). `revision` передаётся в NavMesh кеш как часть ключа пути. Если меняется приказ, старый маршрут не обязан оставаться пригодным.

Почти все автономные системы проверяют активную команду. `StateSystem::UpdateNpc` делает это первым условием и немедленно `return` после `UpdateManualCommand`. StateSystem.h:395–413 (../StateSystem.h#L395). Это правило сильнее текущего `state`: даже если другой проход случайно поставил `Repairing`, активная команда `Stop` всё равно остановит движение. `Stop`, `Wander`, `FollowEntity` остаются до отмены; `MoveToPosition`, `MoveToRoom`, `RepairEntity` сами очищаются после выполнения. Если цель Follow потеряла Transform, система показывает `Blocked`, но команду не удаляет: сцену можно исправить и продолжить. StateSystem.h:1345–1458 (../StateSystem.h#L1345).

2.3. Движение, двери, рабочая зона

`NPCMovementComponent` задаёт авторские настройки ходьбы 2, бега 4.5 единицы/с, порог бега 8, обычный допуск прибытия 0.45, дистанции открытия и ожидания двери 2.25 и 1.25, глубину выхода за дверь 1.5. NPCComponents.h:616–641 (../NPCComponents.h#L616). `StateSystem` переносит эти настройки в `CharacterMovement` и пишет `walkDirection`, `currentSpeed`, `isMoving`, `isRunning`, `locomotionState`. Поле `canFly` сейчас не влияет на `MoveTowards`: вертикальная компонента направления обнуляется.

В `NpcBrainComponent` лежат `currentRoom`, `navigationTargetRoom`, `doorRoute`, `doorRouteIndex`, `activeDoor`, `routeDirty`, `repathTimer`, параметры ремонта и диагностические строки. Это не просто «большой компонент»; эти поля отвечают на конкретные вопросы: где я сейчас, куда хочу, какие двери запланированы, какую прохожу и нужно ли перестроить путь. NPCComponents.h:646 и далее (../NPCComponents.h#L646). `WorkZoneComponent` на отдельной Entity указывает `owner` оборудования. `StateSystem` ищет её и подбирает точку на поверхности, достижимую по NavMesh, если он доступен. RepairComponent.h:17–20 (../src/Gameplay/Repair/RepairComponent.h#L17), StateSystem.h:135–203 (../StateSystem.h#L135).

2.4. Бой, память, тактика и оружие

`NPCCombatComponent` в NPCComponents.h:475–599 (../NPCComponents.h#L475) хранит `targetEntity`, `targetVisible`, `visibleAimPoint`, радиус зрения и атаки, таймеры, последнее **лично увиденное** положение, наблюдавшуюся скорость и план поиска. `visibleAimPoint` актуальна только при `targetVisible=true`; не используйте старую точку без проверки. `NpcTacticalComponent` хранит авторский и активный стиль, позицию укрытия, позицию выглядывания, таймеры фаз, короткую очередь стрельбы на ходу, счётчик неудачных выглядываний и диагностический статус. NPCComponents.h:305–395 (../NPCComponents.h#L305). `WeaponComponent` находится на Entity оружия; `WeaponSystem::GetEquippedWeaponEntity(registry, owner)` разрешает владельца в оружие. `WeaponOwnerComponent` в событии урона позволяет обратное разрешение оружия в стрелка.

Не смешивайте память наблюдения и текущий Transform цели. Сам `NpcCombatSystem` смотрит на текущую геометрию кандидата, чтобы **проверить**, может ли NPC его увидеть. Но после потери прямой видимости `StateSystem` идёт к `lastKnownPosition`, а `NpcCoverSystem` оценивает укрытие относительно неё. Новую координату скрытой цели эти слои не записывают. NpcCombatSystem.h:373–379 (../NpcCombatSystem.h#L373), NpcCoverSystem.h:199–229 (../NpcCoverSystem.h#L199).

3. Мирный NPC: как задача ремонта превращается в назначение

3.1. `NpcUtilitySystem::Update`: допуск к автоматике

Сигнатура `void Update(ECSRegistry& registry, float deltaTime)` в NpcUtilitySystem.h:54–104 (../NpcUtilitySystem.h#L54) показывает два входа, но `deltaTime` пока намеренно не используется: score строится из текущего мира. `ForEach<NPCComponent,NpcBrainComponent>` получает каждую пару данных. Внутри дополнительно читаются `CrewMemberComponent`, `NPCCombatComponent`, `FactionComponent`. Комбинация боевого компонента с ненейтральной фракцией считается боевым NPC и выводится из старой crew utility. Неактивная сущность, `isHostile` и член экипажа `onDuty=false` также не получают автоматический ремонт. Для них система очищает диагностический score и при отсутствии ручного приказа приводит автоматическое ремонтное состояние к бездействию. Это защита от случайного «дёргания» боевого персонажа между погоней и пробоиной.

Вызов `EvaluateNpc(registry,npc,brain)` происходит только для допущенного NPC. Его выход — изменения `brain.idleScore`, `brain.repairScore`, `brain.repairTarget`, `brain.routeDirty`, `brain.state`, `brain.status`. Он **не добавляет заявку** и не изменяет `RepairableComponent`. Это важно для трассировки: увидев `brain.repairScore=0.8`, нельзя заключать, что цель уже принадлежит этому NPC.

3.2. `EvaluateNpc`: два прохода и арбитраж

В NpcUtilitySystem.h:115–301 (../NpcUtilitySystem.h#L115) базовый `idleScore` принудительно ставится в 0.10. Затем код перебирает все `HullBreachComponent`, считает оценку, запоминает максимум; отдельно перебирает `DamageableComponent`, пропуская объекты, которые одновременно являются пробоинами. Перед оценкой обоих видов проверяется `CrewTaskCoordinator::IsClaimedByOther`, чтобы предварительная utility не отправляла второго NPC на чужую заявку. Внутренний цикл не создаёт маршрут и не вызывает Bullet. Его результат — ID одного лучшего кандидата и численная оценка.

`CalculateBreachRepairScore` в NpcUtilitySystem.h:406–523 (../NpcUtilitySystem.h#L406) сначала проверяет наличие пробоины, `RepairableComponent`, Transform NPC и Brain; запечатанная или помеченная `repaired` пробоина получает 0. Далее считается трёхмерное расстояние `sqrt(dx²+dy²+dz²)`. Если `detectionRange>0` и дистанция больше диапазона, оценка 0. В пределах диапазона `distanceScore = 1 - clamp(distance/detectionRange,0,1)`, `severity = clamp(leakRate/0.15,0.15,1)`, итог `clamp(0.75*severity + 0.25*distanceScore,0,1)`. Коэффициенты выражают политику: интенсивность утечки весит втрое больше близости. При расстоянии 6 из 30 и `leakRate=0.12` получаются `distanceScore=0.8`, `severity=0.8`, итог `0.75*0.8+0.25*0.8=0.8`.

`CalculateEquipmentRepairScore` в NpcUtilitySystem.h:532–702 (../NpcUtilitySystem.h#L532) требует `Damageable`, `Repairable`, оба Transform и Brain; полное здоровье либо `repaired=true` даёт 0. `damageScore` — доля потерянного здоровья `1 - clamp(health/maxHealth,0,1)`. Тип устройства задаёт `importance`: обычное 0.40, генератор 1.0, кислородный генератор 0.90, свет 0.15. Итог `clamp(0.45*damageScore + 0.40*importance + 0.15*distanceScore,0,1)`. Для генератора с 50/100 HP на расстоянии 6 из 30 получаем `0.45*0.5+0.40*1+0.15*0.8=0.745`. Эта таблица зашита в utility; если добавить новый критичный тип

оборудования, он получит 0.40, пока вы не расширите правило. Формула координатора другая, поэтому debug `brain.repairScore` не обязан равняться `CrewTaskClaimComponent::priority`.

После сканирования `brain.repairScore` сохраняется для диагностики. Активная ручная команда останавливает дальнейшую запись состояния: score всё равно виден в UI, но команду нельзя перехватить. Если NPC уже `Repairing` и цель ещё пригодна, цель сохраняется. Иначе при новом лучшем `repairTarget` ставится `routeDirty=true`; score больше `idleScore` переводит в `MoveToRepair`, иначе `Idle`. NpcUtilitySystem.h:219–301 (../NpcUtilitySystem.h#L219). Нулевой score не даёт автоматическую работу. Важная особенность: utility рассматривает местную выгодность, но окончательное распределение происходит следующим проходом.

3.3. `CrewTaskCoordinator::Update`: одна цель — один исполнитель

Вход `Update(ECSRegistry& registry,double worldTime)` не получает `dt`. В начале CrewTaskCoordinator.h:110–138 (../src/Gameplay/AI/CrewTaskCoordinator.h#L110) сбрасывается снимок телеметрии, удаляются негодные заявки, собираются незавершённые цели и уже занятые ID. Вектор экипажа строится через `ForEach<NPCComponent,NpcBrainComponent>` и сортируется по Entity ID. Сортировка делает назначение повторяемым при одинаковых входах, несмотря на порядок unordered хранилища. Внутри цикла при отсутствии `CrewMemberComponent` он добавляется с настройками по умолчанию. Это значит, что созданный в редакторе мирный NPC без явной роли всё равно может быть назначен на ремонт.

В CrewTaskCoordinator.h:139–170 (../src/Gameplay/AI/CrewTaskCoordinator.h#L139) неактивный, `isHostile`, `onDuty=false` или управляемый вручную персонаж теряет старую заявку. При ручной команде автоматическое состояние не перезаписывается. Если назначение уже существует и валидно, `KeepExistingAssignment` оставляет его. Эта «липкость» важна: без неё два ремонтника могли бы каждый кадр менять цели при малом изменении расстояния. При смене ID `brain.repairTarget` ставится `routeDirty`, а не перестраивается путь прямо в координаторе.

Здесь есть точная граница между двумя проходами, которую легко пропустить: `NpcUtilitySystem` исключает также сочетание `NPCCombatComponent` с ненейтральной фракцией, а `CrewTaskCoordinator` **не повторяет этот `factionCombatant` фильтр**; он проверяет старый `isHostile`. Значит, боевой персонаж с ненейтральной фракцией, `NPCCombatComponent`, но `isHostile=false` может не участвовать в utility и всё же получить ремонтную заявку от координатора, если он активен и на дежурстве. Для нового вражеского NPC явно задавайте согласованную конфигурацию; для развития кода лучше вынести общий предикат «может работать в экипаже» в одну функцию и вызывать из обоих проходов. Это предложение, а не уже существующая единая политика.

Для свободного NPC цикл по незанятым кандидатам вычисляет `CalculateAssignmentScore` и выбирает только результат выше `brain.idleScore`. При равном score используется меньший ID цели. На выбранную цель ставится `CrewTaskClaimComponent{assignee,type,priority,claimedAtSeconds}`; на NPC пишутся `member.assignedTask`, время, `brain.repairTarget`, `routeDirty`, `MoveToRepair` и статус. `claimedTargets.insert` сразу не даёт следующему NPC этого же кадра занять цель. CrewTaskCoordinator.h:172–220 (../src/Gameplay/AI/CrewTaskCoordinator.h#L172). Инвариант в нормальном проходе: для одной ремонтной цели один `CrewTaskClaimComponent`, и назначенный NPC указывает на неё через `assignedTask`.

Формула координатора CrewTaskCoordinator.h:382–446 (../src/Gameplay/AI/CrewTaskCoordinator.h#L382) **отличается** от формулы utility. Для пробоины серьёзность `clamp(leakRate/0.15,0.2,1)`, для оборудования — доля утраченного HP. Дистанция снова трёхмерная; за `detectionRange` оценка 0. Профессия меняет множитель: инженер 1.25, Operations 1.05,

Security 0.72, Medic 0.62, Generalist 0.90. Operations получает ещё 0.15 для генератора или кислородного генератора. `repairSkill` ограничен диапазоном 0.25–2. Итог:

```
score = clamp((0.65 * severity + 0.35 * distanceScore)
              * roleAffinity * repairSkill, 0, 2)
```

Например, инженер с `repairSkill=1.5`, в 6 единицах от пробоины `leakRate=0.12`, с `detectionRange=30`: `severity=0.8`, `distanceScore=0.8`, базовый score 0.8, итог `0.8*1.25*1.5=1.5`. Он превосходит `idleScore=0.10`. Второй такой же NPC уже увидит цель в `claimedTargets` и должен искать другую. Обратите внимание: оценка использует **геометрическое расстояние**, не длину реального маршрута через двери. Поэтому близкая через стену цель может оказаться назначенной раньше доступной, но более далёкой. Это текущий компромисс алгоритма, а не доказательство достижимости.

`CleanupInvalidClaims` в CrewTaskCoordinator.h:281–308 (../src/Gameplay/AI/CrewTaskCoordinator.h#L281) удаляет заявку, если цель больше не нуждается в ремонте либо исполнитель исчез, выключен, снят с дежурства или получил ручную команду. Удаление делается во втором цикле после сбора ID, чтобы не менять пул во время его перебора. `completedTasks` здесь увеличивается при невалидной цели с совпадающим `assignedTask`; отдельно `RepairSystem` увеличивает его **до уничтожения** успешно закрытой пробоины. Поэтому при аналитике не следует считать `completedTasks` безусловно точным числом только успешных окончаний для всех способов удаления целей. RepairSystem.h:660–690 (../src/Gameplay/Repair/RepairSystem.h#L660).

4. Исполнение ремонта почти по строкам

4.1. `StateSystem::Update`: подготовка физического исполнителя

`StateSystem::Update(ECSRegistry&,float,JobSystem&)` в StateSystem.h:212–357 (../StateSystem.h#L212) проходит по `NpcBrainComponent` и `TransformComponent`. Это шире, чем список `NPCComponent`: любая Entity с мозгом и Transform может дойти до этого кода. Поэтому при добавлении компонентов в редакторе следите за целостностью сущности, а не только за именем класса.

Первые строки защищают паузу и неактивность. Для неактивного NPC система обнуляет `CharacterMovement` и, если есть готовый Bullet контроллер, его walk direction и velocity interval. Неактивность сама по себе не удаляет Entity. Далее проверяется конфликт `RigidBodyComponent` с `btKinematicCharacterController`: оба владельца физики не должны одновременно переписывать один Transform. Код удаляет `RigidBodyComponent` и пишет диагностику. Это изменение состава Entity происходит до получения нижележащих указателей на движение и контроллер.

`EnsureComponent<CharacterMovement>`, `EnsureComponent<CharacterController>`, `EnsureComponent<NPCMovementComponent>` добавляют недостающие runtime компоненты. Контроллер бывает полностью готовым (`controller`, `ghostObj`, `capsuleShape` все есть), полностью пустым либо частично построенным. Пустой спавнится `CharacterSpawnSystem::Spawn`, частичный не пересоздаётся поверх старых Bullet объектов. После этого настройки NPC нормализуются: скорость ходьбы не ниже нуля, бег не ниже ходьбы. Если модель умеет анимации, недостающий `AnimationComponent` добавляется автоматически. Эти шаги важны при странном поведении созданного через редактор NPC: «мозг» сам по себе не создаёт модель и корректный скелет, но часть инфраструктуры движения система действительно создаёт.

Текущая комната определяется каждый кадр `FindContainingRoom` с предпочтением прежней комнаты в перекрывающихся bounds двери. Затем `UpdateNpc` заполняет намерение. После него отдельное разведение капсул добавляет локальное steering направление, если два живых активных NPC слишком близко. Оно не телепортирует Transform, а меняет `walkDirection`. Если в текущем кадре не требовался

временный обход коллизии дверного портала, он отключается. Последним вызывается `UpdateStuckStatus`, который сравнивает физический сдвиг Transform после прошлых шагов, а не просто факт желаемого движения. StateSystem.h:297–355 (../StateSystem.h#L297).

4.2. `UpdateNpc`: диспетчер состояний

Сигнатура StateSystem.h:395–403 (../StateSystem.h#L395) принимает Registry, ID, ссылки на Brain, Transform и `CharacterMovement`, константные настройки NPC движения, `deltaTime` и `JobSystem`. Она **не возвращает значение**: выходом являются изменённые компоненты и, для ремонта, вызов `RepairSystem`. Первым проверяется ручная команда. Ниже `switch(brain.state)` направляет `MoveToRepair`, `Repairing` и `WaitingForDoor` в одну функцию `UpdateRepairTask`; это позволяет после открытия двери продолжить ту же работу без нового назначения. `TakingCover` идёт к `tactical->moveTarget`; `Chasing` идёт к последней увиденной точке; `Searching` исполняет план поиска; `Attacking` останавливает нижнюю часть тела, разворачивает корпус и готовит атаку. StateSystem.h:453–538 (../StateSystem.h#L453).

Почему `WaitingForDoor` не самостоятельная «задача»? Потому что дверь — промежуточная фаза **любого** маршрута, а не новая цель экипажа. При каждом кадре `UpdateRepairTask` сначала вновь проверит, что цель всё ещё нужна, и затем снова вызовет `NavigateTo`. Если дверь теперь открыта, маршрут продолжится. Так ремонтник не может бесконечно «ждать старую дверь», когда утечку уже закрыли другим способом.

4.3. `UpdateRepairTask`: вход и две формы цели

Сигнатура StateSystem.h:1629–1639 (../StateSystem.h#L1629) включает `Entity target` и `bool manualCommand`. Один и тот же механизм обслуживает автоматическое назначение и ручной `RepairEntity`; различается только способ завершить работу. На входе по `target` берётся `RepairableComponent*`. Отсутствие компонента либо `repaired=true` немедленно завершает ручную команду через `CompleteManualCommand` или очищает автоматический `brain.repairTarget`, переводит в `Idle`, помечает маршрут грязным и останавливает ходьбу. Никакой прогресс в таком кадре не начисляется.

Дальше функция локально создаёт `repairPosition`, `repairRoom`, `validTarget`, `targetType`. Для пробоины она требует `HullBreachComponent` и `!sealed`, берёт `worldPosition` и `room`. Для оборудования берёт `DamageableComponent`, требует `health < maxHealth`, затем `TransformComponent`. В качестве запасной точки служит Transform самого устройства. `RoomMembershipComponent` даёт комнату, а `FindRepairZoneForOwner` может найти отдельный `WorkZoneComponent`; тогда `TryPickReachableWorkZonePoint` выберет точку на рабочей зоне. StateSystem.h:1671–1776 (../StateSystem.h#L1671). Порядок `if breach ... else if damageable ...` означает, что сущность с обоими компонентами трактуется как пробоина. Это согласовано с пропуском двойного учёта в utility.

`TryPickReachableWorkZonePoint` в StateSystem.h:135–203 (../StateSystem.h#L135) сначала пробует ближайшую точку в зоне, центр и восемь локальных образцов, переведённых в мир. Если известна комната, каждый кандидат должен спроецироваться через `NpcNavMeshSystem::FindReachablePoint` на связный остров сетки. Из успешных берётся ближайший к NPC по горизонтали. Если комната неизвестна, допускаются исходные точки. Если ни один кандидат не подходит при известной комнате, функция возвращает `false`, и остаётся fallback к Transform оборудования. Поэтому отсутствие рабочей зоны не блокирует ремонт, но может привести NPC слишком близко к коллайдеру оборудования. Это место нужно проверить при авторинге сложных аппаратов.

После распознавания типа `!validTarget` снова приводит к ручному завершению или автоматическому `Idle`. Затем StateSystem.h:1813–1862 (../StateSystem.h#L1813) вычисляет **горизонтальное** расстояние до рабочей позиции. Здесь Y специально исключён: ремонтник ходит по палубе, а точка пробоины

может быть на стене выше центра капсулы. Это отличается от трёхмерной дистанции в оценке utility и координатора. Если `distance > brain.repairDistance`, ставится `MoveToRepair` и вызывается `NavigateTo` с комнатой, допуском и `movingState`. Результат `Blocked` даёт конкретную строку для пробоины или оборудования; в любом случае функция возвращается до начисления прогресса. `Waiting` тоже означает отсутствие прогресса. Если расстояние уже в допуске, маршрут в этом кадре не вызывается: ходьба останавливается, состояние становится `Repairing`.

4.4. От секунд к прогрессу и завершению

`brain.repairDuration` — время ремонта с нуля при `repairSkill=1`, нижний предел 0.05 с. `CrewMemberComponent::repairSkill` ограничивается диапазоном 0.25–2. В StateSystem.h:1875–1915 (../StateSystem.h#L1875) вычисляется:

```
progressDelta = requiredProgress * dt * repairSkill / max(repairDuration, 0.05)
```

В исходнике эта формула записана так:

```
const float progressDelta =
    repairable->requiredProgress *
    deltaTime *
    repairSkill /
    duration;
```

Здесь множитель `requiredProgress` переводит долю работы за кадр в единицы шкалы компонента, `deltaTime` делает начисление пропорциональным прошедшему времени, а деление на `duration` задаёт полное время при навыке 1. Нельзя заменить `deltaTime` константой «за кадр»: частота 30 и 120 кадров/с тогда дала бы разные скорости ремонта.

Если `requiredProgress=1`, `repairDuration=3`, `dt=0.1`, skill=1.5, то `progressDelta=0.05`. Из нуля потребуется примерно 20 таких шагов, то есть 2 секунды симуляции. Причина деления на duration проста: за duration секунд при skill=1 сумма `requiredProgress * dt / duration` достигает `requiredProgress`. При skill=1.5 скорость в полтора раза выше, полное время — duration/1.5. Если `requiredProgress=5`, за тот же шаг добавится 0.25, а нормированная доля всё равно 0.05. Это делает авторскую шкалу прогресса независимой от скорости труда.

Вызов `RepairSystem::ApplyRepairProgress(registry,target,progressDelta,&jobs)` — общий путь ремонта, не отдельная NPC симуляция. RepairSystem.h:189–234 (../src/Gameplay/Repair/RepairSystem.h#L189) он отвергает отсутствующий, уже законченный компонент и `progressDelta<=0`, увеличивает `progress` с ограничением `requiredProgress`, а до порога возвращает `false`. Достижение численного порога запускает `CompleteRepair`. Для пробоины с `HullBreachRepairLinkComponent` там сначала восстанавливается runtime mesh и collision; если асинхронная операция ещё не готова, результат `false`, хотя численный прогресс уже 100%. На следующих кадрах `ApplyRepairProgress` снова вызывает завершение, пока меш не готов. Поэтому статус `Repair: waiting for runtime operation` означает **не** недостаточную скорость NPC, а ожидание структурного восстановления корпуса. RepairSystem.h:526–556 (../src/Gameplay/Repair/RepairSystem.h#L526).

Если `completed=false`, `StateSystem` заново получает указатель `RepairableComponent*` и лишь затем читает `Progress01()`, ставит `brain.repairProgress` и статус `Repairing hull breach`, `Repairing equipment` либо ожидание runtime операции. StateSystem.h:1921–1967 (../StateSystem.h#L1921). Повторное получение важно даже если сегодняшний конкретный путь при `false` обычно сохраняет Entity: это безопасная привычка после функции, которая может структурно менять ECS. При `completed=true` код **не разыменовывает прежний `repairable`**. Он выставляет debug прогресс 1, очищает `repairTarget`, `doorRoute`, `doorDestinationRooms`, индекс и активную дверь; затем либо снимает ручную команду, либо переводит автоматический NPC в `Idle`. StateSystem.h:1970–2024 (../StateSystem.h#L1970).

При реальном закрытии пробоины `RepairSystem::CompleteRepair` помечает `sealed`, обнуляет `leakRate`, корректирует связанную панель, помечает `repairable.repaired`, учитывает завершённую заявку экипажа, затем вызывает `registry.DestroyEntity(repairEntity)` и возвращает `true`. RepairSystem.h:570–690 (../src/Gameplay/Repair/RepairSystem.h#L570). Для оборудования Entity остаётся: `Damageable.health` становится `maxHealth`, `isDestroyed=false`, генератор включается, у кислородного генератора снимается `broken`, `repairable.repaired=true`. RepairSystem.h:698–765 (../src/Gameplay/Repair/RepairSystem.h#L698). В текущем NPC вызове не переданы `eventLog`, `worldTime`, `repairSource`; необязательные параметры остаются значениями по умолчанию. Поэтому не обещайте, что именно автоматический ремонт запишет `BreachSealed` или `EquipmentRepaired` в `WorldEventLog`, пока не проведёте эти аргументы через вызов. Это конкретный пример полезного улучшения для аналитики и миссий.

4.5. Ручной ремонт и отмена

`UpdateManualCommand` в StateSystem.h:1345–1459 (../StateSystem.h#L1345) при `RepairEntity` передаёт `brain.command.targetEntity` в ту же `UpdateRepairTask` с `manualCommand=true`. Он не обязан владеть `CrewTaskClaimComponent`: ручной приказ имеет другой источник приоритета. При завершении `CompleteManualCommand` вызывает `brain.command.Clear()`, возвращает `Idle`, помечает маршрут грязным, очищает дверной путь и останавливает движение. StateSystem.h:3071–3085 (../StateSystem.h#L3071). На следующем кадре utility снова имеет право назначить автоматическую работу. Для ручного `Stop` команда намеренно не очищается; восстановление автономии требует внешнего снятия команды.

Отказы в ремонте нельзя сводить к одному «не работает». Если NPC вообще не получает цель — смотрите `Active`, `CrewMember.onDuty`, фракцию, `isHostile`, `Repairable`, `Damageable`/`HullBreach`, `repaired`, диапазон и score. Если цель есть, но NPC не идёт — смотрите `brain.state`, `routeDirty`, `currentRoom`, дверной граф, NavMesh и `CharacterController`. Если стоит рядом, но не ремонтирует — сравните **горизонтальное** расстояние с `repairDistance` и проверьте, не выбрана ли недоступная fallback позиция устройства. Если `progress=100%`, но цель не исчезла — смотрите завершение async mesh операции.

5. Навигация: граф комнат, локальная сетка и Bullet капсула

5.1. `NavigateTo` — общий контракт для разных задач

Сигнатура StateSystem.h:2184–2196 (../StateSystem.h#L2184) принимает `finalTarget`, `targetRoom`, `arrivalDistance`, `movingState` и `dt`. Возвращаемый внутренний `NavigationResult` может означать `Moving`, `Arrived`, `Waiting` или `Blocked`; вызвавшая ветка решает, что это значит для своей задачи. На входе стоит **мировая** цель. На выходе `NavigateTo` не меняет Transform напрямую: он заполняет `CharacterMovement`. Поэтому если при отладке `walkDirection` ненулевой, а позиция стоит, проблема уже ниже AI — в физике, коллизии или контроллере.

В начале `repathTimer` уменьшается на `dt`, debug поля дверного шага сбрасываются. Если `currentRoom` и `targetRoom` известны и различаются, нужен маршрут по комнатам. Если переход через дверь уже начат, он сохраняется, даже если `currentRoom` на пороге временно сменился: иначе путь обрывался бы посередине проёма. Новый target room либо `routeDirty` после истечения `repathTimer` вызывает `BuildDoorRoute`. Неудача ставит `Blocked` с `No traversable door route to target room`; ручная команда при этом сохраняется. Если маршрут грязный, но кулдаун перестроения ещё не истёк, NPC безопасно стоит. Если обе точки уже в одной комнате, промежуточный дверной путь очищается. StateSystem.h:2201–2270 (../StateSystem.h#L2201).

`BuildDoorRoute` в StateSystem.h:2503–2582 (../StateSystem.h#L2503) строит граф: вершина — `RoomComponent` Entity, ребро — `DoorComponent(roomA,roomB)`. BFS выбирает путь с минимальным **числом дверей**, так как

каждая дверь имеет одинаковую стоимость. Дверь с требованием питания исключается, если она не открыта, не сломана и питания нет; сломанная дверь считается проходимой по контракту `DoorSystem`. `visited` хранит предыдущую комнату и дверь, затем путь разворачивается в порядок прохождения. `doorRoute` и `doorDestinationRooms` — параллельные векторы одинаковых шагов. Если позже появится «опасная дверь», «запертая дверь для определённой роли» или стоимость перехода по времени, BFS уже не выразит это корректно; понадобится весовой алгоритм и политика доступа.

`FindContainingRoom` в StateSystem.h:2584–2632 (../StateSystem.h#L2584) преобразует мировую позицию обратной мировой матрицей комнаты в локальное пространство и сравнивает с локальными bounds. Это принципиально для повернутой или масштабированной комнаты: сравнивать мировую точку прямо с `localMin/max` нельзя. Предпочтение прежней комнаты предотвращает прыжки ID в перекрывающихся объёмах у двери. Возврат `INVALID_ENTITY` допустим; тогда верхний граф не строится и остаётся локальное прямое направление. На сложной геометрии это может довести NPC до стены, поэтому пустой `currentRoom` в диагностике — значимый симптом, а не всегда безобидная мелочь.

5.2. Дверной переход как последовательность фаз

В StateSystem.h:2272–2499 (../StateSystem.h#L2272) цикл берёт очередную дверь, её `DoorComponent` и мировую позицию. Отсутствие двери или Transform помечает маршрут грязным и возвращает `Blocked` без чтения несуществующих данных. Затем вычисляется точка выхода **внутри следующей комнаты** на `doorExitDistance`; точка подхода смещена назад от оси выхода. Эти две точки предотвращают типичную ошибку: кратчайший NavMesh путь к центру двери приводит капсулу по диагонали в косяк.

Когда дистанция до двери не больше `doorOpenDistance`, вызывается `DoorComponent::TryOpen(hasPower)`. NPC не анимирует створки; он задаёт намерение, а `DoorSystem` продвигает `openProgress`. Если открыть нельзя, `Blocked` показывает причину. Когда NPC подошёл ближе `doorWaitDistance`, а `openProgress < 0.65`, он останавливается с `WaitingForDoor`. После открытия сначала выравнивается к approach point; затем короткий участок через портал проходит напрямую к exit point, потому что NavMesh исходной комнаты не может проектировать точку в соседней. В этой фазе код временно игнорирует коллизию **конкретной открытой двери** для капсулы и восстанавливает её вне перехода. Дверной шаг считается законченным после достаточного смещения за плоскость двери или близости к exit point. Только после этого начинается движение к следующей двери или финальной цели.

Для понимания проекции у двери: `progressBeyondDoor` — скалярное произведение вектора от двери к NPC с осью выхода, делённое на длину оси. Положительное значение означает продвижение в сторону следующей комнаты. Сравнивать только `currentRoom` недостаточно, потому что bounds комнат могут пересекаться прямо в проёме. Поэтому дополнительно требуется `progressBeyondDoor >= max(0.35,0.75*doorExitDistance)` или фактическая близость к точке выхода. Это локальный пример того, зачем в игре нужна векторная математика, а не только «расстояние до цели».

5.3. `MoveTowardsNavigated`, NavMesh и движение

`MoveTowardsNavigated` в StateSystem.h:2950–3020 (../StateSystem.h#L2950) сначала запрашивает `NpcNavMeshSystem::FindNextWaypoint`. Если запрос успешен, текущей целью становится ближайший waypoint; иначе используется исходная цель. Поверх него `FindDynamicAvoidanceWaypoint` может выдать короткий объезд подвижного Bullet AABB. Для подхода к двери `requireForwardProgress` дополнительно проверяет положительное скалярное произведение направления кандидата с направлением к двери: боковой обход не должен отправить NPC назад от единственного прохода. Это не решение всей задачи планирования пути; это локальная защита конкретного портального участка.

В конфигурации проекта DX_FrameWork.vcxproj:336–338 (../DX_FrameWork.vcxproj#L336) свойство `NpcNavMeshEnabled` по умолчанию 1, а `NpcNavMeshSystem.cpp` включён в компиляцию. Заголовок NpcNavMeshSystem.h:85–180 (../src/Gameplay/AI/NpcNavMeshSystem.h#L85) имеет условные заглушки при `NPC_NAVMESH_ENABLED=0`. Тогда запросы возвращают `false`, `StateSystem` идёт напрямую, а не падает. При включённом слое NpcNavMeshSystem.cpp:1495–1541 (../src/Gameplay/AI/NpcNavMeshSystem.cpp#L1495) кеширует путь по NPC, комнате, `command.revision` и изменению цели больше 0.60. Неудачный запрос не повторяется каждый кадр до изменения ключа или явной инвалидизации. `FindReachablePoint` проектирует желаемую точку на связный остров. Значит, в отладке «NavMesh не нашёл путь» и «StateSystem не пытался идти» — разные случаи: при первом может действовать прямой fallback.

`MoveTowards` в StateSystem.h:3024–3056 (../StateSystem.h#L3024) берёт `to-from`, обнуляет Y, вычисляет длину, делит X и Z на неё и записывает единичное направление. Без нормализации NPC ускорялся бы с расстоянием до цели: вектор `(10,0,0)` сильнее `(1,0,0)` при одной и той же настройке скорости. Нулевая длина вызывает `StopMovement`, чтобы не делить на ноль. Выбор Walk/Run идёт по `allowRunning && distance >= runWhenFartherThan`; это решение по локальной текущей цели движения. Физический сдвиг происходит позднее через `CharacterControllerSystem` и Bullet, а анимация читает `locomotionState`. ApplicationClass.cpp:1537–1548 (../src/Core/ApplicationClass.cpp#L1537).

`UpdateStuckStatus` в StateSystem.h:3108–3180 (../StateSystem.h#L3108) при неподвижном намерении обнуляет таймер; при движении сравнивает новую позицию с `lastProgressPosition`. Порог в коде `0.08` единицы, несмотря на комментарий «3 см» рядом с ним; ориентируйтесь на число в условии. Если прогресса нет в течение `stuckTimeout` (в Brain по умолчанию 2 с), собирается подробный snapshot двери, позиции, фаз и направления. Не пытайтесь чинить это простым бесконечным `repath` каждый кадр: если геометрия запирает капсулу, новые одинаковые пути не сдвинут физический объект.

6. Противник: выбор, зрение, память и состояния боя

6.1. `NpcCombatSystem::Update`: выбор кандидата

Сигнатура NpcCombatSystem.h:42–47 (../NpcCombatSystem.h#L42) принимает Registry, `deltaTime`, Bullet `dynamicsWorld` и настройки отладочных лучей. Главный `ForEach<NPCCombatComponent>` означает: участие в **этом** восприятии определяется наличием боевого компонента. Внутри получаются `NPCComponent`, `NpcBrainComponent`, `TransformComponent`; без одного из них NPC пропускается. Активная ручная команда сбрасывает время стационарного прицеливания и не даёт автоматическому бою принять решение. Отсутствие `FactionComponent` сбрасывает `targetVisible` и прицеливание. NpcCombatSystem.h:57–97 (../NpcCombatSystem.h#L57).

Далее внутренний `ForEach<FactionComponent,TransformComponent>` перебирает возможные цели. Кандидат не может быть самим наблюдателем, должен быть активным, иметь `NPCComponent` или `PlayerComponent`, иметь здоровье больше нуля и быть враждебным по `AreEnemies`. Сравнивается `candidateDistanceSquared = dx²+dy²+dz²`, выбирается ближайшая по трёхмерной дистанции Entity. Квадратный корень не нужен, так как для неотрицательных чисел `a<b` тогда и только тогда, когда `sqrt(a)<sqrt(b)`. Это небольшая, но правильная экономия внутри вложенного цикла. NpcCombatSystem.h:101–169 (../NpcCombatSystem.h#L101).

Важное ограничение текущей реализации: **сначала** выбирается ближайший живой враг, и **только затем** проверяются FOV и заслонение. Если близкий враг находится за стеной, а другой дальше, но виден, второй не проверяется как запасная визуальная цель в этом проходе. Это не сознательная система приоритета угроз, а следствие порядка фильтров. Для улучшения позже выбирайте

ближайшего среди **видимых** кандидатов либо отдельно храните ближайшего и лучшего видимого; при этом не ломайте личную память цели. При полном отсутствии живых врагов `targetEntity` очищается, `targetVisible=false`, `rangedAimTime=0`.

Ещё одно точное различие guard условий: внутри `NpcCombatSystem::Update` для **наблюдателя** проверяется наличие `NPCComponent`, Brain, Transform, команды и фракции, но нет отдельного раннего `ActiveComponent` или `npcData.health>0` guard. Поздний `NpcRangedCombatSystem` и `NpcCoverSystem` такие ограничения ставят. Поэтому выключенный или мёртвый NPC теоретически может ещё обновить свои поля восприятия, хотя стрелять и выполнять движение не должен. При рефакторинге добавьте единый предикат жизнеспособного наблюдателя и проверьте обратную совместимость с отключением Entity. Это предложение; сегодняшнюю трассу выстраивайте по существующим ранним выходам.

Указатель `candidateTransform` из внутренней лямбды не сохраняется наружу. После выбора ID код заново получает `targetTransform` — правильный способ избежать ссылки на временный контекст выбора. `stanceTimeRemaining` уменьшается на `dt`. Прежде чем считать дистанцию атаки, система отдельно готовит геометрию наблюдения. NpcCombatSystem.h:171–205 (../NpcCombatSystem.h#L171).

6.2. Сектор зрения и три луча

Глаза NPC по умолчанию находятся на `position.y+1.6`. При доступной анимации и кости `mixamorig:Head` точка берётся из актуальной матрицы головы, умноженной на мировую матрицу персонажа. NpcCombatSystem.h:208–221 (../NpcCombatSystem.h#L208). Это объясняет, почему один и тот же NPC может видеть поверх низкой стены при подходящей позе, а константная точка на высоте корпуса дала бы ложное заслонение. Для цели пробуются высоты груди 1.0, головы игрока по `PlayCameraComponent::firstPersonEyeOffset.y` или 1.6 для NPC, затем нижней части корпуса 0.55.

До дорогих лучей вычисляются расстояние от глаз к груди и горизонтальный сектор. Текущий forward для yaw модели:

```
XMFLOAT3 forward = { -sin(rotation.y), 0.0f, -cos(rotation.y) };
bool inCone = dot >= cos(radians(fieldOfViewDegrees * 0.5f));
```

При `fieldOfViewDegrees=120` половина сектора 60°, `cos(60°)=0.5`. Цель точно впереди даёт dot около 1 и проходит, под углом 75° даёт примерно 0.259 и отбрасывается. Вектор к цели нормализуется **по XZ**, потому что сектор должен быть горизонтальным; высота цели сама по себе не меняет yaw обзора. Если горизонтальные координаты почти совпали, сектор считается пройденным, чтобы избежать деления на почти ноль. NpcCombatSystem.h:258–298 (../NpcCombatSystem.h#L258).

После теста диапазона и сектора цикл пробует три высоты. Для каждой требуется ненулевой `dynamicsWorld`, длина луча больше 0.001 и не больше `seeRange`. `PhysicsRaycast::CastRay` идёт от глаз к точке, исключая самого NPC; цель видна, если луч никого не задел или первым задел именно целевую Entity. При первой успешной высоте `visibleAimPoint` запоминает **эту** открытую точку, debug renderer может нарисовать луч. NpcCombatSystem.h:300–352 (../NpcCombatSystem.h#L300). Отсутствие Bullet мира здесь не означает «видно всё»: цикл продолжает без успешного результата. Это полезная безопасная семантика при загрузке/остановке мира.

6.3. Память не равна всеведению

После лучей `previouslyVisible` истинно, только если в прошлом кадре была видна **эта же** цель. Затем записывается новый `targetVisible`. Если видимость подтверждена, `combat.targetEntity=bestTargetEntity`, и `ObserveVisibleTarget(targetTransform->position,dt,previouslyVisible)` обновляет факт наблюдения. NpcCombatSystem.h:354–379 (../NpcCombatSystem.h#L354). В NPCComponents.h:535–551 (../NPCComponents.h#L535) `ObserveVisibleTarget` сначала обнуляет `lastSeenVelocity`, а при непрерывной видимости вычисляет `Δx/dt` и `Δz/dt` по двум наблюдениям; Y не предсказывается. После повторного

выхода из за стены скорость начинается заново, чтобы прыжок скрытой цели не превратился в «измеренную» скорость. Функция обнуляет таймер забывания и старый поисковый план.

При потере видимости новая текущая позиция цели не попадает в память. `EstimateSearchPosition()` в NPCComponents.h:555–570 (../NPCComponents.h#L555) берёт `lastKnownPosition + lastSeenVelocity*predictionSeconds` по горизонтали, ограничивая длину прогноза `maxPredictionDistance` (по умолчанию 3). Например, последняя лично увиденная точка `(10,0,5)`, скорость `(4,0,0)` единиц/с и горизонт 0.8 с дают сырое упреждение 3.2; ограничение до 3 даёт план `(13,0,5)`. Это гипотеза, не новое наблюдение. Пока цель скрыта, её реальный Transform не должен «таскать» план вслед за ней.

Когда цель видна, вооружённый NPC входит в `Attacking` при расстоянии не больше `min(combat.rangedAttackRange,weapon.range)`; иначе `Chasing`. Без оружия используется `attackRange` и гистерезис `ShouldHoldMeleeStance`. При входе в стойку `EnterMeleeStance` устанавливает минимальную паузу до удара, не сокращая уже более длинный cooldown. Система не сбрасывает активный путь к укрытию, если NPC сейчас исполняет неоткрытую cover фазу: `executingCoverOrder` удерживает прежнее состояние. NpcCombatSystem.h:407–435 (../NpcCombatSystem.h#L407). Это пример арбитража между сенсором и тактическим исполнителем: видимость — факт, но не любой факт немедленно отменяет уже выданную фазу движения.

Если ранее выбранная цель пропала из зрения, но Entity остаётся, NPC продолжает поиск. `timeSinceTargetSeen` растёт только в `Searching`, когда план уже готов и NPC больше не идёт к предполагаемой точке. `loseTargetDelay` по умолчанию 3 с; до забывания завершается круговой осмотр `searchTurnRemaining` и при соответствующем стиле назначенные сектора. Только затем `targetEntity` и личная память очищаются, `state=Idle`. NpcCombatSystem.h:436–467 (../NpcCombatSystem.h#L436). Не путайте этот таймер с «прошло 3 секунды с первого кадра невидимости»: на пути к последней позиции время ожидания не расходуется.

6.4. `StateSystem` в погоне, поиске и атаке

В `Chasing` StateSystem.h:642–817 (../StateSystem.h#L642) проверяется цель и память. Для маршрута используется `combat.lastKnownPosition`, не скрытый текущий Transform, хотя Transform цели первоначально читается для валидации существования. У безоружного NPC включается бег уже на коротком расстоянии; вооружённый учитывает стиль, входящий огонь и желаемую дистанцию. `Defensive` удерживает позицию, прекращает наступательные поисковые приказы и при потере контакта переходит в `Searching`, где не бежит вслед за предположением. `Assault` и `Balanced` могут исследовать продолжение наблюдаемой скорости и ограниченное число секторов. При достижении последней точки без видимой цели состояние становится `Searching`, а не автоматически `Attacking`.

В `Searching` StateSystem.h:819–915 (../StateSystem.h#L819) сначала останавливается ходьба и проверяется существование наблюдения. План строится один раз на эпизод потери цели: `searchPlanReady=true`, `BeginSearchTurn()` задаёт полный оборот `2π`, таймеры сбрасываются. `AdvanceSearchTurn(yaw,rotationSpeed,dt)` ограничивает шаг остатком угла и возвращает нормализованный угол через `atan2(sin,cos)`. При скорости 90°/с и `dt=0.1` один шаг 9°, полный поворот требует примерно 40 шагов. Если скорость настройки равна нулю, используется безопасный запас 90°/с. NPCComponents.h:511–530 (../NPCComponents.h#L511). Важна конечность этого действия: угловой остаток уменьшается, и NPC не вращается вечно, если дальше должен забыть цель.

В `Attacking` StateSystem.h:1156–1335 (../StateSystem.h#L1156) сначала проверяются живой ID, личная видимость, Transform цели и дистанция. При выходе за дальность NPC снова `Chasing`; при потере прямого зрения стоящий стрелок останавливается и тоже переходит к погоне/поиску. В состоянии атаки `StopMovement` сразу делает нижнюю анимацию Idle, корпус разворачивается к цели ограниченным

угловым шагом `rotationSpeed*dt`. Допуск 8° проверяется **после** поворота. Для вооружённого NPC этот проход только накапливает `rangedAimTime`; фактический луч из дула будет построен после обновления скелета. Для безоружного проход отсчитывает `attackTimer`, уважает паузу стойки и активный Punch клип, выбирает взвешенный вариант анимации и запрашивает `AnimationState::Punch`. Сейчас эта ветка, согласно коду и его собственному комментарию, **создаёт анимацию, но сама не наносит melee урон**. Не объявляйте рукопашный бой готовым для релиза без отдельной проверки hit window, дальности и события урона.

7. Укрытие, оружие и очередь урона

7.1. `NpcCoverSystem::Update`: тактика одного NPC

Сигнатура NpcCoverSystem.h:36–46 (../NpcCoverSystem.h#L36) требует Bullet мир и положительное конечное `dt`. `ForEach<NPCCombatComponent>` дополнительно требует `NpcBrainComponent`, `NpcTacticalComponent`, `FactionComponent`, `TransformComponent`, `NPCComponent`. Нет одного — нет решения укрытия. Система уменьшает `underFireTime`, `decisionTimer`, `phaseTimer`, таймер отклонённой точки и другие индивидуальные интервалы. `damageReactionPending` будит новое решение, но сам сигнал не сообщает точную позицию стрелявшего: для геометрии используется лично наблюдавшаяся `lastKnownPosition`. Неактивный, мёртвый, управляемый вручную NPC или персонаж без подтверждённой вражеской цели и памяти выходит до поиска позиции. NpcCoverSystem.h:90–229 (../NpcCoverSystem.h#L90).

Выбранный `reservedCover` — авторская Entity с `NpcCoverPointComponent`; динамическое укрытие Entity не имеет и отмечено `dynamicCoverActive`. При существующем укрытии `NpcCoverQuery::IsProtectedFromTarget` проверяет защиту относительно последней наблюдавшейся позиции. Один неудачный raycast увеличивает `coverValidationFailureTime`, но позиция сохраняется до `coverValidationGraceDuration`; это гасит однокадровое дрожание у края коллайдера. NpcCoverSystem.h:230–285 (../NpcCoverSystem.h#L230). Если NPC открыт в peek фазе, истёк таймер или он получил попадание, цель движения возвращается к `coverPosition`, состояние становится `TakingCover`. Попадания во время peek увеличивают `failedPeekCount`; после порога запрашивается смена укрытия, но она ждёт окончания `coverHoldTimer`. Когда скрытая фаза заканчивается, `moveTarget=peekPosition`, NPC идёт выглянуть. NpcCoverSystem.h:287–375 (../NpcCoverSystem.h#L287).

Когда укрытие нужно выбрать, `NpcTacticalDecisionSystem::EvaluateCoverRequest` учитывает стиль и угрозу. `NpcCoverQuery::FindBest` ищет авторскую или динамическую точку по текущей комнате, позиции NPC, памяти о цели и параметрам тактики. Если не найдено — ставится `noSuitableCover` и cooldown, чтобы не делать дорогой поиск каждый кадр. Если найдено — сохраняются `coverPosition`, `peekPosition`, `moveTarget`, тип укрытия, таймер удержания и `brain.state=TakingCover`; старый NavMesh путь инвалидируется. NpcCoverSystem.h:390–479 (../NpcCoverSystem.h#L390). `NpcCoverSystem` не перемещает NPC. На следующем проходе **того же кадра** `StateSystem` увидит `TakingCover` и построит движение. При прибытии к скрытой точке он ставит `InCover` и таймер ожидания, при прибытии к peek — `exposed=true`, `Attacking` и окно огня не короче `rangedAimDelay+1.2`. StateSystem.h:539–600 (../StateSystem.h#L539).

`ReleaseCurrentCover` NpcCoverSystem.h:490–506 (../NpcCoverSystem.h#L490) временно запрещает немедленное повторное избрание старой авторской точки на 4 с и очищает runtime флаги. Это лучше, чем бесконечный цикл «эта же точка снова лучшая — она же снова не защищает». Параметры `hideDuration`, `exposeDuration`, `minimumCoverHoldDuration`, `failedPeeksBeforeRelocate` из `NpcTacticalComponent` дают четыре разных рычага. Не подменяйте их одним общим «агрессивность»: один управляет ритмом выглядывания, другой устойчивостью к смене позиции.

7.2. `NpcRangedCombatSystem::Update`: серия обязательных ворот

Сигнатура NpcRangedCombatSystem.h:30–37 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) получает Registry, Bullet мир, `DamageSystem`, D3D устройство, `JobSystem` и debug параметры. `ForEach<NPCComponent,NPCCombatComponent>` начинает с `rangedStatus="Not shooting"`. Потом последовательно отсекаются отсутствующий Brain, мир, `Active`, `PlayerGameTag`, фракция, нулевое здоровье, активная команда, отсутствие экипированного оружия. NpcRangedCombatSystem.h:39–118 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L39). Это полезная отладочная лестница: если ранний guard вернул управление без специальной строки, новый статус останется `Not shooting`.

При `weapon.isReloading` стрельба заканчивается статусом `Reloading`. При пустом магазине запрашивается `WeaponSystem::RequestReload`; статус сообщает, началась ли перезарядка или боеприпасы исчерпаны. Затем определяется `movingFire`: требуется тактический компонент, подходящее состояние `Chasing` или `TakingCover` и личная видимость. На ходу используются собственные `movingAimTime`, размер и пауза короткой очереди. Стационарный NPC обязан быть в `Attacking`, и **в обоих режимах** нужен `combat.targetVisible` именно в текущем кадре. Пока соответствующий таймер меньше `rangedAimDelay`, выстрел не создаётся. NpcRangedCombatSystem.h:120–189 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L120).

После этого цель вновь проверяется: живая Entity, активна, не мертва, имеет фракцию и остаётся враждебной. Повторная фракционная проверка непосредственно перед огнём защищает от устаревшего `targetEntity`, если отношения изменились между выбором и выстрелом. NpcRangedCombatSystem.h:192–245 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L192). Для вычисления мирового дула нужны Transform, `CharacterMovement`, модель и анимация. Кость руки берётся по `weapon.rightHandBone`; если она отсутствует, статус `Missing right-hand bone`. `muzzlePosition` — локальная точка оружия; цепочка `GripMatrix(worldGrip) * hand * transform.GetWorldMatrix()` переводит её через хват, кость и мир. Порядок умножения связан с принятой в DirectXMath конвенцией проекта. Если заменить порядок на `world * hand * grip`, дуло окажется не там, где рука. NpcRangedCombatSystem.h:247–307 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L247).

Направление выстрела — `visibleAimPoint - origin`. `WeaponFiringRules::ValidRay` отбрасывает некорректный или нулевой вектор. Дальность сверяется с `weapon.range`, затем корпус должен смотреть на цель с ошибкой не более 8°. Ошибка yaw нормализуется через `atan2(sin(Δ),cos(Δ))`, иначе при переходе через ±π обычное вычитание выбрало бы длинный оборот. NpcRangedCombatSystem.h:309–363 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L309). Наконец короткий луч от груди к дулу проверяет, не оказался ли длинный ствол за укрытием, а отдельный луч от дула к видимой точке блокирует огонь, если первым на линии стоит живой NPC дружественной фракции. NpcRangedCombatSystem.h:365–451 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L365).

Только после всех ворот ставится `weapon.triggerHeld=true` и вызывается `WeaponSystem::TryFire`. Результат записывается как `Fired`, `Reloading`, `Weapon cooling down` либо `Weapon unavailable`. Если это успешный выстрел на ходу при свободной линии, `firedOnRoute` и счётчик очереди обновляются; по исчерпании очереди выставляется `movingBurstPauseTimer`. NpcRangedCombatSystem.h:453–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L453). **Подтверждённый дефект текущего кода:** `NpcRangedCombatSystem` проверяет положительный `movingBurstPauseTimer` перед следующей очередью, но ни он, ни другой проход его не уменьшает. Пока `movingFire` остаётся активным, стрельба на ходу может зависнуть после первой очереди со статусом `Moving: pause between bursts`; переход к обычной стрельбе сбрасывает таймер. NpcRangedCombatSystem.h:155–174 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L155). `triggerHeld` — признак попытки спуска в

этом шаге, а не доказательство попадания. Доказательство состоявшегося выстрела — возврат `TryFire=true`; доказательство попадания — результат физического луча внутри оружейной системы.

Короткая цитата из NpcRangedCombatSystem.h:176–181 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L176) фиксирует главный запрет:

```
if ((!movingFire && brain.state != NpcState::Attacking) || !combat.targetVisible)
{
    combat.rangedStatus = "No visible target in firing stance";
    return;
}
```

Внешние скобки важны: при разрешённом `movingFire` стационарная стойка не нужна, но `!combat.targetVisible` запрещает **оба** режима. Это место стоит перечитать перед добавлением подавляющего огня: простой новый state не превратится в возможность стрельбы по невидимой цели.

7.3. `WeaponSystem::TryFire` и `DamageSystem::Update`

`WeaponSystem::TryFire` в WeaponSystem.h:75–155 (../src/Gameplay/Combat/WeaponSystem.h#L75) — общая точка игрока и NPC. Он проверяет мир, живого и активного стрелка, корректный луч, оружие, патроны и `WeaponFiringRules::CanFire`. При пустом магазине пытается начать механическую перезарядку. Затем нормализует направление и рассчитывает множитель разброса от движения. Две случайные выборки дают направление внутри конуса с учётом `spreadDegrees` и текущей отдачи; `CommitShot` расходует патрон и запускает cooldown. После этого Bullet луч длиной `weapon.range` останавливается на **первом** объекте. Даже промах считается состоявшимся выстрелом и возвращает `true`. Если луч задел стену перед выбранным AI врагом, урон цели за стеной не переносится: `DamageEvent.target` — Entity физического первого попадания.

Две точные строки из WeaponSystem.h:121–131 (../src/Gameplay/Combat/WeaponSystem.h#L121) показывают различие между фактом выстрела и фактом попадания:

```
const bool hit = PhysicsRaycast::CastRay(origin, normalized, world,
    weapon->range, shooterEntity, shooterObject, rayHit);
```

После необязательной отладочной отрисовки стоит отдельная строка:

```
if (!hit) return true;
```

Здесь `true` у `TryFire` означает, что патрон уже израсходован и луч создан. Логический `hit=false` означает отсутствие первого пересечения, поэтому события урона нет. В этом движке «пуля» обычного оружия — **hitscan луч**, не отдельная летящая Entity с траекторией и временем полёта. `Bullet` в названиях физического мира — библиотека коллизий; она проверяет пересечение луча и управляет капсулой NPC. Это два разных смысла слова «bullet». Если позже понадобится медленный плазменный снаряд, ему потребуется собственный компонент движения, время жизни и обработка столкновения; текущий `TryFire` сам такой снаряд не создаёт.

`DamageEvent` содержит Entity оружия как `source`, Entity попадания как `target`, число урона, тип, точку, нормаль и параметры возможной пробоины. Для корпуса дополнительно вычисляются разрешение и порог резки меша. Затем событие добавляется в очередь `DamageSystem`. WeaponSystem.h:133–155 (../src/Gameplay/Combat/WeaponSystem.h#L133). `DamageSystem::Update` позже обрабатывает очередь: проверяет валидную активную цель и положительный урон; ставит `NpcTacticalComponent::underFireTime=3`, считает попадания за окно 4 с и может поставить `damageReactionPending`. Событие оружия разрешается через `WeaponOwnerComponent` во владельца, но боевой `targetEntity` жертвы не заполняется магически — направление входящего огня лишь короткая ориентировка. DamageSystem.cpp:104–189 (../src/Gameplay/Combat/DamageSystem.cpp#L104). `InvulnerableComponent` и god mode не уменьшают здоровье; обычная тактическая реакция на попадание при этом может всё же появиться, кроме специально тихого события `silentKill`.

Для NPC без общего `Damageable` здоровье уменьшается с ограничением от 0 до max; при нуле снимается `ActiveComponent`. Для устройства `Damageable.health` уменьшается, `Normalize` приводит состояние к допустимому диапазону, при переходе в уничтоженное состояние создаётся взрыв и событие уничтожения, а `Repairable.progress` сбрасывается в 0, `repaired=false`. DamageSystem.cpp:204–211 (../src/Gameplay/Combat/DamageSystem.cpp#L204), DamageSystem.cpp:331–369 (../src/Gameplay/Combat/DamageSystem.cpp#L331). Последнее связывает бой с работой экипажа: после попадания повреждённое оборудование снова становится кандидатом ремонта на следующем AI проходе.

8. Прогоним данные руками: два проверяемых сценария

8.1. Один кадр назначения и один кадр фактического ремонта

Зададим **учебную сцену** с активным мирным NPC `Entity=10` в `(0,0,0)` и активной незапечатанной пробоиной `Entity=40` в `(6,0,0)`. У NPC есть `NPCComponent{isHostile=false}`, `NpcBrainComponent` с `detectionRange=30`, `idleScore` после utility 0.10, `repairDistance=1`, `repairDuration=3`, `CrewMemberComponent{role=Engineer,onDuty=true,repairSkill=1.5}`. У пробоины есть `HullBreachComponent{leakRate=0.12,room=5}`, `RepairableComponent{progress=0,requiredProgress=1,repaired=false}`. Путь по комнатам и Bullet капсула считаются корректными. Пусть `dt=0.1` с. Это выбранные числа для расчёта, а не обещание о конкретной сохранённой сцене.

На первом проходе `NpcUtilitySystem` видит `Entity=40`, заявок другого NPC нет. `distance=6`, `distanceScore=1-6/30=0.8`, `severity=clamp(0.12/0.15,0.15,1)=0.8`, `repairScore=0.8`. Так как `0.8>0.10`, он пишет `brain.repairTarget=40`, `state=MoveToRepair`, `routeDirty=true`. Если рядом есть второй мирный NPC, на этом предварительном этапе **оба** могут посчитать одну пробоину лучшей; это пока только заявка в голове каждого.

Затем координатор заново собирает кандидатов. Для инженера базовая смесь `0.65*0.8+0.35*0.8=0.8`, роль 1.25, skill 1.5, итог `1.5`. Он создаёт `CrewTaskClaimComponent` на пробоине: `assignee=10`, `type=RepairBreach`, `priority=1.5`, время текущего мира. У NPC `assignedTask=40`. Если следующий NPC в том же проходе увидит `claimedTargets` с 40, он уже не возьмёт эту пробоину. Обратите внимание: `brain.repairScore=0.8` и `claim.priority=1.5` одновременно корректны, потому что это разные формулы и разные задачи проходов.

В том же кадре `StateSystem` до физики видит расстояние 6>1. `UpdateRepairTask` ставит `MoveToRepair`, вызывает `NavigateTo` к `(6,0,0)`. Допустим комната та же и первый waypoint совпадает с финальной точкой. `MoveTowards` пишет `walkDirection=(1,0,0)`, `currentSpeed=2` при обычной ходьбе; `Repairable.progress` всё ещё 0. Позднее `CharacterControllerSystem` передаст движение Bullet. Если нет коллизий, ожидаемый шаг порядка `2*0.1=0.2` единицы; фактическая позиция может отличаться из за физики. Нельзя требовать увеличения прогресса уже в кадре назначения при дистанции 6.

Предположим, после нескольких шагов Transform NPC на **начале** следующего AI кадра находится в `(5.1,0,0)`, то есть горизонтальная дистанция до пробоины 0.9. `Utility` сохраняет или повторно выбирает цель, `KeepExistingAssignment` сохраняет заявку без скачка к другой аварии. `StateSystem::UpdateRepairTask` теперь не вызывает `NavigateTo`, а останавливает ходьбу, ставит `Repairing`. За `dt=0.1` вычисляется `progressDelta=1*0.1*1.5/3=0.05`; `RepairSystem` записывает `progress=0.05` и возвращает `false`, потому что до `requiredProgress=1` ещё далеко. `brain.repairProgress=0.05`. Через 20 таких рабочих шагов сумма достигнет 1.0, если авария не была повторно повреждена и dt не менялся. При пробоине с async mesh восстановлением число уже может показывать 1.0, а `completed=false` продлится до готовности

операции. После фактического `DestroyEntity(40)` нельзя читать прежний `repairable*`; следующий координатор увидит отсутствие цели и не должен сохранять старую заявку.

Этот прогон проверяет два инварианта. Во первых, оценка полезности не равна физическому ремонту; между ними есть арбитраж и маршрут. Во вторых, сумма `progressDelta` зависит от **симуляционного** времени, поэтому пауза и `dt=0` не чинят корпус. Если в реальной сцене прогресс идёт у NPC за стеной, измерьте `repairPosition`, `currentRoom`, выбранный work zone point и горизонтальный `repairDistance`: формула сама по себе не проверяет физическую видимость рабочей поверхности.

8.2. Один кадр подтверждённого выстрела и следующий кадр урона

Зададим другой учебный снимок. Вооружённый активный NPC в `(0,0,0)`, yaw=0, его модель смотрит в направлении `(0,0,-1)` согласно формуле forward. Единственный живой вражеский игрок в `(0,0,-8)`, оба имеют разные ненейтральные `FactionId`. У NPC `seeRange=10`, `fieldOfViewDegrees=120`, `rangedAttackRange=12`, `rangedAimDelay=0.35`. Экипированное оружие имеет `range=20`, 5 патронов, `currentCooldown=0`, `spreadDegrees=0` и нулевую текущую отдачу; анимация содержит нужную кость руки. Предположим реальный Bullet мир содержит коллайдер игрока, но не содержит стены или союзника на луче. `dt=0.1` с; перед кадром `rangedAimTime=0.30` от уже видимой цели.

`WeaponSystem::UpdateTimers` сначала сбрасывает `triggerHeld=false` и обновляет оружейный таймер. `NpcCombatSystem` в поиске кандидатов вычисляет `distanceSquared=64`; игрок единственный вражеский персонаж, поэтому выбирается его ID. Вектор до груди примерно `(0,-0.6,-8)` от фиксированной точки глаз `(0,1.6,0)`; горизонтальное направление `(0,0,-1)`, скалярное произведение с forward равно 1, порог `cos(60°)=0.5`, сектор пройден. Луч к груди (или к следующей открытой высоте, если грудь закрыта) подтверждает видимость. Система записывает `targetVisible=true`, `visibleAimPoint` и наблюдаемую позицию `(0,0,-8)`; дальность центр–центр 8 меньше `min(12,20)=12`, значит `brain.state=Attacking`.

Пусть `NpcCoverSystem` в этом кадре не выдал приказа уйти в укрытие. `StateSystem` останавливает ходьбу, проверяет дальность и оставшуюся ошибку yaw. Поскольку корпус уже повернут на цель, ошибка 0<8°, `rangedAimTime` становится 0.40. Далее контроллер, анимация и физика обновляют позу. `NpcRangedCombatSystem` в конце кадра видит готовое оружие, `Attacking`, личную видимость и 0.40≥0.35. Из матрицы хвата, руки и мира он получает **фактический** origin дула; для примера пусть он равен `(0,1.4,-0.4)`. Точка груди игрока `(0,1.0,-8)` даёт ненормализованный луч `(0,-0.4,-7.6)` с длиной около 7.61, меньше 20. Короткий луч грудь NPC–дуло свободен, полный луч до цели не встречает союзника.

`WeaponSystem::TryFire` нормализует луч; при нулевом разбросе он остаётся направлен к выбранной точке. `CommitShot` расходует один патрон и устанавливает кулдаун согласно параметрам оружия. Bullet луч от дула первым пересекает коллайдер игрока. `TryFire` создаёт `DamageEvent{source=weaponEntity,target=playerEntity,damageAmount=weapon.damage,...}`, добавляет его в очередь и возвращает `true`; `combat.rangedStatus="Fired"`. Если бы физический луч промахнулся из за разброса или движения цели после проверки зрения, возврат всё равно был бы `true`, но `DamageEvent` бы не появился. Если бы первым попал в стену, событие адресовалось бы стене, не игроку.

В этом же кадре `DamageSystem::Update` уже был вызван **раньше**, перед поздним выстрелом. При обычном цикле новый `DamageEvent` ждёт следующего шага. Допустим `weapon.damage=20`, здоровье игрока перед очередью 100 и нет god mode. Следующий `DamageSystem::Update` уменьшит его до 80. Если целью вместо игрока был NPC, его `NpcTacticalComponent` получит `underFireTime=3` и сигнал реакции; при нуле HP обычный NPC потеряет `ActiveComponent`. Эта задержка на границе очереди важна при тестах: проверка HP сразу после `NpcRangedCombatSystem::Update` ожидает ещё старое здоровье, проверка после следующего `DamageSystem::Update` — новое.

9. Как добавлять поведение, не разрывая цепочку

9.1. Сначала определите место нового факта

Предложение: добавить задачу «проверить аварийный клапан», которая доступна мирному экипажу, требует подхода к устройству и занимает 2 секунды. Не начинайте с нового `case` в `StateSystem`, пока не ответили на четыре вопроса: **что делает цель пригодной, кто может её взять, как подтвердить достижение и что меняется в мире после выполнения**. Для этой задачи потребовался бы компонент цели, например `ValveFaultComponent{faulted,room,interactionPosition}`, авторский параметр длительности и runtime прогресс. ID исполнителя можно хранить через обобщённую заявку либо расширить нынешний `CrewTaskClaimComponent`. Пока текущий `CrewTaskCoordinator` собирает исключительно `RepairableComponent` цели через `IsRepairTargetValid`, новый компонент сам по себе не попадёт в очередь. CrewTaskCoordinator.h:225–276 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225).

Для небольшого расширения **существующего** ремонта, например нового типа генератора, путь короче. Если устройство уже имеет `DamageableComponent`, `RepairableComponent`, Transform и нужный WorkZone, оно попадёт в кандидаты. Нужно решить, какую `importance` оно получит в utility, нужна ли поправка `roleAffinity` в координаторе и что `RepairSystem::CompleteRepair` должен делать с его собственным состоянием после восстановления HP. Иначе `Damageable.health` станет полным, но функциональный флаг нового устройства может остаться `broken=true` и игра увидит противоречие. Добавьте сохранение его авторских полей в `SceneManager` и редактирование в Editor, если настройка должна жить между запусками.

Для **нового вида задачи** сначала опишите состояние данных, затем интерфейс оценки, затем резервирование и лишь после этого исполнение. Оценщик не должен менять мир; координатор должен выпускать ровно одного исполнителя; исполнитель должен быть идемпотентным по отношению к повторным кадрам и безопасным к удалению цели. Если действие занимает время, храните прогресс на цели, если несколько исполнителей могут способствовать одной работе, либо на назначении, если прогресс принадлежит одному исполнителю и должен обнуляться при его уходе. Нынешний ремонт хранит прогресс на `RepairableComponent` цели, поэтому при смене ремонтника работа продолжается. Это осознанное поведение, которое стоит явно выбрать для новой системы.

9.2. Практический маршрут изменения проекта

1. **Данные.** Добавьте минимальный компонент цели и авторские параметры. Разделите сохраняемые настройки и runtime память. Пример разделения уже есть у `CrewMemberComponent`: `role/onDuty/repairSkill` сохраняются, `assignedTask` и счётчик текущего запуска не записываются. SceneManager.cpp:3552–3558 (../src/Scene/SceneManager.cpp#L3552), SceneManager.cpp:4629–4640 (../src/Scene/SceneManager.cpp#L4629).
2. **Валидация цели.** В одном месте проверьте `IsAlive`, `Active`, обязательные компоненты и незавершённость. Нельзя считать «есть Transform» эквивалентом «клапан сломан». Выберите политику при удалении цели: освободить claim, поставить `Idle`, прекратить старый путь.
3. **Оценка.** Задайте численную формулу в понятных единицах и проверьте её на ручных примерах: близкая мелкая авария, дальняя критическая, два NPC разных ролей, порог `idleScore`. Если доступность зависит от маршрута, геометрическую дистанцию можно использовать как дешёвый предварительный фильтр, а достижимость проверять до закрепления назначения.
4. **Резервирование.** Запишите, кто владеет задачей, и решите, удерживается ли назначение при колебаниях score. Для работ вроде ремонта липкость полезна; для тушения быстро растущего пожара может понадобиться прерывание по аварийному приоритету. Не переназначайте каждый кадр без cooldown или гистерезиса.
5. **Исполнитель.** Добавьте состояние и ветку в `StateSystem::UpdateNpc` или отдельную исполнительную систему в конкретной точке `ApplicationClass`. Для перемещения используйте общий `NavigateTo` и `CharacterMovement`, для изменения мира — специализированную систему действия. Границу после потенциального `DestroyEntity` обрабатывайте по ID и результату, не через сохранённый указатель.
6. **Авторинг.** Добавьте поля сохранения и загрузки в SceneManager.cpp (../src/Scene/SceneManager.cpp#L3951), UI создания и редактирования в EditorUI.cpp (../src/Editor/EditorUI.cpp#L6737), диагностику в NPC Studio или Object Debug. Не сохранённый авторский параметр будет возвращаться к значению по умолчанию при перезагрузке сцены.
7. **Проверка.** Пройдите по сценарию «один NPC–одна задача», затем «два NPC–одна задача», удаление цели в пути, ручной Stop посередине, потеря питания двери, pause и повторная загрузка. В каждой проверке фиксируйте ID, состояние, статус, маршрут и фактическое изменение мира.

Если понадобится новый **боевой** тип поведения, например точечный подавляющий огонь, отделите его решение от факта выстрела. Решение должно выставить намерение, таймер и допустимую точку, но не вычитать HP непосредственно. Общий `WeaponSystem::TryFire` уже отвечает за патрон, разброс, физический первый hit, эффекты и очередь `DamageEvent`. Новый режим обязан сохранить проверки фракции, личной видимости либо явно описанного способа подавления, блокировки дула, союзника в линии и разрешения оружия. Если он должен стрелять в точку без визуального контакта, это **новое правило дизайна**, которого текущий `NpcRangedCombatSystem` не допускает. Его нельзя получить простым изменением `brain.state=Attacking`.

9.3. Инварианты, которые стоит проверять в коде ревью

Для ремонта: у пригодной цели есть `RepairableComponent` и конкретный тип (`HullBreach` или `Damageable`); у незавершённой автоматической задачи один claim; `member.assignedTask`, `brain.repairTarget` и claim указывают на согласованные ID; ручной приказ имеет приоритет; до достижения `repairDistance` прогресс не меняется; после уничтожения пробоины нет чтения старых ссылок. Для боя: цель живой активный враг; `targetVisible` соответствует лучу текущего кадра; `visibleAimPoint` читается только при этой видимости; скрытая цель не обновляет `lastKnownPosition`; патрон расходуется только при успешном `TryFire`; попадание адресовано первому Bullet коллайдеру; урон проходит через очередь. Для

движения: Brain хранит цель и маршрут, `CharacterMovement` хранит намерение, Bullet владеет фактической позицией; при неактивности намерение обнулено.

Эти правила полезнее огромного списка «возможных тестов» без причин: каждый инвариант можно привязать к конкретной границе системы. Если он нарушен, ищите **первый проход, после которого данные стали неправильными**, а не последний визуальный симптом. NPC, который не стреляет, может быть исправен как оружие, но не иметь `targetVisible`; NPC, который не чинит, может правильно выбрать цель, но не иметь достижимой рабочей точки.

10. Диагностика по наблюдаемым данным

10.1. Ремонтник не начинает работу

В Object Debug и NPC Studio сначала найдите NPC Entity и ремонтную Entity. Для NPC проверьте `ActiveComponent`, `NPCComponent`, Brain, `isHostile=false`, `CrewMember.onDuty=true`, отсутствие активной `NpcCommand`. Для цели проверьте `Active`, `Repairable`, `!repaired` и либо `HullBreach&&!sealed`, либо `Damageable.health<maxHealth`. Эти условия прямо соответствуют `IsRepairTargetValid`, CrewTaskCoordinator.h:225–243 (../src/Gameplay/AI/CrewTaskCoordinator.h#L225). Затем сравните `brain.repairScore` с `brain.idleScore=0.10`, `CrewTaskCoordinatorSnapshot.openTasks`, `assignmentsThisFrame`, и `CrewTaskClaimComponent.assignee` на цели. Наличие score без claim означает, что локальная utility видит работу, но координатор мог назначить её другому или предпочесть другую пару NPC–задача.

Если claim есть и `brain.state=MoveToRepair`, но персонаж стоит, проверьте `brain.currentRoom`, `navigationTargetRoom`, `doorRouteIndex`, `activeDoor`, `debugDoorPhase`, `blockedReason`, `routeDirty` и `repathTimer`. `No traversable door route` означает проблему графа комнат/питания двери; `WAITING_FOR_OPEN` означает нормальное ожидание `openProgress`; `CROSSING_DIRECT` при нулевом физическом прогрессе указывает на коллизию портала или капсулы. StateSystem.h:2201–2499 (../StateSystem.h#L2201). `NpcNavMeshSystem::GetDebugInfo(npc)` сообщает `compiledIn`, `roomMeshBuilt`, `polygonCount`, путь, динамический обход и статус. NpcNavMeshSystem.h:29–60 (../src/Gameplay/AI/NpcNavMeshSystem.h#L29). При `compiledIn=false` прямой fallback ожидаем, но на сложном корабле он может вести в стену.

Если `brain.state=Repairing`, посмотрите фактическую `Repairable.progress`, `requiredProgress`, `brain.repairProgress`, `repairDuration`, `repairSkill`, `dt`. При `progress=requiredProgress` и статусе ожидания runtime операции исследуйте `HullBreachRepairLinkComponent` и завершение mesh repair. При полном HP оборудования, но `repaired=false`, проверьте, не было ли нового попадания, сбросившего прогресс; если функциональный флаг устройства остаётся сломанным после полного ремонта, расширьте `RepairSystem::CompleteRepair` именно для этого типа.

10.2. Противник не видит или не стреляет

Для восприятия последовательно проверьте `NPCCombatComponent`, `FactionComponent` на обоих участниках, здоровье и Active цели, расстояние до **ближайшего выбранного кандидата**, yaw наблюдателя, `fieldOfViewDegrees`, `seeRange`, Bullet мир и debug sight rays. Если видимый дальний враг игнорируется, посмотрите, не выбран ли более близкий враг за стеной: текущая система не ищет второго после отказа луча. `combat.targetVisible` должен быть истинным именно в нужный кадр, а `visibleAimPoint` — одна из проверенных высот цели. Если луч начинается внутри стены, проверьте кость головы и мировую матрицу модели.

Для стрельбы идите по NpcRangedCombatSystem.h:30–500 (../src/Gameplay/Combat/NpcRangedCombatSystem.h#L30) в порядке guard: Active, `PlayerGameTag` отсутствует у NPC, Brain, фракция, здоровье, ручная команда, экипированное оружие, патроны,

перезарядка, `Attacking` или разрешённый movingFire, `targetVisible`, время прицеливания, живой вражеский target, Transform и анимация, правильное имя кости руки, дальность от **дула**, ошибка yaw не более 8°, свободный короткий участок грудь–дуло, отсутствие союзника на линии. `rangedStatus` часто прямо говорит, где остановился проход: `Unarmed`, `Aiming`, `Turning toward target`, `Missing right-hand bone`, `Muzzle blocked by cover`, `Holding fire: ally in firing lane`, `Weapon cooling down`.

При статусе `Fired` и отсутствии потери HP проверьте shot ray: промах всё равно возвращает `true`. Если физический луч задел другое тело первым, `DamageEvent.target` не равен выбранной AI цели. Если событие есть, проверьте следующий шаг `DamageSystem::Update`, `InvulnerableComponent`, `PlayerComponent::godMode` и фактический тип здоровья цели. Для NPC без оружия `AnimationState::Punch` пока нельзя считать доказательством урона: этот код запускает клип, а не `DamageEvent`.

10.3. Короткая таблица симптомов

Наблюдение	Первое поле для проверки	Вероятная граница системы
`repairScore > 0`, назначения нет	`CrewTaskClaimComponent`, `assignedTask`, `onDuty`	Utility → Coordinator
Назначение есть, `progress=0`	`state`, `blockedReason`, дистанция до work zone	Coordinator → State/NavMesh
`progress=1`, пробоина открыта	статус ожидания runtime операции	State → RepairSystem/MeshCutting
NPC идёт на месте	`walkDirection`, фактический Transform, `stuckTimer`	State → Bullet controller
Враг близко, `targetVisible=false`	yaw, FOV, три sight ray, ближайший кандидат	Combat perception → Bullet raycast
`targetVisible=true`, выстрела нет	`rangedStatus`, aim time, кость, дуло, союзник	State/Animation → Ranged combat
`Fired`, HP не изменилось	weapon shot ray, `DamageEvent.target`, следующий кадр	Weapon → Damage queue

11. Что сейчас является пределом реализации

Эта глава различает подтверждённый путь и идеи развития. **Подтверждено кодом:** мирные NPC сами оценивают и получают ремонтные работы, маршрутизируются через комнаты и двери, чинят пробоины и оборудование через общий `RepairSystem`; боевые NPC выбирают враждебную цель по фракциям, проверяют личную видимость, помнят последнее наблюдение, ищут, занимают индивидуальное укрытие, могут стрелять через общий hitscan `WeaponSystem` с Bullet raycast и очередью урона. **Ограничения текущего пути:** групповой `NpcSquadCombatSystem::Update` отключён; безоружная NPC атака в `StateSystem` сейчас анимационная; выбор ближайшего врага предшествует проверке видимости; оценка ремонта использует прямое расстояние, а не фактическую стоимость маршрута; NPC вызов `ApplyRepairProgress` не передаёт `WorldEventLog` и автора действия.

Предложения к развитию, не объявленные готовым функционалом: объединить или документировать расхождение двух ремонтных scoring формул; выбирать лучшую **видимую** угрозу; добавить честное окно melee попадания и `DamageEvent`; подключить стоимость/доступность маршрута в назначение; провести источник NPC ремонта в WorldEventLog; вводить squad слой только после фиксации контрактов индивидуального боя. Каждое такое изменение должно сопровождаться конкретным сценарием, в котором проверяются входные компоненты, выходные поля, физический

результат и поведение при отмене/удалении цели. Так учебник остаётся точной картой движка, а новые идеи не маскируются под уже существующий код.

Архитектура сложного игрового AI без конфликтов систем

Хороший AI — не тот, у которого больше условий. Хороший AI знает только доступные персонажу факты, принимает устойчивое объяснимое решение, резервирует нужные ресурсы, выполняет действие через штатные системы и умеет признать неудачу. Эта глава проектирует следующий слой поверх уже существующих компонентов DX Framework.

1 Почему системы начинают конфликтовать

Конфликт появляется, когда несколько систем считают себя владельцами одного результата.

Пример плохого кадра:

- perception увидел противника и записал `Attacking`;
- cover system записал `TakingCover`;
- weapon search решил идти к винтовке и записал `Chasing`;
- damage reaction повернул transform;
- animation controller увидел скорость и включил run;
- squad system в конце кадра снова выдал flank target.

Каждая локальная причина выглядит разумно, но общего решения нет. Последняя система в порядке кадра случайно побеждает.

Исправление — не «правильнее отсортировать все if», а разделить полномочия.

2 Шесть слоёв AI

Устойчивый pipeline:

```
World truth
   ↓ sensing
Perception memory
   ↓ interpretation
Situation and needs
   ↓ arbitration
Chosen goal and intent
   ↓ planning
Action sequence
   ↓ execution
Movement Weapon Interaction Animation
   ↓ feedback
Success Failure Blocked Interrupted
```

2.1 World truth

Registry, Transform, physics world, health и inventory содержат реальность симуляции. NPC не должен автоматически читать всё это как знание.

2.2 Perception memory

Сюда попадают только подтверждённые наблюдения: видимая цель, услышанный выстрел, направление близкого попадания, сообщение союзника с уровнем доверия.

2.3 Situation

Интерпретация отвечает: угроза близко, патроны заканчиваются, NPC окружён, союзник ранен, выход заблокирован. Это производные факты, а не действие.

2.4 Goal и intent

Goal — желаемое состояние: выжить, получить оружие, удержать дистанцию, обезвредить противника. Intent — выбранное направление на ближайший период с причиной и обязательством.

2.5 Plan и action

Plan раскладывает цель на действия: найти оружие → зарезервировать → добежать → поднять → экипировать → вернуться в бой. Executor выполняет только текущее действие.

2.6 Feedback

Исполнитель не переписывает глобальную цель. Он сообщает `Succeeded`, `Failed`, `Blocked`, `Interrupted` и подробную причину. Planner или arbiter решает, что дальше.

3 Владение данными

Ниже рекомендуемая граница.

Данные	Единственный писатель	Читатели
Видимые контакты	PerceptionSystem	planner, combat, Studio
Последняя известная позиция	ContactMemorySystem	search, tactics
Выбранная стратегическая цель	GoalArbiter	planner, Studio
Текущий action	ActionPlanner	executors, animation
Путь и steering	Navigation executor	movement, Studio
Trigger и reload request	Weapon executor	WeaponSystem
Фаза оружия и ammo	WeaponSystem	AI, UI, animation
Animation action request	Action/animation bridge	AnimationController
Transform персонажа	movement/physics	все остальные
Tactical cover reservation	ReservationSystem	cover query, planner
Причина решения	модуль, создавший intent	Studio

Слово «единственный» относится к смысловому полю. Разные системы могут изменять разные части одного компонента, но по мере роста лучше разделять компоненты по владельцам.

4 Истина мира не равна знанию NPC

В текущем боевом коде уже заложено правильное правило: `targetEntity` может оставаться активным, но стрелять можно только при личной проверке `targetVisible`, а поиск использует `lastKnownPosition`.

Нужны три разных понятия:

- Entity identity: кого NPC считает вероятной целью.
- Confirmed observation: где цель действительно видна сейчас.
- Belief: где цель могла оказаться после потери контакта.

Belief не обновляется скрытым `Transform` игрока. Иначе NPC постоянно знает, где он.

4.1 Контакт от союзника

Сообщение напарника должно создавать `reported contact`, а не `targetVisible = true`. Получатель знает район и время сообщения, но обязан сам подтвердить цель перед точным огнём.

Полезные поля контакта:

```
struct ContactMemory
{
    Entity suspectedEntity = INVALID_ENTITY;
    DirectX::XMFLOAT3 lastPosition{};
    float confidence = 0.0f;
    float age = 0.0f;
    bool personallyConfirmed = false;
    ContactSource source = ContactSource::Sight;
};
```

Confidence убывает со временем. Оно влияет на решение проверить сектор, но не превращает догадку в линию прицеливания.

5 FSM Utility Behavior Tree и GOAP

Это не конкурирующие религии. Они отвечают на разные уровни.

5.1 FSM

Finite State Machine удобна для фазы исполнения: `Moving`, `Aiming`, `Reloading`, `Interacting`. Она плоха как единственный мозг, когда переходы между десятками целей образуют паутину.

5.2 Utility AI

Utility оценивает несколько целей числами. Хорошо подходит для выбора «бежать, искать оружие, продолжать бой, сдаться». Нужны нормализация, hard constraints, hysteresis и объяснение score.

5.3 Behavior Tree

Дерево удобно описывает реактивную последовательность с fallback. Без дисциплины blackboard и владения оно тоже может конфликтовать с другими системами.

5.4 GOAP

GOAP строит цепочку действий к желаемому состоянию на основе preconditions и effects. Полезен для редких составных целей, но избыточен для каждого короткого peek.

5.5 Рекомендация для DX Framework

Гибрид:

- Utility Goal Arbiter выбирает высокоуровневую цель.
- Небольшой planner или явная action sequence строит план.
- FSM каждого executor выполняет действие.
- Чистые tactical rules вычисляют локальные разрешения.
- Animation controller отображает semantic action в клипы.

Так сохраняется нынешняя понятная структура и появляется место для новых целей.

6 Blackboard без глобальной свалки

Blackboard полезен как структурированный снимок данных для решения. Он не должен быть `unordered_map<string, any>` без владельцев.

```
struct NpcDecisionContext
{
    Entity self = INVALID_ENTITY;
    float healthFraction = 1.0f;
    int ammoInMagazine = 0;
    int reserveAmmo = 0;
    bool hasWeapon = false;
    bool targetVisible = false;
    bool underFire = false;
    float targetDistance = 0.0f;
    float nearestWeaponPathCost = 0.0f;
    bool canSurrender = false;
};
```

Context строится один раз для цикла решения и передаётся как `const&`. Модули не ходят сами по Registry и не получают тайное знание. Результат становится тестируемым.

7 Goal Arbiter

Arbiter рассматривает кандидатов и выбирает один intent.

```
enum class NpcGoal
{
    None,
    Survive,
    Fight,
    KeepDistance,
    AcquireWeapon,
    SearchContact,
    HelpAlly,
    CaptureEnemy,
    Surrender,
    Escape
};

struct GoalCandidate
{
    NpcGoal goal = NpcGoal::None;
    float score = 0.0f;
    bool allowed = false;
    const char* reason = "";
};
```

7.1 Сначала запреты потом score

Нельзя компенсировать запрет огромным score. Если нет пути к оружию, `AcquireWeapon` не allowed. Если фракция запрещает плен, `CaptureEnemy` отсутствует. Если NPC не видел сдачу, он не знает, что противник готов.

Порядок:

1. hard constraints;
2. emergency overrides;
3. score допустимых целей;
4. hysteresis и commitment;
5. выбор;
6. причина решения.

7.2 Нормализация score

Каждый фактор приводится к `[0,1]`. Иначе расстояние в метрах случайно победит health fraction.

Пример функции срочности оружия:

```
unarmed = 1 если оружия нет, иначе 0
reachable = 1 - saturate(pathCost / maxUsefulCost)
dangerPenalty = saturate(exposureRisk)
weaponScore = unarmed * reachable * (1 - dangerPenalty)
```

Формула не обязана быть «умной». Она должна давать предсказуемые крайние случаи.

8 Устойчивость решения

Если два score близки, выбор может меняться каждый кадр. Используются четыре механизма.

8.1 Commitment

После выбора цель удерживается минимальное время, если не произошло критическое событие. Текущий `tacticCommitTimer` уже решает похожую задачу для стиля.

8.2 Hysteresis

Новая цель должна превысить текущую не на 0.001, а на margin:

```
switch if newScore > currentScore + 0.15
```

8.3 Cooldown

После отказа конкретному ресурсу даётся cooldown. `rejectedCoverTimer` — существующий пример.

8.4 Interrupt policy

Каждый action сообщает, чем его можно прервать.

- Reload можно прервать критической опасностью, но не слабым изменением score.
- Vault нельзя прервать в середине физического перехода.
- Aim можно прервать потерей видимости.
- Surrender может быть необратим после отброшенного оружия.

9 Тактические модули как пазл

Модуль не должен сам управлять NPC. Он предлагает intent.

```
struct TacticalProposal
{
    TacticalAction action;
    float score;
    DirectX::XMFLOAT3 targetPosition{};
    Entity targetEntity = INVALID_ENTITY;
    float commitment = 0.0f;
    const char* reason = "";
};

class ITacticalModule
{
public:
    virtual void Collect(
        const NpcDecisionContext& context,
        std::vector<TacticalProposal>& output) const = 0;
};
```

Примеры модулей:

- HoldCover;
- PeekAndFire;
- RetreatToRange;
- ChangeAngle;
- Flank;
- Suppress;
- CloseDistance;
- AcquireWeapon;
- Surrender;
- ArrestSurrenderedEnemy.

Arbiter выбирает одну заявку. Только executor меняет движение и trigger.

10 Как встроить текущие боевые системы

Не нужно выбрасывать работающий код.

- `NpcCombatSystem` остаётся perception/target-selection слоем и поддерживает confirmed visibility и память.
- `NpcTacticalDecisionSystem` остаётся чистым высокоуровневым правилом стиля и запроса укрытия.
- `NpcCoverQuery` остаётся сервисом поиска и оценки кандидатов.
- `NpcCoverSystem` постепенно становится executor жизненного цикла cover/peek.
- `NpcRangedCombatSystem` остаётся weapon executor: aim, muzzle, fire lane, ammo, reload.
- `NpcNavMeshSystem` остаётся навигационным сервисом и исполнителем пути.
- `NpcSquadCombatSystem` не должен напрямую перезаписывать индивидуальный intent; позже он публикует role/order proposal.

Первый рефакторинг — не перенос кода, а запись таблицы «кто имеет право менять поле» и устранение двойных писателей.

11 Бегство как цель Survive

NPC не должен убегать только по условию `health < 20`. Решение зависит от возможности и цены.

Факторы:

- health fraction;
- recent damage rate;
- число видимых угроз;
- наличие укрытия;
- дистанция до безопасной зоны;
- боезапас;
- наличие союзников;
- morale;
- приказ удерживать позицию;
- возможность сдачи.

11.1 Score бегства

Пример:

```
healthNeed = 1 - healthFraction
damageNeed = saturate(recentDamagePerSecond / dangerousDps)
outnumbered = saturate((enemyCount - allyCount) / 3)
escapeChance = 1 - pathRisk

escapeScore =
    0.35 * healthNeed +
    0.30 * damageNeed +
    0.20 * outnumbered +
    0.15 * escapeChance
```

Это начало настройки, не универсальная истина.

11.2 Safe destination

«Бежать от врага» не равно «бежать в точку по обратному вектору». Точка должна:

- находиться на NavMesh;
- увеличивать дистанцию или разрывать line of sight;
- иметь путь с приемлемым риском;
- не вести в тупик рядом с другой угрозой;
- не быть занята союзником, если узкая;
- иметь следующую точку выхода.

Можно оценивать кандидаты по кольцам NavMesh или tactical graph. Cover является одним фактором, но не единственной целью.

11.3 Отступление и паника

Тактическое отступление сохраняет оружие и может вести огонь короткими очередями. Паника снижает точность, игнорирует строй и выбирает ближайший очевидный выход. Это разные action profiles, даже если оба увеличивают дистанцию.

12 Поиск оружия

Текущий `InventorySystem` уже предоставляет надёжную операцию pickup, владение и слоты. AI-слою не нужно копировать оружие. Ему надо выбрать доступную сущность и выполнить обычное взаимодействие.

12.1 Когда искать

Кандидат `AcquireWeapon` допустим, если:

- NPC способен использовать оружие;
- активного подходящего оружия нет или оно бесполезно;
- существует известный pickup;
- pickup не принадлежит другому;
- есть свободный слот или разрешена замена;
- путь достижим;
- риск не превышает полезность.

Не надо искать Transform всех weapons через стены как знание. Источник знания: увиденный pickup, известный шкаф, команда, память об уроненном оружии.

12.2 Оценка оружия

```
weaponUtility =
    combatPower * ammoFactor * roleFit * condition -
    pathCost * distanceWeight -
    exposureRisk -
    reservationPenalty
```

Combat power учитывает damage, fire rate, range и тип цели. Role fit не должен быть жёстким запретом: оборонщик предпочитает дальнее оружие, но возьмёт пистолет вместо пустых рук.

12.3 Резервация

Два NPC не должны одновременно бежать к одной винтовке. Нужен общий сервис:

```
struct Reservation
{
    Entity resource = INVALID_ENTITY;
    Entity owner = INVALID_ENTITY;
    ReservationKind kind{};
    float expiresAt = 0.0f;
    std::uint32_t revision = 0;
};
```

Резервация имеет TTL и освобождается при успехе, отказе, смерти владельца или смене цели. Она не является владением предметом: `WeaponOwnerComponent` появляется только после pickup.

12.4 План

```
ReserveWeapon
→ NavigateToPickup
→ FacePickup
→ PlayPickupAction
→ InventorySystem::PickupWeapon
→ ReleaseReservation
→ Equip
```

Если оружие исчезло на третьем шаге, action возвращает `ResourceGone`. Planner не продолжает анимацию поднятия пустого места.

13 Боезапас и решение

Ammo принадлежит `WeaponComponent`. AI читает:

- ammo in magazine;
- reserve ammo;
- is reloading;
- reload timer;
- эффективную дальность.

Пустой магазин при наличии резерва создаёт `Reload` action. Нулевой общий запас может повысить `AcquireWeapon`, `TakeAmmo`, `Retreat` или `Surrender`. Он не должен заставлять `NpcRangedCombatSystem` притворяться, что оружия нет: executor сообщает состояние, arbiter выбирает цель.

14 Сдача

Сдача — изменение социального и боевого состояния, а не просто анимация рук вверх.

Условия:

- мораль ниже порога;
- противник демонстрирует превосходство;
- NPC понимает требование сдаться;
- есть право на сдачу по фракции/характеру;
- нет активной невыполнимой клятвы или scripted override;
- NPC не находится в физически непрерываемом действии.

Последствия:

- прекращается огонь;
- оружие блокируется или отбрасывается;
- faction relation для targeting меняется на `Surrendered`, а не обязательно Neutral;
- включается surrender animation action;
- появляется interaction для ареста;
- союзники получают наблюдаемое событие, но не телепатическую истину.

Нужен отдельный компонент состояния:

```
enum class ComplianceState
{
    None,
    ConsideringSurrender,
    Surrendering,
    Surrendered,
    Restrained,
    EscapingCustody
};
```

Не стоит кодировать это через `health = 0` или `isHostile = false`: потеряется смысл и появятся невозможные комбинации.

15 Взять в плен или убить

Это решение разделено между правилами мира и локальной тактикой.

15.1 Rules of engagement

Фракция или миссия задаёт:

- разрешена ли летальная сила;
- надо ли предлагать сдачу;
- какие цели должны быть захвачены живыми;
- считается ли сдающийся защищённым;
- можно ли стрелять по бегущему пленнику;
- кто имеет полномочие ареста.

AI не должен выводить моральные правила только из score.

15.2 Tactical feasibility

Даже при приказе захватить живым NPC оценивает:

- действительно ли цель сдалась;
- есть ли прикрытие во время подхода;
- не держит ли цель скрытое оружие;
- есть ли свободный союзник для overwatch;
- достижим ли interaction range;
- не появилась ли новая смертельная угроза.

15.3 Capture plan

```
ConfirmSurrender
→ ReservePrisonerInteraction
→ AssignCoveringActor optional
→ ApproachFromSafeArc
→ OrderDropWeapon
→ WaitForCompliance
→ RestrainInteraction
→ MarkRestrained
→ EscortToDestination
```

Если цель снова берёт оружие, compliance state меняется, interaction прерывается, а rules of engagement заново определяет допустимую силу.

16 Нелетальная сила

«Не убивать» не означает стрелять обычным оружием до 1 HP. Нужны отдельные damage effects:

- stun;
- suppression;
- pain/compliance;
- stamina loss;
- incapacitation;
- restraint.

Damage pipeline может передавать tags и magnitude. Health остаётся физическим повреждением, а consciousness/morale — отдельными ресурсами. Тогда электрошокер, удар, газ и команда сдаться не конкурируют за одно число.

17 Stamina

Stamina полезна, если она создаёт решения, а не просто полоску.

Расход:

- sprint;
- melee attack;
- удержание тяжёлого оружия;
- быстрый peek/lean;
- vault;
- сопротивление захвату.

Восстановление зависит от движения, posture, suppression и injuries.

AI не должен ждать полного нуля. Utility учитывает прогноз стоимости плана:

```
canSprintToCover = stamina >= pathSprintCost + safetyReserve
```

Владельцем stamina является character resource system. Movement executor запрашивает режим, resource system списывает фактический расход, planner читает снимок.

18 Morale

Morale медленнее краткого fear. На него влияют:

- лидер рядом;
- численное преимущество;
- смерть союзника;
- попадания и suppression;
- отсутствие боезапаса;
- изоляция;
- успех собственных действий;
- характер и обучение.

Morale не должен мгновенно прыгать. Используются накопление, decay и thresholds с hysteresis. Он влияет на score, но не напрямую записывает state.

19 Укрытие и выбор стороны

Текущий `NpcCoverQuery` уже проверяет protection, peek sight и path. Для укрытия с двумя сторонами одна Entity не обязана иметь один peek. Лучше представить cover как позицию защиты плюс набор firing slots:

```
struct CoverSlot
{
    DirectX::XMFLOAT3 peekPosition{};
    PeekSide side = PeekSide::Left;
    Stance stance = Stance::Stand;
    float exposure = 0.0f;
};
```

Каждый slot оценивается отдельно. Low cover может иметь crouch hide и stand peek. Высокая стена — left/right lean. Один slot резервируется конкретным NPC.

Важно проверять защиту несколькими лучами к телу: голова, грудь, таз. Один луч, упёршийся в низкую стену, не доказывает, что стоящий NPC защищён.

20 Навигация и tactical cost

Кратчайший путь не всегда лучший в бою. Ребро NavMesh может иметь дополнительные стоимости:

- exposure to known enemy;
- noise;
- narrow doorway;
- fire/hazard;
- crowd density;
- distance from desired combat range.

Полная динамическая стоимость каждого polygon может быть дорогой. Начать можно с оценки нескольких готовых путей по sampled points. Planner выбирает цель, NavMesh строит путь, tactical evaluator измеряет риск, затем при необходимости проверяется следующий кандидат.

21 Движение и стрельба

Текущие правила правильно разрешают moving fire только для боевого движения и при `targetVisible`. Executor должен дополнительно учитывать:

- animation/posture supports firing;
- muzzle не перекрыто;
- fire lane не содержит союзника;
- weapon не reloading;
- spread допустим;
- burst budget и pause;
- действие не требует обеих рук.

Стрельба является параллельным каналом только там, где контракты совместимы. Нельзя стрелять во время pickup, surrender или vault. Вместо огромной таблицы `if` action может публиковать capability mask:

```
enum ActionCapability
{
    CanAim = 1 << 0,
    CanFire = 1 << 1,
    CanSprint = 1 << 2,
    CanInteract = 1 << 3
};
```

Weapon executor читает разрешение текущего action.

22 Параллельные каналы действий

Один NPC может одновременно идти, смотреть и говорить. Но некоторые действия конфликтуют по телу и ресурсам.

Удобные каналы:

- locomotion;
- facing/aim;
- hands/weapon;
- interaction;
- speech;
- upper-body animation.

Action объявляет, какие каналы захватывает. Reload захватывает hands/weapon, но может разрешить walk. Vault захватывает locomotion и body animation. Surrender захватывает weapon и upper body.

Арбитр каналов предотвращает случай, когда reload и pickup одновременно управляют руками.

23 Групповая работа как отдельный слой

Индивидуальный AI должен работать без squad. Group coordinator читает публичные возможности и предлагает:

- role assignment;
- shared contact report;
- reservation of sectors;
- suppress/flank timing;
- prisoner overwatch.

Он не должен напрямую вести каждого по точкам каждый кадр. Приказ группы становится ограничением или proposal индивидуального arbiter. Боец может отклонить его по аварийной причине и записать отказ.

24 Объяснимость решения

Для каждого выбранного intent надо сохранить:

- имя цели;
- winning score;
- предыдущую цель и score;
- hard constraints;
- три главных фактора;
- commitment remaining;
- interrupt cause;
- plan step;
- последний failure.

Пример журнала:

```
Goal Fight → AcquireWeapon
score 0.82 vs 0.34
reason: unarmed 1.00, weapon reachable 0.91, exposure risk 0.18
reserved entity 42 for 4.0 s
```

Это полезнее строки «Moving to target».

25 Ошибки как данные

Action failure должен иметь тип:

```
enum class ActionFailure
{
    None,
    TargetGone,
    NoPath,
    PathBlocked,
    ReservationLost,
    InteractionRejected,
    WeaponUnavailable,
    InterruptedByDanger,
    AnimationMissing,
    Timeout
};
```

Planner может реагировать по-разному. `NoPath` запрещает кандидата на некоторое время. `ReservationLost` запускает поиск другого предмета. `AnimationMissing` обычно не отменяет gameplay и использует fallback.

26 Частота обновления

Не все слои работают каждый кадр.

- sensing raycasts: распределённо или по budget;
- target visibility для текущей цели: чаще;
- emergency reaction: каждый simulation tick;
- goal arbitration: 2–5 раз в секунду или по событию;
- plan rebuild: при смене цели или failure;
- steering и weapon execution: каждый tick;
- Studio capture: после штатных систем.

Событие будит редкий decision loop, но всё равно проходит arbiter. Так критический урон реагирует быстро, не превращая полный planner в 60 запусков в секунду.

27 Детерминизм и случайность

Случайность нужна для вариативности, но должна воспроизводиться. У каждого NPC или сценария хранится seed. Решение записывает выбранное случайное значение или индекс варианта.

Не использовать глобальный `rand()` из разных систем: порядок вызовов изменится после любого рефакторинга. Лучше отдельный RNG stream по назначению: tactics, animation variety, audio.

28 Сохранение игры

Не все runtime поля стоит сохранять.

Сохраняются:

- долгосрочная цель, если она важна;
- inventory и ammo;
- health, stamina, morale;
- compliance state;
- память контактов с timestamp относительно мира;
- selected style и характер;
- активный action только если он умеет безопасно восстановиться.

Не сохраняются:

- сырые указатели;
- временный путь NavMesh, если его можно перестроить;
- UI selection Studio;
- debug rays;
- pointer на reservation без revision и validation.

После загрузки выполняется revalidation: сущность ещё существует, путь строится, ресурс свободен.

29 Тестирование AI

29.1 Чистые unit tests

Тестировать utility и rules без мира:

- нулевое здоровье;
- нет оружия и один reachable pickup;
- pickup занят;
- surrender forbidden;
- два score на границе hysteresis;
- commitment и emergency interrupt.

29.2 Scenario tests

Маленькая сцена с фиксированным seed и входными событиями. Проверяется не точная позиция каждого кадра, а инварианты:

- NPC не стреляет без visibility;
- два NPC не владеют одной reservation;
- surrendered NPC не нажимает trigger;
- planner не перестраивается каждый кадр;
- отсутствующий clip не отменяет arrest.

29.3 Soak test

Десятки минут ускоренной симуляции ищут deadlock, утечки reservations, бесконечные планы и рост журнала.

30 Пошаговый план внедрения без большого взрыва

Этап 1 Карта владельцев

Записать все поля `NpcBrainComponent`, `NPCCombatComponent`, `NpcTacticalComponent` и системы-писатели. Найти поля с двумя писателями.

Этап 2 Intent рядом с текущим state

Добавить небольшой `NpcIntentComponent` с goal, action, reason, revision и commitment. Сначала только отображать в Studio, не менять поведение.

Этап 3 Один модуль

Перенести решение `AcquireWeapon` или `RetreatToRange` в чистый proposal. Старый combat path остаётся fallback.

Этап 4 Executor и feedback

Навигация к оружию возвращает typed result. Только после успеха вызывается `InventorySystem::PickupWeapon`.

Этап 5 Reservation service

Один общий сервис для weapon, cover slot и prisoner interaction.

Этап 6 Goal Arbiter

Подключить Fight, AcquireWeapon и Escape. Ввести hysteresis и журнал score.

Этап 7 Compliance

Добавить surrender/capture после того, как action ownership уже работает. Иначе социальные состояния умножат существующие конфликты.

Этап 8 Squad proposals

Вернуть командную работу поверх индивидуальных возможностей.

31 Пример полного решения

Ситуация: Sirius NPC имеет 25% здоровья, магазин пуст, резерв пуст, видит вооружённого Iridian на 9 м и знает о винтовке за стеной в 6 м.

1. Perception подтверждает Iridian и последнюю позицию.
2. Inventory snapshot сообщает отсутствие боезапаса.
3. Weapon knowledge содержит pickup и свежесть наблюдения.
4. Goal candidates:

- Fight allowed, score 0.18;
- AcquireWeapon allowed, score 0.62;
- Escape allowed, score 0.78;
- Surrender allowed, score 0.55.

5. Escape побеждает и получает commitment 3 с.
6. Safe-position query находит путь, разрывающий line of sight.
7. Executor резервирует destination slot и запускает sprint.
8. Weapon channel запрещает fire, потому что ammo 0.
9. При достижении безопасности score пересчитываются. AcquireWeapon теперь 0.81 из-за низкого риска.
10. Резервируется винтовка, строится plan pickup.
11. NPC поднимает реальную entity через `InventorySystem`.
12. Studio показывает обе смены целей и факторы.

Ни одна система не записывает всё поведение. Каждая предоставляет часть доказательства.

32 Практические задания

Задание 1 Владельцы

Для текущих `targetEntity`, `targetVisible`, `moveTarget`, `triggerHeld` и `currentState` выпиши всех писателей. Если писателей несколько, реши, кто должен стать владельцем, а кто — отправлять запрос.

Задание 2 Acquire weapon

На бумаге создай три оружия: близкое без патронов, далёкое хорошее, среднее занятое союзником. Рассчитай простые score и объясни reservation.

Задание 3 Escape

Придумай пять кандидатов NavMesh. Для каждого оцени расстояние, разрыв видимости, риск и тупик. Выбери не обязательно ближайший.

Задание 4 Capture

Нарисуй state diagram `Armed → Considering → Surrendering → Surrendered → Restrained`. Добавь прерывание «снова взял оружие» и «арестующий получил новую угрозу».

Задание 5 Конфликт каналов

Составь таблицу для Reload, Sprint, Peek, Pickup, Surrender и Arrest. Отметь locomotion, aim, hands, interaction и animation. Найди действия, которые могут выполняться параллельно.

Задание 6 Studio

Спроектируй вкладку Decision: goal scores, current commitment, plan steps, reservations и last failure. Каждый текст должен отвечать на вопрос, а не повторять имя state.

33 Критерий хорошего AI

AI можно считать архитектурно здоровым, если:

- скрытая цель не передвигает память NPC;
- одна система владеет одним смысловым выходом;
- решение отделено от исполнения;
- действие возвращает typed feedback;
- ресурсы резервируются и освобождаются;
- новый модуль предлагает intent, а не переписывает пять компонентов;
- score устойчивы commitment и hysteresis;
- анимация отображает действие, но не определяет игровую истину;
- Studio объясняет выбор и не подменяет pipeline;
- любой сложный сценарий можно воспроизвести фиксированным seed и журналом.

Тогда «лучший AI» означает не всезнающего и идеально стреляющего противника. Это персонаж, который знает ограниченную информацию, делает понятные ошибки, адаптируется к последствиям и остаётся расширяемым для разработчика.

Построчный разбор систем DX Framework

Эта часть разбирает текущий снимок проекта, а не абстрактный учебный движок. Номера строк относятся к версии кода на момент сборки книги. После правок номера могут сместиться, поэтому главным ориентиром остаются имя файла, имя метода и сама строка.

1 Как читать систему до изменения кода

Читай каждый `Update` в пять проходов:

1. Какие сущности система перебирает.
2. Какие компоненты только читает, а какие изменяет.
3. Какие guard clauses досрочно прекращают обработку.
4. Где принимается решение и где оно исполняется.
5. Какие значения надо показать в инспекторе, чтобы увидеть причину решения.

Не пытайся сразу удержать весь файл в голове. Сначала проследи один конкретный путь: например, «NPC увидел цель, дошёл до края укрытия и сделал один выстрел».

2 ECSRegistry и перебор двух компонентов

Файл `src/ECS/ECSRegistry.h`, метод `ForEach<T1, T2>`.

```
template<typename T1, typename T2, typename Fn>
void ForEach(Fn&& fn)
{
    auto* pool1 = FindPool<T1>();
    auto* pool2 = FindPool<T2>();
    if (!pool1 || !pool2) return;
    for (auto& [entity, comp1] : *pool1)
    {
        if (pool2->Has(entity))
        {
            auto& comp2 = *pool2->Get(entity);
            fn(entity, comp1, comp2);
        }
    }
}
```

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`template<typename T1, typename T2, typename Fn>`	Метод получает два типа компонентов и тип функции обратного вызова	Один код работает для любых пар компонентов	Жёстко заданные типы породят отдельный цикл на каждую систему
`void ForEach(Fn&& fn)`	Принимается callable без лишнего копирования	Лямбда может захватывать данные системы	Передача тяжёлой функции по значению создаст копии
`pool1 = FindPool<T1>()`	Ищется существующее хранилище первого типа	Перебирать можно только созданный pool	Создание pool при чтении скроет отсутствие компонентов
`pool2 = FindPool<T2>()`	Ищется хранилище второго типа	Оно будет проверяться для каждой сущности	Без него нельзя получить `comp2`
`if (!pool1 || !pool2) return`	Вся операция прекращается, если одного типа вообще нет в мире	Нет ни одного совпадения	Разыменование `nullptr` приведёт к падению

Строка	Что происходит	Зачем это здесь	Что сломается при ошибке
`for (... : *pool1)`	Перебирается только первый pool	Стоимость зависит от размера первого pool	Если первым выбран очень массовый компонент, цикл будет дороже
`pool2->Has(entity)`	Проверяется наличие второго компонента у той же сущности	Формируется пересечение двух множеств	Без проверки `Get` может вернуть `nullptr`
`comp2 = *pool2->Get(entity)`	Получается ссылка на второй компонент	Лямбда может читать и изменять его	Копия не сохранит изменения в ECS
`fn(entity, comp1, comp2)`	Управление передаётся системе	Registry не знает игровой логики	Неверный порядок аргументов даст путаницу типов

Важная деталь производительности

Этот `ForEach` всегда перебирает `pool1`. Поэтому полезно первым указывать более редкий компонент. Если `TransformComponent` есть у 10 000 сущностей, а `NPCCombatComponent` у 40, вызов `ForEach<NPCCombatComponent, TransformComponent>` выгоднее обратного порядка.

Мини задание

Найди вызов `ForEach<FactionComponent, TransformComponent>` в выборе цели. Ответь: сколько сущностей попадёт во внешний цикл и сколько проверок будет сделано, если Faction есть у 20 сущностей, а Transform у 5000.

3 TransformComponent и мировая матрица

Файл `src/ECS/Components/Components.h`, метод `GetWorldMatrix`.

```
XMMATRIX GetWorldMatrix() const
{
    XMMATRIX scaleMatrix = XMMatrixScaling(scale.x, scale.y, scale.z);
    XMMATRIX rotX = XMMatrixRotationX(rotation.x);
    XMMATRIX rotY = XMMatrixRotationY(rotation.y);
    XMMATRIX rotZ = XMMatrixRotationZ(rotation.z);
    XMMATRIX rotationMatrix = rotX * rotY * rotZ;
    XMMATRIX translationMatrix = XMMatrixTranslation(position.x, position.y, position.z);
    return scaleMatrix * rotationMatrix * translationMatrix;
}
```

Строка	Что происходит	Откуда берутся данные	Проверка
`GetWorldMatrix() const`	Строится матрица из текущего Transform	`position`, `rotation`, `scale` принадлежат компоненту	Метод не должен менять компонент
`XMMatrixScaling`	Создаётся масштаб по трём осям	`scale` по умолчанию `(1,1,1)`	Нулевой масштаб мешает обратной матрице
`XMMatrixRotationX`	Поворот вокруг X	`rotation.x` должен быть в радианах	Градусы дадут неверный огромный угол
`XMMatrixRotationY`	Поворот вокруг вертикальной оси	Для NPC это обычно yaw	Проверь знак и направление модели
`XMMatrixRotationZ`	Поворот вокруг Z	Roll или наклон	Неверная ось наклонит не туда
`rotX * rotY * rotZ`	Три поворота соединяются	Порядок является соглашением проекта	Перестановка изменит ориентацию
`XMMatrixTranslation`	Создаётся перенос в мировую позицию	`position` должна быть мировой	Локальная позиция родителя потребует иерархии

Строка	Что происходит	Откуда берутся данные	Проверка
`scale * rotation * translation`	Возвращается полный локально-мировой конвейер	DirectXMath использует выбранную проектом конвенцию	Другой порядок заставит объект вращаться вокруг иной точки

Как проверить без веры в формулу

Поставь объект в `(5, 0, 0)`, масштаб `(2, 1, 1)`, поворот Y на 90 градусов. Возьми локальную точку `(1, 0, 0)` и выведи её мировую позицию после `XMVector3TransformCoord`. Затем по одному отключай масштаб, поворот и перенос. Так видно вклад каждой матрицы.

4 NpcTacticalDecisionSystem и выбор стиля

Файл `NpcTacticalDecisionSystem.h`, метод `UpdateStyle`.

```
const float dx = threatPosition.x - npcPosition.x;
const float dz = threatPosition.z - npcPosition.z;
const float threatDistance = std::sqrt(dx * dx + dz * dz);

if (tactical.hasThreatDistanceSample && deltaTime > 0.0001f)
{
    tactical.enemyClosingSpeed =
        (tactical.lastThreatDistance - threatDistance) / deltaTime;
}
else
{
    tactical.enemyClosingSpeed = 0.0f;
    tactical.hasThreatDistanceSample = true;
}
tactical.lastThreatDistance = threatDistance;
```

Строка	Состояние до строки	Результат строки	Почему важно
`dx = threat.x - npc.x`	Есть две мировые позиции	Горизонтальная разница по X	Знак показывает сторону, квадрат убирает знак для длины
`dz = threat.z - npc.z`	То же по Z	Горизонтальная разница по Z	Y намеренно исключён из тактики дистанции
`sqrt(dx*dx + dz*dz)`	Есть катеты	Получено расстояние на плоскости	Для вывода скорости нужна настоящая длина, не квадрат
`hasThreatDistanceSample`	Первый кадр ещё не имеет истории	Определяется, можно ли сравнивать измерения	Без истории скорость сближения будет мусором
`deltaTime > 0.0001f`	Кадр мог быть паузой или нулевым шагом	Защита деления	Деление на ноль создаст бесконечность
`last - current`	Известны две дистанции	Положительный результат означает сближение	Обратный порядок поменяет смысл знака
`/ deltaTime`	Есть изменение за кадр	Получаются метры в секунду	Без деления значение зависит от FPS
`enemyClosingSpeed = 0`	Нет достоверной пары измерений	Безопасное начальное значение	NPC не принимает случайное решение в первый кадр
`hasThreatDistanceSample = true`	Первый образец принят	Следующий кадр сможет вычислить скорость	Флаг должен ставиться только после реального измерения
`lastThreatDistance = threatDistance`	Старое значение больше не нужно	Сохранена база для следующего кадра	Если записать раньше расчёта, скорость всегда будет нулём

Далее система уважает ручной выбор:

```
if (tactical.selectedStyle != NpcTacticalStyle::Auto)
{
    tactical.activeStyle = tactical.selectedStyle;
    tactical.tacticCommitTimer = 0.0f;
    tactical.tacticReason = "Стиль принудительно выбран в инспекторе";
    return;
}
```

Строка	Смысл
`selectedStyle != Auto`	Дизайнер хочет детерминированный тест, а не авто выбор
`activeStyle = selectedStyle`	Исполняемый стиль синхронизируется с настройкой
`commitTimer = 0`	Для ручного режима не нужна фиксация авто решения
`tacticReason = ...`	Инспектор объясняет источник решения
`return`	Ни одно авто правило ниже не имеет права перезаписать стиль

Затем вычисляются понятные булевы факты, а уже из них выбирается стиль. Это лучше одного гигантского `if`: каждый факт можно показать в отладчике и протестировать отдельно.

```
const bool enemyIsDangerouslyClose =
    threatDistance < (std::max)(2.5f, preferredRange * 0.35f);
const bool enemyIsCloseForCautiousNpc =
    threatDistance < preferredRange * 0.75f && aggression < 0.65f;
const bool enemyIsPushing = tactical.enemyClosingSpeed > 0.75f;
const bool cautiousUnderFire =
    tactical.underFireTime > 0.0f && aggression < 0.65f;
```

Порядок последующих `if` задаёт приоритет. Опасная близость должна проверяться раньше высокой агрессии, иначе агрессивный NPC продолжит пушить даже в неудобной почти рукопашной дистанции.

5 NpcCoverQuery и доказательство защиты

Файл `NpcCoverQuery.h`, метод `IsProtectedFromTarget`.

```
targetPosition.y += 1.0f;
coverPosition.y += (std::max)(0.2f, protectionHeight);

const XMFLOAT3 direction{
    coverPosition.x - targetPosition.x,
    coverPosition.y - targetPosition.y,
    coverPosition.z - targetPosition.z
};
const float distance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&direction)));

if (distance <= 0.01f)
    return false;
```

Строка	Геометрический смысл	Почему не иначе
`target.y += 1.0f`	Источник поднимается примерно на уровень оружия	Луч от пола ошибочно признает низкий бордюр полной защитой
`cover.y += protectionHeight`	Проверяется конкретная высота защищаемого тела	Одна высота не доказывает защиту головы и груди одновременно
`direction = cover - target`	Луч направлен от угрозы к кандидату	Обратный вектор полетит от укрытия прочь от врага
`Length(direction)`	Получается конечная дальность луча	Бесконечный ray может подобрать геометрию за кандидатом

Строка	Геометрический смысл	Почему не иначе
`distance <= 0.01f`	Отклоняется вырожденная проверка	Нормализация и raycast нулевого отрезка ненадёжны

Продолжение:

```
    const auto* targetController =
        registry.GetComponent<CharacterController>(targetEntity);
    const btCollisionObject* targetBody =
        targetController ? targetController->ghostObj : nullptr;

    PhysicsRaycastHit hit{};
    PhysicsRaycast::CastRay(
        targetPosition,
        direction,
        world,
        distance,
        targetEntity,
        targetBody,
        hit);

    return hit.collisionObject &&
        hit.collisionObject->isStaticOrKinematicObject();
```

Строка	Что происходит	Возможная ошибка
`GetComponent<CharacterController>`	Ищется физическое представление цели	Не все сущности обязаны иметь controller
тернарный оператор	Безопасно выбирается `ghostObj` или `nullptr`	Разыменование отсутствующего controller уронит игру
`PhysicsRaycastHit hit{}`	Результат обнуляется перед вызовом	Старое попадание не должно пережить новый луч
`CastRay`	Проверяется только отрезок до позиции укрытия	Большая дальность даст ложную стену за точкой
`targetEntity`, `targetBody`	Цель исключается из собственного луча	Луч иначе столкнётся со стрелком в самом начале
`hit.collisionObject`	Должно существовать препятствие	Свободная линия означает отсутствие защиты
`isStaticOrKinematicObject()`	Защитой признаётся стабильная геометрия	Живой NPC не должен считаться стеной

Ограничение одной высоты

Этот метод отвечает только на вопрос про одну выбранную высоту. Для стойки нужны минимум лучи к груди и голове. Для приседа можно проверять более низкие точки. Поэтому будущая оценка укрытия должна возвращать не просто `bool`, а профиль: `standingProtected`, `crouchProtected`, `leftPeekValid`, `rightPeekValid`.

6 StateSystem и исполнение движения в укрытие

Файл `StateSystem.h`, ветка `NpcState::TakingCover`.

```
auto* tactical = registry.GetComponent<NpcTacticalComponent>(npc);
auto* combat = registry.GetComponent<NPCCombatComponent>(npc);
if (!tactical || !combat || !combat->hasLastKnownPosition)
{
    brain.state = NpcState::Chasing;
    break;
}
```

Строка	Кто владеет данными	Решение
получение `tactical`	Тактический компонент хранит цель движения и фазу укрытия	Без него состояние не может исполниться
получение `combat`	Боевой компонент хранит контакт	Укрытие оценивается относительно угрозы
guard clause	Проверяет минимальный контракт состояния	Нельзя продолжать с неполными данными
`state = Chasing`	Выбирается восстановительное состояние	Сейчас это политика; защитнику позже может подойти `Searching` или удержание позиции
`break`	Завершается только текущая ветка `switch`	`return` мог бы прекратить обработку всей сущности или системы в зависимости от контекста

Выбор режима перемещения:

```
NPCMovementComponent tacticalMovement = npcMovement;
if (tactical->movingToPeek)
{
    tacticalMovement.speed = (std::max)(
        tacticalMovement.speed,
        tactical->peekMoveSpeed);
    tacticalMovement.allowRunning = false;
}
else
{
    tacticalMovement.allowRunning = npcMovement.allowRunning;
    tacticalMovement.runWhenFartherThan = 0.0f;
}
```

Строка	Смысл	Почему создана копия
`tacticalMovement = npcMovement`	Берутся обычные настройки NPC	Исходный компонент не меняется навсегда
`movingToPeek`	NPC идёт к точке выглядывания, а не к безопасной точке	Эти движения должны выглядеть по-разному
`max(speed, peekMoveSpeed)`	Выглядывание не станет медленнее обычной ходьбы	Резкая анимация требует короткого перемещения
`allowRunning = false`	Выглядывание не переключает locomotion в Run	Позже его заменит отдельный lean или peek clip
ветка `else`	NPC реально бежит к укрытию	Под огнём медленная прогулка неверна
`runWhenFartherThan = 0`	Бег разрешён на любой ненулевой дистанции	Обычный порог дальности здесь слишком поздний

Навигация и прибытие:

```
const NavigationResult result = NavigateTo(
    registry, npc, brain, transform, movement,
    tacticalMovement, tactical->moveTarget,
    room, npcMovement.arrivalDistance,
    NpcState::TakingCover, deltaTime);

if (result == NavigationResult::Arrived)
{
    StopMovement(movement);
    if (tactical->movingToPeek)
    {
        tactical->exposed = true;
        tactical->crouched = false;
        tactical->phaseTimer = (std::max)(
            combat->rangedAimDelay + 1.2f,
            tactical->exposeDuration);
        brain.state = NpcState::Attacking;
    }
    else
    {
        tactical->exposed = false;
        tactical->crouched = tactical->coverUsesCrouch;
        tactical->phaseTimer = (std::max)(0.1f, tactical->hideDuration);
        brain.state = NpcState::InCover;
    }
}
```

Строка	Изменение состояния	Инвариант после строки
`NavigateTo`	Исполнитель пробует приблизиться к `moveTarget`	Решение о точке уже принято другой системой
`Arrived`	Путь закончен	Нельзя оставлять старую скорость
`StopMovement`	Скорость и флаг движения сбрасываются	Анимация не продолжает бег на месте
`movingToPeek`	Различаются край укрытия и защищённая позиция	Одна координата не должна означать обе фазы
`exposed = true`	Тактика знает, что NPC доступен для огня	Получение урона в этой фазе может увеличить `failedPeekCount`
`crouched = false`	Для текущего peek используется стойка	Будущий профиль укрытия сможет выбрать crouch-peek
`phaseTimer = max(...)`	Окно включает прицеливание и короткую очередь	Слишком короткий таймер вернёт NPC до выстрела
`state = Attacking`	Управление передаётся боевой ветке	Сам `TakingCover` не стреляет
защищённая ветка	NPC скрыт и ждёт	`phaseTimer` становится временем ожидания до peek
`state = InCover`	Управление передаётся фазе укрытия	Следующий переход должен быть явным

Если `NavigateTo` возвращает `Blocked`, резервирование сбрасывается, таймер решения обнуляется и поиск должен получить шанс выбрать другую точку. Это пример восстановления после ошибки исполнения, а не нового тактического решения.

7 NpcRangedCombatSystem и точка дула

Файл `src/Gameplay/Combat/NpcRangedCombatSystem.h`.

```
XMMATRIX hand;
if (!mesh->model->TryGetBoneModelTransform(
        *animation,
        weapon->rightHandBone,
        hand))
{
    combat.rangedStatus = "Missing right-hand bone";
    return;
}
```

Строка	Что проверяется	Почему выстрел запрещается
`XMMATRIX hand`	Подготавливается место для матрицы кости	Дуло привязано к текущей анимационной позе
`TryGetBoneModelTransform`	Модель ищет кость, заданную оружием	Имена костей отличаются между моделями
`!result`	Кость не найдена	Вычислять мировое дуло не из чего
`rangedStatus`	Причина сохраняется для инспектора	Иначе NPC молча перестанет стрелять
`return`	Текущая попытка выстрела прекращается	Подстановка нулевой матрицы даст луч из неверного места

Мировая точка дула:

```
XMStoreFloat3(
    &origin,
    XMVector3TransformCoord(
        XMLoadFloat3(&weapon->muzzlePosition),
        WeaponIKSystem::GripMatrix(weapon->worldGrip) *
            hand *
            transform->GetWorldMatrix()));
```

Читаем изнутри наружу:

1. `muzzlePosition` — точка в локальном пространстве оружия.
2. `GripMatrix` учитывает хват и ориентацию оружия.
3. `hand` переносит оружие в текущую позу кости.
4. `GetWorldMatrix` переносит персонажа в мир уровня.
5. `TransformCoord` применяет полный конвейер к точке.
6. `XMStoreFloat3` сохраняет SIMD-вектор в обычный `XMFLOAT3`.

Направление строится только к точке, подтверждённой личным зрением:

```
XMFLOAT3 aimPoint = combat.visibleAimPoint;
XMFLOAT3 direction{
    aimPoint.x - origin.x,
    aimPoint.y - origin.y,
    aimPoint.z - origin.z
};
```

`targetEntity` может оставаться активным после ухода игрока за стену. Поэтому он не является разрешением стрелять. `visibleAimPoint` должен обновляться только при текущем подтверждённом контакте.

8 Проверка chest to muzzle

```
XMFLOAT3 chest = transform->position;
chest.y += 1.45f;

const XMFLOAT3 toMuzzle{
    origin.x - chest.x,
    origin.y - chest.y,
    origin.z - chest.z
};

const float muzzleDistance = XMVectorGetX(
    XMVector3Length(XMLoadFloat3(&toMuzzle)));
```

Строка	Геометрия	Ошибка, которую ловит
`chest = position`	Берётся мировая база персонажа	Нужна стабильная точка внутри тела
`chest.y += 1.45`	Точка поднимается к груди	Луч от ног пересёк бы пол или низ стены
`origin - chest`	Направление идёт к дулу	Проверяем, не высунулся ли ствол сквозь геометрию
`Length`	Получается длина короткого отрезка	Raycast не должен идти дальше дула

После raycast любое препятствие на этом отрезке ставит статус `Muzzle blocked by cover` и запрещает выстрел. Эта проверка не заменяет линию огня: она отвечает только за участок внутри позы NPC и укрытия.

9 Проверка линии огня и союзника

```
PhysicsRaycastHit fireLane{};
PhysicsRaycast::CastRay(
    origin,
    direction,
    world,
    distance,
    npc,
    ownBody,
    fireLane);
```

Аргумент	Точное назначение
`origin`	мировое дуло
`direction`	ненормализованный вектор к видимой точке; обёртка raycast знает длину отдельно
`world`	физический мир
`distance`	конец проверки находится на цели, а не за ней
`npc`	сущность стрелка исключается
`ownBody`	физический ghost стрелка исключается
`fireLane`	первое попадание возвращается сюда

```
const auto* blockingNpc =
    registry.GetComponent<NPCComponent>(fireLane.entity);
const auto* allyFaction =
    registry.GetComponent<FactionComponent>(fireLane.entity);

const bool friendlyInFireLane =
    blockingNpc &&
    blockingNpc->health > 0.0f &&
    allyFaction &&
    GetFactionRelation(
        observerFaction->faction,
        allyFaction->faction) == FactionRelation::Friendly;
```

Булево выражение читается слева направо благодаря short circuit:

1. Первый объект луча должен быть NPC.
2. Он должен быть жив.
3. У него должна быть фракция.
4. Отношение фракций должно быть Friendly.

Только после успешной проверки указателя читаются его поля. Переставлять `blockingNpc->health` перед `blockingNpc` нельзя.

10 WeaponSystem и одна попытка выстрела

Файл `src/Gameplay/Combat/WeaponSystem.h`, метод `TryFire`.

```
if (!world || !registry.IsAlive(shooterEntity) ||
    !registry.HasComponent<ActiveComponent>(shooterEntity) ||
    !WeaponFiringRules::ValidRay(origin, direction))
    return false;
```

Проверка	Что гарантирует следующему коду
`world`	физический raycast можно вызвать
`IsAlive`	entity не уничтожен
`ActiveComponent`	выключенная сущность не стреляет
`ValidRay`	координаты конечны и направление ненулевое

Поиск оружия и перезарядка:

```
const Entity weaponEntity =
    GetEquippedWeaponEntity(registry, shooterEntity);
auto* weapon = registry.GetComponent<WeaponComponent>(weaponEntity);
if (!weapon)
    return false;

if (weapon->ammoInMagazine <= 0)
{
    WeaponFiringRules::StartReload(*weapon);
    return false;
}

if (!WeaponFiringRules::CanFire(*weapon))
    return false;
```

Строка	Состояние оружия	Результат
`GetEquippedWeaponEntity`	Стрелок может иметь инвентарь и несколько предметов	Находится именно экипированное оружие
`GetComponent<WeaponComponent>`	Entity мог быть неверным или предмет не оружие	Получается механическое состояние

Строка	Состояние оружия	Результат
`ammo <= 0`	Магазин пуст	Запускается таймер перезарядки
`return false`	В этот кадр пуля не создаётся	Перезарядка не считается выстрелом
`CanFire`	Проверяет кулдаун, reload и другие правила	Все стрелки используют один контракт

Разброс и фиксация выстрела:

```
XMStoreFloat3(&normalized,
    XMVector3Normalize(XMLoadFloat3(&direction)));

const auto* movement =
    registry.GetComponent<CharacterMovement>(shooterEntity);
const float motionScale = WeaponFiringRules::MotionMultiplier(
    *weapon,
    movement && movement->isMoving);

const float radial = sample(random);
const float angular = sample(random);
const XMFLOAT3 muzzleDirection = normalized;
normalized = WeaponFiringRules::ConeDirection(
    normalized,
    weapon->spreadDegrees * motionScale +
        weapon->currentRecoilDegrees,
    radial,
    angular);

WeaponFiringRules::CommitShot(*weapon, motionScale);
```

Строка	Почему расположена именно здесь
нормализация	Дальность и конус ожидают единичное направление
чтение движения	Разброс зависит от состояния стрелка
`movement && ...`	Отсутствующий компонент безопасно трактуется как неподвижность
две случайные выборки	Одна задаёт радиус, другая угол внутри конуса
сохранение `muzzleDirection`	Muzzle flash должен смотреть по стволу, а не по случайному отклонению пули
`ConeDirection`	Получается фактический луч этого выстрела
`CommitShot`	Списывается патрон, ставится кулдаун и отдача

Сначала фиксируется выстрел, затем делается raycast. Поэтому промах всё равно тратит патрон и создаёт кулдаун — это правильная механика.

Попадание превращается в событие:

```
PhysicsRaycastHit rayHit{};
const bool hit = PhysicsRaycast::CastRay(
    origin,
    normalized,
    world,
    weapon->range,
    shooterEntity,
    shooterObject,
    rayHit);

if (!hit)
    return true;

DamageEvent damageEvent{};
damageEvent.source = weaponEntity;
damageEvent.target = rayHit.entity;
damageEvent.damageAmount = weapon->damage;
damageSystem.AddDamageEvent(damageEvent);
return true;
```

`true` означает «выстрел состоялся», а не «цель получила урон». Это различие нужно для анимации, звука, расхода патронов и тактического поведения.

11 Один кадр боя от восприятия до урона

Полезно читать не отдельные классы, а движение данных:

1. Система восприятия подтверждает личную видимость и обновляет `visibleAimPoint`.
2. Боевая система хранит `targetEntity`, но не считает его вечной видимостью.
3. Тактическое решение выбирает стиль и запрос укрытия.
4. Cover query оценивает кандидатов лучами относительно подтверждённой угрозы.
5. `StateSystem` исполняет `moveTarget` через навигацию.
6. В состоянии атаки `NpcRangedCombatSystem` вычисляет мировое дуло.
7. Проверяются поворот, chest-to-muzzle и дружественная линия огня.
8. `WeaponSystem::TryFire` проверяет оружие, создаёт разброс и физический луч.
9. Первое попадание превращается в `DamageEvent`.
10. `DamageSystem` применяет урон и записывает факт угрозы для следующего тактического решения.

Если NPC не стреляет, проверяй цепочку сверху вниз. Не начинай сразу с `TryFire`: до него мог не дойти вызов.

12 Метод локализации бага

Для каждого перехода храни три строки отладки:

- `decision`: чего NPC хотел;
- `reason`: почему он этого захотел;
- `execution`: что исполнитель реально сделал.

Пример:

```
decision: перейти к укрытию 41
reason: два попадания во время выглядывания
execution: путь не найден, cover 41 отклонено на 4 секунды
```

Это различает ошибку решения и ошибку исполнения. Если решение правильное, но NPC стоит, смотри NavMesh и состояние движения. Если путь есть, но выбрана плохая сторона стены, смотри геометрию cover query и score.

13 Практика по чтению кода

Задание 1 ECS

Нарисуй два множества сущностей: с `NPCCombatComponent` и с `TransformComponent`. Руками отметь пересечение, которое получит лямбда.

Задание 2 матрица дула

В отладке покажи четыре точки разными цветами: локальное дуло после grip, после hand, после world и окончательный `origin`. Если одна стадия прыгает, ты сразу узнаешь виноватую матрицу.

Задание 3 укрытие

Для одного кандидата нарисуй лучи к ногам, груди и голове. В журнале напиши, какой луч встретил статическую геометрию первым.

Задание 4 выстрел

Поставь союзника между двумя врагами. Запиши по кадрам `fireLane.entity`, отношение фракций и `rangedStatus`. Затем убери союзника и сравни цепочку.

Задание 5 объяснение

Не глядя в книгу объясни вслух, почему активный `targetEntity` не даёт права стрелять. Правильный ответ должен упомянуть текущую личную видимость, укрытие и первое физическое попадание луча.

14 Контрольный список

После разбора ты должен уметь:

- отличить выбор сущностей ECS от игровой логики лямбды;
- назвать пространство каждой точки в расчёте дула;
- объяснить, почему `lastKnownPosition` не является всезнанием;
- разделить решение об укрытии, поиск геометрии и исполнение движения;
- прочитать guard clause как контракт следующего участка;
- отличить «выстрел состоялся» от «луч попал»;
- найти, какая система владеет изменяемым полем;
- построить цепочку данных одного кадра;
- добавить отладку причины, а не только результата.

Справочник полезных алгоритмов

Этот раздел нужен не для заучивания. Перед новой механикой найди похожую задачу, выпиши входы и выходы, затем реализуй минимальный вариант и обязательно добавь визуальную проверку.

1 Безопасная нормализация

Задача: получить единичное направление без `NaN`.

Вход: произвольный вектор `v`.

Выход: единичный вектор и признак успеха.

```
bool TryNormalize(const XMFLOAT3& value, XMFLOAT3& result)
{
    const XMVECTOR vector = XMLoadFloat3(&value);
    const float lengthSquared = XMVectorGetX(XMVector3LengthSq(vector));
    if (lengthSquared <= 0.000001f)
        return false;

    XMStoreFloat3(&result, XMVector3Normalize(vector));
    return true;
}
```

Сложность: O(1).

Ловушка: проверять длину уже после нормализации.

Применение: лучи зрения, движение, направление ствола, steering.

2 Квадрат расстояния

Задача: быстро сравнить кандидатов или радиус.

```
float DistanceSquared(const XMFLOAT3& a, const XMFLOAT3& b)
{
    const float dx = a.x - b.x;
    const float dy = a.y - b.y;
    const float dz = a.z - b.z;
    return dx * dx + dy * dy + dz * dz;
}
```

Сравнивай с `radius * radius`. Квадратный корень нужен только для настоящего расстояния.

Сложность: O(1) на кандидата, O(n) для полного перебора.

3 Поле зрения через dot

Вход: нормализованный `forward`, направление к цели, полный угол FOV.

```
const float halfFov = XMConvertToRadians(fovDegrees * 0.5f);
const float threshold = std::cos(halfFov);
const bool inside = Dot(forward, normalizedToTarget) >= threshold;
```

Ловушка: использовать полный угол вместо половинного.

Применение: дешёвый фильтр перед raycast. После dot всё равно проверяется препятствие.

4 Кратчайшая разница углов

```
float ShortestAngle(float current, float target)
{
    return std::atan2(
        std::sin(target - current),
        std::cos(target - current));
}
```

Выход: значение от `-pi` до `pi`.

Применение: поворот NPC, турели, оценка ошибки прицеливания.

5 Луч и отрезок

Луч бесконечен при `t >= 0`. Для оружия и зрения обычно нужен отрезок с максимальным расстоянием.

```
struct SegmentRay
{
    XMFLOAT3 origin{};
    XMFLOAT3 direction{}; // Единичный вектор.
    float maxDistance = 0.0f;
};
```

Создание из двух точек:

1. `delta = end - start`;
2. `distance = length(delta)`;
3. отклонить почти нулевую длину;
4. `direction = delta / distance`;
5. `maxDistance = distance`.

Ловушка: передать ненормализованное направление в функцию, которая считает `t` метрами.

6 Ray plane

Формула:

$$t = ((Q - O) \cdot N)/(D \cdot N)$$

Порядок: проверить параллельность, получить `t`, проверить диапазон, вычислить точку.

Сложность: O(1).

Применение: приказ на поверхность, маркер мыши, проекция тактической точки.

7 Ray sphere

Порядок: построить квадратное уравнение, проверить дискриминант, выбрать ближайший неотрицательный корень.

Сложность: O(1).

Применение: грубый broad phase, выбор персонажа, отладочные цели.

Ловушка: вернуть отрицательный корень, лежащий позади начала.

8 Ray AABB slab method

Порядок: для каждой оси найти интервал `t1..t2`, пересечь его с общим `tMin..tMax`, отклонить пустое пересечение.

Сложность: O(1), ровно три оси.

Применение: редактор, broad phase, грубая проверка видимости объекта.

Ловушка: делить на почти нулевую компоненту направления. Параллельная ось проверяется отдельно.

9 Lerp

$$\mathrm{lerp}(a, b, t) = a + (b - a)t$$

Применение: цвет, позиция, числовой параметр.

Ловушка: использовать один и тот же `t` каждый кадр как будто это постоянная скорость. `current = lerp(current, target, 0.1)` зависит от FPS.

10 Экспоненциальное приближение

```
const float alpha = 1.0f - std::exp(-speed * deltaTime);
current += (target - current) * alpha;
```

Применение: камера, плавный поворот, UI, шумные оценки AI.

Плюс: почти одинаковая реакция при разном FPS.

11 Fixed timestep

Физику удобно обновлять фиксированным шагом.

```
accumulator += frameDelta;
while (accumulator >= fixedDelta)
{
    SimulateFixedStep(fixedDelta);
    accumulator -= fixedDelta;
}
```

Ловушка: бесконечно догонять после долгой паузы. Ограничивай `frameDelta` и число шагов.

Применение: физика, детерминированные таймеры, воспроизводимые тесты.

12 Cooldown и one shot trigger

```
cooldown = (std::max)(0.0f, cooldown - deltaTime);
if (request && cooldown <= 0.0f)
{
    ExecuteAction();
    cooldown = cooldownDuration;
}
```

Применение: выстрел, смена укрытия, повторный звук, пересчёт пути.

Ловушка: ставить cooldown каждый кадр, пока запрос активен. Тогда он никогда не достигнет нуля.

13 Конечный автомат

Минимальная модель:

```
Idle -> Searching -> Chasing -> TakingCover -> InCover -> Attacking
```

Для каждого состояния зафиксируй:

- условия входа;
- действия во время состояния;
- условия выхода;
- обязательные компоненты;
- восстановительное состояние при ошибке.

Сложность: O(1) на сущность за кадр, если состояние исполняет ограниченную работу.

Ловушка: менять состояние в нескольких системах без единого владельца перехода.

14 Utility scoring

Каждое действие получает нормализованный score.

$$\text{score} = \text{w_1f_1} + \text{w_2f_2} + \ldots + \text{w_nf_n}$$

Пример выбора укрытия:

```
score = protection * 5
      + rangeFit * 2
      + peekQuality * 3
      - pathCost * 0.2
      - exposureDuringMove * 4
```

Порядок: сначала жёсткие фильтры, затем score. Нельзя компенсировать отсутствующую защиту красивой дистанцией.

Ловушка: факторы разных масштабов. Приводи их примерно к 0..1 или явно учитывай диапазоны весов.

15 Гистерезис

Гистерезис не даёт переключаться между почти равными решениями.

```
if (newScore > currentScore + switchMargin)
    SelectNewCandidate();
```

Можно добавить минимальное время фиксации решения.

Применение: стиль боя, выбор укрытия, оружия, цели.

Ловушка: лечить гистерезисом плохие входные данные. Сначала проверь score и геометрию.

16 A star

A star ищет путь по графу.

$$f(n) = g(n) + h(n)$$

`g` — уже пройденная стоимость, `h` — оценка оставшегося пути.

Структуры: open set, closed set, `cameFrom`, стоимость `g`.

Сложность: зависит от графа и очереди; с приоритетной очередью обычно близка к O(E log V).

Применение: NavMesh или сетка.

Ловушки:

- пересчитывать полный путь каждый кадр;
- считать точку достижимой только потому, что она близко геометрически;
- не инвалидировать путь при смене `moveTarget`;
- не различать `NoPath`, `Blocked`, `Arrived`.

17 Arrival steering

Обычный seek летит к цели на полной скорости и перелетает её. Arrival снижает скорость рядом с целью.

```
const float distance = Length(toTarget);
const float desiredSpeed = maxSpeed *
    (std::min)(1.0f, distance / slowRadius);
const Vec3 desiredVelocity = Normalize(toTarget) * desiredSpeed;
```

Применение: остановка у укрытия и точки ремонта.

Ловушка: `slowRadius` меньше `arrivalDistance` или равен нулю.

18 Separation

Для каждого близкого соседа создаётся отталкивание, обычно сильнее на малой дистанции.

```
separation += away / max(distanceSquared, epsilon)
```

Применение: NPC не слипаются на пути.

Ловушка: запускать полный O(n^2) перебор для тысяч сущностей. Используй spatial hash или соседей навигации.

19 Spatial hash

Мир делится на ячейки. Координата превращается в целочисленный ключ:

```
cellX = floor(position.x / cellSize);
cellZ = floor(position.z / cellSize);
```

Для поиска в радиусе проверяются соседние ячейки, а не все сущности мира.

Средняя сложность: близка к O(1 + k), где k — число локальных кандидатов, при удачном размере ячейки.

Применение: восприятие, separation, поиск ближайших объектов.

20 Случайное направление внутри конуса

Равномерный разброс по площади нельзя получать равномерным углом по двум осям: это искажает плотность.

Практическая схема:

1. две независимые выборки `u` и `v` от 0 до 1;
2. радиальная часть использует `sqrt(u)`;
3. угол равен `2*pi*v`;
4. строится локальное отклонение;
5. оно переносится в базис `right`, `up`, `forward`.

Применение: `WeaponFiringRules::ConeDirection`.

Ловушка: делать новую случайную выборку отдельно для дебаг-луча и урона. Один выстрел должен иметь одно фактическое направление.

21 Семплирование динамических укрытий

Если авторские точки не покрывают все варианты боя, кандидаты могут генерироваться вокруг NPC или препятствий.

Кольцо вокруг NPC

```
for (int i = 0; i < sampleCount; ++i)
{
    const float angle = twoPi * i / sampleCount;
    candidate.x = npc.x + std::cos(angle) * radius;
    candidate.z = npc.z + std::sin(angle) * radius;
}
```

Для каждого кандидата:

1. спроецировать на NavMesh;
2. проверить достижимость;
3. проверить защиту несколькими лучами;
4. найти левый и правый peek;
5. оценить расстояние до угрозы;
6. оценить открытость пути;
7. применить score и hysteresis.

Ловушка: считать точку укрытием только потому, что один низкий луч встретил стену.

22 Профиль укрытия вместо bool

Полезная структура результата:

```
struct CoverProfile
{
    bool standingProtected = false;
    bool crouchingProtected = false;
    bool leftPeekValid = false;
    bool rightPeekValid = false;
    float pathCost = 0.0f;
    float exposureCost = 0.0f;
    float score = -FLT_MAX;
};
```

Так решение не теряет информацию. Анимационный модуль позднее выберет стойку и сторону выглядывания, не повторяя геометрический поиск.

23 Reservoir sampling для случайного кандидата

Если кандидатов много и надо равновероятно выбрать один без массива:

```
int seen = 0;
Entity selected = INVALID_ENTITY;
for (Entity candidate : candidates)
{
    ++seen;
    if (RandomInt(1, seen) == 1)
        selected = candidate;
}
```

Применение: разнообразие патрульных точек или равных укрытий.

24 Ring buffer журнала решений

Храни последние N событий в кольцевом буфере, а не одну быстро меняющуюся строку.

```
timestamp | state before | decision | reason | state after
```

Сложность: O(1) на запись, фиксированная память.

Применение: NPC Logic Studio. Это решает проблему мелькающих статусов, которые невозможно прочесть даже в пошаговом режиме.

25 Приоритетная очередь событий угрозы

События можно оценивать по важности и свежести:

```
priority = damageWeight * damage
         + proximityWeight * proximity
         + recencyWeight * freshness
```

Тихое убийство может не создавать общую тревогу, но локальное событие попадания всё равно разворачивает выжившего NPC к близкой угрозе.

Ловушка: передавать точную текущую позицию врага через событие навсегда. Получатель должен подтвердить контакт собственным зрением.

26 Чек лист выбора алгоритма

Перед реализацией ответь:

1. Какие точные входы?
2. Какой точный выход?
3. В каком пространстве координат данные?
4. Нужен ли точный результат или дешёвый фильтр?
5. Сколько сущностей участвует?
6. Какова сложность на кадр?
7. Как алгоритм ведёт себя на нулевых и граничных значениях?
8. Как визуально доказать его результат?
9. Какая система владеет состоянием?
10. Что записать в журнал причины?

Математика и алгоритмы для игры: второй курс с выводами и расчётами

Первый математический курс дал арифметику векторов и матриц. Теперь цель — научиться превращать игровое правило в алгоритм, заранее оценивать его стоимость и проверять результат числами. Примеры ниже учебные, если явно не назван метод движка. Алгоритм хорош, когда понятны входные данные, единицы, инвариант, крайние случаи и цена выполнения. Формула без этих пяти вещей легко превращается в красивый, но неисправный код.

1. Линейная интерполяция и экспоненциальное сглаживание

Пусть дверь находится в начале хода в позиции 0 м и должна за секунду дойти до 2 м. Линейная интерполяция между `a` и `b` при параметре `t` равна `lerp(a,b,t)=a+(b-a)t`, где `t` ограничен [0,1]. При `t=0` результат 0, при `t=0.25` — 0.5 м, при `t=1` — 2 м. Почему формула устроена так? Разность `b-a` показывает полный путь, умножение на долю показывает пройденную часть, прибавление `a` возвращает нас к началу координат. Для вектора действие выполняется по каждой координате. Если `A=(1;0;2)`, `B=(5;0;6)`, `t=0.25`, получаем `(2;0;3)`.

Частая ошибка — каждый кадр вызывать `position=lerp(position,target,0.1)`. Это не ход длительностью десять кадров. После первого кадра остаётся 90% расстояния, после второго 81%, после n кадров `0.9^n` исходного расстояния. На 120 FPS за секунду шагов больше, поэтому движение приблизится к цели сильнее, чем на 30 FPS. Если нужен путь за фиксированную секунду, храни стартовую позицию и `elapsed/duration`. Если нужно именно плавное приближение, используй коэффициент, зависящий от времени: `alpha=1-exp(-lambda*dt)`, затем `x += (target-x)*alpha`. При `lambda=2/c` и `dt=0.5c` получаем `alpha=1-e^-1≈0.632`, оставшаяся доля 0.368. Два шага по 0.25 с дают ту же общую долю `1-e^-1`, если нет округления и target не менялся. Здесь размерность `lambda*dt` должна быть безразмерной.

В движке подобные операции встречаются при визуальном сглаживании, цветах частиц и повороте. Но угол нельзя бездумно интерполировать как обычное число через границу ±π: от 179° к −179° арифметическая середина 0° идёт длинным путём. Сначала вычисли кратчайшую разницу `atan2(sin(b-a),cos(b-a))`, затем прибавляй её долю. Для полной 3D ориентации вместо смешения коэффициентов матриц применяй интерполяцию кватернионов с проверкой нормировки и выбора короткой дуги.

2. Численные производная и интеграл: скорость из позиции, путь из скорости

Производная отвечает на вопрос «как быстро меняется величина сейчас?». Если позиция в t=1 с равна 2 м, а в t=1.1 с — 2.3 м, средняя скорость на интервале `(2.3-2)/0.1=3 м/с`. Чем меньше интервал, тем ближе оценка к мгновенной скорости, но слишком малый шаг усиливает ошибки `float` и шум измерения. Поэтому для velocity физического тела лучше читать согласованное значение Bullet, чем каждый кадр вычитать два почти одинаковых Transform без учёта телепорта и физического шага.

Интеграл собирает маленькие вклады за время. Если скорость постоянна 3 м/с в течение 0.1 с, путь 0.3 м. В коде это `position += velocity*dt`. Если ускорение 2 м/с^2, скорость за шаг меняется на `acceleration*dt`. Для начальных `v=0`, `x=0`, `dt=0.5`, два полунеявных шага дают: первый `v=1`, `x=0.5`; второй `v=2`, `x=1.5`. Аналитическое положение при постоянном ускорении за 1 с равно `x=0+0·1+0.5·2·1²=1 м`. Разница 0.5 м — численная погрешность крупного шага, а не другая физика. При меньшем фиксированном шаге ошибка падает. Bullet решает больше задач, чем эта простая формула, но понимание интегратора помогает объяснить влияние `dt`.

Проверяй размерности: `a` в м/с², `a*dt` в м/с, `v*dt` в м. Не прибавляй силу в ньютонах сразу к скорости: сначала `a=F/m`. Для массы 10 кг и силы 20 Н ускорение 2 м/с². При импульсе 20 Н·с изменение скорости равно `J/m=2 м/с`. Сила действует в течение времени, импульс уже содержит время. Этим объясняется, почему взрывной толчок лучше задавать импульсом, если он должен произойти мгновенно.

3. Алгоритм маршрута: граф, стоимость и A*

Комнаты корабля можно представить вершинами графа, двери — рёбрами. Пусть стартовая комната A связана с B стоимостью 2 м, B с целью C стоимостью 2 м, а A с C через длинный проход стоимостью 10 м. Минимальный путь — A→B→C стоимостью 4, даже если на экране прямая до C кажется короткой. Если дверь A–B заперта, ребро отсутствует или получает очень высокую стоимость; решение меняется. Для NavMesh один waypoint не является фактическим движением: после планирования капсула ещё должна пройти через физический мир.

Алгоритм Дейкстры хранит лучшую известную цену `g(v)` от старта и выбирает вершину с наименьшей текущей ценой. A* добавляет оценку оставшегося пути `h(v)` и выбирает минимум `f(v)=g(v)+h(v)`. Эвристика `h` не должна завышать реальную остаточную стоимость, если нужна гарантия кратчайшего пути. Для движения по плоскости с ценой не меньше геометрической длины прямая евклидова дистанция до цели подходит. Если за закрытую дверь назначен дополнительный штраф, прямая всё ещё нижняя граница. Если в игре есть телепорт с нулевой ценой, обычная дистанция уже может завышать реальную стоимость и гарантия нарушится.

Пройди выбор вручную. Из A доступны B с `g=2,h=2,f=4` и D с `g=1,h=8,f=9`. Сначала раскрывается B. Если из B в C ребро 2, новая цель получает `g=4,h=0,f=4`. Алгоритм находит маршрут стоимостью 4. Для NPC после двери важно не только построить путь, но и зафиксировать причину его устаревания: дверь изменила состояние, NavMesh перестроен, цель сместилась. Кэшируй путь вместе с revision данных, от которых он зависит. Бесконечный repath каждый кадр может быть дороже самого движения и не решит проблему запертой двери.

4. Выбор работы экипажа как задача оптимизации

Один NPC может выбрать работу по максимальному score, но распределение двух NPC между двумя работами требует учитывать конкуренцию. Пусть инженер A даёт полезность 9 для пробоины X и 8 для Y, инженер B — 7 для X и 1 для Y. Жадный выбор A→X оставит B→Y, сумма 10. Другой вариант A→Y, B→X даёт сумму 15. Значит локально лучший первый выбор не обязательно максимизирует пользу группы. Для маленького экипажа можно перебрать допустимые назначения или использовать алгоритм оптимального паросочетания. Для большой сцены можно оставить детерминированный жадный вариант, если скорость и понятность важнее оптимальности, но это осознанный компромисс.

Формула score должна иметь понятную шкалу. Если severity в [0,1], а расстояние в метрах от 0 до 100, сырое выражение `severity-distance` практически всегда определяется дистанцией. Нормализуй: например `distanceScore=1/(1+d/10м)`. При d=0 он равен 1, при d=10м — 0.5, при d=30м — 0.25. `score=4*severity+2*distanceScore+roleBonus` теперь имеет сравнимые слагаемые. Но score не должен отменять жёсткие запреты: отсутствует путь, нет нужного ресурса, цель уже занята, опасная среда без скафандра. Сначала фильтруй допустимость, затем ранжируй.

Инвариант claim: на эксклюзивную задачу назначен максимум один живой исполнитель. После смерти, ручного приказа, уничтожения цели или истечения срока claim должен освободиться. Если цель удаляется во время ремонта, хранить указатель на компонент через удаление опасно. Сохрани Entity ID, после структурного изменения перечитай Registry. В текущем движке цепь

Utility/Coordinator/StateSystem уже существует; задача оптимального паросочетания здесь предлагается как развитие, а не заявляется готовой функцией.

5. Вероятность, разброс оружия и воспроизводимость

Случайное число из [0,1) может выбирать событие с вероятностью p: `u<p`. При `p=0.2` из 100 независимых испытаний ожидается около 20 успехов, но не ровно 20. Вероятность нуля попаданий за n независимых попыток равна `(1-p)^n`: при p=0.2 и n=5 это `0.8^5≈0.32768`. Нельзя проверять случайную стрельбу тестом «из пяти выстрелов обязаны попасть ровно один раз». Проверяй распределение на большом числе запусков, а конкретный баг воспроизводи с сохранённым seed.

Равномерный выбор угла не всегда даёт равномерную площадь на диске. Если радиус r выбирать просто равномерно от 0 до R, слишком много точек окажется близко к центру относительно площади. Для равномерной площади диска возьми `r=R*sqrt(u)`, угол `ϕ=2πv`, где u и v независимы и равномерны. Тогда `x=r*cosϕ`, `y=r*sinϕ`. Проверка: доля попаданий внутри половины радиуса должна быть площадью маленького круга относительно большого, `(R/2)²/R²=1/4`, а не 1/2. Для конуса направления нужна отдельная геометрия и нормировка; у текущего `WeaponSystem` есть собственное правило разброса, поэтому перед заменой сначала измерь его и не меняй баланс по учебной формуле вслепую.

Для NPC различай наблюдение и случайность. Пусть противник не видит игрока за стеной; случайный разброс не должен выдавать точную скрытую позицию. Он меняет направление луча вокруг уже допустимой точки прицеливания, а не создаёт знание. Запиши seed, shooter Entity, target Entity, исходный aim и фактический ray direction. Тогда один промах можно воспроизвести до пикселя и отличить ошибку геометрии дула от случайного отклонения.

6. Пространственные алгоритмы: AABB и отсев

Ось-ориентированный ограничивающий параллелепипед AABB хранит минимум и максимум по каждой оси. Точка `(x;y;z)` внутри, если `minX≤x≤maxX`, `minY≤y≤maxY`, `minZ≤z≤maxZ`. Для двух AABB пересечение есть, если интервалы перекрываются по всем трём осям. Если хотя бы по одной оси `A.maxX < B.minX`, коробки разделены, и дорогую проверку треугольников можно не делать. Это широкая фаза физики и полезный предварительный отсев видимости/взаимодействия.

Пример: A по X `[0,2]`, B по X `[3,4]`. Разделение уже доказано, Y и Z можно не читать. Если B по X `[1,3]`, X совместима, но ещё нужно проверить Y и Z. Для вращающегося объекта локальный AABB нельзя переносить только двумя диагональными углами; после вращения экстремальная вершина может быть другой. Либо трансформируй все восемь углов, либо используй центр и абсолютные компоненты матрицы для вычисления новой половины размера. Ошибочный world bounds может скрыть видимый mesh на краю камеры или пропустить raycast предварительной фазы.

Для множества объектов наивное сравнение каждого с каждым стоит `n(n-1)/2`. При n=1000 это 499500 пар. Разбиение пространства на клетки, дерево BVH или broadphase Bullet сокращает число реальных кандидатов, особенно если объекты разбросаны. Но структура ускоряет запрос ценой обновления: если каждый объект постоянно движется, её надо поддерживать. Не оптимизируй наугад; сначала измерь число проверяемых пар, стоимость обновления и время кадра.

7. Численная устойчивость и тесты свойств

При вычислениях с `float` не проверяй геометрию через абсолютное `==`. Для длины в метрах выбери допустимую ошибку в метрах; для нормированного dot — безразмерную ошибку. `epsilon=1e-6` не

является универсальным ответом: для корабля километрового масштаба и для миллиметрового muzzle одна и та же абсолютная ошибка имеет разный смысл. Проверь конечность входов через `isfinite`, прежде чем NaN попадёт в Transform, Bullet или shader buffer. Сравнение `NaN < 0` ложно, поэтому обычный `clamp` не всегда спасёт испорченное значение.

Тест свойства проверяет закон, а не копирует реализацию. Для случайных совместимых матриц `transpose(transpose(A))` должно вернуть A в пределах погрешности. Для ненулевого вектора `length(normalize(v))≈1`. Для пары преобразований `p*(M*M^-1)≈p`, если M обратима. Для распределения энергии сумма выданной мощности не превосходит доступную. Для двух комнат без утечек общий запас вещества в физической модели сохраняется. Эти проверки полезны после рефакторинга: они находят ошибку даже если конкретные числа в примере изменились.

Закончи курс мини-проектом: возьми одну пробоину, двух инженеров и две двери. На бумаге вычисли расстояния, score и допустимые назначения; нарисуй граф комнат и лучший маршрут; затем запусти сцену и сравни debug числа. Разница между расчётом и игрой должна иметь имя: физическое препятствие, изменившееся состояние двери, другой `dt`, конкурирующий claim или ошибка формулы. Так математика становится инструментом исследования собственного кода.

8. Луч, плоскость и сфера: вычисляем попадание до Bullet

Луч описывается началом O, единичным направлением D и параметром `t≥0`: `P(t)=O+tD`. Если D нормирован, t измеряется в метрах; если нет, t просто параметр и его нельзя сравнивать с дальностью оружия без пересчёта. Возьмём O=`(0;1;0)`, D=`(0;0;1)`, плоскость стены `z=5`. У луча z-координата `0+t*1=t`, значит t=5 и точка попадания `(0;1;5)`. Для направления `(0;0;2)` тот же контакт имеет t=2.5, но расстояние всё ещё 5 м. Вот почему оружейный `ValidRay` и нормировка направления важны до проверки range.

Плоскость в общем виде задаётся точкой Q и единичной нормалью N: точка P лежит на ней, когда `dot(P-Q,N)=0`. Подставим луч: `dot(O+tD-Q,N)=0`, раскроем линейность, получим `t=dot(Q-O,N)/dot(D,N)`. Знаменатель около нуля означает, что луч почти параллелен плоскости. Если одновременно числитель около нуля, луч лежит в плоскости, и «единственная точка попадания» не определена. Если t отрицателен, пересечение находится позади начала луча. Для оружия дополнительно требуется `t≤range`. Для picking редактора t может ограничиваться ближней и дальней плоскостью камеры.

Сфера с центром C и радиусом r задаётся `|P-C|²=r²`. Подстановка `P=O+tD` даёт квадратное уравнение. При нормированном D обозначь `m=O-C`; тогда `t²+2dot(m,D)t+dot(m,m)-r²=0`. Дискриминант меньше нуля — нет пересечения; равен нулю — касание; положителен — два пересечения, вход и выход. Выбирай наименьший неотрицательный t. Если начало луча внутри сферы, первый корень отрицателен, а второй положителен: это точка выхода. В Bullet тебе не нужно повторять эту математику для каждого объекта, но она помогает понять, почему отладочный луч попал в ближний collider, а не в выбранную NPC цель.

9. Столкновение прямоугольников и разделяющая ось

Для AABB проверка интервалов по X, Y, Z достаточна. Для повернутых коробок оси этих объектов различаются, поэтому простое сравнение мировых min/max может давать ложное пересечение. Теорема разделяющей оси говорит: две выпуклые фигуры не пересекаются, если существует ось, на которой их проекции не пересекаются. Для двумерных прямоугольников кандидаты — нормали к сторонам обоих. Проекция вершины P на единичную ось n — `dot(P,n)`. Для каждого многоугольника найди минимум и максимум этой проекции. Если интервалы разошлись хотя бы на одной оси, столкновения нет. Если не разошлись на всех необходимых осях, фигуры пересекаются.

Пример на оси X: проекция первого прямоугольника `[0,2]`, второго `[3,5]`. Условие `2<3` сразу доказывает отсутствие контакта. На другой оси проверять не нужно. Если интервалы `[0,2]` и `[1,3]` перекрываются, это ещё не доказывает столкновение: по Y фигуры могут быть разделены. Для трёхмерных OBB нужны оси граней обоих объектов и оси попарных векторных произведений их базисных осей; это уже более дорогая проверка. Bullet выполняет специализированную collision работу, но понимание SAT полезно для editor gizmo, UI геометрии и предварительной диагностики collider.

Есть тонкий крайний случай: «касаются» или «пересекаются»? Для интервалов `[0,2]` и `[2,4]` общая точка 2. Если условие разделения строгое `<`, касание считается контактом; если `≤`, нет. В физике обычно нужен допуск и контактный зазор, поэтому символ сравнения — часть игрового правила. На высоких скоростях дискретная проверка положения до и после шага может пропустить тонкую стену: объект окажется по одну сторону на t=0 и по другую на t=dt. Для такого случая используют swept collision/continuous detection или ограничивают шаг, а не просто увеличивают радиус объекта.

10. Управление дверью или камерой: ошибка, скорость и устойчивость

Пусть привод двери должен довести её до позиции 1 м, сейчас она в 0.6 м. Ошибка `e=target-current=0.4 м`. Простейшее пропорциональное управление задаёт скорость `v=Kp*e`. Чтобы результат был в м/с, `Kp` измеряется в 1/с. При `Kp=2/с`, v=0.8 м/с; за `dt=0.1с` дверь сместится на 0.08 м до 0.68 м. Следующая ошибка станет 0.32 м, новая скорость 0.64 м/с. Движение постепенно замедляется и асимптотически приближается к цели. Чтобы не ждать бесконечно, введи допуск, например 1 мм: внутри него установи конечную позицию и закончи переход.

Если применить `current += Kp*e*dt`, устойчивость простого дискретного правила зависит от `Kp*dt`. При значении 1 результат достигается за шаг, при значении 2 прыгает через цель на противоположную сторону, при ещё больших значениях может расходиться. Поэтому изменение dt без проверки может превратить плавную дверь в дрожащую. Для критичного движения используй фиксированный шаг, ограничение скорости или аналитическое сглаживание из главы 1. Для физического тела не записывай Transform поверх Bullet каждый кадр; применяй согласованные силы/импульсы или кинематический контроллер.

PID добавляет интегральный член для накопленной ошибки и дифференциальный член для реакции на скорость её изменения. Но он не является обязательной «улучшенной версией» для каждой двери. В игре простая интерполяция с известной длительностью часто понятнее и легче тестируется. PID полезнее там, где есть инерция и возмущения, например стабилизация корабля. Перед его добавлением измерь реальное поведение, выбери единицы коэффициентов, зажми интегратор при насыщении привода и проверь устойчивость при фиксированном dt. Три буквы без числового эксперимента только скрывают проблему.

Инженерные практики для развития проекта

Эта часть добавляет темы, которые становятся важнее по мере роста движка. Они не создают новую видимую механику за один вечер, но не дают механикам превратиться в неуправляемый монолит.

1 Контракт до класса

Перед новым файлом ответь письменно.

Вопрос	Пример для stamina
Кто владеет данными	`StaminaComponent` на персонаже
Кто меняет значение	`StaminaSystem`
Кто только читает	movement UI tactical decision
Когда обновляется	после intent до выбора фактической скорости
Какая единица	нормализованный диапазон или points
Какие инварианты	`0 <= current <= maximum`
Как сохраняется	authoring maximum да runtime current для save game
Как отлаживается	current drain reason regen delay

Если два класса заявляют право окончательно писать одно поле, контракт ещё не готов.

2 Правило одного источника истины

У оружия magazine живёт в одном `WeaponComponent`. У комнаты pressure живёт в одном `RoomComponent`. У dynamic body физическая позиция после fixed step живёт в Bullet и копируется в Transform. Кэш допустим, если он помечен как derived и имеет понятную процедуру invalidation.

Плохой признак: баг исправляют синхронизацией двух почти одинаковых полей каждый кадр. Обычно лучше удалить одного владельца.

3 RAII и внешние библиотеки

DirectX и Bullet часто возвращают raw pointers, но это не означает отсутствие владельца. Для каждого runtime ресурса в компоненте или manager запиши destroy sequence.

```
Bullet body
    removeRigidBody from world
    delete motion state
    delete body
    delete shape children
    delete shape
    delete backing triangle mesh
D3D resource
    release references held by views
    release view
    release buffer or texture
```

Когда возможно, используй `unique_ptr` с custom deleter или небольшой wrapper. Если component остаётся move only, ECS уже имеет move overload `AddComponent`.

4 Инварианты сильнее комментариев

Комментарий сообщает намерение, но проверка ловит нарушение. Полезные debug assertions:

```
assert(component.maximum >= 0.0f);
assert(component.current >= 0.0f);
assert(component.current <= component.maximum + epsilon);
```

В release код всё равно должен безопасно обрабатывать внешние данные из JSON. Assert помогает разработчику, validation защищает пользователя и save file.

5 Guard clauses

Ранний выход делает основной путь плоским.

```
if (!world)
    return;
if (!registry.IsAlive(entity))
    return;
auto* transform = registry.GetComponent<TransformComponent>(entity);
if (!transform)
    return;

// основной путь начинается здесь
```

Не все ошибки равны. Ожидаемое отсутствие optional component допускает тихий return. Нарушение обязательного инварианта должно оставить diagnostic reason, assert или error log.

6 Именование величин и единиц

Добавляй единицу в имя там, где ошибка опасна: `reloadSeconds`, `outputMw`, `energyMwh`, `angleRadians`, `distanceMeters`. `value`, `speed` и `time` слишком легко смешать.

Величина	Хорошее имя	Типичная ошибка
угол	`yawRadians`	передать degrees в `sin`
длительность	`cooldownSeconds`	уменьшать на milliseconds
мощность	`requiredMw`	считать её запасом энергии
квадрат расстояния	`distanceSquared`	сравнить с обычным radius
нормализованная доля	`powerRatio`	принять за проценты 0..100

7 Координатные пространства как часть типа

Имя `position` не сообщает пространство. Для сложного кода используй `worldPosition`, `localOffset`, `viewDirection`, `modelNormal`. Особенно это важно для muzzle, IK, picking ray и joint frames.

Минимальный комментарий перед формулой должен сообщать переход: local muzzle point умножается на bone matrix, затем на entity world matrix и становится world origin выстрела.

8 Численная устойчивость

Основные правила проекта:

- проверяй длину до нормализации;
- clamp аргумент `acos` в `[-1, 1]`;
- сравнивай squared distance со squared threshold;
- ограничивай большой `deltaTime` для простой симуляции;
- используй epsilon только с понятной единицей;
- проверяй `isfinite` на границе физики и файлов;
- не дели на количество элементов до проверки нуля.

Epsilon `0.0001` не является универсальной магией. Для позиции в метрах и для normalized dot это разные масштабы ошибки.

9 Детерминизм и случайность

Random влияет на spread, wandering, effects и utility tie break. Для воспроизводимого теста используй seed, который сохраняется в debug run.

```
seed + sequence of calls + same inputs -> same decisions
```

Если разные системы используют один глобальный generator, добавление случайной частицы может изменить AI. Лучше иметь отдельные streams или deterministic sampler по Entity и event index.

10 State machine и причина перехода

Храни не только state, но и `lastTransitionReason` для debug. Переход должен выполняться в одном месте или через функцию.

```
void SetState(NpcBrainComponent& brain, NpcState next, std::string reason)
{
    if (brain.state == next)
        return;
    brain.previousState = brain.state;
    brain.state = next;
    brain.lastTransitionReason = std::move(reason);
}
```

Так журнал показывает факт, а не пытается восстановить причину по текущим полям спустя несколько кадров.

11 Decision и execution

Decision выбирает цель и намерение. Execution проверяет возможность и выполняет. Результат execution возвращается как success, running или failure reason.

```
tactical module -> MoveToCover intent
navigation      -> path or no path
movement        -> displacement intent
Bullet          -> actual position
feedback        -> reached blocked stuck
```

Если decision каждый кадр перезаписывает intent до получения feedback, NPC дёргается между решениями.

12 Cooldown hysteresis и commitment

Cooldown ограничивает частоту повторной оценки. Hysteresis требует, чтобы новый вариант был заметно лучше текущего. Commitment запрещает смену на короткой фазе, например резкое выглядывание.

```
switch when
newScore > currentScore + hysteresis
and decisionCooldown <= 0
and not in committed action
```

Эти три механизма решают разные проблемы. Один огромный timer скрывает симптомы, но делает AI медленным.

13 Ring buffer журналов

Журнал решения ограничивается последними N событиями. Поля события: world time, Entity, category, previous state, next state, reason и важные числа.

Не форматируй всю строку заранее, если UI может переключать язык. Храни структурированные поля и локализуй представление.

14 Structured debug snapshot

Inspector не должен читать десятки компонентов несогласованно и придумывать смысл. Система может публиковать компактный snapshot.

```
NpcDebugSnapshot
    state
    target and visibility
    tactical order and reason
    move target and path status
    weapon ammo cooldown reload
    cover candidate rejection counts
```

Snapshot является derived debug data и не участвует в решениях.

15 Тестовая пирамида движка

Уровень	Пример	Что ловит
чистая функция	cone direction limited flow utility score	математику и границы
component system	PowerSystem на маленьком registry	порядок чтения и записи
scenario	два NPC и одна стена	интеграцию нескольких систем
soak	30 минут симуляции	утечки рост журнала редкие застревания
save load	round trip сцены	пропущенные поля и remap
visual regression	shader test scene	presentation изменения

Не каждый тест должен запускать DirectX window. Чистые правила выноси в функции без renderer и world, чтобы они проверялись быстро.

16 Arrange act assert

```
Arrange
    создать registry и минимальные компоненты
Act
    вызвать одну систему с известным dt
Assert
    проверить поля и события
```

В Arrange не копируй целую игровую сцену. Чем меньше обязательных данных, тем лучше тест показывает контракт системы.

17 Golden scenario для NPC

Создай несколько фиксированных сцен.

- прямая видимость без укрытий;
- низкая стена и высокая стена;
- два подходящих укрытия с разной дистанцией;
- цель исчезает за углом;
- empty magazine во время движения;
- friendly в firing lane;
- defender и assault profile;
- входящий near miss без урона;
- повреждение во время exposed phase;
- путь временно недоступен.

Для каждого сценария запиши ожидаемую последовательность состояний, а не только конечную позицию.

18 Property tests для математики

Полезные свойства:

- нормализованный ненулевой vector имеет length около единицы;
- dot normalized vectors находится в `[-1,1]`;
- ray hit fraction находится в `[0,1]`;
- `LimitedFlow` не переносит больше половины разницы за один парный обмен;
- utility score finite и ограничен заданным диапазоном;
- matrix inverse возвращает исходную точку в пределах epsilon;
- power supplied не превышает generation;
- сумма transfer между комнатами сохраняется без утечки до clamp.

19 Save version и migration

Добавь номер schema. При загрузке старой версии применяй migration defaults. Не определяй версию по отсутствию случайного поля, если формат уже развивается.

```
version 3
    add faction = Neutral when absent
version 4
    convert reloadTimeMs to reloadSeconds
version 5
    split coverPoint into cover and peek positions
```

После migration runtime resources всё равно пересоздаются обычным load pipeline.

20 Атомарное сохранение

Безопасная запись:

1. записать временный файл рядом;
2. flush и закрыть;
3. проверить что JSON читается;
4. заменить основной файл;
5. при необходимости сохранить backup.

Падение во время прямой перезаписи не должно уничтожать единственный save.

21 Совместимость компонентов

При добавлении обязательного компонента старые сцены его не имеют. Выбери одну политику: default при load, editor migration tool или система `EnsureComponent`. Не размазывай три разных default по Inspector, load и runtime.

22 Профилирование до оптимизации

Сначала измеряй время систем и количество элементов.

```
Frame
    AI 2.1 ms for 84 NPC
    Physics 3.8 ms for 612 bodies
    Render preparation 1.4 ms for 2300 candidates
    GPU 7.6 ms
```

Среднее скрывает spikes. Записывай median, 95 percentile и maximum. Отдельно считай query counts: raycasts NPC, cover candidates, path rebuilds и draw calls.

23 Пространственное ускорение

Если каждый NPC проверяет каждого кандидата, стоимость растёт квадратично. Spatial hash или grid уменьшает набор соседей.

```
cell = floor(position / cellSize)
query radius -> neighboring cells
exact distance -> final filter
```

Grid только создаёт кандидатов. Точная проверка distance, FOV и raycast всё равно выполняется после.

24 Бюджетирование AI

Не все решения требуют 60 обновлений в секунду.

Слой	Частота пример	Причина
движение controller	каждый simulation frame	плавность и контакт
weapon cooldown	каждый simulation frame	точное время
target candidate scan	5 или 10 Hz	дорогая пространственная оценка
cover rescore	по событию и cooldown	стабильность решения
дальний idle NPC	редкий tick	simulation LOD

Распределяй NPC по фазам, чтобы все не делали дорогой scan в одном кадре.

25 Memory и invalidation

Raw pointer на component действителен только пока pool не изменён способом, который перераспределяет storage, и пока Entity жива. Не сохраняй component pointer между кадрами без контракта контейнера. Сохраняй Entity и повторно получай pointer после `IsAlive`.

Модель, Bullet object и D3D resource имеют отдельные жизни. Смена сцены обязана инвалидировать caches, debug selection и async requests предыдущей сцены.

26 Ошибки фоновых задач

Worker error должен стать состоянием задачи, а не исчезнуть в debug output. Main thread получает success или structured error и может показать asset path, stage и message.

При закрытии сцены long job либо отменяется, либо результат проверяет generation token и не применяется к новому миру.

27 Generation token

```
sceneGeneration = 12
job captures 12
scene reload increments to 13
job completes with 12
main thread rejects stale result
```

Этот простой механизм предотвращает применение старой async геометрии к Entity новой сцены с совпавшим ID.

28 Dependency direction

Желательное направление:

```
Core and math
    <- ECS data
        <- gameplay rules
            <- presentation and editor
```

Renderer может читать gameplay snapshot, но gameplay не должен включать EditorUI для принятия решения. Debug hooks оформляй интерфейсом или данными, а не обратной зависимостью.

29 Модульная тактика NPC

Каждый tactical module реализует общий контракт.

```
CanEvaluate(context)
Score(context)
BuildIntent(context)
Explain(context)
```

Context является read only snapshot восприятия, оружия, расстояния, stamina и укрытий. Module не двигает NPC напрямую. Arbiter выбирает intent, execution layers выполняют.

Новые модули: hold distance, assault, retreat, suppress, flank, reload safety, investigate sound, protect ally. Групповая координация может добавлять context facts, но не должна быть обязательной для одиночного NPC.

30 Capability компоненты

Не раздувай один `NPCComponent` сотней bool. Возможности оформляй компонентами или data profiles.

```
CanUseCoverComponent
CanLeanComponent
StaminaComponent
HearingComponent
SquadMemberComponent
MedicCapability
```

Отсутствие компонента означает отсутствие возможности. System перебирает только нужный pool.

31 Data driven параметры

Порог дистанции, время exposed phase и скорость stamina не должны быть случайными literals в нескольких системах. Собери profile с понятными единицами и validation. Editor меняет profile, runtime читает его.

Баланс параметров не заменяет алгоритм. Если NPC меняет укрытие каждый две секунды из-за отсутствия hysteresis, увеличение таймера лишь маскирует причину.

32 Телеметрия решений

Для каждого дорогого выбора полезны counters.

- candidates total;
- rejected by distance;
- rejected by protection;
- rejected by path;
- rejected reserved;
- chosen score;
- evaluation time microseconds;
- time since last change;
- execution failure reason.

Так видно, что NPC не тупит, а все кандидаты отвергнуты конкретным фильтром.

33 Локализация журналов

Храни event id и параметры, например `CoverRejectedLowProtection` и `protectionHeight=0.82`. Русский и английский UI форматируют их отдельно. Если хранить только готовую английскую строку, перевод быстро станет неполным.

34 Комментарии которые помогают

Хороший комментарий отвечает почему, указывает единицы или контракт внешней библиотеки.

```
// Bullet transform хранит центр капсулы, а ECS позиция находится у ног.
```

Плохой комментарий повторяет синтаксис.

```
// Увеличиваем i на один.
++i;
```

После изменения поведения обновляй комментарий вместе с кодом. Устаревшее объяснение опаснее отсутствующего.

35 Маленький рефакторинг

Безопасная последовательность:

1. зафиксировать scenario и debug output;
2. вынести чистую функцию без изменения результата;
3. сравнить поведение;
4. вынести данные;
5. сравнить снова;
6. только потом менять правило.

Не совмещай перенос архитектуры, новый алгоритм и баланс параметров в одном непроверяемом шаге.

36 Definition of Done

Механика готова, когда:

- работает основной путь;
- отказ имеет понятную причину;
- runtime resources удаляются;
- save load сохраняет нужные данные;
- Inspector показывает важные поля;
- есть минимальный scenario test;
- стоимость измерена;
- комментарии объясняют нетривиальный контракт;
- старые сцены либо совместимы, либо мигрируются;
- место расширения записано в атласе.

37 Рекомендуемый порядок развития проекта

1. Закончить наблюдаемость: единый журнал, counters, debug rays и state reasons.
2. Закрепить golden scenarios NPC и физики.
3. Ввести profiles и единицы для разбросанных параметров.
4. Довести save version и migration.
5. Разделить длинные системы по decision, query и execution.
6. Добавить stamina как учебный вертикальный срез.
7. Добавить lean и crouch как capability и animation action.
8. Добавить hearing и investigate sound.
9. Добавить новые одиночные tactical modules.
10. Только после стабильного одиночного AI подключать squad coordination.

38 Практикум по самостоятельной разработке

Задание 1 Stamina

Самостоятельно опиши component, system order, units, invariants, debug fields и save policy. Код начинай только после схемы.

Задание 2 Physics ownership

Для двери, ящика, NPC и отсека заполни таблицу: тип body, владелец Transform до и после fixed step, group, mask, cleanup.

Задание 3 Scenario test

Создай тест: NPC идёт к новому укрытию, стреляет в движении, начинает reload, доходит, прячется и только потом выглядывает. Запиши ожидаемые состояния по времени.

Задание 4 Save migration

Придумай старую версию `WeaponComponent` без reserve ammo и новую с reserve ammo. Опиши migration и default.

Задание 5 Performance

Добавь counters raycast, cover query и path rebuild. Сравни два NPC и двадцать NPC, не меняя алгоритм.

39 Финальный чек лист разработчика

- Я знаю владельца каждого изменяемого поля.
- Я знаю единицу каждой физической величины.
- Я могу назвать порядок системы в кадре.
- Я отделяю intent от подтверждённого факта.
- Я не сохраняю runtime pointers.
- Я удаляю внешний ресурс до уничтожения его владельца.
- Я могу воспроизвести случайный тест через seed.
- Я вижу причину отказа в журнале.
- Я измеряю прежде чем оптимизировать.
- Я изменяю одно правило за проверяемый шаг.

Эти привычки полезнее любого отдельного паттерна. Они позволяют добавлять новые механики как модули, а не как новые ветви одного бесконечного `Update`.

Как превратить растущий проект в устойчивый движок

Новые возможности становятся полезными только тогда, когда их можно повторяемо загрузить, проверить, отладить и изменить без цепной поломки. Эта глава собирает инженерные улучшения, которые особенно пригодятся DX Framework на следующем этапе.

1 Вертикальный срез вместо большого слоя

Вертикальный срез проходит через все необходимые уровни одной маленькой способности.

Например, «NPC ищет оружие»:

- компонент знания о pickup;
- чистая оценка кандидата;
- reservation;
- action и навигация;
- штатный `InventorySystem::PickupWeapon`;
- animation request;
- Inspector и журнал;
- сохранение только долгоживущих данных;
- один scenario test.

Горизонтальный подход сначала пишет огромный универсальный planner, потом универсальные actions, а увидеть результат можно через месяц. Вертикальный срез раньше выявляет плохие границы.

2 Архитектурная запись решения

Для заметного изменения полезен короткий ADR-файл:

```
Название: Кто владеет NpcIntent
Контекст: cover и combat пишут state независимо
Решение: только GoalArbiter меняет goal, executors меняют phase
Альтернативы: последний writer wins; единая огромная система
Последствия: нужен feedback и migration старых states
```

ADR не должен быть бюрократией. Пять абзацев сохраняют причину через месяц, когда код уже не объясняет, почему выбран именно такой путь.

3 Data driven настройки

Числа тактики не стоит размазывать по системам. Профиль может хранить:

- preferred range;
- aggression и morale baseline;
- cover weights;
- burst policy;
- допустимые цели;
- animation semantics;
- rules of engagement.

Runtime component хранит выбранный profile id и изменяемое состояние. JSON schema имеет version. При загрузке значения clamp и validate.

Data driven не означает «вынести каждый bool в JSON». Инварианты и алгоритмы остаются в C++. В данные уходят параметры и выбор модулей.

4 Semantic names

Система становится хрупкой, если gameplay знает конкретный clip `Kevin_Melee_Jab_L`. Лучше semantic action:

```
Locomotion.Run.Rifle
Combat.Peek.Left.Stand
Interaction.Pickup.Weapon
Compliance.Surrender
Compliance.Restrain
```

Профиль модели связывает semantic с clip. NPC action публикует semantic. Animation controller выбирает доступный clip и fallback. Animation Studio редактирует связь.

Тот же подход полезен для shader/material features и audio events.

5 События и команды

Событие сообщает, что произошло. Команда просит что-то сделать.

- `DamageApplied` — событие прошлого факта.
- `RequestReload` — команда.
- `WeaponReloaded` — событие результата.
- `TargetSpotted` — событие perception.
- `MoveIntent` — команда movement executor.

Смешивание приводит к циклам. Если `DamageApplied` сразу телепортирует NPC в cover, оно перестаёт быть событием и скрывает решение.

Event queue должна иметь понятный момент доставки. Событие этого кадра либо обрабатывается позже в том же кадре по документированному порядку, либо на следующем тике. Иначе появляются ошибки «иногда на один кадр».

6 Версии компонентов и сцен

Сцена живёт дольше структуры C++. Нужны:

- номер версии scene format;
- версия больших component payload;
- значения по умолчанию для отсутствующих полей;
- явная migration для изменённого смысла;
- warning для неизвестных полей или enum;
- round-trip test load → save → load.

Нельзя полагаться только на то, что JSON библиотека проигнорирует отсутствующее поле. Если раньше `isHostile` определял врага, а теперь это делает `FactionComponent`, migration должна назначить faction осмысленно или сообщить о ручной проверке.

7 Asset database

По мере роста файлов полезен реестр assets:

```
AssetId
source path
type
content hash
dependencies
import settings
runtime resource state
last error
```

Scene сохраняет стабильный AssetId, а не случайный абсолютный путь. Importer отслеживает texture dependencies. Изменение файла инвалидирует только зависящие assets.

Asset database также позволяет найти, какие сцены используют модель перед несовместимым изменением skeleton.

8 Асинхронная загрузка

Полезное разделение:

- IO и декодирование на worker;
- создание Direct3D resources на render/main thread;
- atomic публикация готового handle;
- fallback asset до готовности;
- cancel, если requester уничтожен.

Состояния:

```
Unloaded → Queued → LoadingCPU → WaitingGPU → Ready
                                    ↘ Failed
```

Нельзя отдавать наружу наполовину созданный mesh. Handle возвращает Ready resource или fallback.

9 Handles вместо сырых указателей

Модель или texture может hot reload. Сырой pointer в сотне компонентов легко становится висячим. Handle содержит id и generation. Resource manager проверяет generation и возвращает актуальный объект.

ECS Entity уже использует похожую идею идентичности. Для assets она особенно важна при асинхронности.

10 Renderer graph и проходы

Когда renderer растёт, список вызовов превращается в неявную зависимость ресурсов. Render graph описывает passes:

- shadow;
- opaque PBR;
- transparent;
- particles;
- debug;
- postprocess;
- UI.

Каждый pass объявляет inputs/outputs. Graph вычисляет порядок и время жизни временных targets. Для небольшого DirectX 11 renderer не нужен универсальный фреймворк сразу. Начать можно с явной структуры `RenderFramePlan` и списка ресурсов.

11 Material feature set

Один гигантский shader со всеми bool ветками становится дорогим и трудным. Тысячи полностью отдельных файлов тоже неудобны.

Компромисс:

- небольшой набор shader families;
- compile-time defines для крупных features;
- runtime parameters для дешёвых изменений;
- cache варианта по feature mask;
- fallback variant при ошибке.

Feature mask может включать skinning, normal map, alpha test, damage overlay и shadow receiving. Вариант компилируется и отражается один раз.

12 Профилирование

Измерять надо время и количество работы.

CPU markers:

- total frame;
- AI sensing;
- goal arbitration;
- NavMesh queries;
- physics step;
- animation sampling;
- render submission.

Счётчики:

- NPC evaluated;
- visibility rays;
- cover candidates;
- path rebuilds;
- draw calls;
- vertices/triangles;
- shader switches;
- constant buffer updates;
- loaded texture memory.

Среднее скрывает spike. Нужны median, 95/99 percentile и worst recent frame.

13 Budget мышление

Вместо «оптимизировать AI» задаётся бюджет:

```
60 FPS frame: 16.67 ms
AI total target: 2.0 ms
visibility rays per frame: 40
new path queries per frame: 4
cover full searches per frame: 2
```

Requests ставятся в очередь по срочности. Emergency target visibility выше фонового wander sensing. Если budget закончился, неэкстренное решение использует свежий предыдущий результат с возрастом.

14 Fixed timestep и replay

Physics и важная симуляция стабильнее на фиксированном шаге. Rendering интерполирует между состояниями.

Для replay записываются:

- initial scene hash;
- fixed timestep;
- random seeds;
- команды игрока и Studio;
- внешние события;
- важные version numbers.

Полный Transform каждого кадра записывать необязательно. При детерминированной симуляции достаточно входов. Bullet может иметь платформенные различия, поэтому для тестов полезны tolerance и high-level invariants.

15 Debug draw как API

Сейчас tactical component хранит векторы debug rays. При росте лучше общий временный DebugDrawQueue:

```
DebugLine{from, to, color, lifetime, category, entity}
DebugSphere{center, radius, color, lifetime, category, entity}
DebugText{position, text, category, entity}
```

Категории: perception, cover, navigation, weapon, animation IK, physics. UI включает фильтр. Очередь не сохраняется в scene и имеет предел памяти.

16 Ошибки кодировки и документации

В текущих shader/model исходниках встречаются повреждённые комментарии. Это сигнал не только о тексте, но и о toolchain.

Рекомендуется:

- сохранить исходники UTF-8;
- включить `/utf-8` в MSVC;
- добавить `.editorconfig`;
- не смешивать OEM/Windows-1251;
- проверять JSON и logs на UTF-8;
- постепенно исправлять комментарии рядом с изменяемым кодом, не выполняя огромную механическую замену без diff review.

17 Автоматические проверки

Минимальная лестница:

1. форматирование изменённых файлов;
2. compile headers/targets;
3. unit tests чистых rules;
4. asset validation;
5. headless scenario ticks;
6. render smoke scene;
7. ручная проверка Studio.

Проверка должна соответствовать риску. Изменение текста книги не требует сборки игры. Изменение HLSL требует compile всех variants и эталонного кадра.

18 Golden scenes

Небольшие сцены, каждая проверяет один контракт:

- PBR spheres с сеткой roughness/metalness;
- skinned skeleton и extreme pose;
- shadows и transparent glass;
- NPC line of sight behind cover;
- two cover sides and low cover;
- weapon pickup competition;
- surrender/capture;
- doors/NavMesh traversal.

Сцена должна быстро загружаться и иметь фиксированную камеру. Она является лабораторным прибором, а не красивым уровнем.

19 Работа с большим компонентом

`NpcTacticalComponent` уже хранит настройки, runtime timers, debug rays, group fields и status. Он полезен, но будет расти.

Безопасное разбиение по владельцам:

- `NpcTacticalProfileComponent` — авторские параметры;
- `NpcTacticalRuntimeComponent` — фазы, timers, current cover;
- `NpcPerceptionMemoryComponent` — контакты;
- `NpcIntentComponent` — goal/action/reason;
- `NpcReservationComponent` — handles;
- `NpcDebugComponent` — optional debug state.

Разбивать надо не по числу строк, а когда поля имеют разные жизненные циклы и писателей. Миграция выполняется вертикально, по одной группе.

20 Код и комментарии

Комментарий должен объяснять причину, единицы, владельца или неочевидный контракт.

Плохой:

```
// Уменьшаем таймер.
timer -= dt;
```

Хороший:

```
// Commitment использует simulation time. Насыщение до нуля позволяет
// всем правилам проверять один порог даже после длинного кадра.
timer = std::max(0.0f, timer - std::max(0.0f, dt));
```

Построчные комментарии полезны в учебной копии, но production-код не должен повторять синтаксис каждой строки. Имена и маленькие функции несут основное объяснение.

21 API редакторских инструментов

EditorUI сейчас большой и содержит много логики Studio. Следующий шаг — отделить модели инструментов:

- `AnimationStudioSession` хранит selection, playhead, edit layer, undo stack;
- `AnimationProfileSerializer` читает/пишет versioned JSON;
- `AnimationStudioPanel` только рисует UI и отправляет команды session;
- `NpcStudioSession` управляет capture/filter/export;
- `NpcStudioPanel` отображает snapshot;
- viewport gizmo adapter преобразует координаты.

Так UI можно перестраивать без изменения runtime animation.

22 Undo redo

Редакторская операция оформляется командой с `Do` и `Undo`:

```
MovePoseKey
AddPoseKey
DeletePoseKey
ChangeClipBinding
ChangeWeaponGrip
```

При drag gizmo не надо создавать сотни команд. На mouse down запоминается before, во время drag идёт preview, на mouse up фиксируется одна команда after.

23 Идеи новых учебных систем

23.1 Sound propagation

Звук имеет источник, громкость, категорию и затухание через комнаты/двери. NPC получает неопределённый район, не точную позицию. Это отличная практика графов, perception memory и отладки.

23.2 Suspicion

Подозрение растёт от частичных наблюдений и падает со временем. Порог меняет idle → investigate → confirmed combat. Оно связывает stealth и существующий search без всезнания.

23.3 Smart objects

Дверь, турель, аптечка и терминал публикуют actions с preconditions, duration и slots. Planner использует общий интерфейс взаимодействия.

23.4 Tactical influence map

Сетка хранит видимость, опасность, союзную поддержку и звук. Это не истина NPC автоматически: карта строится из его памяти или является командным ресурсом с задержкой.

23.5 Damage anatomy

Отдельные injuries влияют на скорость, aim и stamina, не превращая здоровье в единственный показатель. Начинать стоит с двух-трёх gameplay-эффектов, а не полной медицинской симуляции.

24 Долгий roadmap

Фаза A Наблюдаемость

- единая таблица владельцев AI полей;
- intent и action в NPC Studio;
- structured failure;
- общий debug draw;
- фиксированные scenario scenes.

Фаза B Ресурсы и действия

- reservation service;
- acquire weapon vertical slice;
- action channels;
- semantic animation requests;
- stamina.

Фаза C Социальное поведение

- morale;
- surrender;
- rules of engagement;
- capture and escort;
- group overwatch proposal.

Фаза D Контент и renderer

- asset validator;
- shader reflection;
- material variants;
- transactional hot reload;
- GLB dependency database.

Фаза E Масштабирование

- profiling budgets;
- spatial target queries;
- scheduled sensing;
- replay;
- selective multithreading.

25 Как самому изучать любую новую область

Используй цикл:

1. Нарисовать поток данных.
2. Найти владельца каждого состояния.
3. Выписать инварианты и единицы.
4. Пройти один конкретный пример вручную.
5. Добавить наблюдаемость.
6. Сделать минимальный вертикальный срез.
7. Проверить крайние случаи.
8. Только затем обобщать.

Этот метод одинаково работает для шейдера, импорта, физики, AI и editor tool.

26 Финальный чек лист новой системы

- Есть одно предложение о назначении системы.
- Определены входы и выходы.
- Известен единственный владелец изменяемого состояния.
- Отсутствующие данные имеют явный результат.
- Используются правильные часы и единицы.
- Действие можно прервать по документированной политике.
- Ошибка возвращается как данные.
- Состояние диагностируется в Studio или логе.
- Есть маленькая тестовая сцена.
- Сохранение и migration продуманы.
- Нет скрытого чтения знания, которого персонаж не получал.
- UI не подменяет runtime pipeline.
- Оптимизация основана на измерении.

Когда эти пункты выполняются, система обычно остаётся понятной даже после добавления следующих пяти возможностей.

Полный индекс runtime исходников

Этот индекс нужен для проверки полноты. В нём перечислены собственные runtime-модули проекта и глава, где объясняется их код. Парные `.h/.cpp` считаются одной реализацией. Простые component-файлы читаются вместе с системой, которая владеет их полями.

1 Явно исключено

1. Весь `Editor` и editor-only методы UI, gizmo, selection и authoring.
2. `FloodingSystem.h/.cpp` как рудимент старой концепции подлодки.
3. Исходники сторонних библиотек: ImGui, ImGuizmo, stb, tinygltf header, Assimp, Bullet, Recast и Detour.
4. `tests` и `tools` не являются игровым runtime; инженерные главы объясняют принципы их использования.
5. HLSL-файлы не исключены: они разобраны в shader-главах как собственный код проекта.

2 Core и platform

Файл или модуль	Где разобран
`main.cpp`	V12 Core runtime, раздел 1
`SystemClass`	V12 Core runtime, разделы 2–7
`TimeManager`	V12 Core runtime, раздел 8
`GameTimer`	V12 Core runtime, раздел 9
`PhysicsWorld`	V12 Core runtime, раздел 10
`JobSystem`	V12 Core runtime, раздел 11
`FileSystem`	V12 Core runtime, разделы 12–15
`AssetsLoadSystem`	V12 Core runtime, разделы 16–20
`WorldEvent` и `WorldEventLog`	V12 Core runtime, раздел 21
`DebugHud`	V12 Core runtime, раздел 21
`ApplicationClass` runtime	V12 Application ECS input scene, разделы 1–6
`CameraController`	V12 Application ECS input scene, раздел 9
`InputClass`	V12 Application ECS input scene, раздел 7
`InputManager`	V12 Application ECS input scene, раздел 8
`GameSettings`	V7 foundations и главы соответствующих settings consumers
`CollisionUtils`	V10 Bullet physics и V12 Physics geometry AI
`helper.h`, `FileName.h`	V7 engine systems и V12 Core runtime, вспомогательные преобразования

3 ECS

Файл или модуль	Где разобран
`ECSTypes`	V7 foundations, сущность и INVALID sentinel
`IComponentPool`	V7 foundations и V12 Application ECS input scene, раздел 10
`ComponentManager<T>`	V12 Application ECS input scene, раздел 11
`ECSRegistry`	V12 Application ECS input scene, раздел 10
`Components.h`	V7 foundations и главы систем-владельцев
`TransformSystem`	V12 Application ECS input scene, раздел 12
`RenderSystem`	V12 Application ECS input scene, раздел 13
`DoorSystem`	V12 Gameplay runtime, раздел 20
`PhysicsSystem`	V12 Physics geometry AI, разделы 1–7
`JointConnectionSystem`	V12 World effects runtime, раздел 12
`SubmarineJointSystem`	V12 World effects runtime, раздел 11, историческое имя
`PressureSystem`	V12 World effects runtime, раздел 8
`MeshCuttingTest`	V12 Physics geometry AI, разделы 9–17
`RenderMesh`	V12 Render runtime, model и buffer contracts

4 Render и camera

Файл или модуль	Где разобран
`D3DClass`	V12 Render runtime, разделы 1–10
`Renderer`	V12 Render runtime, раздел 45
`CameraClass`	V8 shaders from zero, view/projection; V12 camera controller
`ViewPointClass`	V8 shaders from zero и V12 camera/controller path
`FrustumCulling` и `RenderCandidateSystem`	V12 Render runtime, раздел 42
`TextureClass`	V12 Render runtime, раздел 11
`RenderTextureClass`	V12 Render runtime, раздел 12
`ShadowMap`	V12 Render runtime, раздел 13
`BitmapClass`	V12 Render runtime, раздел 15
`WorldSpriteClass`	V12 Render runtime, раздел 16
`DWriteTextSystem`	V12 Render runtime, раздел 24
`GrassRenderer`	V12 Render runtime, разделы 20–23
`ParticleRenderer`	V12 Render runtime, раздел 25
`MaterialClass`	V12 Render runtime, раздел 47
`ModelClass` legacy	V12 Render runtime, раздел 17

Файл или модуль	Где разобран
`ModelManager`	V12 Render runtime, раздел 18
`TinyGltfModelClass`	V12 Render runtime, раздел 19, experimental placeholder
`AssimpModelClass`	V11 Assets animation studios и V12 Render runtime, раздел 48
`ModelScaleUtils`	V11 import, нормализация масштаба

5 Shader wrappers и HLSL

Файл или модуль	Где разобран
`ShaderBaseClass`	V12 Render runtime, раздел 46
`ShaderManager`	V12 Render runtime, раздел 41
`ColorShaderClass`	V12 Render runtime, раздел 27
`TextureShaderClass`	V12 Render runtime, раздел 28
`LightShaderClass`	V12 Render runtime, раздел 29
`PBRShaderClass`	V12 Render runtime, раздел 30
`NormalMapShaderClass`	V12 Render runtime, раздел 31
`LightMapShaderClass`	V12 Render runtime, раздел 32
`MultiTextureShaderClass`	V12 Render runtime, раздел 32
`FogShaderClass`	V12 Render runtime, раздел 33
`ClipPlaneShaderClass`	V12 Render runtime, раздел 34
`ReflectionShaderClass`	V12 Render runtime, раздел 35
`WaterShaderClass`	V12 Render runtime, раздел 36; графический эффект, не FloodingSystem
`GlassShaderClass`	V12 Render runtime, раздел 37
`ProjectionShaderClass`	V12 Render runtime, раздел 38
`TerrainBlendShaderClass`	V12 Render runtime, раздел 39
`SkyShaderClass`	V12 Render runtime, раздел 40
`ShadowShaderClass`	V12 Render runtime, раздел 14
Все собственные `.vs/.ps/.hlsl`	V8 shaders from zero и V11 shader pipeline, по стадиям и buffers

6 Scene и assets

Файл или модуль	Где разобран
`SceneManager` runtime ownership	V12 Application ECS input scene, разделы 14–15
`GameObject` legacy	V12 Application ECS input scene, раздел 16
`LightClass`	V8 shaders и V11 shader pipeline

Файл или модуль	Где разобран
`PickingSystem` math/runtime	V12 Application ECS input scene, раздел 17; editor selection исключён
Static asset async path	V12 Core runtime, `AssetsLoadSystem`
GLB/FBX import	V11 Assets animation studios
Embedded/external textures	V11 Assets animation studios и V12 Render runtime, раздел 48
Scene JSON save/load	V12 Application ECS input scene, раздел 14

7 Animation

Файл или модуль	Где разобран
`AnimationComponent`	V11 Assets animation studios, data ownership
`AnimationController`	V12 Application ECS input scene, раздел 18
`AnimationSystem`	V12 Application ECS input scene, раздел 19
`TwoBoneIK`	V12 Application ECS input scene, раздел 20
`WeaponIKSystem`	V12 Application ECS input scene, раздел 21
`MeleeActionJson`	V12 Application ECS input scene, раздел 22
Bone loading/interpolation	V11 Assets animation studios и V12 Render runtime, раздел 48
Animation Studio	V11 Assets animation studios, snapshot, profiles, events и live scene

8 Player combat inventory interaction

Файл или модуль	Где разобран
`GameplayController`	V12 Gameplay runtime, раздел 1
`GameplayInputSystem`	V12 Gameplay runtime, раздел 2
`CharacterSpawnSystem`	V12 Gameplay runtime, раздел 3
`CharacterMovementSystem`	V12 Gameplay runtime, раздел 4
`CharacterControllerSystem`	V12 Gameplay runtime, раздел 5
`CharacterExternalVelocitySystem`	V12 Gameplay runtime, раздел 6
`CharacterFacingSystem`	V12 Gameplay runtime, раздел 7
`FirstPersonViewSystem`	V12 Gameplay runtime, раздел 8
`PlayerVitalsSystem`	V12 Gameplay runtime, раздел 9
`PlayerComponents` и physics tuning	Разобраны рядом с системами 3–9
`WeaponComponent`	V12 Gameplay runtime, разделы 10–11
`WeaponSystem`	V12 Gameplay runtime, раздел 10

Файл или модуль	Где разобран
WeaponFiringRules	V12 Gameplay runtime, раздел 11
PlayerPunchSystem	V12 Gameplay runtime, раздел 12
DamageEvent и DamageSystem	V12 Gameplay runtime, раздел 13
InventoryComponent и InventorySystem	V12 Gameplay runtime, раздел 14
WeaponPickupSystem	V12 Gameplay runtime, раздел 15
EventSystem	V12 Gameplay runtime, раздел 16
InteractionSystem	V12 Gameplay runtime, раздел 17
TriggerZoneComponent/System	V12 Gameplay runtime, раздел 18
TriggerActionSystem	V12 Gameplay runtime, раздел 19

9 World power repair effects UI

Файл или модуль	Где разобран
PowerSystem	V12 World effects runtime, раздел 1
GeneratorComponent/System	V12 World effects runtime, раздел 2
EmergencyBatterySystem	V12 World effects runtime, раздел 3
PowerGridComponent, PowerConsumerComponent, ShipPowerGridSystem	V12 World effects runtime, раздел 4
OxygenGeneratorComponent/System	V12 World effects runtime, раздел 5
RoomSystem	V12 World effects runtime, раздел 6
RoomEnvironmentSystem	V12 World effects runtime, раздел 7
DecompressionSourceComponent	V12 World effects runtime, раздел 7
RoomCullingSystem	V12 World effects runtime, раздел 9
ShipAlertSystem	V12 World effects runtime, раздел 10
WorldAreaComponent	V12 World effects runtime, room/world classification
SubmarineSystem	V12 World effects runtime, исторические transform/group helpers
RepairComponent, RuntimeMeshDamageComponent, RepairSystem	V12 World effects runtime, раздел 13
ParticleSystem	V12 World effects runtime, раздел 14
ParticleRenderer	V12 World effects runtime, раздел 15 и Render runtime 25
ExplosionSystem	V12 World effects runtime, раздел 16
GameHud, GeneratorHud, OxygenGeneratorHud	V12 World effects runtime, раздел 17
SpotLightComponent	V8/V11 lighting и shader pipeline

10 NPC и crew AI

Файл или модуль	Где разобран
NPCComponents	V7 NPC и V11 AI architecture, владельцы данных
NpcCommandSystem	V12 Physics geometry AI, раздел 18
NpcUtilitySystem	V12 Physics geometry AI, разделы 19–20
StateSystem	V7 NPC, исполнение state/route/repair
NpcCombatSystem	V12 Physics geometry AI, раздел 22
NpcTacticalDecisionSystem	V12 Physics geometry AI, раздел 23
NpcCoverQuery	V12 Physics geometry AI, разделы 24–25
NpcCoverSystem	V12 Physics geometry AI, раздел 26
NpcCombatDecisionRules	V12 Physics geometry AI, раздел 27
NpcRangedCombatSystem	V12 Physics geometry AI, раздел 28
NpcNavMeshSystem	V12 Physics geometry AI, разделы 29–32
NpcCrowdSpacing	V12 Physics geometry AI, раздел 33
NpcSquadCombatSystem	V12 Physics geometry AI, раздел 34; необязательный group layer
CrewTaskCoordinator	V12 Physics geometry AI, раздел 21
NpcSimulationStudio	V12 Physics geometry AI, раздел 35 и V11 Studio chapter

11 Как пользоваться индексом при изучении файла

1. Найди файл в таблице.
2. Сначала прочитай его профильную V12 главу.
3. Открой исходник и подпиши возле каждого значимого блока: вход, ранний выход, чтение, вычисление, запись, событие.
4. Для непонятной формулы вернись в математическую главу.
5. Для внешнего API вернись в DirectX, Bullet или import chapter.
6. Если обнаружен файл runtime, которого нет в индексе, добавь его сюда одновременно с новой системой. Так документация остаётся проверяемой.

Учебный маршрут по DX Framework и NPC

Цель маршрута — не «прочитать побольше», а научиться объяснять код, предсказывать его поведение и самостоятельно добавлять механики без разрастания одной огромной системы.

1 Правило одного учебного блока

Каждый блок длится от 60 до 90 минут:

1. 20 минут теории.
2. 20 минут чтения текущего кода.
3. 30 минут маленькой самостоятельной реализации.
4. 10 минут теста и записи вывода.

После двух блоков делай перерыв. В конце дня сохраняй не только код, но и короткий отчёт:

```
Что хотел получить
Что предсказал до запуска
Что увидел
Какая строка объясняет результат
Что осталось непонятным
```

2 Интенсив на один свободный день

Это лучший старт перед четырёхнедельным маршрутом.

Блок 1 Векторы и луч

Теория: точка, вектор, `end - start`, длина, нормализация, конечная дальность.

Чтение: `NpcRangedCombatSystem.h`, участок от `visibleAimPoint` до вычисления `distance`.

Практика: вручную посчитать направление для двух точек, затем вывести значение DirectXMath.

Тест: нулевая длина, цель впереди, цель позади, цель выше.

Критерий готовности: можешь без подсказки объяснить каждый аргумент `CastRay`.

Блок 2 Матрица дула

Теория: матрица 4 на 4, сложение, умножение строка на столбец, порядок преобразований.

Чтение: `TransformComponent::GetWorldMatrix` и цепочка `Grip * hand * world`.

Практика: нарисовать промежуточные точки преобразования дула разными цветами.

Тест: повернуть NPC, сменить масштаб, проиграть анимацию, убедиться, что дуло следует за рукой.

Критерий готовности: можешь назвать пространство точки после каждого множителя.

Блок 3 Отладка одного выстрела

Теория: guard clause, первое попадание, дружественная линия огня.

Чтение: `NpcRangedCombatSystem` от chest-to-muzzle до `TryFire`.

Практика: поставить стену и союзника в разные участки луча. До запуска записать ожидаемый статус.

Тест: свободное дуло, заблокированное дуло, союзник в линии, враг за стеной.

Критерий готовности: по журналу видно точную первую причину запрета.

Блок 4 Маленькое самостоятельное изменение

Практика: добавить только новое поле отладки, не новую механику. Например, вывести `yawError`, `muzzleDistance`, `fireLane.entity`.

Проверка: значение сохраняется достаточно долго или попадает в ring buffer, поэтому его можно прочесть.

Что прислать на разбор: изменённый участок, один скрин инспектора, собственное объяснение из пяти предложений.

3 Неделя 1 Математика которую видно

День 1 Точки и векторы

Теория: сложение, вычитание, масштабирование.

Код: расчёты `dx`, `dy`, `dz` в боевых системах.

Задание: функция направления и длины между точками с защитой нулевого случая.

Выходной тест: пять наборов координат, включая одинаковые точки.

День 2 Dot и поле зрения

Теория: скалярное произведение, cosine, половинный FOV.

Код: текущая проверка видимости NPC.

Задание: нарисовать границы FOV и луч к кандидату.

Критерий: цель за стеной проходит FOV, но отклоняется raycast; ты можешь объяснить оба этапа.

День 3 Матрицы вручную

Теория: индексы, сложение ячеек, строка на столбец.

Задание: решить четыре матрицы 2 на 2 на бумаге, затем написать учебную функцию умножения без DirectXMath.

Критерий: все ячейки совпадают с ответом, порядок A B и B A даёт разные результаты.

День 4 Трансформации

Теория: scale, rotation, translation, world и inverse world.

Код: `GetWorldMatrix`.

Задание: преобразовать локальную контрольную точку и вернуть её обратной матрицей.

Критерий: возвращённая точка совпадает с исходной с малой погрешностью.

День 5 Лучи

Теория: параметрическое уравнение, ray против segment, ray-plane и ray-AABB.

Задание: отладочный луч из двух точек и ограниченная длина.

Критерий: препятствие за конечной точкой не считается попаданием.

День 6 Время

Теория: `deltaTime`, cooldown, fixed step, экспоненциальное сглаживание.

Задание: одинаковый таймер при 30, 60 и 144 FPS в тестовой симуляции.

Критерий: расхождение не превышает один шаг симуляции.

День 7 Проверка недели

Без книги объясни: нормализацию, dot, умножение матриц, порядок world transform, создание луча и таймер. Повтори только темы, которые не можешь объяснить на собственном примере.

4 Неделя 2 Архитектура текущего кода

День 8 ECS и владельцы данных

Код: `ECSRegistry::ForEach`, `GetComponent`, компоненты NPC.

Задание: таблица «поле → владелец → кто читает → кто пишет» для `targetEntity`, `lastKnownPosition`, `activeStyle`, `moveTarget`, `ammoInMagazine`.

Критерий: у каждого поля один понятный владелец записи.

День 9 Guard clauses

Код: `WeaponSystem::TryFire`.

Задание: для каждой ранней остановки записать, какой контракт она защищает.

Тест: искусственно вызвать каждую остановку и увидеть отдельную причину.

День 10 Состояния

Код: ветки `TakingCover`, `InCover`, `Attacking`.

Задание: нарисовать переходы и подписать условие каждого ребра.

Критерий: нет перехода «просто потому что так получилось».

День 11 Решение против исполнения

Код: `NpcTacticalDecisionSystem`, `NpcCoverQuery`, `StateSystem`.

Задание: подписать для каждой системы: вход, выход и запрещённые обязанности.

Критерий: query не меняет state, decision не строит путь, state не пересчитывает тактику.

День 12 Журнал решений

Задание: спроектировать ring buffer последних 64 решений с временем, состоянием до и после, причиной и entity кандидата.

Тест: пошаговый режим не теряет короткие промежуточные статусы.

День 13 Одна трасса кадра

Проследи один выстрел от восприятия до `DamageEvent`. Запиши функцию, поле и значение на каждом шаге.

День 14 Рефакторинг без изменения поведения

Выбери один длинный участок и выдели маленькую чистую функцию проверки. До и после сравни одинаковый сценарий и журнал.

Критерий: поведение не изменилось, причина стала читаться легче.

5 Неделя 3 Индивидуальный боевой NPC

День 15 Подтверждение контакта

Теория: perceived data против world truth.

Задание: проверить, что союзник передаёт только сообщение о контакте, а получатель сам подтверждает видимость.

Критерий: NPC не знает точную текущую позицию скрытого врага.

День 16 Выбор цели

Задание: разделить жёсткие фильтры и score цели: жив, враждебен, допустимая комната, расстояние, угроза.

Тест: игрок и NPC одновременно являются кандидатами.

День 17 Профиль укрытия

Задание: на бумаге спроектировать `CoverProfile` для стойки, приседа, левого и правого peek. Код писать только после проверки структуры.

Критерий: низкая стена не считается защитой стоящего NPC, но может защитить сидящего.

День 18 Score укрытия

Задание: жёстко отклонять незащищённые точки, остальные оценивать по защите, пути, дистанции и peek.

Тест: два NPC по одну сторону стены не считают её защитой друг от друга.

День 19 Фазы укрытия

Задание: записать таймлайн `move → hide → peek → aim → burst → return`.

Критерий: таймер экспозиции не истекает до завершения прицеливания.

День 20 Смена укрытия

Задание: смена только по понятным причинам: укрытие потеряло защиту, путь заблокирован, серия опасных peek, враг критически близко.

Критерий: NPC не бегает между укрытиями каждые две секунды без причины.

День 21 Дистанция и отступление

Задание: Defensive удерживает диапазон, Assault получает право сближаться, Balanced использует гистерезис.

Тест: при пуше одного NPC второй не повторяет симметричный пуш автоматически.

6 Неделя 4 Дружественные NPC и расширяемые модули

День 22 Память контакта

Спроектируй данные: кто сообщил, когда, где был подтверждён контакт, уверенность, время устаревания. Точная мировая позиция скрытого врага не читается напрямую.

День 23 Локальная реакция на выстрел

NPC слышит или получает попадание, поворачивается к близкому источнику, но не получает вечную цель. Оставь отдельное правило тишины для будущих тихих убийств.

День 24 Дружественная линия огня

Проверь перестроение союзника, пересечение траекторий и запрет огня. Добавь в журнал entity блокирующего союзника и длительность блокировки.

День 25 Простые роли без группового разума

Добавь роль как параметр индивидуальной тактики: Assault, Defensive, Support. Роль меняет веса и пороги, но не телепатически координирует NPC.

День 26 Контракт тактического модуля

Новый модуль должен возвращать предложение:

```
struct TacticalProposal
{
    TacticalAction action;
    float score;
    const char* reason;
};
```

Арбитр выбирает предложение, а исполнитель выполняет. Это позволяет добавлять Retreat, Suppress, Flank, ReloadInCover как пазлы.

День 27 Полигон

Создай сценарии:

- открытая площадка;
- высокая стена;
- низкая стена;
- двустороннее укрытие;
- узкий проход с союзником;
- две альтернативы пути;
- потеря контакта;
- противник быстро сближается.

Каждый сценарий должен иметь ожидаемый результат до запуска.

День 28 Вертикальный срез

Один NPC должен:

1. обнаружить врага;
2. подтвердить линию зрения;
3. выбрать роль и укрытие;
4. добежать с подходящей скоростью;
5. выбрать стойку и сторону peek;
6. проверить дуло и союзника;
7. сделать ограниченную очередь;
8. перезарядиться за укрытием;
9. сменить позицию только по объяснимой причине;
10. потерять контакт без всезнания.

7 Очерёдность будущих модулей

Добавляй по одному модулю и не смешивай их правила:

Порядок	Модуль	Вход	Предложение
1	Reload safety	патроны, укрытие, угроза	перезарядиться сейчас
2	Retreat	дистанция, здоровье, путь	увеличить дистанцию
3	Suppression	последняя подтверждённая зона, боезапас	подавлять область
4	Flank	профиль укрытия врага, маршруты	обойти сбоку
5	Grenade	группа целей, траектория, союзники	бросить гранату
6	Revive	раненый союзник, риск пути	помочь союзнику
7	Squad coordinator	предложения отдельных NPC	распределить роли

Групповая тактика идёт последней. Сначала одиночный NPC должен быть предсказуемым и объяснимым.

8 Формат мини задания для совместной проверки

Когда присылаешь решение, достаточно пяти частей:

1. Что хотел изменить.
2. Какое условие написал своими словами.
3. Сам изменённый участок кода.
4. Что ожидал до запуска.
5. Что реально произошло и какой статус был в журнале.

Так можно проверить твоё мышление, не отбирая у тебя самостоятельную работу готовым большим куском кода.

9 Критерии окончания маршрута

Маршрут закончен, когда ты можешь:

- вручную построить вектор, матрицу и луч;
- построчно объяснить выбранную систему;
- назвать владельца каждого изменяемого поля;
- отделить восприятие от истинного состояния мира;
- отделить решение от запроса геометрии и исполнения;
- добавить новый тактический модуль через общий контракт;
- доказать решение NPC журналом и трассерами;
- написать тестовый сценарий до запуска;
- найти баг по цепочке данных, а не случайными правками.